MDN JavaScript Documentation

Volume 3 of 9 — Reference: Functions · Classes · Regular expressions · Errors · Misc

Unofficial print compilation. Functions, classes, regular expression syntax, the complete error reference, and the miscellaneous reference pages (lexical grammar, execution model, iteration protocols, strict mode and others). This volume contains 179 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

Functions 5
- Arrow function expressions 14
- Default parameters 23
- get 29
- Method definitions 34
- Rest parameters 39
- set 44
- The arguments object 47
  - arguments[Symbol.iterator]() 52
  - arguments.callee 54
  - arguments.length 58

Classes 60
- constructor 67
- extends 73
- Private elements 82
- Public class fields 91
- static 96
- Static initialization blocks 101

Regular expressions 106
- Backreference: \1, \2 111
- Buffer boundary assertion: \A, \z, \Z 114
- Capturing group: (...) 116
- Character class escape: \d, \D, \w, \W, \s, \S 119
- Character class: [...], [^...] 121
- Character escape: \n, \u{...} 127
- Disjunction: | 130
- Input boundary assertion: ^, $ 132
- Literal character: a, b 134
- Lookahead assertion: (?=...), (?!...) 137
- Lookbehind assertion: (?<=...), (?<!...) 140
- Modifier: (?ims-ims:...) 142
- Named backreference: \k<name> 145
- Named capturing group: (?<name>...) 146
- Non-capturing group: (?:...) 149
- Quantifier: *, +, ?, {n}, {n,}, {n,m} 151
- Unicode character class escape: \p{...}, \P{...} 155
- Wildcard: . 160
- Word boundary assertion: \b, \B 162

JavaScript error reference 164
- AggregateError: No Promise in Promise.any was resolved 167
- Error: Permission denied to access property "x" 169
- InternalError: too much recursion 170
- RangeError: argument is not a valid code point 172
- RangeError: BigInt division by zero 173
- RangeError: BigInt negative exponent 174
- RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD' 175
- RangeError: invalid array length 175
- RangeError: invalid date 177
- RangeError: precision is out of range 179
- RangeError: radix must be an integer 180
- RangeError: repeat count must be less than infinity 181
- RangeError: repeat count must be non-negative 182
- RangeError: x can't be converted to BigInt because it isn't an integer 183
- ReferenceError: "x" is not defined 184
- ReferenceError: assignment to undeclared variable "x" 185
- ReferenceError: can't access lexical declaration 'X' before initialization 187
- ReferenceError: must call super constructor before using 'this' in derived class constructor 189
- ReferenceError: super() called twice in derived class constructor 190
- SyntaxError: 'arguments'/'eval' can't be defined or assigned to in strict mode code 192
- SyntaxError: "0"-prefixed octal literals are deprecated 193
- SyntaxError: "use strict" not allowed in function with non-simple parameters 194
- SyntaxError: "x" is a reserved identifier 196
- SyntaxError: \ at end of pattern 197
- SyntaxError: a declaration in the head of a for-of loop can't have an initializer 198
- SyntaxError: applying the 'delete' operator to an unqualified name is deprecated 200
- SyntaxError: arguments is not valid in fields 201
- SyntaxError: await is only valid in async functions, async generators and modules 202
- SyntaxError: await/yield expression can't be used in parameter 204
- SyntaxError: cannot use ?? unparenthesized within || and && expressions 205
- SyntaxError: character class escape cannot be used in class range in regular expression 207
- SyntaxError: continue must be inside loop 208
- SyntaxError: duplicate capture group name in regular expression 209
- SyntaxError: duplicate formal argument x 210
- SyntaxError: for-in loop head declarations may not have initializers 212
- SyntaxError: function statement requires a name 213
- SyntaxError: functions cannot be labelled 215
- SyntaxError: getter and setter for private name #x should either be both static or non-static 217
- SyntaxError: getter functions must have no arguments 218
- SyntaxError: identifier starts immediately after numeric literal 220
- SyntaxError: illegal character 221
- SyntaxError: import declarations may only appear at top level of a module 223
- SyntaxError: incomplete quantifier in regular expression 224
- SyntaxError: invalid assignment left-hand side 225
- SyntaxError: invalid BigInt syntax 227
- SyntaxError: invalid capture group name in regular expression 228
- SyntaxError: invalid character in class in regular expression 229

SyntaxError: invalid class set operation in regular expression 230
SyntaxError: invalid decimal escape in regular expression 231
SyntaxError: invalid identity escape in regular expression 232
SyntaxError: invalid named capture reference in regular expression 233
SyntaxError: invalid property name in regular expression 234
SyntaxError: invalid range in character class 235
SyntaxError: invalid regexp group 236
SyntaxError: invalid regular expression flag "x" 238
SyntaxError: invalid unicode escape in regular expression 239
SyntaxError: JSON.parse: bad parsing 240
SyntaxError: label not found 242
SyntaxError: missing : after property id 243
SyntaxError: missing) after argument list 244
SyntaxError: missing) after condition 246
SyntaxError: missing] after element list 247
SyntaxError: missing } after function body 248
SyntaxError: missing } after property list 250
SyntaxError: missing = in const declaration 251
SyntaxError: missing formal parameter 252
SyntaxError: missing name after . operator 253
SyntaxError: missing variable name 255
SyntaxError: negated character class with strings in regular expression 256
SyntaxError: new keyword cannot be used with an optional chain 257
SyntaxError: nothing to repeat 259
SyntaxError: numbers out of order in {} quantifier. 260
SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code 261
SyntaxError: parameter after rest parameter 262
SyntaxError: private fields can't be deleted 263
SyntaxError: property name __proto__ appears more than once in object literal 264
SyntaxError: raw bracket is not allowed in regular expression with unicode flag 265
SyntaxError: redeclaration of formal parameter "x" 266
SyntaxError: reference to undeclared private field or method #x 267
SyntaxError: rest parameter may not have a default 269
SyntaxError: return not in function 270
SyntaxError: setter functions must have one argument 271
SyntaxError: string literal contains an unescaped line break 272
SyntaxError: super() is only valid in derived class constructors 273
SyntaxError: tagged template cannot be used with optional chain 275
SyntaxError: Unexpected '#' used outside of class body 277
SyntaxError: Unexpected token 278
SyntaxError: unlabeled break must be inside loop or switch 280
SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**' 283
SyntaxError: use of super property/member accesses only valid within methods or eval code within methods 284
SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead 286
TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed 287
TypeError: 'x' is not iterable 289
TypeError: "x" is (not) "y" 292
TypeError: "x" is not a constructor 293
TypeError: "x" is not a function 295
TypeError: "x" is not a non-null object 298
TypeError: "x" is read-only 299
TypeError: already executing generator 301
TypeError: BigInt value can't be serialized in JSON 302
TypeError: calling a builtin X constructor without new is forbidden 304
TypeError: can't access/set private field or method: object is not the right class 305
TypeError: can't assign to property "x" on "y": not an object 309
TypeError: can't convert BigInt to number 310
TypeError: can't convert x to BigInt 311
TypeError: can't define property "x": "obj" is not extensible 312
TypeError: can't delete non-configurable array element 314
TypeError: can't redefine non-configurable property "x" 316
TypeError: can't set prototype of this object 317
TypeError: can't set prototype: it would cause a prototype chain cycle 318
TypeError: cannot use 'in' operator to search for 'x' in 'y' 319
TypeError: class constructors must be invoked with 'new' 320
TypeError: cyclic object value 321
TypeError: derived class constructor returned invalid value x 323
TypeError: getting private setter-only property 324
TypeError: Initializing an object twice is an error with private fields/methods 325
TypeError: invalid 'instanceof' operand 'x' 326
TypeError: invalid Array.prototype.sort argument 328
TypeError: invalid assignment to const "x" 329
TypeError: Iterator/AsyncIterator constructor can't be used directly 331
TypeError: matchAll/replaceAll must be called with a global RegExp 332
TypeError: More arguments needed 333
TypeError: null/undefined has no properties 334
TypeError: property "x" is non-configurable and can't be deleted 335

TypeError: Reduce of empty array with no initial value 336
TypeError: setting getter-only property "x" 338
TypeError: WeakSet key/WeakMap value 'x' must be an object or an unregistered symbol 340
TypeError: X.prototype.y called on incompatible type 341
URIError: malformed URI sequence 343
Warning: -file- is being assigned a //# sourceMappingURL, but already has one 344
Warning: unreachable code after return statement 345

JavaScript technologies overview **347**
JavaScript execution model **352**
Lexical grammar **361**
Magic comments **379**
Iteration protocols **385**
Strict mode **398**
Template literals (Template strings) **408**
Trailing commas **416**
Deprecated and obsolete features **422**

JavaScript › Reference

Functions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since July 2015.

Generally speaking, a function is a "subprogram" that can be *called* by code external (or internal, in the case of recursion) to the function. Like the program itself, a function is composed of a sequence of statements called the *function body*. Values can be *passed* to a function as parameters, and the function will *return* a value.

In JavaScript, functions are first-class objects, because they can be passed to other functions, returned from functions, and assigned to variables and properties. They can also have properties and methods just like any other object. What distinguishes them from other objects is that functions can be called.

For more examples and explanations, see the JavaScript guide about functions.

Description

Function values are typically instances of `Function`. See `Function` for information on properties and methods of `Function` objects. Callable values cause `typeof` to return `"function"` instead of `"object"`.

> **Note:** Not all callable values are `instanceof Function`. For example, the `Function.prototype` object is callable but not an instance of `Function`. You can also manually set the prototype chain of your function so it no longer inherits from `Function.prototype`. However, such cases are extremely rare.

Return value

By default, if a function's execution doesn't end at a `return` statement, or if the `return` keyword doesn't have an expression after it, then the return value is `undefined`. The `return` statement allows you to return an arbitrary value from the function. One function call can only return one value, but you can simulate the effect of returning multiple values by returning an object or array and destructuring the result.

> **Note:** Constructors called with `new` have a different set of logic to determine their return values.

Passing arguments

Parameters and arguments have slightly different meanings, but in MDN web docs, we often use them interchangeably. For a quick reference:

```js
function formatNumber(num) {
  return num.toFixed(2);
}

formatNumber(2);
```

In this example, the `num` variable is called the function's *parameter*: it's declared in the parenthesis-enclosed list of the function's definition. The function expects the `num` parameter to be a number — although this is not enforceable in JavaScript without writing runtime validation code. In the `formatNumber(2)` call, the number `2` is the function's *argument*: it's the value that is actually passed to the function in the function call. The argument value can be accessed inside the function body through the corresponding parameter name or the `arguments` object.

Arguments are always *passed by value* and never *passed by reference*. This means that if a function reassigns a parameter, the value won't change outside the function. More precisely, object arguments are *passed by sharing*, which means if the object's properties are mutated, the change will impact the outside of the function. For example:

```js
function updateBrand(obj) {
  // Mutating the object is visible outside the function
  obj.brand = "Toyota";
  // Try to reassign the parameter, but this won't affect
  // the variable's value outside the function
  obj = null;
}

const car = {
  brand: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(car.brand); // Honda

// Pass object reference to the function
updateBrand(car);

// updateBrand mutates car
console.log(car.brand); // Toyota
```

The `this` keyword refers to the object that the function is accessed on — it does not refer to the currently executing function, so you must refer to the function value by name, even within the function body.

Defining functions

Broadly speaking, JavaScript has four kinds of functions:

- Regular function: can return anything; always runs to completion after invocation
- Generator function: returns a `Generator` object; can be paused and resumed with the `yield` operator
- Async function: returns a `Promise`; can be paused and resumed with the `await` operator
- Async generator function: returns an `AsyncGenerator` object; both the `await` and `yield` operators can be used

For every kind of function, there are multiple ways to define it:

Declaration

`function`, `function*`, `async function`, `async function*`

Expression

`function`, `function*`, `async function`, `async function*`

Constructor

`Function()`, `GeneratorFunction()`, `AsyncFunction()`, `AsyncGeneratorFunction()`

In addition, there are special syntaxes for defining arrow functions and methods, which provide more precise semantics for their usage. Classes are conceptually not functions (because they throw an error

when called without `new`), but they also inherit from `Function.prototype` and have `typeof MyClass === "function"`.

```js
// Constructor
const multiply = new Function("x", "y", "return x * y");

// Declaration
function multiply(x, y) {
  return x * y;
} // No need for semicolon here

// Expression; the function is anonymous but assigned to a variable
const multiply = function (x, y) {
  return x * y;
};
// Expression; the function has its own name
const multiply = function funcName(x, y) {
  return x * y;
};

// Arrow function
const multiply = (x, y) => x * y;

// Method
const obj = {
  multiply(x, y) {
    return x * y;
  },
};
```

All syntaxes do approximately the same thing, but there are some subtle behavior differences.

- The `Function()` constructor, `function` expression, and `function` declaration syntaxes create full-fledged function objects, which can be constructed with `new`. However, arrow functions and methods cannot be constructed. Async functions, generator functions, and async generator functions are not constructible regardless of syntax.
- The `function` declaration creates functions that are *hoisted*. Other syntaxes do not hoist the function and the function value is only visible after the definition.
- The arrow function and `Function()` constructor always create *anonymous* functions, which means they can't easily call themselves recursively. One way to call an arrow function recursively is by assigning it to a variable.
- The arrow function syntax does not create its own bindings for `arguments` or `this`.
- The `Function()` constructor cannot access any local variables — it only has access to the global scope.
- The `Function()` constructor causes runtime compilation and is often slower than other syntaxes.

For `function` expressions, there is a distinction between the function name and the variable the function is assigned to. The function name cannot be changed, while the variable the function is assigned to can be reassigned. The function name can be different from the variable the function is assigned to — they have no relation to each other. The function name can be used only within the function's body. Attempting to use it outside the function's body results in an error (or gets another value, if the same name is declared elsewhere). For example:

```js
const y = function x() {};
console.log(x); // ReferenceError: x is not defined
```

On the other hand, the variable the function is assigned to is limited only by its scope, which is guaranteed to include the scope in which the function is declared.

A function declaration also creates a variable with the same name as the function name. Thus, unlike those defined by function expressions, functions defined by function declarations can be accessed by their name in the scope they were defined in, as well as in their own body.

A function defined by `new Function` will dynamically have its source assembled, which is observable when you serialize it. For example, `console.log(new Function().toString())` gives:

```js
function anonymous(
) {

}
```

This is the actual source used to compile the function. However, although the `Function()` constructor will create the function with name `anonymous`, this name is not added to the scope of the body. The body only ever has access to global variables. For example, the following would result in an error:

```js
new Function("alert(anonymous);")();
```

A function defined by a function expression or by a function declaration inherits the current scope. That is, the function forms a closure. On the other hand, a function defined by a `Function` constructor does not inherit any scope other than the global scope (which all functions inherit).

```js
// p is a global variable
globalThis.p = 5;
function myFunc() {
  // p is a local variable
  const p = 9;

  function decl() {
    console.log(p);
  }
  const expr = function () {
    console.log(p);
  };
  const cons = new Function("\tconsole.log(p);");

  decl();
  expr();
  cons();
}
myFunc();

// Logs:
// 9 (for 'decl' by function declaration (current scope))
// 9 (for 'expr' by function expression (current scope))
// 5 (for 'cons' by Function constructor (global scope))
```

Functions defined by function expressions and function declarations are parsed only once, while a function defined by the `Function` constructor parses the string passed to it each and every time the constructor is called. Although a function expression creates a closure every time, the function body is not reparsed, so function expressions are still faster than `new Function(...)`. Therefore the `Function` constructor should generally be avoided whenever possible.

A function declaration may be unintentionally turned into a function expression when it appears in an expression context.

js

```
// A function declaration
function foo() {
  console.log("FOO!");
}

doSomething(
  // A function expression passed as an argument
  function foo() {
    console.log("FOO!");
  },
);
```

On the other hand, a function expression may also be turned into a function declaration. An expression statement cannot begin with the `function` or `async function` keywords, which is a common mistake when implementing IIFEs (Immediately Invoked Function Expressions).

js · ✗ bad example

```
function () { // SyntaxError: Function statements require a function name
  console.log("FOO!");
}();

function foo() {
  console.log("FOO!");
}(); // SyntaxError: Unexpected token ')'
```

Instead, start the expression statement with something else, so that the `function` keyword unambiguously starts a function expression. Common options include grouping and using `void`.

js · ✓ good example

```
(function () {
  console.log("FOO!");
})();

void function () {
  console.log("FOO!");
}();
```

Function parameters

Each function parameter is a simple identifier that you can access in the local scope.

js

```
function myFunc(a, b, c) {
  // You can access the values of a, b, and c here
}
```

There are three special parameter syntaxes:

- *Default parameters* allow formal parameters to be initialized with default values if no value or `undefined` is passed.
- The *rest parameter* allows representing an indefinite number of arguments as an array.
- *Destructuring* allows unpacking elements from arrays, or properties from objects, into distinct variables.

```js
function myFunc({ a, b }, c = 1, ...rest) {
  // You can access the values of a, b, c, and rest here
}
```

There are some consequences if one of the above non-simple parameter syntaxes is used:

- You cannot apply `"use strict"` to the function body — this causes a syntax error.
- Even if the function is not in strict mode, certain strict mode function features apply, including that the `arguments` object stops syncing with the named parameters, `arguments.callee` throws an error when accessed, and duplicate parameter names are not allowed.

The arguments object

You can refer to a function's arguments within the function by using the `arguments` object.

`arguments`
: An array-like object containing the arguments passed to the currently executing function.

`arguments.callee`
: The currently executing function.

`arguments.length`
: The number of arguments passed to the function.

Getter and setter functions

You can define accessor properties on any standard built-in object or user-defined object that supports the addition of new properties. Within object literals and classes, you can use special syntaxes to define the getter and setter of an accessor property.

`get`
: Binds an object property to a function that will be called when that property is looked up.

`set`
: Binds an object property to a function to be called when there is an attempt to set that property.

Note that these syntaxes create an *object property*, not a *method*. The getter and setter functions themselves can only be accessed using reflective APIs such as `Object.getOwnPropertyDescriptor()`.

Block-level functions

In strict mode, functions inside blocks are scoped to that block. Prior to ES2015, block-level functions were forbidden in strict mode.

```js
"use strict";

function f() {
  return 1;
}

{
  function f() {
    return 2;
  }
}

f() === 1; // true

// f() === 2 in non-strict mode
```

Block-level functions in non-strict code

In a word: **Don't.**

In non-strict code, function declarations inside blocks behave strangely. For example:

```js
if (shouldDefineZero) {
  function zero() {
    // DANGER: compatibility risk
    console.log("This is zero.");
  }
}
```

The semantics of this in strict mode are well-specified — `zero` only ever exists within that scope of the `if` block. If `shouldDefineZero` is false, then `zero` should never be defined, since the block never executes. However, historically, this was left unspecified, so different browsers implemented it differently in non-strict mode. For more information, see the `function` declaration reference.

A safer way to define functions conditionally is to assign a function expression to a variable:

```js
// Using a var makes it available as a global variable,
// with closer behavior to a top-level function declaration
var zero;
if (shouldDefineZero) {
  zero = function () {
    console.log("This is zero.");
  };
}
```

Examples

Returning a formatted number

The following function returns a string containing the formatted representation of a number padded with leading zeros.

```js
// This function returns a string padded with leading zeros
function padZeros(num, totalLen) {
  let numStr = num.toString(); // Initialize return value as string
  const numZeros = totalLen - numStr.length; // Calculate no. of zeros
  for (let i = 1; i <= numZeros; i++) {
    numStr = `0${numStr}`;
  }
  return numStr;
}
```

The following statements call the `padZeros` function.

```js
let result;
result = padZeros(42, 4); // returns "0042"
result = padZeros(42, 2); // returns "42"
result = padZeros(5, 4); // returns "0005"
```

Determining whether a function exists

You can determine whether a function exists by using the `typeof` operator. In the following example, a test is performed to determine if the `window` object has a property called `noFunc` that is a function. If so, it is used; otherwise, some other action is taken.

```js
if (typeof window.noFunc === "function") {
  // use noFunc()
} else {
  // do something else
}
```

Note that in the `if` test, a reference to `noFunc` is used — there are no parentheses `()` after the function name so the actual function is not called.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
functions	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
arguments	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
[Symbol.iterator]	52	12	46	39	9	52	46	41	9	6.0	52	9	1.0.0	1.0	4.0.0
callee Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
length	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Arrow functions	45	12	22[1,2]	32	10	45	22[3,4]	32	10	5.0	45	10	1.0.0	1.0	4.0.0
Trailing comma in parameters	58	12	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
Block-level functions	49	12	46	36	10	49	46	36	10	5.0	49	10	1.0.0	1.0	4.0.0
Default parameters	49	14	15	36	10	49	15	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Destructured parameter with default value assignment	49	14	41	36	10	49	41	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Parameters without defaults after default parameters	49	14	26	36	10	49	26	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`get`	1	12	1.5	9.5	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	46	12	34	47	9.1	46	34	33	9.3	5.0	46	9.3	1.0.0	1.0	4.0.0
Method definitions	39	12	34	26	9	39	34	26	9	4.0	39	9	1.0.0	1.0	4.0.0
Async generator methods	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0
Async methods	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Generator methods are not constructable (ES2016)	42	13	43	29	9.1	42	43	29	9.3	4.0	42	9.3	1.0.0	1.0	6.0.0
Rest parameters	47	12	15	34	10	47	15	34	10	5.0	47	10	1.0.0	1.0	6.0.0
Destructuring rest parameters	49	79	52	36	10	49	52	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`set`	1	12	1.5	9.5	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	46	12	34	47	9.1	46	34	33	9.3	5.0	46	9.3	1.0.0	1.0	4.0.0

1. The initial implementation of arrow functions in Firefox made them automatically strict. This has been changed as of Firefox 24. The use of `'use strict';` is now required.
2. Before Firefox 39, a line terminator (`\n`) was incorrectly allowed after arrow function arguments. This has been fixed to conform to the ES2015 specification and code like `() \n => {}` will now throw a `SyntaxError` in this and later versions.
3. The initial implementation of arrow functions in Firefox for Android made them automatically strict. This has been changed as of Firefox for Android 24. The use of `'use strict';` is now required.
4. Before Firefox for Android 39, a line terminator (`\n`) was incorrectly allowed after arrow function arguments. This has been fixed to conform to the ES2015 specification and code like `() \n => {}` will now throw a `SyntaxError` in this and later versions.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Classes
- `function`
- `function` expression
- `Function`

JavaScript › Reference › Functions

Arrow function expressions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions · MDN page last modified 2026-09-04

Baseline: Widely available — available across major browsers since September 2016.

An **arrow function expression** is a compact alternative to a traditional function expression, with some semantic differences and deliberate limitations in usage:

- Arrow functions don't have their own bindings to `this`, `arguments`, or `super`, and should not be used as methods.
- Arrow functions cannot be used as constructors. Calling them with `new` throws a `TypeError`. They also don't have access to the `new.target` keyword.
- Arrow functions cannot use `yield` within their body and cannot be created as generator functions.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Arrow function expressions". It runs live in the browser there; its source code follows.

js · interactive example source

```
const materials = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

console.log(materials.map((material) => material.length));
// Expected output: Array [8, 6, 7, 9]
```

Syntax

js

```
() => expression

param => expression

(param) => expression

(param1, paramN) => expression

() => {
  statements
}

param => {
  statements
}

(param1, paramN) => {
  statements
}
```

Rest parameters, default parameters, and destructuring within params are supported, and always require parentheses:

```js
(a, b, ...r) => expression
(a = 400, b = 20, c) => expression
([a, b] = [10, 20]) => expression
({ a, b } = { a: 10, b: 20 }) => expression
```

Arrow functions can be async by prefixing the expression with the `async` keyword.

```js
async param => expression
async (param1, param2, ...paramN) => {
  statements
}
```

Description

Let's decompose a traditional anonymous function down to the simplest arrow function step-by-step. Each step along the way is a valid arrow function.

> **Note:** Traditional function expressions and arrow functions have more differences than their syntax. We will introduce their behavior differences in more detail in the next few subsections.

```js
// Traditional anonymous function
(function (a) {
  return a + 100;
});

// 1. Remove the word "function" and place arrow between the argument and opening body brace
(a) => {
  return a + 100;
};

// 2. Remove the body braces and word "return" — the return is implied.
(a) => a + 100;

// 3. Remove the parameter parentheses
a => a + 100;
```

In the example above, both the parentheses around the parameter and the braces around the function body may be omitted. However, they can only be omitted in certain cases.

The parentheses can only be omitted if the function has a single simple parameter. If it has multiple parameters, no parameters, or default, destructured, or rest parameters, the parentheses around the parameter list are required.

```js
// Traditional anonymous function
(function (a, b) {
  return a + b + 100;
});

// Arrow function
(a, b) => a + b + 100;
```

(code continues on the next page)

(code continued from the previous page)

```
const a = 4;
const b = 2;

// Traditional anonymous function (no parameters)
(function () {
  return a + b + 100;
});

// Arrow function (no parameters)
() => a + b + 100;
```

The braces can only be omitted if the function directly returns an expression. If the body has statements, the braces are required. In this case, return values must be explicitly specified with the `return` keyword. Arrow functions cannot guess what or when you want to return.

js

```
// Traditional anonymous function
(function (a, b) {
  const chuck = 42;
  return a + b + chuck;
});

// Arrow function
(a, b) => {
  const chuck = 42;
  return a + b + chuck;
};
```

Arrow functions are not inherently associated with a name. If the arrow function needs to call itself, use a named function expression instead. You can also assign the arrow function to a variable, allowing you to refer to it through that variable.

js

```
// Traditional Function
function bob(a) {
  return a + 100;
}

// Arrow Function
const bob2 = (a) => a + 100;
```

Function body

Arrow functions can have either an *expression body* or the usual *block body*.

In an expression body, only a single expression is specified, which becomes the implicit return value. The block body is analogous to traditional function bodies, where return values must be explicitly specified with the `return` keyword. Arrow functions are not required to return a value. If execution of the block body reaches the end without encountering a `return` statement, the function returns `undefined` like other functions.

```js
// Expression body
const add = (a, b) => a + b; // Implicitly returns a + b

// Block body
const add2 = (a, b) => {
  console.log(a, b);
  return a + b; // Must explicitly return a value
};

// No return value
const add3 = (b) => {
  a += b;
  // No return statement, so returns undefined
};
```

Returning object literals using the expression body syntax `(params) => { object: literal }` does not work as expected.

js · ✗ bad example

```
const func = () => { foo: 1 };
// Calling func() returns undefined!

const func2 = () => { foo: function () {} };
// SyntaxError: function statement requires a name

const func3 = () => { foo() {} };
// SyntaxError: Unexpected token '{'
```

This is because JavaScript only sees the arrow function as having an expression body if the token following the arrow is not a left brace, so the code inside braces ({}) is parsed as a sequence of statements, where `foo` is a label, not a key in an object literal.

To fix this, wrap the object literal in parentheses:

js · ✓ good example

```
const func = () => ({ foo: 1 });
```

Cannot be used as methods

Arrow function expressions should only be used for non-method functions because they do not have their own `this`. Let's see what happens when we try to use them as methods:

```js
"use strict";

const obj = {
  i: 10,
  b: () => console.log(this.i, this),
  c() {
    console.log(this.i, this);
  },
};

obj.b(); // logs undefined, Window { /* … */ } (or the global object)
obj.c(); // logs 10, Object { /* … */ }
```

Another example involving `Object.defineProperty()`:

```js
"use strict";

const obj = {
  a: 10,
};

Object.defineProperty(obj, "b", {
  get: () => {
    console.log(this.a, typeof this.a, this); // undefined 'undefined' Window { /* … */ }
(or the global object)
    return this.a + 10; // represents global object 'Window', therefore 'this.a' returns
'undefined'
  },
});
```

Because a class's body has a `this` context, arrow functions as class fields close over the class's `this` context, and the `this` inside the arrow function's body will correctly point to the instance (or the class itself, for static fields). However, because it is a closure, not the function's own binding, the value of `this` will not change based on the execution context.

```js
class C {
  a = 1;
  autoBoundMethod = () => {
    console.log(this.a);
  };
}

const c = new C();
c.autoBoundMethod(); // 1
const { autoBoundMethod } = c;
autoBoundMethod(); // 1
// If it were a normal method, it should be undefined in this case
```

Arrow function properties are often said to be "auto-bound methods", because the equivalent with normal methods is:

```js
class C {
  a = 1;
  constructor() {
    this.method = this.method.bind(this);
  }
  method() {
    console.log(this.a);
  }
}
```

> **Note:** Class fields are defined on the *instance*, not on the *prototype*, so every instance creation would create a new function reference and allocate a new closure, potentially leading to more memory usage than a normal unbound method.

For similar reasons, the `call()`, `apply()`, and `bind()` methods are not useful when called on arrow functions, because arrow functions establish `this` based on the scope the arrow function is defined within, and the `this` value does not change based on how the function is invoked.

No binding of arguments

Arrow functions do not have their own arguments object. Thus, in this example, arguments is a reference to the arguments of the enclosing scope:

```js
function foo(n) {
  const f = () => arguments[0] + n; // foo's implicit arguments binding. arguments[0] is n
  return f();
}

foo(3); // 3 + 3 = 6
```

In most cases, using rest parameters is a good alternative to using an arguments object.

```js
function foo(n) {
  const f = (...args) => args[0] + n;
  return f(10);
}

foo(1); // 11
```

Cannot be used as constructors

Arrow functions cannot be used as constructors and will throw an error when called with new. They also do not have a prototype property.

```js
const Foo = () => {};
const foo = new Foo(); // TypeError: Foo is not a constructor
console.log("prototype" in Foo); // false
```

Cannot be used as generators

The yield keyword cannot be used in an arrow function's body (except when used within generator functions further nested within the arrow function). As a consequence, arrow functions cannot be used as generators.

Line break before arrow

An arrow function cannot contain a line break between its parameters and its arrow.

js · ✗ bad example

```
const func = (a, b, c)
  => 1;
// SyntaxError: Unexpected token '=>'
```

For the purpose of formatting, you may put the line break after the arrow or use parentheses/braces around the function body, as shown below. You can also put line breaks between parameters.

```js
const func = (a, b, c) =>
  1;

const func2 = (a, b, c) => (
  1
);

const func3 = (a, b, c) => {
  return 1;
};

const func4 = (
  a,
  b,
  c,
) => 1;
```

Precedence of arrow

Although the arrow in an arrow function is not an operator, arrow functions have special parsing rules that interact differently with operator precedence compared to regular functions.

js · ✗ bad example

```
let callback;

callback = callback || () => {};
// SyntaxError: invalid arrow-function arguments
```

Because `=>` has a lower precedence than most operators, parentheses are necessary to avoid `callback || ()` being parsed as the arguments list of the arrow function.

js · ✓ good example

```
callback = callback || (() => {});
```

Examples

Using arrow functions

```js
// An empty arrow function returns undefined
const empty = () => {};

(() => "foobar")();
// Returns "foobar"
// (this is an Immediately Invoked Function Expression)

const simple = (a) => (a > 15 ? 15 : a);
simple(16); // 15
simple(10); // 10

const max = (a, b) => (a > b ? a : b);
```

(code continues on the next page)

(code continued from the previous page)

```
// Easy array filtering, mapping, etc.
const arr = [5, 6, 13, 0, 1, 18, 23];

const sum = arr.reduce((a, b) => a + b);
// 66

const even = arr.filter((v) => v % 2 === 0);
// [6, 0, 18]

const double = arr.map((v) => v * 2);
// [10, 12, 26, 0, 2, 36, 46]

// More concise promise chains
promise
  .then((a) => {
    // …
  })
  .then((b) => {
    // …
  });

// Arrow functions without parameters
setTimeout(() => {
  console.log("I happen sooner");
  setTimeout(() => {
    // deeper code
    console.log("I happen later");
  }, 1);
}, 1);
```

Using call, bind, and apply

The `call()`, `apply()`, and `bind()` methods work as expected with traditional functions, because we establish the scope for each of the methods:

js

```
const obj = {
  num: 100,
};

// Setting "num" on globalThis to show how it is NOT used.
globalThis.num = 42;

// A traditional function to operate on "this"
function add(a, b, c) {
  return this.num + a + b + c;
}

console.log(add.call(obj, 1, 2, 3)); // 106
console.log(add.apply(obj, [1, 2, 3])); // 106
const boundAdd = add.bind(obj);
console.log(boundAdd(1, 2, 3)); // 106
```

With arrow functions, since our `add` function is essentially created on the `globalThis` (global) scope, it will assume `this` is the `globalThis`.

```js
const obj = {
  num: 100,
};

// Setting "num" on globalThis to show how it gets picked up.
globalThis.num = 42;

// Arrow function
const add = (a, b, c) => this.num + a + b + c;

console.log(add.call(obj, 1, 2, 3)); // 48
console.log(add.apply(obj, [1, 2, 3])); // 48
const boundAdd = add.bind(obj);
console.log(boundAdd(1, 2, 3)); // 48
```

Perhaps the greatest benefit of using arrow functions is with methods like `setTimeout()` and `EventTarget.prototype.addEventListener()` that usually require some kind of closure, `call()`, `apply()`, or `bind()` to ensure that the function is executed in the proper scope.

With traditional function expressions, code like this does not work as expected:

```js
const obj = {
  count: 10,
  doSomethingLater() {
    setTimeout(function () {
      // the function executes on the window scope
      this.count++;
      console.log(this.count);
    }, 300);
  },
};

obj.doSomethingLater(); // logs "NaN", because the property "count" is not in the window
scope.
```

With arrow functions, the `this` scope is more easily preserved:

```js
const obj = {
  count: 10,
  doSomethingLater() {
    // The method syntax binds "this" to the "obj" context.
    setTimeout(() => {
      // Since the arrow function doesn't have its own binding and
      // setTimeout (as a function call) doesn't create a binding
      // itself, the "obj" context of the outer method is used.
      this.count++;
      console.log(this.count);
    }, 300);
  },
};

obj.doSomethingLater(); // logs 11
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-arrow-function-definitions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Arrow functions	45	12	22[1,2]	32	10	45	22[3,4]	32	10	5.0	45	10	1.0.0	1.0	4.0.0
Trailing comma in parameters	58	12	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0

1. The initial implementation of arrow functions in Firefox made them automatically strict. This has been changed as of Firefox 24. The use of `'use strict';` is now required.
2. Before Firefox 39, a line terminator (`\n`) was incorrectly allowed after arrow function arguments. This has been fixed to conform to the ES2015 specification and code like `() \n => {}` will now throw a `SyntaxError` in this and later versions.
3. The initial implementation of arrow functions in Firefox for Android made them automatically strict. This has been changed as of Firefox for Android 24. The use of `'use strict';` is now required.
4. Before Firefox for Android 39, a line terminator (`\n`) was incorrectly allowed after arrow function arguments. This has been fixed to conform to the ES2015 specification and code like `() \n => {}` will now throw a `SyntaxError` in this and later versions.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.arrow_functions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `function`
- `function` expression
- ES6 In Depth: Arrow functions on hacks.mozilla.org (2015)

JavaScript › Reference › Functions

Default parameters

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2016.

Default function parameters allow named parameters to be initialized with default values if no value or `undefined` is passed.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Default parameters". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function multiply(a, b = 1) {
  return a * b;
}

console.log(multiply(5, 2));
// Expected output: 10

console.log(multiply(5));
// Expected output: 5
```

Syntax

js

```
function fnName(param1 = defaultValue1, /* …, */ paramN = defaultValueN) {
  // …
}
```

Description

In JavaScript, function parameters default to `undefined`. However, it's often useful to set a different default value. This is where default parameters can help.

In the following example, if no value is provided for `b` when `multiply` is called, `b`'s value would be `undefined` when evaluating `a * b` and `multiply` would return `NaN`.

js

```
function multiply(a, b) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // NaN !
```

In the past, the general strategy for setting defaults was to test parameter values in the function body and assign a value if they are `undefined`. In the following example, `b` is set to `1` if `multiply` is called with only one argument:

js

```
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // 5
```

With default parameters, checks in the function body are no longer necessary. Now, you can assign `1` as the default value for `b` in the function head:

```js
function multiply(a, b = 1) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // 5
multiply(5, undefined); // 5
```

Parameters are still set left-to-right, overwriting default parameters even if there are later parameters without defaults.

```js
function f(x = 1, y) {
  return [x, y];
}

f(); // [1, undefined]
f(2); // [2, undefined]
```

> **Note:** The first default parameter and all parameters after it will not contribute to the function's `length`.

The default parameter initializers live in their own scope, which is a parent of the scope created for the function body.

This means that earlier parameters can be referred to in the initializers of later parameters. However, functions and variables declared in the function body cannot be referred to from default value parameter initializers; attempting to do so throws a run-time `ReferenceError`. This also includes `var`-declared variables in the function body.

For example, the following function will throw a `ReferenceError` when invoked, because the default parameter value does not have access to the child scope of the function body:

js · ✗ bad example

```js
function f(a = go()) {
  function go() {
    return ":P";
  }
}

f(); // ReferenceError: go is not defined
```

This function will print the value of the *parameter* `a`, because the variable `var a` is hoisted only to the top of the scope created for the function body, not the parent scope created for the parameter list, so its value is not visible to `b`.

js · ✗ bad example

```js
function f(a, b = () => console.log(a)) {
  var a = 1;
  b();
}

f(); // undefined
f(5); // 5
```

The default parameter allows any expression, but you cannot use `await` or `yield` that would pause the evaluation of the default expression. The parameter must be initialized *synchronously*.

js · ✗ bad example

```
async function f(a = await Promise.resolve(1)) {
  return a;
}
```

> **Note:** Because the default parameter is evaluated when the function is called, not when the function is defined, the validity of the `await` and `yield` operators depends on the function itself, not its surrounding function. For example, if the current function is not `async`, `await` will be parsed as an identifier and follow normal identifier syntax rules, even when this function is nested in an `async` function.

Examples

Passing undefined vs. other falsy values

In the second call in this example, even if the first argument is set explicitly to `undefined` (though not `null` or other falsy values), the value of the `num` argument is still the default.

js

```
function test(num = 1) {
  console.log(typeof num);
}

test(); // 'number' (num is set to 1)
test(undefined); // 'number' (num is set to 1 too)

// test with other falsy values:
test(""); // 'string' (num is set to '')
test(null); // 'object' (num is set to null)
```

Evaluated at call time

The default argument is evaluated at *call time*. Unlike with Python (for example), a new object is created each time the function is called.

js

```
function append(value, array = []) {
  array.push(value);
  return array;
}

append(1); // [1]
append(2); // [2], not [1, 2]
```

This even applies to functions and variables:

```js
function callSomething(thing = something()) {
  return thing;
}

let numberOfTimesCalled = 0;
function something() {
  numberOfTimesCalled += 1;
  return numberOfTimesCalled;
}

callSomething(); // 1
callSomething(); // 2
```

Earlier parameters are available to later default parameters

Parameters defined earlier (to the left) are available to later default parameters:

```js
function greet(name, greeting, message = `${greeting} ${name}`) {
  return [name, greeting, message];
}

greet("David", "Hi"); // ["David", "Hi", "Hi David"]
greet("David", "Hi", "Happy Birthday!"); // ["David", "Hi", "Happy Birthday!"]
```

This functionality can be approximated like this, which demonstrates how many edge cases are handled:

```js
function go() {
  return ":P";
}

function withDefaults(
  a,
  b = 5,
  c = b,
  d = go(),
  e = this,
  f = arguments,
  g = this.value,
) {
  return [a, b, c, d, e, f, g];
}

function withoutDefaults(a, b, c, d, e, f, g) {
  switch (arguments.length) {
    case 0:
    case 1:
      b = 5;
    case 2:
      c = b;
    case 3:
      d = go();
    case 4:
      e = this;
    case 5:
      f = arguments;
```

(code continues on the next page)

(code continued from the previous page)

```
    case 6:
      g = this.value;
  }
  return [a, b, c, d, e, f, g];
}

withDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", {value:"=^_^="}, arguments, "=^_^="]

withoutDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", {value:"=^_^="}, arguments, "=^_^="]
```

Destructured parameter with default value assignment

You can use default value assignment with the destructuring syntax.

A common way of doing that is to set an empty object/array as the default value for the destructured parameter; for example: `[x = 1, y = 2] = []`. This makes it possible to pass nothing to the function and still have those values prefilled:

js

```
function preFilledArray([x = 1, y = 2] = []) {
  return x + y;
}

preFilledArray(); // 3
preFilledArray([]); // 3
preFilledArray([2]); // 4
preFilledArray([2, 3]); // 5

// Works the same for objects:
function preFilledObject({ z = 3 } = {}) {
  return z;
}

preFilledObject(); // 3
preFilledObject({}); // 3
preFilledObject({ z: 2 }); // 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Default parameters	49	14	15	36	10	49	15	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Destructured parameter with default value assignment	49	14	41	36	10	49	41	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Parameters without defaults after default parameters	49	14	26	36	10	49	26	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.default_parameters`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- Rest parameters
- Nullish coalescing operator (??)

JavaScript › Reference › Functions

get

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/get · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since July 2015.

The **`get`** syntax binds an object property to a function that will be called when that property is looked up. It can also be used in classes.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Getter declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const obj = {
  log: ["a", "b", "c"],
  get latest() {
    return this.log[this.log.length - 1];
  },
};

console.log(obj.latest);
// Expected output: "c"
```

Syntax

js

```
{ get prop() { /* … */ } }
{ get [expression]() { /* … */ } }
```

There are some additional syntax restrictions:

- A getter must have exactly zero parameters.

Parameters

`prop`

The name of the property to bind to the given function. In the same way as other properties in object initializers, it can be a string literal, a number literal, or an identifier.

`expression`

You can also use expressions for a computed property name to bind to the given function.

Description

Sometimes, it is desirable to allow access to a property that returns a dynamically computed value, or you may want to reflect the status of an internal variable without requiring the use of explicit method calls. In JavaScript, this can be accomplished with the use of a *getter*.

An object property is either a data property or an accessor property, but it cannot simultaneously be both. Read `Object.defineProperty()` for more information. The getter syntax allows you to specify the getter function in an object initializer.

```js
const obj = {
  get prop() {
    // getter, the code executed when reading obj.prop
    return someValue;
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable and enumerable.

Examples

Defining a getter on new objects in object initializers

This will create a pseudo-property `latest` for object `obj`, which will return the last array item in `log`.

```js
const obj = {
  log: ["example", "test"],
  get latest() {
    return this.log.at(-1);
  },
};
console.log(obj.latest); // "test"
```

Note that attempting to assign a value to `latest` will not change it.

Using getters in classes

You can use the exact same syntax to define public instance getters that are available on class instances. In classes, you don't need the comma separator between methods.

```js
class ClassWithGetSet {
  #msg = "hello world";
  get msg() {
    return this.#msg;
  }
  set msg(x) {
    this.#msg = `hello ${x}`;
  }
}

const instance = new ClassWithGetSet();
console.log(instance.msg); // "hello world"

instance.msg = "cake";
console.log(instance.msg); // "hello cake"
```

Getter properties are defined on the `prototype` property of the class and are thus shared by all instances of the class. Unlike getter properties in object literals, getter properties in classes are not enumerable.

Static getters and private getters use similar syntaxes, which are described in the `static` and private elements pages.

Deleting a getter using the `delete` operator

If you want to remove the getter, you can just `delete` it:

```js
delete obj.latest;
```

Defining a getter on existing objects using `defineProperty`

To append a getter to an existing object later at any time, use `Object.defineProperty()`.

```js
const o = { a: 0 };

Object.defineProperty(o, "b", {
  get() {
    return this.a + 1;
  },
});

console.log(o.b); // Runs the getter, which yields a + 1 (which is 1)
```

Using a computed property name

```js
const expr = "foo";

const obj = {
  get [expr]() {
    return "bar";
  },
};

console.log(obj.foo); // "bar"
```

Defining static getters

```js
class MyConstants {
  static get foo() {
    return "foo";
  }
}

console.log(MyConstants.foo); // 'foo'
MyConstants.foo = "bar";
console.log(MyConstants.foo); // 'foo', a static getter's value cannot be changed
```

Smart / self-overwriting / lazy getters

Getters give you a way to *define* a property of an object, but they do not *calculate* the property's value until it is accessed. A getter defers the cost of calculating the value until the value is needed. If it is never needed, you never pay the cost.

An additional optimization technique to lazify or delay the calculation of a property value and cache it for later access is *smart* (or *memoized*) getters. The value is calculated the first time the getter is called and is then cached so subsequent accesses return the cached value without recalculating it. This is useful in the following situations:

- If the calculation of a property value is expensive (takes much RAM or CPU time, spawns worker threads, retrieves remote file, etc.).
- If the value isn't needed just now. It will be used later, or in some cases, it's not used at all.
- If it's used, it will be accessed several times, and there is no need to re-calculate that value because it will never be changed or shouldn't be re-calculated.

> **Note:** This means that you shouldn't write a lazy getter for a property whose value you expect to change, because if the getter is lazy, then it will not recalculate the value.
>
> Note that getters are not "lazy" or "memoized" by nature; you must implement this technique if you desire this behavior.

In the following example, the object has a getter as its own property. On getting the property, the property is removed from the object and re-added, but implicitly as a data property this time. Finally, the value gets returned.

```js
const obj = {
  get notifier() {
    delete this.notifier;
    this.notifier = document.getElementById("bookmarked-notification-anchor");
    return this.notifier;
  },
};
```

Feature detection

Many functions accept an object and retrieve individual properties from it as separate parameters (this object parameter is known as an *options bag*). You can detect whether a specific option is supported by using a getter to track if the property has been retrieved. This example checks if the `colorType` option is supported by the `HTMLCanvasElement.getContext()` method.

```js
function isColorTypeSupported() {
  let supported = false;
  const obj = {
    get colorType() {
      supported = true;
      return undefined;
    },
  };
  document.createElement("canvas").getContext("2d", obj);
  return supported;
}
```

get vs. defineProperty

While using the `get` keyword and `Object.defineProperty()` have similar results, there is a subtle difference between the two when used on classes. The `get` syntax defines the property on the instance's prototype, while `Object.defineProperty()` defines the property on the instance it is applied to.

```js
class Example {
  get hello() {
    return "world";
  }
}

const obj = new Example();
console.log(obj.hello);
// "world"

console.log(Object.getOwnPropertyDescriptor(obj, "hello"));
// undefined

console.log(
  Object.getOwnPropertyDescriptor(Object.getPrototypeOf(obj), "hello"),
);
// { configurable: true, enumerable: false, get: function get hello() { return 'world'; },
set: undefined }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-method-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
get	1	12	1.5	9.5	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	46	12	34	47	9.1	46	34	33	9.3	5.0	46	9.3	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.get`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Working with objects guide
- Functions
- `set`
- `Object.defineProperty()`
- Object initializer
- `class`
- Property accessors
- Incompatible ES5 change: literal getter and setter functions must now have exactly zero or one arguments by Jeff Walden (2010)
- More SpiderMonkey changes: ancient, esoteric, very rarely used syntax for creating getters and setters is being removed by Jeff Walden (2010)

JavaScript › Reference › Functions

Method definitions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Method_definitions · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2015.

Method definition is a shorter syntax for defining a function property in an object initializer. It can also be used in classes.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Method definitions". It runs live in the browser there; its source code follows.

js · interactive example source

```
const obj = {
  foo() {
    return "bar";
  },
};

console.log(obj.foo());
// Expected output: "bar"
```

Syntax

```js
({
  property(parameters) {},
  *generator(parameters) {},
  async property(parameters) {},
  async *generator(parameters) {},

  // with computed keys
  [expression](parameters) {},
  *[expression](parameters) {},
  async [expression](parameters) {},
  async *[expression](parameters) {},
})
```

Description

The shorthand syntax is similar to the getter and setter syntax.

Given the following code:

```js
const obj = {
  foo: function () {
    // …
  },
  bar: function () {
    // …
  },
};
```

You are now able to shorten this to:

```js
const obj = {
  foo() {
    // …
  },
  bar() {
    // …
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable, enumerable, and writable, just like normal properties.

`function*`, `async function`, and `async function*` properties all have their respective method syntaxes; see examples below.

However, note that the method syntax is not equivalent to a normal property with a function as its value — there are semantic differences. This makes methods defined in object literals more consistent with methods in classes.

Method definitions are not constructable

Methods cannot be constructors! They will throw a `TypeError` if you try to instantiate them. On the other hand, a property created as a function can be used as a constructor.

js · ✗ bad example

```
const obj = {
  method() {},
};
new obj.method(); // TypeError: obj.method is not a constructor
```

Using super in method definitions

Only functions defined as methods have access to the `super` keyword. `super.prop` looks up the property on the prototype of the object that the method was initialized on.

js · ✗ bad example

```
const obj = {
  __proto__: {
    prop: "foo",
  },
  notAMethod: function () {
    console.log(super.prop); // SyntaxError: 'super' keyword unexpected here
  },
};
```

Examples

Using method definitions

js

```
const obj = {
  a: "foo",
  b() {
    return this.a;
  },
};
console.log(obj.b()); // "foo"
```

Method definitions in classes

You can use the exact same syntax to define public instance methods that are available on class instances. In classes, you don't need the comma separator between methods.

js

```
class ClassWithPublicInstanceMethod {
  publicMethod() {
    return "hello world";
  }
  secondPublicMethod() {
    return "goodbye world";
  }
}

const instance = new ClassWithPublicInstanceMethod();
console.log(instance.publicMethod()); // "hello world"
```

Public instance methods are defined on the `prototype` property of the class and are thus shared by all instances of the class. They are writable, non-enumerable, and configurable.

Inside instance methods, `this` and `super` work like in normal methods. Usually, `this` refers to the instance itself. In subclasses, `super` lets you access the prototype of the object that the method is

attached to, allowing you to call methods from the superclass.

```js
class BaseClass {
  msg = "hello world";
  basePublicMethod() {
    return this.msg;
  }
}

class SubClass extends BaseClass {
  subPublicMethod() {
    return super.basePublicMethod();
  }
}

const instance = new SubClass();
console.log(instance.subPublicMethod()); // "hello world"
```

Static methods and private methods use similar syntaxes, which are described in the `static` and private elements pages.

Computed property names

The method syntax also supports computed property names.

```js
const bar = {
  foo0: function () {
    return 0;
  },
  foo1() {
    return 1;
  },
  ["foo".toUpperCase()]() {
    return 2;
  },
};

console.log(bar.foo0()); // 0
console.log(bar.foo1()); // 1
console.log(bar.FOO()); // 2
```

Generator methods

Note that the asterisk (*) in the generator method syntax must be *before* the generator property name. (That is, `* g(){}` will work, but `g *(){}` will not.)

```js
// Using a named property
const obj = {
  g: function* () {
    let index = 0;
    while (true) {
      yield index++;
    }
  },
};
```

(code continues on the next page)

(code continued from the previous page)

```
// The same object using shorthand syntax
const obj2 = {
  *g() {
    let index = 0;
    while (true) {
      yield index++;
    }
  },
};

const it = obj2.g();
console.log(it.next().value); // 0
console.log(it.next().value); // 1
```

Async methods

js

```
// Using a named property
const obj = {
  f: async function () {
    await somePromise;
  },
};

// The same object using shorthand syntax
const obj2 = {
  async f() {
    await somePromise;
  },
};
```

Async generator methods

js

```
// Using a named property
const obj = {
  f: async function* () {
    yield 1;
    yield 2;
    yield 3;
  },
};

// The same object using shorthand syntax
const obj2 = {
  async *f() {
    yield 1;
    yield 2;
    yield 3;
  },
};
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-method-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Method definitions	39	12	34	26	9	39	34	26	9	4.0	39	9	1.0.0	1.0	4.0.0
Async generator methods	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0
Async methods	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Generator methods are not constructable (ES2016)	42	13	43	29	9.1	42	43	29	9.3	4.0	42	9.3	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.method_definitions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Working with objects guide
- Functions
- `get`
- `set`
- Object initializer
- `class`

JavaScript › Reference › Functions

Rest parameters

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/rest_parameters · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2016.

The **rest parameter** syntax allows a function to accept an indefinite number of arguments as an array, providing a way to represent variadic functions in JavaScript.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Rest parameters". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function sum(...theArgs) {
  let total = 0;
  for (const arg of theArgs) {
    total += arg;
  }
  return total;
}

console.log(sum(1, 2, 3));
// Expected output: 6

console.log(sum(1, 2, 3, 4));
// Expected output: 10
```

Syntax

```js
function f(a, b, ...theArgs) {
  // …
}
```

There are some additional syntax restrictions:

- A function definition can only have one rest parameter.
- The rest parameter must be the last parameter in the function definition.
- Trailing commas are not allowed after the rest parameter.
- The rest parameter cannot have a default value.

Description

A function definition's last parameter can be prefixed with `...` (three U+002E FULL STOP characters), which will cause all remaining (user supplied) parameters to be placed within an `Array` object.

```js
function myFun(a, b, ...manyMoreArgs) {
  console.log("a", a);
  console.log("b", b);
  console.log("manyMoreArgs", manyMoreArgs);
}

myFun("one", "two", "three", "four", "five", "six");

// Console Output:
// a, one
// b, two
// manyMoreArgs, ["three", "four", "five", "six"]
```

The rest parameter may be destructured, which allows you to ignore certain parameter positions.

```js
function ignoreFirst(...[, b, c]) {
  return b + c;
}
```

However, the following are all syntax errors:

```
js · ✗ bad example
function wrong1(...one, ...wrong) {}
function wrong2(...wrong, arg2, arg3) {}
function wrong3(...wrong,) {}
function wrong4(...wrong = []) {}
```

The rest parameter is not counted towards the function's `length` property.

The difference between rest parameters and the arguments object

There are four main differences between rest parameters and the `arguments` object:

- The `arguments` object is **not a real array**, while rest parameters are `Array` instances, meaning methods like `sort()`, `map()`, `forEach()` or `pop()` can be applied on it directly.

- The `arguments` object has the additional (deprecated) `callee` property.
- In a non-strict function with simple parameters, the `arguments` object syncs its indices with the values of parameters. The rest parameter array never updates its value when the named parameters are re-assigned.
- The rest parameter bundles all the *extra* parameters into a single array, but does not contain any named argument defined *before* the `...restParam`. The `arguments` object contains all of the parameters — including the parameters in the `...restParam` array — bundled into one array-like object.

Examples

Using rest parameters

In this example, the first argument is mapped to `a` and the second to `b`, so these named arguments are used as normal.

However, the third argument, `manyMoreArgs`, will be an array that contains the third, fourth, fifth, sixth, …, nth — as many arguments as the user specifies.

```js
function myFun(a, b, ...manyMoreArgs) {
  console.log("a", a);
  console.log("b", b);
  console.log("manyMoreArgs", manyMoreArgs);
}

myFun("one", "two", "three", "four", "five", "six");

// a, "one"
// b, "two"
// manyMoreArgs, ["three", "four", "five", "six"] <-- an array
```

Below, even though there is just one value, the last argument still gets put into an array.

```js
// Using the same function definition from example above

myFun("one", "two", "three");

// a, "one"
// b, "two"
// manyMoreArgs, ["three"] <-- an array with just one value
```

Below, the third argument isn't provided, but `manyMoreArgs` is still an array (albeit an empty one).

```js
// Using the same function definition from example above

myFun("one", "two");

// a, "one"
// b, "two"
// manyMoreArgs, [] <-- still an array
```

Below, only one argument is provided, so `b` gets the default value `undefined`, but `manyMoreArgs` is still an empty array.

```js
// Using the same function definition from example above

myFun("one");

// a, "one"
// b, undefined
// manyMoreArgs, [] <-- still an array
```

Argument length

Since `theArgs` is an array, a count of its elements is given by the `length` property. If the function's only parameter is a rest parameter, `restParams.length` will be equal to `arguments.length`.

```js
function fun1(...theArgs) {
  console.log(theArgs.length);
}

fun1(); // 0
fun1(5); // 1
fun1(5, 6, 7); // 3
```

Using rest parameters in combination with ordinary parameters

In the next example, a rest parameter is used to collect all parameters after the first parameter into an array. Each one of the parameter values collected into the array is then multiplied by the first parameter, and the array is returned:

```js
function multiply(multiplier, ...theArgs) {
  return theArgs.map((element) => multiplier * element);
}

const arr = multiply(2, 15, 25, 42);
console.log(arr); // [30, 50, 84]
```

From arguments to an array

`Array` methods can be used on rest parameters, but not on the `arguments` object:

```js
function sortRestArgs(...theArgs) {
  const sortedArgs = theArgs.sort();
  return sortedArgs;
}

console.log(sortRestArgs(5, 3, 7, 1)); // 1, 3, 5, 7

function sortArguments() {
  const sortedArgs = arguments.sort();
  return sortedArgs; // this will never happen
}

console.log(sortArguments(5, 3, 7, 1));
// throws a TypeError (arguments.sort is not a function)
```

Rest parameters were introduced to reduce the boilerplate code that was commonly used for converting a set of arguments to an array.

Before rest parameters, `arguments` need to be converted to a normal array before calling array methods on them:

```js
function fn(a, b) {
  const normalArray = Array.prototype.slice.call(arguments);
  // — or —
  const normalArray2 = [].slice.call(arguments);
  // — or —
  const normalArrayFrom = Array.from(arguments);

  const first = normalArray.shift(); // OK, gives the first argument
  const firstBad = arguments.shift(); // ERROR (arguments is not a normal array)
}
```

Now, you can easily gain access to a normal array using a rest parameter:

```js
function fn(...args) {
  const normalArray = args;
  const first = normalArray.shift(); // OK, gives the first argument
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Rest parameters	47	12	15	34	10	47	15	34	10	5.0	47	10	1.0.0	1.0	6.0.0
Destructuring rest parameters	49	79	52	36	10	49	52	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.rest_parameters`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Functions guide
- Functions
- Spread syntax (`...`)
- Default parameters
- `arguments`
- `Array`

JavaScript › Reference › Functions

set

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/set · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since July 2015.

The **`set`** syntax binds an object property to a function to be called when there is an attempt to set that property. It can also be used in classes.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Setter declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const language = {
  set current(name) {
    this.log.push(name);
  },
  log: [],
};

language.current = "EN";
language.current = "FA";

console.log(language.log);
// Expected output: Array ["EN", "FA"]
```

Syntax

js

```
{ set prop(val) { /* … */ } }
{ set [expression](val) { /* … */ } }
```

There are some additional syntax restrictions:

- A setter must have exactly one parameter.

Parameters

`prop`

The name of the property to bind to the given function. In the same way as other properties in object initializers, it can be a string literal, a number literal, or an identifier.

`val`

An alias for the variable that holds the value attempted to be assigned to `prop`.

`expression`

You can also use expressions for a computed property name to bind to the given function.

Description

In JavaScript, a setter can be used to execute a function whenever an attempt is made to change a property's value. Setters are most often used in conjunction with getters.

An object property is either a data property or an accessor property, but it cannot simultaneously be both. Read `Object.defineProperty()` for more information. The setter syntax allows you to specify the

setter function in an object initializer.

```js
const obj = {
  set prop(val) {
    // setter, the code executed when setting obj.prop
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable and enumerable.

Examples

Defining a setter on new objects in object initializers

The following example defines a pseudo-property `current` of object `language`. When `current` is assigned a value, it updates `log` with that value:

```js
const language = {
  set current(name) {
    this.log.push(name);
  },
  log: [],
};

language.current = "EN";
console.log(language.log); // ['EN']

language.current = "FA";
console.log(language.log); // ['EN', 'FA']
```

Note that `current` has no getter, and any attempts to access it will result in `undefined`.

Using setters in classes

You can use the exact same syntax to define public instance setters that are available on class instances. In classes, you don't need the comma separator between methods.

```js
class ClassWithGetSet {
  #msg = "hello world";
  get msg() {
    return this.#msg;
  }
  set msg(x) {
    this.#msg = `hello ${x}`;
  }
}

const instance = new ClassWithGetSet();
console.log(instance.msg); // "hello world"

instance.msg = "cake";
console.log(instance.msg); // "hello cake"
```

Setter properties are defined on the `prototype` property of the class and are thus shared by all instances of the class. Unlike setter properties in object literals, setter properties in classes are not enumerable.

Static setters and private setters use similar syntaxes, which are described in the `static` and private elements pages.

Removing a setter with the `delete` operator

If you want to remove the setter, you can just `delete` it:

```js
delete language.current;
```

Defining a setter on existing objects using `defineProperty`

To append a setter to an *existing* object, use `Object.defineProperty()`.

```js
const o = { a: 0 };

Object.defineProperty(o, "b", {
  set(x) {
    this.a = x / 2;
  },
});

o.b = 10;
// Runs the setter, which assigns 10 / 2 (5) to the 'a' property

console.log(o.a); // 5
```

Using a computed property name

```js
const expr = "foo";

const obj = {
  baz: "bar",
  set [expr](v) {
    this.baz = v;
  },
};

console.log(obj.baz); // "bar"

obj.foo = "baz";
// Run the setter

console.log(obj.baz); // "baz"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-method-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`set`	1	12	1.5	9.5	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	46	12	34	47	9.1	46	34	33	9.3	5.0	46	9.3	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.set`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Working with objects guide
- Functions
- `get`
- `Object.defineProperty()`
- Object initializer
- `class`
- Property accessors
- Incompatible ES5 change: literal getter and setter functions must now have exactly zero or one arguments by Jeff Walden (2010)
- More SpiderMonkey changes: ancient, esoteric, very rarely used syntax for creating getters and setters is being removed by Jeff Walden (2010)

JavaScript › Reference › Functions

The arguments object

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/arguments · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since July 2015.

`arguments` is an array-like object accessible inside functions that contains the values of the arguments passed to that function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: The arguments object". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function func1(a, b, c) {
  console.log(arguments[0]);
  // Expected output: 1

  console.log(arguments[1]);
  // Expected output: 2

  console.log(arguments[2]);
  // Expected output: 3
}

func1(1, 2, 3);
```

Description

> **Note:** In modern code, rest parameters should be preferred.

The `arguments` object is a local variable available within all non-arrow functions. You can refer to a function's arguments inside that function by using its `arguments` object. It has entries for each argument the function was called with, with the first entry's index at `0`.

For example, if a function is passed 3 arguments, you can access them as follows:

js

```
arguments[0]; // first argument
arguments[1]; // second argument
arguments[2]; // third argument
```

The `arguments` object is useful for functions called with more arguments than they are formally declared to accept, called *variadic functions*, such as `Math.min()`. This example function accepts any number of string arguments and returns the longest one:

js

```
function longestString() {
  let longest = "";
  if (arguments.length === 0) {
    throw new TypeError("At least one string is required");
  }
  for (const arg of arguments) {
    if (arg.length > longest.length) {
      longest = arg;
    }
  }
  return longest;
}
```

You can use `arguments.length` to count how many arguments the function was called with. If you instead want to count how many parameters a function is declared to accept, inspect that function's `length` property.

Assigning to indices

Each argument index can also be set or reassigned:

```js
arguments[1] = "new value";
```

Non-strict functions that only have simple parameters (that is, no rest, default, or destructured parameters) will sync the new value of parameters with the `arguments` object, and vice versa:

```js
function func(a) {
  arguments[0] = 99; // updating arguments[0] also updates a
  console.log(a);
}
func(10); // 99

function func2(a) {
  a = 99; // updating a also updates arguments[0]
  console.log(arguments[0]);
}
func2(10); // 99
```

Non-strict functions that *are* passed rest, default, or destructured parameters will not sync new values assigned to parameters in the function body with the `arguments` object. Instead, the `arguments` object in non-strict functions with complex parameters will always reflect the values passed to the function when the function was called.

```js
function funcWithDefault(a = 55) {
  arguments[0] = 99; // updating arguments[0] does not also update a
  console.log(a);
}
funcWithDefault(10); // 10

function funcWithDefault2(a = 55) {
  a = 99; // updating a does not also update arguments[0]
  console.log(arguments[0]);
}
funcWithDefault2(10); // 10

// An untracked default parameter
function funcWithDefault3(a = 55) {
  console.log(arguments[0]);
  console.log(arguments.length);
}
funcWithDefault3(); // undefined; 0
```

This is the same behavior exhibited by all strict-mode functions, regardless of the type of parameters they are passed. That is, assigning new values to parameters in the body of the function never affects the `arguments` object, nor will assigning new values to the `arguments` indices affect the value of parameters, even when the function only has simple parameters.

> **Note:** You cannot write a `"use strict";` directive in the body of a function definition that accepts rest, default, or destructured parameters. Doing so will throw a syntax error.

arguments is an array-like object

`arguments` is an array-like object, which means that `arguments` has a `length` property and properties indexed from zero, but it doesn't have `Array`'s built-in methods like `forEach()` or `map()`. However, it can be converted to a real `Array`, using one of `slice()`, `Array.from()`, or spread syntax.

```js
const args = Array.prototype.slice.call(arguments);
// or
const args = Array.from(arguments);
// or
const args = [...arguments];
```

For common use cases, using it as an array-like object is sufficient, since it both is iterable and has `length` and number indices. For example, `Function.prototype.apply()` accepts array-like objects.

```js
function midpoint() {
  return (
    (Math.min.apply(null, arguments) + Math.max.apply(null, arguments)) / 2
  );
}

console.log(midpoint(3, 1, 4, 1, 5)); // 3
```

Properties

`arguments.callee` Deprecated

Reference to the currently executing function that the arguments belong to. Forbidden in strict mode.

`arguments.length`

The number of arguments that were passed to the function.

`arguments[Symbol.iterator]()`

Returns a new array iterator object that contains the values for each index in `arguments`.

Examples

Defining a function that concatenates several strings

This example defines a function that concatenates several strings. The function's only formal argument is a string containing the characters that separate the items to concatenate.

```js
function myConcat(separator) {
  const args = Array.prototype.slice.call(arguments, 1);
  return args.join(separator);
}
```

You can pass as many arguments as you like to this function. It returns a string list using each argument in the list:

```js
myConcat(", ", "red", "orange", "blue");
// "red, orange, blue"

myConcat("; ", "elephant", "giraffe", "lion", "cheetah");
// "elephant; giraffe; lion; cheetah"

myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley");
// "sage. basil. oregano. pepper. parsley"
```

Defining a function that creates HTML lists

This example defines a function that creates a string containing HTML for a list. The only formal argument for the function is a string that is `"u"` if the list is to be unordered (bulleted), or `"o"` if the list is to be ordered (numbered). The function is defined as follows:

```js
function list(type) {
  let html = `<${type}l><li>`;
  const args = Array.prototype.slice.call(arguments, 1);
  html += args.join("</li><li>");
  html += `</li></${type}l>`; // end list
  return html;
}
```

You can pass any number of arguments to this function, and it adds each argument as a list item to a list of the type indicated. For example:

```js
list("u", "One", "Two", "Three");
// "<ul><li>One</li><li>Two</li><li>Three</li></ul>"
```

Using typeof with arguments

The `typeof` operator returns `'object'` when used with `arguments`:

```js
console.log(typeof arguments); // 'object'
```

The type of individual arguments can be determined by indexing `arguments`:

```js
console.log(typeof arguments[0]); // returns the type of the first argument
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-arguments-exotic-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`arguments`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
[Symbol.iterator]	52	12	46	39	9	52	46	41	9	6.0	52	9	1.0.0	1.0	4.0.0
`callee` Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`length`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.arguments`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- Rest parameters

JavaScript › Reference › Functions › The arguments object

arguments[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/arguments/Symbol.iterator · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2016.

The **`[Symbol.iterator]()`** method of `arguments` objects implements the iterable protocol and allows `arguments` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the `arguments` object.

The initial value of this property is the same function object as the initial value of the `Array.prototype.values` property (and also the same as `Array.prototype[Symbol.iterator]()`).

Syntax

```
arguments[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Array.prototype.values()`: a new iterable iterator object that yields the value of each index in the `arguments` object.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `arguments` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```js
function f() {
  for (const letter of arguments) {
    console.log(letter);
  }
}
f("w", "y", "k", "o", "p");
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
function f() {
  const argsIter = arguments[Symbol.iterator]();
  console.log(argsIter.next().value); // w
  console.log(argsIter.next().value); // y
  console.log(argsIter.next().value); // k
  console.log(argsIter.next().value); // o
  console.log(argsIter.next().value); // p
}
f("w", "y", "k", "o", "p");
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-createunmappedargumentsobject
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-createmappedargumentsobject

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	52	12	46	39	9	52	46	41	9	6.0	52	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.arguments.@@iterator`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Functions guide
- Functions
- `arguments`
- `Array.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

JavaScript › Reference › Functions › The arguments object

arguments.callee

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/arguments/callee · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** Accessing `arguments.callee` in strict mode will throw a `TypeError`. If a function must reference itself, either give the function expression a name or use a function declaration.

The **`arguments.callee`** data property contains the currently executing function that the arguments belong to.

Value

A reference to the currently executing function.

Property attributes of `arguments.callee`	
Writable	yes
Enumerable	no
Configurable	yes

> **Note:** `callee` is a data property only in non-strict functions with simple parameters (in which case the `arguments` object is also auto-syncing). Otherwise, it is an accessor property whose getter and setter both throw a `TypeError`.

Description

`callee` is a property of the `arguments` object. It can be used to refer to the currently executing function inside the function body of that function. This is useful when the name of the function is unknown, such as within a function expression with no name (also called "anonymous functions").

(The text below is largely adapted from a Stack Overflow answer by olliej)

Early versions of JavaScript did not allow named function expressions, and for this reason you could not make a recursive function expression.

For example, this syntax worked:

```js
function factorial(n) {
  return n <= 1 ? 1 : factorial(n - 1) * n;
}

[1, 2, 3, 4, 5].map(factorial);
```

but:

```js
[1, 2, 3, 4, 5].map(function (n) {
  return n <= 1 ? 1 : /* what goes here? */ (n - 1) * n;
});
```

did not. To get around this `arguments.callee` was added so you could do

```js
[1, 2, 3, 4, 5].map(function (n) {
  return n <= 1 ? 1 : arguments.callee(n - 1) * n;
});
```

However, the design of `arguments.callee` has multiple issues. The first problem is that the recursive call will get a different `this` value. For example:

```js
function sillyFunction(recursed) {
  if (this !== globalThis) {
    console.log("This is:", this);
  } else {
    console.log("This is the global");
  }

  if (!recursed) {
    return arguments.callee(true);
  }
}

sillyFunction();
// This is the global
// This is: [object Arguments]
```

In addition, references to `arguments.callee` make inlining and tail recursion impossible in the general case. (You can achieve it in select cases through tracing, etc., but even the best code is suboptimal due to checks that would not otherwise be necessary.)

ECMAScript 3 resolved these issues by allowing named function expressions. For example:

```js
[1, 2, 3, 4, 5].map(function factorial(n) {
  return n <= 1 ? 1 : factorial(n - 1) * n;
});
```

This has numerous benefits:

- the function can be called like any other from inside your code
- it does not create a variable in the outer scope (except for IE 8 and below)
- it has better performance than accessing the arguments object

Strict mode has banned other properties that leak stack information, like the `caller` property of functions. This is because looking at the call stack has one single major effect: it makes a large number of optimizations impossible, or much more difficult. For example, if you cannot guarantee that a function `f` will not call an unknown function, it is not possible to inline `f`.

```js
function f(a, b, c, d, e) {
  return a ? b * c : d * e;
}
```

If the JavaScript interpreter cannot guarantee that all the provided arguments are numbers at the point that the call is made, it needs to either insert checks for all the arguments before the inlined code, or it cannot inline the function. This means any call site that may have been trivially inlinable accumulates a large number of guards. Now in this particular case a smart interpreter should be able to rearrange the checks to be more optimal and not check any values that would not be used. However in many cases that's just not possible and therefore it becomes impossible to inline.

Examples

Using arguments.callee in an anonymous recursive function

A recursive function must be able to refer to itself. Typically, a function refers to itself by its name. However, an anonymous function (which can be created by a function expression or the `Function` constructor) does not have a name. Therefore if there is no accessible variable referring to it, the only way the function can refer to itself is by `arguments.callee`.

The following example defines a function, which, in turn, defines and returns a factorial function. This example isn't very practical, and there are nearly no cases where the same result cannot be achieved with named function expressions.

```js
function create() {
  return function (n) {
    if (n <= 1) {
      return 1;
    }
    return n * arguments.callee(n - 1);
  };
}

const result = create()(5); // returns 120 (5 * 4 * 3 * 2 * 1)
```

Recursion of anonymous functions with a Y-combinator

Although function expressions can now be named, arrow functions always remain anonymous, which means they cannot reference themselves without being assigned to a variable first. Fortunately, in Lambda calculus there's a very good solution which allows a function to both be anonymous and self-referential. The technique is called a Y-combinator. Here we will not explain *how* it works, only *that* it works.

```js
// The Y-combinator: a utility function!
const Y = (hof) => ((x) => x(x))((x) => hof((y) => x(x)(y)));

console.log(
  [1, 2, 3, 4, 5].map(
    // Wrap the higher-order function in the Y-combinator
    // "factorial" is not a function's name: it's introduced as a parameter
    Y((factorial) => (n) => (n <= 1 ? 1 : factorial(n - 1) * n)),
  ),
);
// [ 1, 2, 6, 24, 120 ]
```

> **Note:** This method allocates a new closure for every iteration, which may significantly increase memory usage. It's only here to demonstrate the possibility, but should be avoided in production. Use a temporary variable or a named function expression instead.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-arguments-exotic-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`callee` Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.arguments.callee`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Functions guide
- Functions
- `arguments`
- `Function.prototype.caller`

JavaScript › Reference › Functions › The arguments object

arguments.length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/arguments/length · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since July 2015.

The **`arguments.length`** data property contains the number of arguments passed to the function.

Value

A non-negative integer.

Property attributes of `arguments.length`	
Writable	yes
Enumerable	no
Configurable	yes

Description

The `arguments.length` property provides the number of arguments actually passed to a function. This can be more or less than the number of defined parameters (see `Function.prototype.length`). For example, for the function below:

```js
function func1(a, b, c) {
  console.log(arguments.length);
}
```

`func1.length` returns `3`, because `func1` declares three formal parameters. However, `func1(1, 2, 3, 4, 5)` logs `5`, because `func1` was called with five arguments. Similarly, `func1(1)` logs `1`, because `func1` was called with one argument.

Examples

Using arguments.length

In this example, we define a function that can add two or more numbers together.

```js
function adder(base /*, num1, …, numN */) {
  base = Number(base);
  for (let i = 1; i < arguments.length; i++) {
    base += Number(arguments[i]);
  }
  return base;
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-arguments-exotic-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`length`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.functions.arguments.length`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `arguments`
- `Function: length`

JavaScript › Reference

Classes

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since March 2016.

Classes are a template for creating objects. They encapsulate data with code to work on that data. Classes in JS are built on prototypes but also have some syntax and semantics that are unique to classes.

For more examples and explanations, see the Using classes guide.

Description

Defining classes

Classes are in fact "special functions", and just as you can define function expressions and function declarations, a class can be defined in two ways: a class expression or a class declaration.

```js
// Declaration
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}

// Expression; the class is anonymous but assigned to a variable
const Rectangle = class {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};

// Expression; the class has its own name
const Rectangle = class Rectangle2 {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};
```

Like function expressions, class expressions may be anonymous, or have a name that's different from the variable that it's assigned to. However, unlike function declarations, class declarations have the same temporal dead zone restrictions as `let` or `const` and behave as if they are not hoisted.

Class body

The body of a class is the part that is in curly braces `{}`. This is where you define class members, such as methods or constructor.

The body of a class is executed in strict mode even without the `"use strict"` directive.

A class element can be characterized by three aspects:

- Kind: Getter, setter, method, or field
- Location: Static or instance

- Visibility: Public or private

Together, they add up to 16 possible combinations. To divide the reference more logically and avoid overlapping content, the different elements are introduced in detail in different pages:

Method definitions
: Public instance method

getter
: Public instance getter

setter
: Public instance setter

Public class fields
: Public instance field

`static`
: Public static method, getter, setter, and field

Private elements
: Everything that's private

> **Note:** Private elements have the restriction that all private names declared in the same class must be unique, except for a getter/setter pair where both are static or both are instance. All other public properties do not have this restriction — you can have multiple public properties with the same name, and the last one overwrites the others. This is the same behavior as in object initializers.

In addition, there are two special class element syntaxes: `constructor` and static initialization blocks, with their own references.

Constructor

The `constructor` method is a special method for creating and initializing an object created with a class. There can only be one special method with the name "constructor" in a class — a `SyntaxError` is thrown if the class contains more than one occurrence of a `constructor` method.

A constructor can use the `super` keyword to call the constructor of the super class.

You can create instance properties inside the constructor:

```js
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}
```

Alternatively, if your instance properties' values do not depend on the constructor's arguments, you can define them as class fields.

Static initialization blocks

Static initialization blocks allow flexible initialization of static properties, including the evaluation of statements during initialization, while granting access to the private scope.

Multiple static blocks can be declared, and these can be interleaved with the declaration of static fields and methods (all static items are evaluated in declaration order).

Methods

Methods are defined on the prototype of each class instance and are shared by all instances. Methods can be plain functions, async functions, generator functions, or async generator functions. For more information, see method definitions.

```js
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
  // Getter
  get area() {
    return this.calcArea();
  }
  // Method
  calcArea() {
    return this.height * this.width;
  }
  *getSides() {
    yield this.height;
    yield this.width;
    yield this.height;
    yield this.width;
  }
}

const square = new Rectangle(10, 10);

console.log(square.area); // 100
console.log([...square.getSides()]); // [10, 10, 10, 10]
```

Static methods and fields

The `static` keyword defines a static method or field for a class. Static properties (fields and methods) are defined on the class itself instead of each instance. Static methods are often used to create utility functions for an application, whereas static fields are useful for caches, fixed-configuration, or any other data that doesn't need to be replicated across instances.

```js
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  static displayName = "Point";
  static distance(a, b) {
    const dx = a.x - b.x;
    const dy = a.y - b.y;

    return Math.hypot(dx, dy);
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const p1 = new Point(5, 5);
const p2 = new Point(10, 10);
p1.displayName; // undefined
p1.distance; // undefined
p2.displayName; // undefined
p2.distance; // undefined

console.log(Point.displayName); // "Point"
console.log(Point.distance(p1, p2)); // 7.0710678118654755
```

Field declarations

With the class field declaration syntax, the constructor example can be written as:

js

```
class Rectangle {
  height = 0;
  width;
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}
```

Class fields are similar to object properties, not variables, so we don't use keywords such as `const` to declare them. In JavaScript, private elements use a special identifier syntax, so modifier keywords like `public` and `private` should not be used either.

As seen above, the fields can be declared with or without a default value. Fields without default values default to `undefined`. By declaring fields up-front, class definitions become more self-documenting, and the fields are always present, which help with optimizations.

See public class fields for more information.

Private elements

Using private fields, the definition can be refined as below.

js

```
class Rectangle {
  #height = 0;
  #width;
  constructor(height, width) {
    this.#height = height;
    this.#width = width;
  }
}
```

It's an error to reference private fields from outside of the class; they can only be read or written within the class body. By defining things that are not visible outside of the class, you ensure that your classes' users can't depend on internals, which may change from version to version.

Private fields can only be declared up-front in a field declaration. They cannot be created later through assigning to them, the way that normal properties can.

Private methods and accessors can also be defined using the same syntax as their public counterparts, but with the identifier starting with `#`.

For more information, see private elements.

Inheritance

The `extends` keyword is used in *class declarations* or *class expressions* to create a class as a child of another constructor (either a class or a function).

```js
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name); // call the super class constructor and pass in the name parameter
  }

  speak() {
    console.log(`${this.name} barks.`);
  }
}

const d = new Dog("Mitzie");
d.speak(); // Mitzie barks.
```

If there is a constructor present in the subclass, it needs to first call `super()` before using `this`. The `super` keyword can also be used to call corresponding methods of super class.

```js
class Cat {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Lion extends Cat {
  speak() {
    super.speak();
    console.log(`${this.name} roars.`);
  }
}

const l = new Lion("Fuzzy");
l.speak();
// Fuzzy makes a noise.
// Fuzzy roars.
```

Evaluation order

When a `class` declaration or `class` expression is evaluated, its various components are evaluated in the following order:

1. The `extends` clause, if present, is first evaluated. It must evaluate to a valid constructor function or `null`, or a `TypeError` is thrown.
2. The `constructor` method is extracted, substituted with a default implementation if `constructor` is not present. However, because the `constructor` definition is only a method definition, this step is not observable.
3. The class elements' property keys are evaluated in the order of declaration. If the property key is computed, the computed expression is evaluated, with the `this` value set to the `this` value surrounding the class (not the class itself). None of the property values are evaluated yet.
4. Methods and accessors are installed in the order of declaration. Instance methods and accessors are installed on the `prototype` property of the current class, and static methods and accessors are installed on the class itself. Private instance methods and accessors are saved to be installed on the instance directly later. This step is not observable.
5. The class is now initialized with the prototype specified by `extends` and implementation specified by `constructor`. For all steps above, if an evaluated expression tries to access the name of the class, a `ReferenceError` is thrown because the class is not initialized yet.
6. The class elements' values are evaluated in the order of declaration:
   - For each instance field (public or private), its initializer expression is saved. The initializer is evaluated during instance creation, at the start of the constructor (for base classes) or immediately before the `super()` call returns (for derived classes).
   - For each static field (public or private), its initializer is evaluated with `this` set to the class itself, and the property is created on the class.
   - Static initialization blocks are evaluated with `this` set to the class itself.
7. The class is now fully initialized and can be used as a constructor function.

For how instances are created, see the `constructor` reference.

Examples

Binding this with instance and static methods

When a static or instance method is called without a value for `this`, such as by assigning the method to a variable and then calling it, the `this` value will be `undefined` inside the method. This behavior is the same even if the `"use strict"` directive isn't present, because code within the `class` body is always executed in strict mode.

```js
class Animal {
  speak() {
    return this;
  }
  static eat() {
    return this;
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const obj = new Animal();
obj.speak(); // the Animal object
const speak = obj.speak;
speak(); // undefined

Animal.eat(); // class Animal
const eat = Animal.eat;
eat(); // undefined
```

If we rewrite the above using traditional function-based syntax in non–strict mode, then `this` method calls are automatically bound to `globalThis`. In strict mode, the value of `this` remains as `undefined`.

js

```
function Animal() {}

Animal.prototype.speak = function () {
  return this;
};

Animal.eat = function () {
  return this;
};

const obj = new Animal();
const speak = obj.speak;
speak(); // global object (in non-strict mode)

const eat = Animal.eat;
eat(); // global object (in non-strict mode)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`classes`	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0
`constructor`	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0
`extends`	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0
Private class fields	74	79	90	62	14.1	74	90	53	14.5	11.0	74	14.5	1.0.0	1.0	12.0.0
Private class fields 'in'	91	91	90	77	15	91	90	64	15	16.0	91	15	1.0.0	1.9	16.4.0
Private class methods	84	84	90	70	15	84	90	60	15	14.0	84	15	1.0.0	1.0	14.6.0
Public class fields	72	79	69	60	16[4,5,6,7]	72	79	51	16[8,9,10,11]	11.0	72	16[8,9,10,11]	1.0.0	1.0	12.0.0
`static` keyword	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0
Static class fields	72	79	75	60	14.1	72	79	51	14.5	11.0	72	14.5	1.0.0	1.0	12.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Static initialization blocks	94	94	93	80	16.4	94	93	66	16.4	17.0	94	16.4	1.0.0	1.14	16.11.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.
3. Also from version 4.0–5.0: Strict mode is required.
4. Also from version 14.1–16: partial support.
5. Also from version 14.1–16: Parentheses in field initializers can lead to `ReferenceErrors`. See bug 236843.
6. Also from version 14–14.1: partial support.
7. Also from version 14–14.1: Doesn't support public static fields. See bug 194095.
8. Also from version 14.5–16: partial support.
9. Also from version 14.5–16: Parentheses in field initializers can lead to `ReferenceErrors`. See bug 236843.
10. Also from version 14–14.5: partial support.
11. Also from version 14–14.5: Doesn't support public static fields. See bug 194095.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Using classes guide
- `class`
- `class` expression
- Functions
- ES6 In Depth: Classes on hacks.mozilla.org (2015)

JavaScript › Reference › Classes

constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/constructor · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since March 2016.

The **`constructor`** method is a special method of a class for creating and initializing an object instance of that class.

> **Note:** This page introduces the `constructor` syntax. For the `constructor` property present on all objects, see `Object.prototype.constructor`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Class constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Polygon {
  constructor() {
    this.name = "Polygon";
  }
}

const poly = new Polygon();

console.log(poly.name);
// Expected output: "Polygon"
```

Syntax

js

```
constructor() { /* … */ }
constructor(argument0) { /* … */ }
constructor(argument0, argument1) { /* … */ }
constructor(argument0, argument1, /* …, */ argumentN) { /* … */ }
```

There are some additional syntax restrictions:

- A non-static class method with the non-computed name `constructor` cannot be a getter, setter, async, or generator.
- A class cannot have more than one `constructor` method.

Description

A constructor enables you to provide any custom initialization that must be done before any other methods can be called on an instantiated object.

js

```
class Person {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const otto = new Person("Otto");

otto.introduce(); // Hello, my name is Otto
```

If you don't provide your own constructor, then a default constructor will be supplied for you. If your class is a base class, the default constructor is empty:

js

```
constructor() {}
```

If your class is a derived class, the default constructor calls the parent constructor, passing along any arguments that were provided:

```js
constructor(...args) {
  super(...args);
}
```

> **Note:** The difference between an explicit constructor like the one above and the default constructor is that the latter doesn't actually invoke the array iterator through argument spreading.

That enables code like this to work:

```js
class ValidationError extends Error {
  printCustomerMessage() {
    return `Validation failed :-( (details: ${this.message})`;
  }
}

try {
  throw new ValidationError("Not a valid phone number");
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(error.name); // This is Error instead of ValidationError!
    console.log(error.printCustomerMessage());
  } else {
    console.log("Unknown error", error);
    throw error;
  }
}
```

The `ValidationError` class doesn't need an explicit constructor, because it doesn't need to do any custom initialization. The default constructor then takes care of initializing the parent `Error` from the argument it is given.

However, if you provide your own constructor, and your class derives from some parent class, then you must explicitly call the parent class constructor using `super()`. For example:

```js
class ValidationError extends Error {
  constructor(message) {
    super(message); // call parent class constructor
    this.name = "ValidationError";
    this.code = "42";
  }

  printCustomerMessage() {
    return `Validation failed :-( (details: ${this.message}, code: ${this.code})`;
  }
}

try {
  throw new ValidationError("Not a valid phone number");
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(error.name); // Now this is ValidationError!
    console.log(error.printCustomerMessage());
```

(code continues on the next page)

(code continued from the previous page)

```
  } else {
    console.log("Unknown error", error);
    throw error;
  }
}
```

Using new on a class goes through the following steps:

1. (If it's a derived class) The constructor body before the super() call is evaluated. This part should not access this because it's not yet initialized.
2. (If it's a derived class) The super() call is evaluated, which initializes the parent class through the same process.
3. The current class's fields are initialized.
4. The constructor body after the super() call (or the entire body, if it's a base class) is evaluated.

Within the constructor body, you can access the object being created through this and access the class that is called with new through new.target. Note that methods (including getters and setters) and the prototype chain are already initialized on this before the constructor is executed, so you can even access methods of the subclass from the constructor of the superclass. However, if those methods use this, the this will not have been fully initialized yet. This means reading public fields of the derived class will result in undefined, while reading private fields will result in a TypeError.

js · ✗ bad example

```
new (class C extends class B {
  constructor() {
    console.log(this.foo());
  }
} {
  #a = 1;
  foo() {
    return this.#a; // TypeError: Cannot read private member #a from an object whose class
did not declare it
    // It's not really because the class didn't declare it,
    // but because the private field isn't initialized yet
    // when the superclass constructor is running
  }
})();
```

The constructor method may have a return value. While the base class may return anything from its constructor, the derived class must return an object or undefined, or a TypeError will be thrown.

js

```
class ParentClass {
  constructor() {
    return 1;
  }
}

console.log(new ParentClass()); // ParentClass {}
// The return value is ignored because it's not an object
// This is consistent with function constructors
```

(code continues on the next page)

(code continued from the previous page)

```
class ChildClass extends ParentClass {
  constructor() {
    return 1;
  }
}

console.log(new ChildClass()); // TypeError: Derived constructors may only return object or
undefined
```

If the parent class constructor returns an object, that object will be used as the `this` value on which class fields of the derived class will be defined. This trick is called "return overriding", which allows a derived class's fields (including private ones) to be defined on unrelated objects.

The `constructor` follows normal method syntax, so parameter default values, rest parameters, etc. can all be used.

js

```
class Person {
  constructor(name = "Anonymous") {
    this.name = name;
  }
  introduce() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const person = new Person();
person.introduce(); // Hello, my name is Anonymous
```

The constructor must be a literal name. Computed properties cannot become constructors.

js

```
class Foo {
  // This is a computed property. It will not be picked up as a constructor.
  ["constructor"]() {
    console.log("called");
    this.a = 1;
  }
}

const foo = new Foo(); // No log
console.log(foo); // Foo {}
foo.constructor(); // Logs "called"
console.log(foo); // Foo { a: 1 }
```

Non-static async methods, generator methods, and accessors are forbidden from having the non-computed name `constructor`. Static and instance class fields also cannot have the non-computed name `constructor`. Private names cannot be called `#constructor`. Any non-static member with the non-computed name `constructor` must be a plain method.

Examples

Using the constructor

This code snippet is taken from the classes sample (live demo).

```js
class Square extends Polygon {
  constructor(length) {
    // Here, it calls the parent class' constructor with lengths
    // provided for the Polygon's width and height
    super(length, length);
    // NOTE: In derived classes, `super()` must be called before you
    // can use `this`. Leaving this out will cause a ReferenceError.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }

  set area(value) {
    this.height = value ** 0.5;
    this.width = value ** 0.5;
  }
}
```

Calling super in a constructor bound to a different prototype

`super()` calls the constructor that's the prototype of the current class. If you change the prototype of the current class itself, `super()` will call the constructor that's the new prototype. Changing the prototype of the current class's `prototype` property doesn't affect which constructor `super()` calls.

```js
class Polygon {
  constructor() {
    this.name = "Polygon";
  }
}

class Rectangle {
  constructor() {
    this.name = "Rectangle";
  }
}

class Square extends Polygon {
  constructor() {
    super();
  }
}

// Make Square extend Rectangle (which is a base class) instead of Polygon
Object.setPrototypeOf(Square, Rectangle);

const newInstance = new Square();

// newInstance is still an instance of Polygon, because we didn't
// change the prototype of Square.prototype, so the prototype chain
// of newInstance is still
//   newInstance --> Square.prototype --> Polygon.prototype
console.log(newInstance instanceof Polygon); // true
console.log(newInstance instanceof Rectangle); // false
```

(code continues on the next page)

(code continued from the previous page)

```
// However, because super() calls Rectangle as constructor, the name property
// of newInstance is initialized with the logic in Rectangle
console.log(newInstance.name); // Rectangle
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-static-semantics-constructormethod

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
constructor	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.
3. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.constructor`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Using classes guide
- Classes
- Static initialization blocks
- `class`
- `super()`
- `Object.prototype.constructor`

JavaScript › Reference › Classes

extends

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2016.

The **`extends`** keyword is used in class declarations or class expressions to create a class that is a child of another class.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Class extends". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class DateFormatter extends Date {
  getFormattedDate() {
    const months = [
```

(code continues on the next page)

(code continued from the previous page)

```
      "Jan",
      "Feb",
      "Mar",
      "Apr",
      "May",
      "Jun",
      "Jul",
      "Aug",
      "Sep",
      "Oct",
      "Nov",
      "Dec",
    ];
    return `${this.getDate()}-${months[this.getMonth()]}-${this.getFullYear()}`;
  }
}

console.log(new DateFormatter("August 19, 1975 23:15:30").getFormattedDate());
// Expected output: "19-Aug-1975"
```

Syntax

js

```
class ChildClass extends ParentClass { /* … */ }
```

`ParentClass`

An expression that evaluates to a constructor function (including a class) or `null`.

Description

The `extends` keyword can be used to subclass custom classes as well as built-in objects.

Any constructor that can be called with `new` and has the `prototype` property can be the candidate for the parent class. The two conditions must both hold — for example, bound functions and `Proxy` can be constructed, but they don't have a `prototype` property, so they cannot be subclassed.

js

```
function OldStyleClass() {
  this.someProperty = 1;
}
OldStyleClass.prototype.someMethod = function () {};

class ChildClass extends OldStyleClass {}

class ModernClass {
  someProperty = 1;
  someMethod() {}
}

class AnotherChildClass extends ModernClass {}
```

The `prototype` property of the `ParentClass` must be an `Object` or `null`, but you would rarely worry about this in practice, because a non-object `prototype` doesn't behave as it should anyway. (It's ignored by the `new` operator.)

```js
function ParentClass() {}
ParentClass.prototype = 3;

class ChildClass extends ParentClass {}
// Uncaught TypeError: Class extends value does not have valid prototype property 3

console.log(Object.getPrototypeOf(new ParentClass()));
// [Object: null prototype] {}
// Not actually a number!
```

`extends` sets the prototype for both `ChildClass` and `ChildClass.prototype`.

	Prototype of `ChildClass`	**Prototype of `ChildClass.prototype`**
`extends` clause absent	`Function.prototype`	`Object.prototype`
`extends null`	`Function.prototype`	`null`
`extends ParentClass`	`ParentClass`	`ParentClass.prototype`

```js
class ParentClass {}
class ChildClass extends ParentClass {}

// Allows inheritance of static properties
Object.getPrototypeOf(ChildClass) === ParentClass;
// Allows inheritance of instance properties
Object.getPrototypeOf(ChildClass.prototype) === ParentClass.prototype;
```

The right-hand side of `extends` does not have to be an identifier. You can use any expression that evaluates to a constructor. This is often useful to create mixins. The `this` value in the `extends` expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

```js
class SomeClass extends class {
  constructor() {
    console.log("Base class");
  }
} {
  constructor() {
    super();
    console.log("Derived class");
  }
}

new SomeClass();
// Base class
// Derived class
```

While the base class may return anything from its constructor, the derived class must return an object or `undefined`, or a `TypeError` will be thrown.

```js
class ParentClass {
  constructor() {
    return 1;
  }
}

console.log(new ParentClass()); // ParentClass {}
// The return value is ignored because it's not an object
// This is consistent with function constructors

class ChildClass extends ParentClass {
  constructor() {
    super();
    return 1;
  }
}

console.log(new ChildClass()); // TypeError: Derived constructors may only return object or
undefined
```

If the parent class constructor returns an object, that object will be used as the `this` value for the derived class when further initializing class fields. This trick is called "return overriding", which allows a derived class's fields (including private ones) to be defined on unrelated objects.

Subclassing built-ins

> **Warning:** The standard committee now holds the position that the built-in subclassing mechanism in previous spec versions is over-engineered and causes non-negligible performance and security impacts. New built-in methods consider less about subclasses, and engine implementers are investigating whether to remove certain subclassing mechanisms. Consider using composition instead of inheritance when enhancing built-ins.

Here are some things you may expect when extending a class:

- When calling a static factory method (like `Promise.resolve()` or `Array.from()`) on a subclass, the returned instance is always an instance of the subclass.
- When calling an instance method that returns a new instance (like `Promise.prototype.then()` or `Array.prototype.map()`) on a subclass, the returned instance is always an instance of the subclass.
- Instance methods try to delegate to a minimal set of primitive methods where possible. For example, for a subclass of `Promise`, overriding `then()` automatically causes the behavior of `catch()` to change; or for a subclass of `Map`, overriding `set()` automatically causes the behavior of the `Map()` constructor to change.

However, the above expectations take non-trivial efforts to implement properly.

- The first one requires the static method to read the value of `this` to get the constructor for constructing the returned instance. This means `[p1, p2, p3].map(Promise.resolve)` throws an error because the `this` inside `Promise.resolve` is `undefined`. A way to fix this is to fall back to the base class if `this` is not a constructor, like `Array.from()` does, but that still means the base class is special-cased.
- The second one requires the instance method to read `this.constructor` to get the constructor function. However, `new this.constructor()` may break legacy code, because the `constructor` property is both writable and configurable and is not protected in any way. Therefore, many copying

built-in methods use the constructor's `[Symbol.species]` property instead (which by default just returns `this`, the constructor itself). However, `[Symbol.species]` allows running arbitrary code and creating instances of arbitrary type, which poses a security concern and greatly complicates subclassing semantics.
- The third one leads to visible invocations of custom code, which makes a lot of optimizations harder to implement. For example, if the `Map()` constructor is called with an iterable of *x* elements, then it must visibly invoke the `set()` method *x* times, instead of just copying the elements into the internal storage.

These problems are not unique to built-in classes. For your own classes, you will likely have to make the same decisions. However, for built-in classes, optimizability and security are a much bigger concern. New built-in methods always construct the base class and call as few custom methods as possible. If you want to subclass built-ins while achieving the above expectations, you need to override all methods that have the default behavior baked into them. Any addition of new methods on the base class may also break the semantics of your subclass because they are inherited by default. Therefore, a better way to extend built-ins is to use *composition*.

Extending null

`extends null` was designed to allow easy creation of objects that do not inherit from `Object.prototype`. However, due to unsettled decisions about whether `super()` should be called within the constructor, it's not possible to construct such a class in practice using any constructor implementation that doesn't return an object. The TC39 committee is working on re-enabling this feature.

```js
new (class extends null {})();
// TypeError: Super constructor null of anonymous class is not a constructor

new (class extends null {
  constructor() {}
})();
// ReferenceError: Must call super constructor in derived class before accessing 'this' or
returning from derived constructor

new (class extends null {
  constructor() {
    super();
  }
})();
// TypeError: Super constructor null of anonymous class is not a constructor
```

Instead, you need to explicitly return an instance from the constructor.

```js
class NullClass extends null {
  constructor() {
    // Using new.target allows derived classes to
    // have the correct prototype chain
    return Object.create(new.target.prototype);
  }
}

const proto = Object.getPrototypeOf;
console.log(proto(proto(new NullClass()))); // null
```

Examples

Using extends

The first example creates a class called `Square` from a class called `Polygon`. This example is extracted from this live demo (source).

```js
class Square extends Polygon {
  constructor(length) {
    // Here, it calls the parent class' constructor with lengths
    // provided for the Polygon's width and height
    super(length, length);
    // Note: In derived classes, super() must be called before you
    // can use 'this'. Leaving this out will cause a reference error.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }
}
```

Extending plain objects

Classes cannot extend regular (non-constructible) objects. If you want to inherit from a regular object by making all properties of this object available on inherited instances, you can instead use `Object.setPrototypeOf()`:

```js
const Animal = {
  speak() {
    console.log(`${this.name} makes a noise.`);
  },
};

class Dog {
  constructor(name) {
    this.name = name;
  }
}

Object.setPrototypeOf(Dog.prototype, Animal);

const d = new Dog("Mitzie");
d.speak(); // Mitzie makes a noise.
```

Extending built-in objects

This example extends the built-in `Date` object. This example is extracted from this live demo (source).

```js
class MyDate extends Date {
  getFormattedDate() {
    const months = [
      "Jan", "Feb", "Mar", "Apr", "May", "Jun",
      "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
    ];
    return `${this.getDate()}-${months[this.getMonth()]}-${this.getFullYear()}`;
  }
}
```

Extending `Object`

All JavaScript objects inherit from `Object.prototype` by default, so writing `extends Object` at first glance seems redundant. The only difference from not writing `extends` at all is that the constructor itself inherits static methods from `Object`, such as `Object.keys()`. However, because no `Object` static method uses the `this` value, there's still no value in inheriting these static methods.

The `Object()` constructor special-cases the subclassing scenario. If it's implicitly called via `super()`, it always initializes a new object with `new.target.prototype` as its prototype. Any value passed to `super()` is ignored.

```js
class C extends Object {
  constructor(v) {
    super(v);
  }
}

console.log(new C(1) instanceof Number); // false
console.log(C.keys({ a: 1, b: 2 })); // [ 'a', 'b' ]
```

Compare this behavior with a custom wrapper that does not special-case subclassing:

```js
function MyObject(v) {
  return new Object(v);
}
class D extends MyObject {
  constructor(v) {
    super(v);
  }
}
console.log(new D(1) instanceof Number); // true
```

Species

You might want to return `Array` objects in your derived array class `MyArray`. The species pattern lets you override default constructors.

For example, when using methods such as `Array.prototype.map()` that return the default constructor, you want these methods to return a parent `Array` object, instead of the `MyArray` object. The `Symbol.species` symbol lets you do this:

```js
class MyArray extends Array {
  // Overwrite species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}

const a = new MyArray(1, 2, 3);
const mapped = a.map((x) => x * x);

console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

This behavior is implemented by many built-in copying methods. For caveats of this feature, see the subclassing built-ins discussion.

Mix-ins

Abstract subclasses or *mix-ins* are templates for classes. A class can only have a single superclass, so multiple inheritance from tooling classes, for example, is not possible. The functionality must be provided by the superclass.

A function with a superclass as input and a subclass extending that superclass as output can be used to implement mix-ins:

```js
const calculatorMixin = (Base) =>
  class extends Base {
    calc() {}
  };

const randomizerMixin = (Base) =>
  class extends Base {
    randomize() {}
  };
```

A class that uses these mix-ins can then be written like this:

```js
class Foo {}
class Bar extends calculatorMixin(randomizerMixin(Foo)) {}
```

Avoiding inheritance

Inheritance is a very strong coupling relationship in object-oriented programming. It means all behaviors of the base class are inherited by the subclass by default, which may not always be what you want. For example, consider the implementation of a `ReadOnlyMap`:

```js
class ReadOnlyMap extends Map {
  set() {
    throw new TypeError("A read-only map must be set at construction time.");
  }
}
```

It turns out that `ReadOnlyMap` is not constructible, because the `Map()` constructor calls the instance's `set()` method.

```js
const m = new ReadOnlyMap([["a", 1]]); // TypeError: A read-only map must be set at
construction time.
```

We may get around this by using a private flag to indicate whether the instance is being constructed. However, a more significant problem with this design is that it breaks the Liskov substitution principle, which states that a subclass should be substitutable for its superclass. If a function expects a `Map` object, it should be able to use a `ReadOnlyMap` object as well, which will break here.

Inheritance often leads to the circle-ellipse problem, because neither type perfectly entails the behavior of the other, although they share a lot of common traits. In general, unless there's a very good reason to use inheritance, it's better to use composition instead. Composition means that a class has a reference to an object of another class, and only uses that object as an implementation detail.

```js
class ReadOnlyMap {
  #data;
  constructor(values) {
    this.#data = new Map(values);
  }
  get(key) {
    return this.#data.get(key);
  }
  has(key) {
    return this.#data.has(key);
  }
  get size() {
    return this.#data.size;
  }
  *keys() {
    yield* this.#data.keys();
  }
  *values() {
    yield* this.#data.values();
  }
  *entries() {
    yield* this.#data.entries();
  }
  *[Symbol.iterator]() {
    yield* this.#data[Symbol.iterator]();
  }
}
```

In this case, the `ReadOnlyMap` class is not a subclass of `Map`, but it still implements most of the same methods. This means more code duplication, but it also means that the `ReadOnlyMap` class is not strongly coupled to the `Map` class, and does not easily break if the `Map` class is changed, avoiding the semantic issues of built-in subclassing. For example, if the `Map` class adds a new utility method (such as `getOrInsert()`) that does not call `set()`, it would cause the `ReadOnlyMap` class to no longer be read-only unless the latter is updated accordingly to override `getOrInsert()` as well. Moreover, `ReadOnlyMap` objects do not have the `set` method at all, which is more accurate than throwing an error at runtime.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`extends`	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.
3. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.extends`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using classes guide
- Classes
- `constructor`
- `class`
- `super`

JavaScript › Reference › Classes

Private elements

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_elements · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2021.

Private elements are counterparts of the regular class elements which are public, including class fields, class methods, etc. Private elements get created by using a hash # prefix and cannot be legally referenced outside of the class. The privacy encapsulation of these class elements is enforced by JavaScript itself. The only way to access a private element is via dot notation, and you can only do so within the class that defines the private element.

Private elements were not native to the language before this syntax existed. In prototypal inheritance, its behavior may be emulated with `WeakMap` objects or closures, but they can't compare to the # syntax in terms of ergonomics.

> **Note:** On MDN, we avoid using the term "private property". A property in JavaScript has a string or symbol key, and has attributes like `writable`, `enumerable`, and `configurable`, but private elements have none. While private elements are accessed with the familiar dot notation, they cannot be proxied, enumerated, deleted, or interacted with using any `Object` method.

Syntax

```js
class ClassWithPrivate {
  #privateField;
  #privateFieldWithInitializer = 42;

  #privateMethod() {
    // …
  }

  static #privateStaticField;
  static #privateStaticFieldWithInitializer = 42;

  static #privateStaticMethod() {
    // …
  }
}
```

There are some additional syntax restrictions:

- All private identifiers declared within a class must be unique. The namespace is shared between static and instance elements. The only exception is when the two declarations define a getter-setter pair.
- The private identifier cannot be `#constructor`.

Description

Most class elements have their private counterparts:

- Private fields
- Private methods
- Private static fields
- Private static methods
- Private getters
- Private setters
- Private static getters
- Private static setters

These features are collectively called *private elements*. However, constructors cannot be private in JavaScript. To prevent classes from being constructed outside of the class, you have to use a private flag.

Private elements are declared with **# names** (pronounced "hash names"), which are identifiers prefixed with #. The hash prefix is an inherent part of the property name — you can draw relationship with the old underscore prefix convention `_privateField` — but it's not an ordinary string property, so you can't dynamically access it with the bracket notation.

It is a syntax error to refer to # names from outside of the class. It is also a syntax error to refer to private elements that were not declared in the class body, or to attempt to remove declared elements with `delete`.

js · ✗ bad example

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    delete this.#privateField; // Syntax error
    this.#undeclaredField = 42; // Syntax error
  }
}

const instance = new ClassWithPrivateField();
instance.#privateField; // Syntax error
```

JavaScript, being a dynamic language, is able to perform this compile-time check because of the special hash identifier syntax, making it different from normal properties on the syntax level.

> **Note:** Code run in the Chrome console can access private elements outside the class. This is a DevTools-only relaxation of the JavaScript syntax restriction.

If you access a private element from an object that doesn't have the element, a `TypeError` is thrown, instead of returning `undefined` as normal properties do.

js · ✗ bad example

```
class C {
  #x;

  static getX(obj) {
    return obj.#x;
  }
}

console.log(C.getX(new C())); // undefined
console.log(C.getX({})); // TypeError: Cannot read private member #x from an object whose
class did not declare it
```

This example also illustrates that you can access private elements within static functions too, and on externally defined instances of the class.

You can use the `in` operator to check whether an externally defined object possesses a private element. This will return `true` if the private field or method exists, and `false` otherwise.

js · ✓ good example

```
class C {
  #x;
  constructor(x) {
    this.#x = x;
  }
  static getX(obj) {
    if (#x in obj) return obj.#x;

    return "obj must be an instance of C";
  }
}
console.log(C.getX(new C("foo"))); // "foo"
console.log(C.getX(new C(0.196))); // 0.196
console.log(C.getX(new C(new Date()))); // the current date and time
console.log(C.getX({})); // "obj must be an instance of C"
```

Note a corollary of private names being always pre-declared and non-deletable: if you found that an object possesses one private element of the current class (either from a `try...catch` or an `in` check), it must possess all other private elements. An object possessing the private elements of a class generally means it was constructed by that class (although not always).

Private elements are not part of the prototypical inheritance model since they can only be accessed within the current class's body and aren't inherited by subclasses. Private elements with the same name within different classes are entirely different and do not interoperate with each other. See them as external metadata attached to each instance, managed by the class. For this reason, `structuredClone()` does not clone private elements, and `Object.freeze()` and `Object.seal()` have no effect on private elements.

For more information on how and when private fields are initialized, see public class fields.

Examples

Private fields

Private fields include private instance fields and private static fields. Private fields are only accessible from inside the class declaration.

Private instance fields

Like their public counterparts, private instance fields:

- are added before the constructor runs in a base class, or immediately after `super()` is invoked in a subclass, and
- are only available on instances of the class.

js

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    this.#privateField = 42;
  }
}

class Subclass extends ClassWithPrivateField {
  #subPrivateField;

  constructor() {
    super();
    this.#subPrivateField = 23;
  }
}

new Subclass(); // In some dev tools, it shows Subclass {#privateField: 42,
#subPrivateField: 23}
```

> **Note:** `#privateField` from the `ClassWithPrivateField` base class is private to `ClassWithPrivateField` and is not accessible from the derived `Subclass`.

Returning overriding object

A class's constructor can return a different object, which will be used as the new `this` for the derived class constructor. The derived class may then define private fields on that returned object — meaning it is possible to "stamp" private fields onto unrelated objects.

```js
class Stamper extends class {
  // A base class whose constructor returns the object it's given
  constructor(obj) {
    return obj;
  }
} {
  // This declaration will "stamp" the private field onto the object
  // returned by the base class constructor
  #stamp = 42;
  static getStamp(obj) {
    return obj.#stamp;
  }
}

const obj = {};
new Stamper(obj);
// `Stamper` calls `Base`, which returns `obj`, so `obj` is
// now the `this` value. `Stamper` then defines `#stamp` on `obj`

console.log(obj); // In some dev tools, it shows {#stamp: 42}
console.log(Stamper.getStamp(obj)); // 42
console.log(obj instanceof Stamper); // false

// You cannot stamp private elements twice
new Stamper(obj); // Error: Initializing an object twice is an error with private fields
```

> **Warning:** This is a potentially very confusing thing to do. You are generally advised to avoid returning anything from the constructor — especially something unrelated to `this`.

Private static fields

Like their public counterparts, private static fields:

- are added to the class constructor at class evaluation time, and
- are only available on the class itself.

```js
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
    return ClassWithPrivateStaticField.#privateStaticField;
  }
}

console.log(ClassWithPrivateStaticField.publicStaticMethod()); // 42
```

There is a restriction on private static fields: only the class which defines the private static field can access the field. This can lead to unexpected behavior when using `this`. In the following example, `this` refers to the `Subclass` class (not the `ClassWithPrivateStaticField` class) when we try to call `Subclass.publicStaticMethod()`, and so causes a `TypeError`.

```js
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
    return this.#privateStaticField;
  }
}

class Subclass extends ClassWithPrivateStaticField {}

Subclass.publicStaticMethod(); // TypeError: Cannot read private member #privateStaticField
from an object whose class did not declare it
```

This is the same if you call the method with `super`, because `super` methods are not called with the super class as `this`.

```js
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
    // When invoked through super, `this` still refers to Subclass
    return this.#privateStaticField;
  }
}

class Subclass extends ClassWithPrivateStaticField {
  static callSuperMethod() {
    return super.publicStaticMethod();
  }
}

Subclass.callSuperMethod(); // TypeError: Cannot read private member #privateStaticField
from an object whose class did not declare it
```

You are advised to always access private static fields through the class name, not through `this`, so inheritance doesn't break the method.

Private methods

Private methods include private instance methods and private static methods. Private methods are only accessible from inside the class declaration.

Private instance methods

Unlike their public counterparts, private instance methods:

- are installed immediately before the instance fields are installed, and
- are only available on instances of the class, not on its `.prototype` property.

```js
class ClassWithPrivateMethod {
  #privateMethod() {
    return 42;
  }

  publicMethod() {
    return this.#privateMethod();
  }
}

const instance = new ClassWithPrivateMethod();
console.log(instance.publicMethod()); // 42
```

Private instance methods may be generator, async, or async generator functions. Private getters and setters are also possible, and follow the same syntax requirements as their public getter and setter counterparts.

```js
class ClassWithPrivateAccessor {
  #message;

  get #decoratedMessage() {
    return `🎬${this.#message}⏺`;
  }
  set #decoratedMessage(msg) {
    this.#message = msg;
  }

  constructor() {
    this.#decoratedMessage = "hello world";
    console.log(this.#decoratedMessage);
  }
}

new ClassWithPrivateAccessor(); // 🎬hello world⏺
```

Unlike public methods, private methods are not accessible on the `.prototype` property of their class.

```js
class C {
  #method() {}

  static getMethod(x) {
    return x.#method;
  }
}

console.log(C.getMethod(new C())); // [Function: #method]
console.log(C.getMethod(C.prototype)); // TypeError: Receiver must be an instance of class C
```

Private static methods

Like their public counterparts, private static methods:

- are added to the class constructor at class evaluation time, and
- are only available on the class itself.

```js
class ClassWithPrivateStaticMethod {
  static #privateStaticMethod() {
    return 42;
  }

  static publicStaticMethod() {
    return ClassWithPrivateStaticMethod.#privateStaticMethod();
  }
}

console.log(ClassWithPrivateStaticMethod.publicStaticMethod()); // 42
```

Private static methods may be generator, async, and async generator functions.

The same restriction previously mentioned for private static fields holds for private static methods, and similarly can lead to unexpected behavior when using `this`. In the following example, when we try to call `Subclass.publicStaticMethod()`, `this` refers to the `Subclass` class (not the `ClassWithPrivateStaticMethod` class) and so causes a `TypeError`.

```js
class ClassWithPrivateStaticMethod {
  static #privateStaticMethod() {
    return 42;
  }

  static publicStaticMethod() {
    return this.#privateStaticMethod();
  }
}

class Subclass extends ClassWithPrivateStaticMethod {}

console.log(Subclass.publicStaticMethod()); // TypeError: Cannot read private member
#privateStaticMethod from an object whose class did not declare it
```

Simulating private constructors

Many other languages include the capability to mark a constructor as private, which prevents the class from being instantiated outside of the class itself — you can only use static factory methods that create instances, or not be able to create instances at all. JavaScript does not have a native way to do this, but it can be accomplished by using a private static flag.

```js
class PrivateConstructor {
  static #isInternalConstructing = false;

  constructor() {
    if (!PrivateConstructor.#isInternalConstructing) {
      throw new TypeError("PrivateConstructor is not constructable");
    }
    PrivateConstructor.#isInternalConstructing = false;
    // More initialization logic
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  static create() {
    PrivateConstructor.#isInternalConstructing = true;
    const instance = new PrivateConstructor();
    return instance;
  }
}

new PrivateConstructor(); // TypeError: PrivateConstructor is not constructable
PrivateConstructor.create(); // PrivateConstructor {}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-lexical-grammar.html#prod-PrivateIdentifier`

Browser compatibility

javascript.classes.private_class_fields

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Private class fields	74	79	90	62	14.1	74	90	53	14.5	11.0	74	14.5	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.private_class_fields`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

javascript.classes.private_class_fields_in

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Private class fields 'in'	91	91	90	77	15	91	90	64	15	16.0	91	15	1.0.0	1.9	16.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.private_class_fields_in`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

javascript.classes.private_class_methods

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Private class methods	84	84	90	70	15	84	90	60	15	14.0	84	15	1.0.0	1.0	14.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.private_class_methods`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using classes guide
- Classes
- Public class fields
- `class`
- Private Syntax FAQ in the TC39 class-fields proposal
- The semantics of all JS class elements by Shu-yu Guo (2018)
- Public and private class fields on v8.dev (2018)

Public class fields

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Public_class_fields · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since September 2022.

Public fields are writable, enumerable, and configurable properties defined on each class instance or class constructor.

Syntax

```js
class ClassWithField {
  instanceField;
  instanceFieldWithInitializer = "instance field";
  static staticField;
  static staticFieldWithInitializer = "static field";
}
```

There are some additional syntax restrictions:

- The name of a static property (field or method) cannot be `prototype`.
- The name of a class field (static or instance) cannot be `constructor`.

Description

This page introduces public instance fields in detail.

- For public static fields, see `static`.
- For private fields, see private elements.
- For public methods, see method definitions.
- For public accessors, see getter and setter.

Public instance fields exist on every created instance of a class. By declaring a public field, you can ensure the field is always present, and the class definition is more self-documenting.

Public instance fields are added to the instance either at construction time in the base class (before the constructor body runs), or just after `super()` returns in a subclass. Fields without initializers are initialized to `undefined`. Like properties, field names may be computed.

```js
const PREFIX = "prefix";

class ClassWithField {
  field;
  fieldWithInitializer = "instance field";
  [`${PREFIX}Field`] = "prefixed field";
}

const instance = new ClassWithField();
console.log(Object.hasOwn(instance, "field")); // true
console.log(instance.field); // undefined
console.log(instance.fieldWithInitializer); // "instance field"
console.log(instance.prefixField); // "prefixed field"
```

Computed field names are only evaluated once, at class definition time. This means that each class always has a fixed set of field names, and two instances cannot have different field names via computed names. The `this` value in the computed expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

```js
class C {
  [Math.random()] = 1;
}

console.log(new C());
console.log(new C());
// Both instances have the same field name
```

In the field initializer, `this` refers to the class instance under construction, and `super` refers to the `prototype` property of the base class, which contains the base class's instance methods, but not its instance fields.

```js
class Base {
  baseField = "base field";
  anotherBaseField = this.baseField;
  baseMethod() {
    return "base method output";
  }
}

class Derived extends Base {
  subField = super.baseMethod();
}

const base = new Base();
const sub = new Derived();

console.log(base.anotherBaseField); // "base field"

console.log(sub.subField); // "base method output"
```

The field initializer expression is evaluated each time a new instance is created. (Because the `this` value is different for each instance, the initializer expression can access instance-specific properties.)

```js
class C {
  obj = {};
}

const instance1 = new C();
const instance2 = new C();
console.log(instance1.obj === instance2.obj); // false
```

The expression is evaluated synchronously. You cannot use `await` or `yield` in the initializer expression. (Think of the initializer expression as being implicitly wrapped in a function.)

Because instance fields of a class are added before the respective constructor runs, you can access the fields' values within the constructor. However, because instance fields of a derived class are defined after `super()` returns, the base class's constructor does not have access to the derived class's fields.

```js
class Base {
  constructor() {
    console.log("Base constructor:", this.field);
  }
}

class Derived extends Base {
  field = 1;
  constructor() {
    super();
    console.log("Derived constructor:", this.field);
    this.field = 2;
  }
}

const instance = new Derived();
// Base constructor: undefined
// Derived constructor: 1
console.log(instance.field); // 2
```

Fields are added one-by-one. Field initializers can refer to field values above them, but not below them. All instance and static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the one being initialized.

```js
class C {
  a = 1;
  b = this.c;
  c = this.a + 1;
  d = this.c + 1;
}

const instance = new C();
console.log(instance.d); // 3
console.log(instance.b); // undefined
```

> **Note:** This is more important with private fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

Because class fields are added using the `[[DefineOwnProperty]]` semantic (which is essentially `Object.defineProperty()`), field declarations in derived classes do not invoke setters in the base class. This behavior differs from using `this.field = ...` in the constructor.

js

```
class Base {
  set field(val) {
    console.log(val);
  }
}

class DerivedWithField extends Base {
  field = 1;
}

const instance = new DerivedWithField(); // No log

class DerivedWithConstructor extends Base {
  constructor() {
    super();
    this.field = 1;
  }
}

const instance2 = new DerivedWithConstructor(); // Logs 1
```

> **Note:** Before the class fields specification was finalized with the `[[DefineOwnProperty]]` semantic, most transpilers, including Babel and tsc, transformed class fields to the `DerivedWithConstructor` form, which has caused subtle bugs after class fields were standardized.

Examples

Using class fields

Class fields cannot depend on arguments of the constructor, so field initializers usually evaluate to the same value for each instance (unless the same expression can evaluate to different values each time, such as `Math.random()` or object initializers).

js · ✗ bad example

```
class Person {
  name = nameArg; // nameArg is out of scope of the constructor
  constructor(nameArg) {}
}
```

js · ✓ good example

```
class Person {
  // All instances of Person will have the same name
  name = "Dragomir";
}
```

However, even declaring an empty class field is beneficial, because it indicates the existence of the field, which allows type checkers as well as human readers to statically analyze the shape of the class.

```js
class Person {
  name;
  age;
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}
```

The code above seems repetitive, but consider the case where `this` is dynamically mutated: the explicit field declaration makes it clear which fields will definitely be present on the instance.

```js
class Person {
  name;
  age;
  constructor(properties) {
    Object.assign(this, properties);
  }
}
```

Because initializers are evaluated after the base class has executed, you can access properties created by the base class constructor.

```js
class Person {
  name;
  age;
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

class Professor extends Person {
  name = `Professor ${this.name}`;
}

console.log(new Professor("Radev", 54).name); // "Professor Radev"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-FieldDefinition

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Public class fields	72	79	69	60	16[1,2,3,4]	72	79	51	16[5,6,7,8]	11.0	72	16[5,6,7,8]	1.0.0	1.0	12.0.0

1. Also from version 14.1–16: partial support.
2. Also from version 14.1–16: Parentheses in field initializers can lead to `ReferenceErrors`. See bug 236843.
3. Also from version 14–14.1: partial support.
4. Also from version 14–14.1: Doesn't support public static fields. See bug 194095.
5. Also from version 14.5–16: partial support.
6. Also from version 14.5–16: Parentheses in field initializers can lead to `ReferenceErrors`. See bug 236843.
7. Also from version 14–14.5: partial support.
8. Also from version 14–14.5: Doesn't support public static fields. See bug 194095.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.public_class_fields`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using classes guide
- Classes
- Private elements
- `class`
- The semantics of all JS class elements by Shu-yu Guo (2018)
- Public and private class fields on v8.dev (2018)

JavaScript › Reference › Classes

static

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/static · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since March 2016.

The **`static`** keyword defines a static method or field for a class, or a static initialization block (see the link for more information about this usage). Static properties cannot be directly accessed on instances of the class. Instead, they're accessed on the class itself.

Static methods are often utility functions, such as functions to create or clone objects, whereas static properties are useful for caches, fixed-configuration, or any other data you don't need to be replicated across instances.

> **Note:** In the context of classes, MDN Web Docs content uses the terms properties and fields interchangeably.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Class static". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class ClassWithStaticMethod {
  static staticProperty = "someValue";
  static staticMethod() {
    return "static method has been called.";
  }
  static {
    console.log("Class static initialization block called");
  }
}

console.log(ClassWithStaticMethod.staticProperty);
// Expected output: "someValue"
console.log(ClassWithStaticMethod.staticMethod());
// Expected output: "static method has been called."
```

Syntax

js

```
class ClassWithStatic {
  static staticField;
  static staticFieldWithInitializer = value;
  static staticMethod() {
    // …
  }
}
```

There are some additional syntax restrictions:

- The name of a static property (field or method) cannot be `prototype`.
- The name of a class field (static or instance) cannot be `constructor`.

Description

This page introduces public static properties of classes, which include static methods, static accessors, and static fields.

- For private static features, see private elements.
- For instance features, see method definitions, getter, setter, and public class fields.

Public static features are declared using the `static` keyword. They are added to the class constructor at the time of class evaluation using the `[[DefineOwnProperty]]` semantic (which is essentially `Object.defineProperty()`). They are accessed again from the class constructor.

Static methods are often utility functions, such as functions to create or clone instances. Public static fields are useful when you want a field to exist only once per class, not on every class instance you create. This is useful for caches, fixed-configuration, or any other data you don't need to be replicated across instances.

Static field names can be computed. The `this` value in the computed expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

Static fields can have an initializer. Static fields without initializers are initialized to `undefined`. Public static fields are not reinitialized on subclasses, but can be accessed via the prototype chain.

```js
class ClassWithStaticField {
  static staticField;
  static staticFieldWithInitializer = "static field";
}

class SubclassWithStaticField extends ClassWithStaticField {
  static subStaticField = "subclass field";
}

console.log(Object.hasOwn(ClassWithStaticField, "staticField")); // true
console.log(ClassWithStaticField.staticField); // undefined
console.log(ClassWithStaticField.staticFieldWithInitializer); // "static field"
console.log(SubclassWithStaticField.staticFieldWithInitializer); // "static field"
console.log(SubclassWithStaticField.subStaticField); // "subclass field"
```

In the field initializer, `this` refers to the current class (which you can also access through its name), and `super` refers to the base class constructor.

```js
class ClassWithStaticField {
  static baseStaticField = "base static field";
  static anotherBaseStaticField = this.baseStaticField;

  static baseStaticMethod() {
    return "base static method output";
  }
}

class SubClassWithStaticField extends ClassWithStaticField {
  static subStaticField = super.baseStaticMethod();
}

console.log(ClassWithStaticField.anotherBaseStaticField); // "base static field"
console.log(SubClassWithStaticField.subStaticField); // "base static method output"
```

The expression is evaluated synchronously. You cannot use `await` or `yield` in the initializer expression. (Think of the initializer expression as being implicitly wrapped in a function.)

Static field initializers and static initialization blocks are evaluated one-by-one. Field initializers can refer to field values above them, but not below them. All static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the one being initialized.

> **Note:** This is more important with private static fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

Examples

Using static members in classes

The following example demonstrates several things:

1. How a static member (method or property) is defined on a class.
2. That a class with a static member can be sub-classed.
3. How a static member can and cannot be called.

js

```
class Triple {
  static customName = "Tripler";
  static description = "I triple any number you provide";
  static calculate(n = 1) {
    return n * 3;
  }
}

class SquaredTriple extends Triple {
  static longDescription;
  static description = "I square the triple of any number you provide";
  static calculate(n) {
    return super.calculate(n) * super.calculate(n);
  }
}

console.log(Triple.description); // 'I triple any number you provide'
console.log(Triple.calculate()); // 3
console.log(Triple.calculate(6)); // 18

const tp = new Triple();

console.log(SquaredTriple.calculate(3)); // 81 (not affected by parent's instantiation)
console.log(SquaredTriple.description); // 'I square the triple of any number you provide'
console.log(SquaredTriple.longDescription); // undefined
console.log(SquaredTriple.customName); // 'Tripler'

// This throws because calculate() is a static member, not an instance member.
console.log(tp.calculate()); // 'tp.calculate is not a function'
```

Calling static members from another static method

In order to call a static method or property within another static method of the same class, you can use the `this` keyword.

```js
class StaticMethodCall {
  static staticProperty = "static property";
  static staticMethod() {
    return `Static method and ${this.staticProperty} has been called`;
  }
  static anotherStaticMethod() {
    return `${this.staticMethod()} from another static method`;
  }
}
StaticMethodCall.staticMethod();
// 'Static method and static property has been called'

StaticMethodCall.anotherStaticMethod();
// 'Static method and static property has been called from another static method'
```

Calling static members from a class constructor and other methods

Static members are not directly accessible using the `this` keyword from non-static methods. You need to call them using the class name: `CLASSNAME.STATIC_METHOD_NAME()` / `CLASSNAME.STATIC_PROPERTY_NAME` or by calling the method as a property of the `constructor`: `this.constructor.STATIC_METHOD_NAME()` / `this.constructor.STATIC_PROPERTY_NAME`

```js
class StaticMethodCall {
  constructor() {
    console.log(StaticMethodCall.staticProperty); // 'static property'
    console.log(this.constructor.staticProperty); // 'static property'
    console.log(StaticMethodCall.staticMethod()); // 'static method has been called.'
    console.log(this.constructor.staticMethod()); // 'static method has been called.'
  }

  static staticProperty = "static property";
  static staticMethod() {
    return "static method has been called.";
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`static` keyword	49[1]	13	45	36[2]	9	49[1]	45	36[2]	9	5.0[3]	49[1]	9	1.0.0	1.0	6.0.0
Static class fields	72	79	75	60	14.1	72	79	51	14.5	11.0	72	14.5	1.0.0	1.0	12.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Static initialization blocks	94	94	93	80	16.4	94	93	66	16.4	17.0	94	16.4	1.0.0	1.14	16.11.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.
3. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.static`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using classes guide
- Classes
- Static initialization blocks
- `class`

JavaScript › Reference › Classes

Static initialization blocks

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Static_initialization_blocks · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since March 2023.

Static initialization blocks are declared within a `class`. They contain statements to be evaluated during class initialization. This permits more flexible initialization logic than `static` properties, such as using `try...catch` or setting multiple fields from a single value. Initialization is performed in the context of the current class declaration, with access to private state, which allows the class to share information of its private elements with other classes or functions declared in the same scope (analogous to "friend" classes in C++).

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Class static initialization blocks". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class ClassWithStaticInitializationBlock {
  static staticProperty1 = "Property 1";
  static staticProperty2;
  static {
    this.staticProperty2 = "Property 2";
  }
}

console.log(ClassWithStaticInitializationBlock.staticProperty1);
// Expected output: "Property 1"
console.log(ClassWithStaticInitializationBlock.staticProperty2);
// Expected output: "Property 2"
```

Syntax

```js
class ClassWithSIB {
  static {
    // …
  }
}
```

Description

Without static initialization blocks, complex static initialization might be achieved by calling a static method after the class declaration:

```js
class MyClass {
  static init() {
    // Access to private static fields is allowed here
  }
}

MyClass.init();
```

However, this approach exposes an implementation detail (the `init()` method) to the user of the class. On the other hand, any initialization logic declared outside the class does not have access to private static fields. Static initialization blocks allow arbitrary initialization logic to be declared within the class and executed during class evaluation.

A `class` can have any number of `static {}` initialization blocks in its class body. These are evaluated, along with any interleaved static field initializers, in the order they are declared. Any static initialization of a super class is performed first, before that of its sub classes.

The scope of the variables declared inside the static block is local to the block. This includes `var`, `function`, `const`, and `let` declarations. `var` declarations will not be hoisted out of the static block.

```js
var y = "Outer y";

class A {
  static field = "Inner y";
  static {
    // var y only hoisted inside block
    console.log(y); // undefined <-- not 'Outer y'

    var y = this.field;
  }
}

// var y defined in static block is not hoisted
// outside the block
console.log(y); // 'Outer y'
```

The `this` inside a static block refers to the constructor object of the class. `super.property` can be used to access static properties of the super class. Note however that it is a syntax error to call `super()` in a class static initialization block, or to use the `arguments` object.

The statements are evaluated synchronously. You cannot use `await` or `yield` in this block. (Think of the initialization statements as being implicitly wrapped in a function.)

The scope of the static block is nested *within* the lexical scope of the class body, and can access private names declared within the class without causing a syntax error.

Static field initializers and static initialization blocks are evaluated one-by-one. The initialization block can refer to field values above it, but not below it. All static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the current block.

> **Note:** This is more important with private static fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

A static initialization block may not have decorators (the class itself may).

Examples

Multiple blocks

The code below demonstrates a class with static initialization blocks and interleaved static field initializers. The output shows that the blocks and fields are evaluated in execution order.

```js
class MyClass {
  static field1 = console.log("static field1");
  static {
    console.log("static block1");
  }
  static field2 = console.log("static field2");
  static {
    console.log("static block2");
  }
}
// 'static field1'
// 'static block1'
// 'static field2'
// 'static block2'
```

Note that any static initialization of a super class is performed first, before that of its sub classes.

Using this and super

The `this` inside a static block refers to the constructor object of the class. This code shows how to access a public static field.

```js
class A {
  static field = "static field";
  static {
    console.log(this.field);
  }
}
// 'static field'
```

The `super.property` syntax can be used inside a `static` block to reference static properties of a super class.

```js
class A {
  static field = "static field";
}

class B extends A {
  static {
    console.log(super.field);
  }
}
// 'static field'
```

Access to private elements

This example below shows how access can be granted to a private instance field of a class from an object outside the class (example from the v8.dev blog):

```js
let getDPrivateField;

class D {
  #privateField;
  constructor(v) {
    this.#privateField = v;
  }
  static {
    getDPrivateField = (d) => d.#privateField;
  }
}

console.log(getDPrivateField(new D("private"))); // 'private'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-ClassStaticBlock`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Static initialization blocks	94	94	93	80	16.4	94	93	66	16.4	17.0	94	16.4	1.0.0	1.14	16.11.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.classes.static.initialization_blocks`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using classes guide
- Classes
- `static`
- `class`
- Class static initialization blocks on v8.dev (2021)
- ES2022 feature: class static initialization blocks by Dr. Axel Rauschmayer (2021)

JavaScript › Reference

Regular expressions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions · MDN page last modified 2026-11-06

A **regular expression** (*regex* for short) allows developers to match strings against a pattern, extract submatch information, or simply test if the string conforms to that pattern. Regular expressions are used in many programming languages, and JavaScript's syntax is inspired by Perl.

You are encouraged to read the regular expressions guide to get an overview of the available regex syntaxes and how they work.

Description

Regular expressions are an important concept in formal language theory. They are a way to describe a possibly infinite set of character strings (called a *language*). A regular expression, at its core, needs the following features:

- A set of *characters* that can be used in the language, called the *alphabet*.
- *Concatenation*: `ab` means "the character `a` followed by the character `b`".
- *Union*: `a|b` means "either `a` or `b`".
- *Kleene star*: `a*` means "zero or more `a` characters".

Assuming a finite alphabet (such as the 26 letters of the English alphabet, or the entire Unicode character set), all regular languages can be generated by the features above. Of course, many patterns are very tedious to express this way (such as "10 digits" or "a character that's not a space"), so JavaScript regular expressions include many shorthands, introduced below.

> **Note:** JavaScript regular expressions are in fact not regular, due to the existence of backreferences (regular expressions must have finite states). However, they are still a very useful feature.

Creating regular expressions

A regular expression is typically created as a literal by enclosing a pattern in forward slashes (`/`):

```js
const regex1 = /ab+c/g;
```

Regular expressions can also be created with the `RegExp()` constructor:

```js
const regex2 = new RegExp("ab+c", "g");
```

They have no runtime differences, although they may have implications on performance, static analyzability, and authoring ergonomic issues with escaping characters. For more information, see the `RegExp` reference.

Regex flags

Flags are special parameters that can change the way a regular expression is interpreted or the way it interacts with the input text. Each flag corresponds to one accessor property on the `RegExp` object.

Flag	Description	Corresponding property
`d`	Generate indices for substring matches.	`hasIndices`

Flag	Description	Corresponding property
`g`	Global search.	`global`
`i`	Case-insensitive search.	`ignoreCase`
`m`	Makes `^` and `$` match the start and end of each line instead of those of the entire string.	`multiline`
`s`	Allows `.` to match newline characters.	`dotAll`
`u`	"Unicode"; treat a pattern as a sequence of Unicode code points.	`unicode`
`v`	An upgrade to the `u` mode with more Unicode features.	`unicodeSets`
`y`	Perform a "sticky" search that matches starting at the current position in the target string.	`sticky`

The `i`, `m`, and `s` flags can be enabled or disabled for specific parts of a regex using the modifier syntax.

The sections below list all available regex syntaxes, grouped by their syntactic nature.

Assertions

Assertions are constructs that test whether the string meets a certain condition at the specified position, but do not consume characters. Assertions cannot be quantified.

Buffer boundary assertion: `\A`, `\z`, `\Z`

Asserts that the current position in the string is strictly at the start or end of the entire string (`\Z` also allows a trailing newline), regardless of the presence of the `m` flag.

Input boundary assertion: `^`, `$`

Asserts that the current position is the start or end of input, or start or end of a line if the `m` flag is set.

Lookahead assertion: `(?=...)`, `(?!...)`

Asserts that the current position is followed or not followed by a certain pattern.

Lookbehind assertion: `(?<=...)`, `(?<!...)`

Asserts that the current position is preceded or not preceded by a certain pattern.

Word boundary assertion: `\b`, `\B`

Asserts that the current position is a word boundary.

Atoms

Atoms are the most basic units of a regular expression. Each atom *consumes* zero or more characters in the string, and either fails the match or allows the pattern to continue matching with the next atom.

Backreference: `\1`, `\2`

Matches a previously matched subpattern captured with a capturing group.

Capturing group: `(...)`

Matches a subpattern and remembers information about the match.

Character class: `[...]`, `[^...]`

Matches any character in or not in a set of characters. When the `v` flag is enabled, it can also be used to match finite-length strings.

Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`

Matches any character in or not in a predefined set of characters.

Character escape: `\n`, `\u{...}`

Matches a character that may not be able to be conveniently represented in its literal form.

Literal character: `a`, `b`

Matches a specific character.

Modifier: `(?ims-ims:...)`

Overrides flag settings in a specific part of a regular expression.

Named backreference: `\k<name>`

Matches a previously matched subpattern captured with a named capturing group.

Named capturing group: `(?<name>...)`

Matches a subpattern and remembers information about the match. The group can later be identified by a custom name instead of by its index in the pattern.

Non-capturing group: `(?:...)`

Matches a subpattern without remembering information about the match.

Unicode character class escape: `\p{...}`, `\P{...}`

Matches a set of characters specified by a Unicode property. When the `v` flag is enabled, it can also be used to match finite-length strings.

Wildcard: `.`

Matches any character except line terminators, unless the `s` flag is set.

Other features

These features do not specify any pattern themselves, but are used to compose patterns.

Disjunction: `|`

Matches any of a set of alternatives separated by the `|` character.

Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

Matches an atom a certain number of times.

Escape sequences

Escape sequences in regexes refer to any kind of syntax formed by `\` followed by one or more characters. They may serve very different purposes depending on what follows `\`. Below is a list of all valid "escape sequences":

Escape sequence	Followed by	Meaning
`\A`	None	Buffer boundary assertion
`\B`	None	Non-word-boundary assertion
`\D`	None	Character class escape representing non-digit characters
`\P`	{, a Unicode property and/or value, then }	Unicode character class escape representing characters without the specified Unicode property
`\S`	None	Character class escape representing non-white-space characters
`\W`	None	Character class escape representing non-word characters
`\Z`	None	Buffer boundary assertion
`\b`	None	Word boundary assertion; inside character classes, represents U+0008 (BACKSPACE)

Escape sequence	Followed by	Meaning
`\c`	A letter from `A` to `Z` or `a` to `z`	A character escape representing the control character with value equal to the letter's character value modulo 32
`\d`	None	Character class escape representing digit characters (`0` to `9`)
`\f`	None	Character escape representing U+000C (FORM FEED)
`\k`	`<`, an identifier, then `>`	A named backreference
`\n`	None	Character escape representing U+000A (LINE FEED)
`\p`	`{`, a Unicode property and/or value, then `}`	Unicode character class escape representing characters with the specified Unicode property
`\q`	`{`, a string, then a `}`	Only valid inside v-mode character classes; represents the string to be matched literally
`\r`	None	Character escape representing U+000D (CARRIAGE RETURN)
`\s`	None	Character class escape representing whitespace characters
`\t`	None	Character escape representing U+0009 (CHARACTER TABULATION)
`\u`	4 hexadecimal digits; or `{`, one or more hexadecimal digits, then `}`	Character escape representing the character with the given code point
`\v`	None	Character escape representing U+000B (LINE TABULATION)
`\w`	None	Character class escape representing word characters (`A` to `Z`, `a` to `z`, `0` to `9`, `_`)
`\x`	2 hexadecimal digits	Character escape representing the character with the given value
`\z`	None	Buffer boundary assertion
`\0`	None	Character escape representing U+0000 (NULL)

`\` followed by `0` and another digit becomes a legacy octal escape sequence, which is forbidden in Unicode-aware mode. `\` followed by any other digit sequence becomes a backreference.

In addition, `\` can be followed by some non-letter-or-digit characters, in which case the escape sequence is always a character escape representing the escaped character itself:

- `\$`, `\(`, `\)`, `\*`, `\+`, `\.`, `\/`, `\?`, `\[`, `\\`, `\]`, `\^`, `\{`, `\|`, `\}`: valid everywhere
- `\-`: only valid inside character classes
- `\!`, `\#`, `\%`, `\&`, `\,`, `\:`, `\;`, `\<`, `\=`, `\>`, `\@`, `` \` ``, `\~`: only valid inside v-mode character classes

The other ASCII characters, namely space character, `"`, `'`, `_`, and any letter character not mentioned above, are not valid escape sequences. In Unicode-unaware mode, escape sequences that are not one of the above become *identity escapes*: they represent the character that follows the backslash. For example, `\a` represents the character `a`. This behavior limits the ability to introduce new escape sequences without causing backward compatibility issues, and is therefore forbidden in Unicode-aware mode.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Atom`

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-RegularExpressionModifiers
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Assertion
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-AtomEscape
Regular Expression Buffer Boundaries for ECMAScript https://tc39.es/proposal-regexp-buffer-boundaries/#sec-patterns
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-patterns-static-semantics-early-errors
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterEscape
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-RegExpUnicodeEscapeSequence
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-DecimalEscape
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Quantifier
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterClassEscape
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-PatternCharacter
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterClass
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Disjunction

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Backreference: `\1`, `\2`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Buffer boundary assertion: `\A`, `\z`, `\Z` Experimental	**No**	**No**	Prev.[1]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Capturing group: `(...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Character class: `[...]`, `[^...]`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Character escape: `\n`, `\x`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unicode character escape: `\u{...}`	50	12	46	37	10	50	46	37	10	5.0	50	10	1.0.0	1.0	6.0.0
Disjunction: `|`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Input boundary assertion: `^`, `$`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Literal character: a, b	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Lookahead assertion: `(?=...)`, `(?!...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Lookbehind assertion: `(?<=...)`, `(?<!...)`	62	79	78	49	16.4	62	79	46	16.4	8.0	62	16.4	1.0.0	1.0	8.10.0
Modifier: `(?ims-ims:...)`	125	125	132	111	26	125	132	83	26	27.0	125	26	1.2.3	1.44	23.0.0
Named backreference: `\k<name>`	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0
Named capture group: `(?<name>...)`	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0
Duplicate names in different disjunction alternatives are allowed	125	125	129	111	17	125	129	83	17	27.0	125	17	1.0.0	1.44	23.0.0
Non-capturing group: `(?:...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unicode character class escape: `\p{...}`, `\P{...}`	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0
Wildcard: `.`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Word boundary assertion: `\b`, `\B`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Disabled by default; the user must enable the preference `javascript.options.experimental.regexp_buffer_boundaries` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Regular expressions guide
- RegExp

JavaScript › Reference › Regular expressions

Backreference: \1, \2

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Backreference · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **backreference** refers to the submatch of a previous capturing group and matches the same text as that group. For named capturing groups, you may prefer to use the named backreference syntax.

Syntax

regex

```
\N
```

> **Note:** `N` is not a literal character.

Parameters

N

A positive integer referring to the number of a capturing group.

Description

A backreference is a way to match the same text as previously matched by a capturing group. Capturing groups count from 1, so the first capturing group's result can be referenced with `\1`, the second with `\2`, and so on. `\0` is a character escape for the NUL character.

In case-insensitive matching, the backreference may match text with different casing from the original text.

```js
/(b)\1/i.test("bB"); // true
```

The backreference must refer to an existent capturing group. If the number it specifies is greater than the total number of capturing groups, a syntax error is thrown.

```js · ✗ bad example
/(a)\2/u; // SyntaxError: Invalid regular expression: Invalid escape
```

The entire sequence of decimal digits is interpreted as the capturing group number. For example, `\12` refers to the twelfth capturing group, not the first capturing group followed by a literal `2`.

In Unicode-unaware mode, if that capturing group does not exist, the escape is interpreted differently depending on its first digit:

- If it starts with `1` through `7`, it becomes a legacy octal escape sequence, consuming up to three octal digits with a maximum value of `0o377`. Any remaining digits are literal characters.
- If it starts with `8` or `9`, it becomes an identity escape, matching the digit itself. Any remaining digits are literal characters.

The parser does not retry a shorter capturing group number.

These are deprecated syntaxes for web compatibility, and you should not rely on them.

```js
/(a)\2/.test("a\x02"); // true
/(a)\12/.test("a\n"); // true; octal 12, not a backreference to group 1 followed by 2
/\18/.test("\x018"); // true; octal 1 followed by a literal 8
/\1234/.test("S4"); // true; octal 123 followed by a literal 4
/\400/.test(" 0"); // true; octal 40 followed by a literal 0
/\89/.test("89"); // true; identity escape followed by a literal 9
```

If the referenced capturing group is unmatched (for example, because it belongs to an unmatched alternative in a disjunction), or the group hasn't matched yet (for example, because it lies to the right of the backreference), the backreference always succeeds (as if it matches the empty string).

```js
/(?:a|(b))\1c/.test("ac"); // true
/\1(a)/.test("a"); // true
```

Examples

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

```js
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups' advantages"
```

Matching duplicate words

The following function finds duplicate words in a string (which are usually typos). Note that it uses the `\w` character class escape, which only matches English letters but not any accented letters or other alphabets. If you want more generic matching, you may want to split the string by whitespace and iterate over the resulting array.

```js
function findDuplicates(text) {
  return text.match(/\b(\w+)\s+\1\b/i)?.[1];
}

findDuplicates("foo foo bar"); // 'foo'
findDuplicates("foo bar foo"); // undefined
findDuplicates("Hello hello"); // 'Hello'
findDuplicates("Hello hellos"); // undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-DecimalEscape`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Backreference: \1, \2	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.backreference`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`
- Named backreference: `\k<name>`

JavaScript › Reference › Regular expressions

Buffer boundary assertion: \A, \z, \Z

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Buffer_boundary_assertion · MDN page last modified 2026-11-06

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

A **buffer boundary assertion** checks if the current position in the string is strictly at the start or end of the entire string (`\Z` also allows a trailing newline), regardless of the presence of the `m` flag (which changes the meanings of the `^` and `$` input boundary assertions). It's only supported in Unicode-aware mode.

Syntax

regex

```
\A
\z
\Z
```

Description

`\A` asserts that the current position is the start of the entire string. `\z` asserts that the current position is the end of the entire string. `\Z` is like `\z`, but also matches before a line terminator or `\r\n` (CRLF) sequence at the end of the string. All of them are *assertions*, so they don't consume any characters.

More precisely, `\A` asserts that the character to the left is out of bounds of the string; `\z` asserts that the character to the right is out of bounds of the string; `\Z` is equivalent to `(?=(?:\r?\n?|[\u{2028}\u{2029}]?)\z)`.

These assertions only make sense when no characters are expected to the left or right of them. For example, `f\Ao` never matches because it's not possible for `\A` to both be at the start of the string and have a character to its left.

The `\A` and `\z` assertions are only useful when the `m` flag is used. Without `m`, they behave just like `^` and `$`. If your pattern only needs to match the start or end of the entire string or a line (and never a mix of both), the recommended approach is to still use the `^` and `$` assertions, and set the `m` flag as needed, because they are more widely supported than the buffer boundary assertions. If you need to match both types of boundaries in the same pattern, you can technically use modifiers to enable or disable the `m` flag in different parts of the pattern, but using these escape sequences makes the code much more readable.

Examples

Mixing buffer and input boundary assertions

Suppose you have a pattern that needs to match either at the start of the whole string or the start of one line. You can turn on `m` so that you can use `^` to refer to the latter, and then use `\A` to refer to the former (you also need to turn on `u` so you can use `\A`).

This example matches line comments on their own lines, which can be a hashbang at start of a file or a line comment on any line. It does not match line comments at the end of a line containing code.

```js
function findLineComments(code) {
  // Matches the hashbang syntax: #!... (which is only valid at start of file)
  // and line comment syntax: //... (which is valid everywhere)
  const pattern = /\A#!.*|^\s*\/\/.*/gmu;
  return code.match(pattern);
}

const program = `#!/usr/env/node

function findLineComments(code) {
  // Matches the hashbang syntax: #!... (which is only valid at start of file)
  // and line comment syntax: //... (which is valid everywhere)
  const pattern = /\\A#!.*|^\\/\\/.*/gmu;
  return code.match(pattern);
}
`;

console.log(findLineComments(program));
// [
//   '#!/usr/env/node',
//   '  // Matches the hashbang syntax: #!... (which is only valid at start of file)',
//   '  // and line comment syntax: //... (which is valid everywhere)'
// ]
```

Another major use case of buffer boundary assertions is when you cannot modify the flags, such as regular expression search in text editors. These cases usually enable `m` by default, so you need to use `\A` and `\z` to "opt out".

Matching the end-of-file but allowing an optional trailing newline

File formats often allow an optional newline at the end of the file. If you want to match an end-of-file pattern that allows the trailing newline, you can use `\Z`. This feature is useful with or without the `m` flag.

```js
const endOfPDF = /%%EOF\Z/u;

console.log(endOfPDF.test("%%EOF")); // true
console.log(endOfPDF.test("%%EOF\n")); // true
console.log(endOfPDF.test("%%EOF\r\n")); // true
console.log(endOfPDF.test("%%EOF\n\n")); // false
console.log(endOfPDF.test("%%EOF\nsomething")); // false
```

Specifications

Specification
Regular Expression Buffer Boundaries for ECMAScript `https://tc39.es/proposal-regexp-buffer-boundaries/#sec-patterns`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Buffer boundary assertion: `\A`, `\z`, `\Z` Experimental	**No**	**No**	Prev.[1]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**

1. Disabled by default; the user must enable the preference `javascript.options.experimental.regexp_buffer_boundaries` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.buffer_boundary_assertion`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`

JavaScript › Reference › Regular expressions

Capturing group: (...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Capturing_group · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **capturing group** groups a subpattern, allowing you to apply a quantifier to the entire group or use disjunctions within it. It memorizes information about the subpattern match, so that you can refer back to it later with a backreference, or access the information through the match results.

If you don't need the result of the subpattern match, use a non-capturing group instead, which improves performance and avoids refactoring hazards.

Syntax

regex

```
(pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A capturing group acts like the grouping operator in JavaScript expressions, allowing you to use a subpattern as a single atom.

Capturing groups are numbered by the order of their opening parentheses. The first capturing group is numbered 1, the second 2, and so on. Named capturing groups are also capturing groups and are numbered together with other (unnamed) capturing groups. The information of the capturing group's match can be accessed through:

- The return value (which is an array) of `RegExp.prototype.exec()`, or `String.prototype.match()` when the regex does not have the `g` flag
- Each match array yielded by the iterator returned by `String.prototype.matchAll()`
- The `pN` parameters of the `String.prototype.replace()` and `String.prototype.replaceAll()` methods' `replacement` callback function
- Backreferences within the same pattern

> **Note:** Even in `exec()`'s result array, capturing groups are accessed by numbers `1`, `2`, etc., because the `0` element is the entire match. `\0` is not a backreference, but a character escape for the NUL character.

Capturing groups in the regex source code correspond to their results one-to-one. If a capturing group is not matched (for example, it belongs to an unmatched alternative in a disjunction), the corresponding result is `undefined`.

```js
/(ab)|(cd)/.exec("cd"); // ['cd', undefined, 'cd']
```

Capturing groups can be quantified. In this case, the match information corresponding to this group is the last match of the group.

```js
/([ab])+/.exec("abc"); // ['ab', 'b']; because "b" comes after "a", this result overwrites
the previous one
```

Capturing groups can be used in lookahead and lookbehind assertions. Because lookbehind assertions match their atoms backwards, the final match corresponding to this group is the one that appears to the *left* end of the string. However, the indices of the match groups still correspond to their relative locations in the regex source.

```js
/c(?=(ab))/.exec("cab"); // ['c', 'ab']
/(?<=(a)(b))c/.exec("abc"); // ['c', 'a', 'b']
/(?<=([ab])+)c/.exec("abc"); // ['c', 'a']; because "a" is seen by the lookbehind after the
lookbehind has seen "b"
```

Capturing groups can be nested, in which case the outer group is numbered first, then the inner group, because they are ordered by their opening parentheses. If a nested group is repeated by a quantifier,

then each time the group matches, the subgroups' results are all overwritten, sometimes with `undefined`.

```js
/((a+)?(b+)?(c))*/.exec("aacbbbcac"); // ['aacbbbcac', 'ac', 'a', undefined, 'c']
```

In the example above, the outer group is matched three times:

1. Matches `"aac"`, with subgroups `"aa"`, `undefined`, and `"c"`.
2. Matches `"bbbc"`, with subgroups `undefined`, `"bbb"`, and `"c"`.
3. Matches `"ac"`, with subgroups `"a"`, `undefined`, and `"c"`.

The `"bbb"` result from the second match is not preserved, because the third match overwrites it with `undefined`.

You can get the start and end indices of each capturing group in the input string by using the `d` flag. This creates an extra `indices` property on the array returned by `exec()`.

You can optionally specify a name to a capturing group, which helps avoid pitfalls related to group positions and indexing. See Named capturing groups for more information.

Parentheses have other purposes in different regex syntaxes. For example, they also enclose lookahead and lookbehind assertions. Because these syntaxes all start with `?`, and `?` is a quantifier which normally cannot occur directly after `(`, this does not lead to ambiguities.

Examples

Matching date

The following example matches a date in the format `YYYY-MM-DD`:

```js
function parseDate(input) {
  const parts = /^(\d{4})-(\d{2})-(\d{2})$/.exec(input);
  if (!parts) {
    return null;
  }
  return parts.slice(1).map((p) => parseInt(p, 10));
}

parseDate("2019-01-01"); // [2019, 1, 1]
parseDate("2019-06-19"); // [2019, 6, 19]
```

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

```js
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups' advantages"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Atom

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Capturing group: `(...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.capturing_group`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Groups and backreferences guide
- Regular expressions
- Non-capturing group: `(?:...)`
- Named capturing group: `(?<name>...)`
- Backreference: `\1`, `\2`

JavaScript › Reference › Regular expressions

Character class escape: \d, \D, \w, \W, \s, \S

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Character_class_escape · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

A **character class escape** is an escape sequence that represents a set of characters.

Syntax

```regex
\d, \D
\s, \S
\w, \W
```

> **Note:** , is not part of the syntax.

Description

Unlike character escapes, character class escapes represent a predefined *set* of characters, much like a character class. The following character classes are supported:

\d
: Matches any digit character. Equivalent to `[0-9]`.

\w
: Matches any word character, where a word character includes letters (A–Z, a–z), numbers (0–9), and underscore (_). If the regex is Unicode-aware and the `i` flag is set, it also matches other Unicode characters that get canonicalized to one of the characters above through case folding.

\s
: Matches any whitespace or line terminator character.

The uppercase forms `\D`, `\W`, and `\S` create complement character classes for `\d`, `\w`, and `\s`, respectively. They match any character that is not in the set of characters matched by the lowercase form.

Unicode character class escapes start with `\p` and `\P`, but they are only supported in Unicode-aware mode. In Unicode-unaware mode, they are identity escapes for the `p` or `P` character.

Character class escapes can be used in character classes. However, they cannot be used as boundaries of character ranges, which is only allowed as a deprecated syntax for web compatibility, and you should not rely on it.

Examples

Splitting by whitespace

The following example splits a string into an array of words, supporting all kinds of whitespace separators:

```js
function splitWords(str) {
  return str.split(/\s+/);
}

splitWords(`Look at the stars
Look  how they\tshine for you`);
// ['Look', 'at', 'the', 'stars', 'Look', 'how', 'they', 'shine', 'for', 'you']
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterClassEscape`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.character_class_escape`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`

JavaScript › Reference › Regular expressions

Character class: [...], [^...]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Character_class · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **character class** matches any character in or not in a custom set of characters. When the `v` flag is enabled, it can also be used to match finite-length strings.

Syntax

regex

```
[]
[abc]
[A-Z]

[^]
[^abc]
[^A-Z]

// `v` mode only
[operand1&&operand2]
[operand1--operand2]
[\q{substring}]
```

Parameters

`operand1`, `operand2`

Can be a single character, another square-bracket-enclosed character class, a character class escape, a Unicode character class escape, or a string using the `\q` syntax.

`substring`

A literal string.

Description

A character class specifies a list of characters between square brackets and matches any character in the list. The `v` flag drastically changes how character classes are parsed and interpreted. The following syntaxes are available in both `v` mode and non-`v` mode:

- A single character: matches the character itself.
- A range of characters: matches any character in the inclusive range. The range is specified by two characters separated by a dash (`-`). The first character must not be larger in character value than the second character. The *character value* is the Unicode code point of the character. Because Unicode code points are usually assigned to alphabets in order, `[a-z]` specifies all lowercase Latin characters, while `[α-ω]` specifies all lowercase Greek characters. In Unicode-unaware mode, regexes are interpreted as a sequence of BMP characters. Therefore, surrogate pairs in character classes represent two characters instead of one; see below for details.
- Escape sequences: `\b`, `\-`, character class escapes, Unicode character class escapes, and other character escapes.

These syntaxes can occur any number of times, and the character sets they represent are unioned. For example, `/[a-zA-Z0-9]/` matches any letter or digit.

The `^` prefix in a character class creates a *complement class*. For example, `[^abc]` matches any character except `a`, `b`, or `c`. The `^` character is a literal character when it appears in the middle of a character class — for example, `[a^b]` matches the characters `a`, `^`, and `b`.

The lexical grammar does a very rough parse of regex literals, so that it does not end the regex literal at a `/` character which appears within a character class. This means `/[/]/` is valid without needing to escape the `/`.

The boundaries of a character range must not specify more than one character, which happens if you use a character class escape. For example:

js · ✗ bad example

```
/[\s-9]/u; // SyntaxError: Invalid regular expression: Invalid character class
```

In Unicode-unaware mode, character ranges where one boundary is a character class make the `-` become a literal character. This is a deprecated syntax for web compatibility, and you should not rely on it.

js

```
/[\s-9]/.test("-"); // true
```

In Unicode-unaware mode, regexes are interpreted as a sequence of BMP characters. Therefore, surrogate pairs in character classes represent two characters instead of one.

```js
/[😊]/.test("\ud83d"); // true
/[😊]/u.test("\ud83d"); // false

/[😊-😛]/.test("😑"); // SyntaxError: Invalid regular expression: /[😊-😛]/: Range out of
order in character class
/[😊-😛]/u.test("😑"); // true
```

Even if the pattern ignores case, the case of the two ends of a range is significant in determining which characters belong to the range. For example, the pattern `/[E-F]/i` only matches `E`, `F`, `e`, and `f`, while the pattern `/[E-f]/i` matches all uppercase and lowercase ASCII letters (because it spans over `E–Z` and `a–f`), as well as `[`, `\`, `]`, `^`, `_`, and `` ` ``.

Non-v-mode character class

Non-v-mode character classes interpret most characters literally and have fewer restrictions about the characters they can contain. For example, `.` is the literal dot character, not the wildcard. The only characters that cannot appear literally are `\`, `]`, and `-`.

- In character classes, most escape sequences are supported, except `\b`, `\B`, and backreferences. `\b` indicates a backspace character instead of a word boundary, while the other two cause syntax errors in Unicode-aware mode.

  In Unicode-unaware mode, `\B` is an identity escape for `B`, and numeric escapes are interpreted as legacy octal escapes or identity escapes instead of backreferences. For example, `\1` matches U+0001 and `\8` matches `8`. To use `\` literally, escape it as `\\`.
- The `]` character indicates the end of the character class. To use it literally, escape it as `\]`.
- The dash (`-`) character, when used between two characters, indicates a range. When it appears at the start or end of a character class, it is a literal character. It's also a literal character when it's used in the boundary of a range. For example, `[a-]` matches the characters `a` and `-`, `[!--]` matches the characters `!` to `-`, and `[--9]` matches the characters `-` to `9`. You can also escape it as `\-` if you want to use it literally anywhere.

v-mode character class

The basic idea of character classes in `v` mode remains the same: you can still use most characters literally, use `-` to denote character ranges, and use escape sequences. One of the most important features of the `v` flag is *set notation* within character classes. As previously mentioned, normal character classes can express unions by concatenating two ranges, such as using `[A-Z0-9]` to mean "the union of the set `[A-Z]` and the set `[0-9]`". However, there's no easy way to represent other operations with character sets, such as intersection and difference.

With the `v` flag, intersection is expressed with `&&`, and subtraction with `--`. The absence of both implies union. The two operands of `&&` or `--` can be a character, character escape, character class escape, or even another character class. For example, to express "a word character that's not an underscore", you can use `[\w--_]`. You cannot mix operators on the same level. For example, `[\w&&[A-z]--_]` is a syntax error. However, because you can nest character classes, you can be explicit by writing `[\w&&[[A-z]--_]]` or `[[\w&&[A-z]]--_]` (which both mean `[A-Za-z]`). Similarly, `[AB--C]` is invalid and you need to write `[A[B--C]]` (which just means `[AB]`).

In `v` mode, the Unicode character class escape `\p` can match finite-length strings, such as emojis. For symmetry, regular character classes can also match more than one character. To write a "string literal" in a character class, you wrap the string in `\q{...}`. The only regex syntax supported here is disjunction

— apart from this, `\q` must completely enclose literals (including escaped characters). This ensures that character classes can only match finite-length strings with finitely many possibilities.

Unlike regular disjunction, the order of alternatives within `\q` does not matter, neither does the order of the `\q` escape with the rest of the operands being unioned. During matching, all alternatives specified in a character class are always attempted in descending order of length, so the match is greedy. For example, `[\q{a|ab}]`, `[\q{ab|a}]`, `[\q{ab}a]`, and `[a\q{ab}]` applied to `"ab"` will all match `"ab"`.

Complement character classes `[^...]` cannot possibly be able to match strings longer than one character. For example, `[\q{ab|c}]` is valid and matches the string `"ab"`, but `[^\q{ab|c}]` is invalid because it's unclear how many characters should be consumed. The check is done by checking if all `\q` contain single characters and all `\p` specify character properties — for unions, all operands must be purely characters; for intersections, at least one operand must be purely characters; for subtraction, the leftmost operand must be purely characters. The check is syntactic without looking at the actual character set being specified, which means although `/[^\q{ab|c}--\q{ab}]/v` is equivalent to `/[^c]/v`, it's still rejected.

Because the character class syntax is now more sophisticated, more characters are reserved and forbidden from appearing literally.

- In addition to `]` and `\`, the following characters must be escaped in character classes if they represent literal characters: `(`, `)`, `[`, `{`, `}`, `/`, `-`, `|`. This list is somewhat similar to the list of syntax characters, except that `^`, `$`, `*`, `+`, and `?` are not reserved inside character classes, while `/` and `-` are not reserved outside character classes (although `/` may delimit a regex literal and therefore still needs to be escaped). All these characters may also be optionally escaped in `u`-mode character classes.
- The following "double punctuator" sequences must be escaped as well (but they don't make much sense without the `v` flag anyway): `&&`, `!!`, `##`, `$$`, `%%`, `**`, `++`, `,,`, `..`, `::`, `;;`, `<<`, `==`, `>>`, `??`, `@@`, `^^`, `` `` ``, `~~`. In `u` mode, some of these characters can only appear literally within character classes and cause a syntax error when escaped. In `v` mode, they must be escaped when appearing in pairs, but can be optionally escaped when appearing alone. For example, `/[\!]/u` is invalid because it's an identity escape, but both `/[\!]/v` and `/[!]/v` are valid, while `/[!!]/v` is invalid. The literal character reference has a detailed table of which characters can appear escaped or unescaped.

Complement classes and case-insensitive matching

Case-insensitive matching works by case-folding both the expected character set and the matched string. When specifying complement classes, the order in which JavaScript performs case-folding and complementing is important. In brief, `[^...]` in `u` mode matches `allCharacters - caseFold(original)`, while in `v` mode matches `caseFold(allCharacters) - caseFold(original)`. This ensures that all complement class syntaxes, including `[^...]`, `\P`, `\W`, etc., cancel each other out.

Consider the following two regexes (to simplify things, let's assume that Unicode characters are one of three kinds: lowercase, uppercase, and caseless, and each uppercase letter has a unique lowercase counterpart, and vice versa):

```js
const r1 = /\p{Lowercase_Letter}/iu;
const r2 = /[^\P{Lowercase_Letter}]/iu;
```

The `r2` is a double negation and seems to be equivalent with `r1`. But in fact, `r1` matches all lower- and uppercase ASCII letters, while `r2` matches none. Here's a step-by-step explanation:

- In `r1`, `\p{Lowercase_Letter}` constructs a set of all lowercase characters. Characters in this set are then case-folded to their lowercase form, so they stay the same. The input string is also case-folded to lowercase. Therefore, `"A"` and `"a"` are both folded to `"a"` and matched by `r1`.
- In `r2`, `\P{Lowercase_Letter}` first constructs a set of all non-lowercase characters, i.e., uppercase letters and caseless characters. Characters in this set are then case-folded to their lowercase form, so the character set becomes all lowercase letters and caseless characters. `[^...]` negates the match, causing it to match anything that's *not* in this set, i.e., an uppercase letter. However, the input is still case-folded to lowercase, so `"A"` is folded to `"a"` and not matched by `r2`.

The main observation here is that after `[^...]` negates the match, the expected character set may not be a subset of the set of case-folded Unicode characters, causing the case-folded input to not be in the expected character set. In `v` mode, the set of all characters is also case-folded. The `\P` character class itself also works slightly differently in `v` mode (see Unicode character class escape). All of these ensure that `[^\P{Lowercase_Letter}]` and `\p{Lowercase_Letter}` are strictly equivalent.

Examples

Matching hexadecimal digits

The following function determines whether a string contains a valid hexadecimal number:

```js
function isHexadecimal(str) {
  return /^[0-9A-F]+$/i.test(str);
}

isHexadecimal("2F3"); // true
isHexadecimal("beef"); // true
isHexadecimal("undefined"); // false
```

Using intersection

The following function matches Greek letters.

```js
function greekLetters(str) {
  return str.match(/[\p{Script_Extensions=Greek}&&\p{Letter}]/gv);
}

// 𐆊 is U+1018A GREEK ZERO SIGN
greekLetters("π𐆊P0零αAΣ"); // [ 'π', 'α', 'Σ' ]
```

Using subtraction

The following function matches all non-ASCII numbers.

```js
function nonASCIINumbers(str) {
  return str.match(/[\p{Decimal_Number}--\d]/gv);
}

// 𑜹 is U+11739 AHOM DIGIT NINE
nonASCIINumbers("𐋡0零1𑜹a"); // [ '𑜹', '𐋡' ]
```

Matching strings

The following function matches all line terminator sequences, including the line terminator characters and the sequence `\r\n` (CRLF). The order in which the alternatives are specified does not matter, because the longest alternative, `\r\n`, is always preferred.

```js
function getLineTerminatorSequences(str) {
  return str.match(/[\r\n\u2028\u2029\q{\r\n}]/gv);
}

getLineTerminatorSequences(`
A poem\r
Is split\rInto many
Stanzas
`); // ['\n', '\r\n', '\r', '\n', '\n']
```

This regular expression is exactly equivalent to `/(?:\r\n|\r|\n|\u2028|\u2029)/gu` or `/(?:\r\n|[\r\n\u2028\u2029])/gu`. However, in regular disjunction, the order of alternatives matters, so `\r\n` must be specified first in order for it to be preferred. Using a character class not only is shorter but also avoids this pitfall.

The most useful case of `\q{}` is when doing subtraction and intersection. Previously, this was possible with multiple lookaheads. The following function matches flags that are not one of the American, Chinese, Russian, British, and French flags.

```js
function notUNSCPermanentMember(flag) {
  return /^[\p{RGI_Emoji_Flag_Sequence}--\q{US|CN|RU|GB|FR}]$/v.test(flag);
}

notUNSCPermanentMember("US"); // false
notUNSCPermanentMember("DE"); // true
```

This example is mostly equivalent to `/^(?!US|CN|RU|GB|FR)\p{RGI_Emoji_Flag_Sequence}$/v`, except perhaps more performant.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterClass

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Character class: `[...]`, `[^...]`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.character_class`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Literal character: `a`, `b`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

JavaScript › Reference › Regular expressions

Character escape: \n, \u{...}

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Character_escape · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **character escape** represents a character that may not be able to be conveniently represented in its literal form.

Syntax

regex

```
\f, \n, \r, \t, \v
\cA, \cB, …, \cz
\0
\^, \$, \\, \., \*, \+, \?, \(, \), \[, \], \{, \}, \|, \/

\xHH
\uHHHH
\u{H…H}
```

> **Note:** `,` is not part of the syntax.

Parameters

H…H

A hexadecimal number representing the Unicode code point of the character. The `\xHH` form must have two hexadecimal digits; the `\uHHHH` form must have four; the `\u{H…H}` form must have one or more hexadecimal digits representing a value no greater than `0x10FFFF`.

Description

The following character escapes are recognized in regular expressions:

`\f`, `\n`, `\r`, `\t`, `\v`

Same as those in string literals, except `\b`, which represents a word boundary in regexes unless in a character class.

`\c` followed by a letter from `A` to `Z` or `a` to `z`

Represents the control character with value equal to the letter's character value modulo 32. For example, `\cJ` represents line break (`\n`), because the code point of `J` is 74, and 74 modulo 32 is 10, which is the code point of line break. Because an uppercase letter and its lowercase form differ by 32, `\cJ` and `\cj` are equivalent. You can represent control characters from 1 to 26 in this form.

`\0`

Represents the U+0000 NUL character. Cannot be followed by a digit (which makes it a legacy octal escape sequence).

`\^, \$, \\, \., \*, \+, \?, \(, \), \[, \], \{, \}, \|, \/`

Represents the character itself. For example, `\\` represents a backslash, and `\(` represents a left parenthesis. These are syntax characters in regexes (`/` is the delimiter of a regex literal), so they require escaping unless in a character class.

`\xHH`

Represents the character with the given hexadecimal Unicode code point. The hexadecimal number must be exactly two digits long.

`\uHHHH`

Represents the character with the given hexadecimal Unicode code point. The hexadecimal number must be exactly four digits long. Two such escape sequences can be used to represent a surrogate pair in Unicode-aware mode. (In Unicode-unaware mode, they are always two separate characters.)

`\u{H…H}`

(Unicode-aware mode only) Represents the character with the given hexadecimal Unicode code point. The hexadecimal number must have one or more digits and a value no greater than `0x10FFFF`. Leading zeros are allowed, so the number can be arbitrarily long.

In Unicode-unaware mode, escape sequences that are not one of the above become *identity escapes*: they represent the character that follows the backslash. For example, `\a` represents the character `a`. This behavior limits the ability to introduce new escape sequences without causing backward compatibility issues, and is therefore forbidden in Unicode-aware mode.

In Unicode-unaware mode, `]`, `{`, and `}` may appear literally if it's not possible to parse them as the end of a character class or quantifier delimiters. This is a deprecated syntax for web compatibility, and you should not rely on it.

In Unicode-unaware mode, escape sequences within character classes of the form `\cX` where `X` is a number or `_` are decoded in the same way as those with ASCII letters: `\c0` is the same as `\cP` when taken modulo 32. In addition, if the form `\cX` is encountered anywhere where `X` is not one of the recognized characters, then the backslash is treated as a literal character. These syntaxes are also deprecated.

```js
/[\c0]/.test("\x10"); // true
/[\c_]/.test("\x1f"); // true
/[\c*]/.test("\\"); // true
/\c/.test("\\c"); // true
/\c0/.test("\\c0"); // true (the \c0 syntax is only supported in character classes)
```

Examples

Using character escapes

Character escapes are useful when you want to match a character that is not easily represented in its literal form. For example, you cannot use a line break literally in a regex literal, so you must use a character escape:

```js
const pattern = /a\nb/;
const string = `a
b`;
console.log(pattern.test(string)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterEscape

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Character escape: \n, \x	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unicode character escape: \u{...}	50	12	46	37	10	50	46	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.character_escape`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Literal character: `a`, `b`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Backreference: `\1`, `\2`
- Named backreference: `\k<name>`
- Word boundary assertion: `\b`, `\B`

JavaScript › Reference › Regular expressions

Disjunction: |

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Disjunction · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

A **disjunction** specifies multiple alternatives. Any alternative matching the input causes the entire disjunction to be matched.

Syntax

regex

```
alternative1|alternative2
alternative1|alternative2|alternative3|…
```

Parameters

`alternativeN`

One alternative pattern, composed of a sequence of atoms and assertions. Successfully matching one alternative causes the entire disjunction to be matched.

Description

The `|` regular expression operator separates two or more *alternatives*. The pattern first tries to match the first alternative; if it fails, it tries to match the second one, and so on. For example, the following matches `"a"` instead of `"ab"`, because the first alternative already matches successfully:

js

```
/a|ab/.exec("abc"); // ['a']
```

The `|` operator has the lowest precedence in a regular expression. If you want to use a disjunction as a part of a bigger pattern, you must group it.

When a grouped disjunction has more expressions after it, the matching begins by selecting the first alternative and attempting to match the rest of the regular expression. If the rest of the regular expression fails to match, the matcher tries the next alternative instead. For example,

js

```
/(?:(a)|(ab))(?:(c)|(bc))/.exec("abc"); // ['abc', 'a', undefined, undefined, 'bc']
// Not ['abc', undefined, 'ab', 'c', undefined]
```

This is because by selecting `a` in the first alternative, it's possible to select `bc` in the second alternative and result in a successful match. This process is called *backtracking*, because the matcher first goes beyond the disjunction and then comes back to it when subsequent matching fails.

Note also that any capturing parentheses inside an alternative that's not matched produce `undefined` in the resulting array.

An alternative can be empty, in which case it matches the empty string (in other words, always matches).

Alternatives are always attempted left-to-right, regardless of the direction of matching (which is reversed in a lookbehind).

Examples

Matching file extensions

The following example matches file extensions, using the same code as the input boundary assertion article:

```js
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Disjunction`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Disjunction: |	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.disjunction`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`
- Character class: `[...]`, `[^...]`

JavaScript › Reference › Regular expressions

Input boundary assertion: ^, $

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Input_boundary_assertion · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

An **input boundary assertion** checks if the current position in the string is an input boundary. An input boundary is the start or end of the string; or, if the `m` flag is set, the start or end of a line.

> **Note:** To match the start and end of the entire string in `m` mode, use the buffer boundary assertions `\A` and `\z`.

Syntax

regex

```
^
$
```

Description

`^` asserts that the current position is the start of input. `$` asserts that the current position is the end of input. Both are *assertions*, so they don't consume any characters.

More precisely, `^` asserts that the character to the left is out of bounds of the string; `$` asserts that the character to the right is out of bounds of the string. If the `m` flag is set, `^` also matches if the character to the left is a line terminator character, and `$` also matches if the character to the right is a line terminator.

Unless the `m` flag is set, these assertions only make sense when no characters are expected to the left or right of them. For example, `f^o` never matches because it's not possible for `^` to both be at the start of the string and have a character to its left.

These assertions don't change the match locations when the `y` flag is used — see also anchored sticky flag.

Examples

Removing trailing slashes

The following example removes trailing slashes from a URL string:

js

```
function removeTrailingSlash(url) {
  return url.replace(/\/$/, "");
}

removeTrailingSlash("https://example.com/"); // "https://example.com"
removeTrailingSlash("https://example.com/docs/"); // "https://example.com/docs"
```

Matching file extensions

The following example checks file types by matching the file extension, which always comes at the end of the string:

```js
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

Matching entire input

Sometimes you want to make sure that your regex matches the entire input, not just a substring of the input. For example, if you are determining if a string is a valid identifier, you can add input boundary assertions to both ends of the pattern:

```js
function isValidIdentifier(str) {
  return /^[$_\p{ID_Start}][$_\p{ID_Continue}]*$/u.test(str);
}

isValidIdentifier("foo"); // true
isValidIdentifier("$1"); // true
isValidIdentifier("1foo"); // false
isValidIdentifier("  foo  "); // false
```

This function is useful when doing codegen (generating code using code), because you can use valid identifiers differently from other string properties, such as dot notation instead of bracket notation:

```js
const variables = ["foo", "foo:bar", "  foo  "];

function toAssignment(key) {
  if (isValidIdentifier(key)) {
    return `globalThis.${key} = undefined;`;
  }
  // JSON.stringify() escapes quotes and other special characters
  return `globalThis[${JSON.stringify(key)}] = undefined;`;
}

const statements = variables.map(toAssignment).join("\n");

console.log(statements);
// globalThis.foo = undefined;
// globalThis["foo:bar"] = undefined;
// globalThis["  foo  "] = undefined;
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Assertion

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Input boundary assertion: ^, $	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.input_boundary_assertion`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assertions guide
- Regular expressions
- Buffer boundary assertion: `\A`, `\z`, `\Z`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`

JavaScript › Reference › Regular expressions

Literal character: a, b

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Literal_character · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **literal character** specifies exactly itself to be matched in the input text.

Syntax

regex

```
c
```

Parameters

c

A single character that is not one of the syntax characters described below.

Description

In regular expressions, most characters can appear literally. They are usually the most basic building blocks of patterns. For example, here is a pattern from the Removing HTML tags example:

js

```
const pattern = /<.+?>/g;
```

In this example, `.`, `+`, and `?` are called *syntax characters*. They have special meanings in regular expressions. The rest of the characters in the pattern (`<` and `>`) are literal characters. They match themselves in the input text: the left and right angle brackets.

The following characters are syntax characters in regular expressions, and they cannot appear as literal characters:

- `^`, `$`
- `\`
- `.`
- `*`, `+`, `?`, `{`, `}`
- `(`, `)`
- `[`, `]`
- `|`

Within character classes, more characters can appear literally. For more information, see the Character class page. For example `\.` and `[.]` both match a literal `.`. In v-mode character classes, however, there is a different set of characters reserved as syntax characters. To be most comprehensive, below is a table of ASCII characters and whether they may appear escaped or unescaped in different contexts, where "✅" means the character represents itself, "❌" means it throws a syntax error, and "⚠️" means the character is valid but means something other than itself.

Characters	Outside character classes in u or v mode		In u-mode character classes		In v-mode character classes	
	Unescaped	Escaped	Unescaped	Escaped	Unescaped	Escaped
`123456789 "' CEFGHIJKLMN OPQRTUVXY_ aceghijklmop quxy`	✅	❌	✅	❌	✅	❌
`!#%&,:;<=>@`~`	✅	❌	✅	❌	✅	✅
`]`	❌	✅	❌	✅	❌	✅
`()[{}`	❌	✅	✅	✅	❌	✅
`*+?`	❌	✅	✅	✅	✅	✅
`/`	✅	✅	✅	✅	❌	✅
`0DSWbdfnrstvw`	✅	⚠️	✅	⚠️	✅	⚠️
`ABZz`	✅	⚠️	✅	❌	✅	❌
`$.`	⚠️	✅	✅	✅	✅	✅
`|`	⚠️	✅	✅	✅	❌	✅
`-`	✅	❌	✅⚠️	✅	❌⚠️	✅
`^`	⚠️	✅	✅⚠️	✅	✅⚠️	✅
`\`	❌⚠️	✅	❌⚠️	✅	❌⚠️	✅

> **Note:** The characters that can both be escaped and unescaped in v-mode character classes are exactly those forbidden as "double punctuators". See v-mode character classes for more information.

Whenever you want to match a syntax character literally, you need to escape it with a backslash (`\`). For example, to match a literal `*` in a pattern, you need to write `\*` in the pattern. Using syntax characters as literal characters either leads to unexpected results or causes syntax errors — for example, `/*/` is not a valid regular expression because the quantifier is not preceded by a pattern. In Unicode-unaware mode, `]`, `{`, and `}` may appear literally if it's not possible to parse them as the end of a character class or quantifier delimiters. This is a deprecated syntax for web compatibility, and you should not rely on it.

Regular expression literals cannot be specified with certain non-syntax literal characters. `/` cannot appear as a literal character outside character classes in a regular expression literal, because `/` is used as the delimiter of the literal itself. You need to escape it as `\/` there if you want to match a literal `/`. Inside a character class, `/` can appear unescaped, as in `/[/]/` (unless the `v` flag is set). Line terminators cannot appear as literal characters in a regular expression literal either, because a literal cannot span multiple lines. You need to use a character escape like `\n` instead. These delimiter and line-terminator restrictions do not apply when using the `RegExp()` constructor, although string literals have their own escaping rules (for example, `"\\"` actually denotes a single backslash character, so `new RegExp("\\*")` and `/\*/` are equivalent).

In Unicode-unaware mode, the pattern is interpreted as a sequence of UTF-16 code units. This means surrogate pairs actually represent two literal characters. This causes unexpected behaviors when paired with other features:

```js
/^[😄]$/.test("😄"); // false, because the pattern is interpreted as /^[\ud83d\ude04]$/
/^😄+$/.test("😄😄"); // false, because the pattern is interpreted as /^\ud83d\ude04+$/
```

In Unicode-aware mode, the pattern is interpreted as a sequence of Unicode code points, and surrogate pairs do not get split. Therefore, you should always prefer to use the `u` flag.

Examples

Using literal characters

The following example is copied from Character escape. The `a` and `b` characters are literal characters in the pattern, and `\n` is an escaped character because it cannot appear literally in a regular expression literal.

```js
const pattern = /a\nb/;
const string = `a
b`;
console.log(pattern.test(string)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-PatternCharacter`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Literal character: a, b	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.literal_character`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Character escape: `\n`, `\u{...}`

Lookahead assertion: (?=...), (?!...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Lookahead_assertion · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **lookahead assertion** "looks ahead": it attempts to match the subsequent input with the given pattern, but it does not consume any of the input — if the match is successful, the current position in the input stays the same.

Syntax

regex

```
(?=pattern)
(?!pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A regular expression generally matches from left to right. This is why lookahead and lookbehind assertions are called as such — lookahead asserts what's on the right, and lookbehind asserts what's on the left.

In order for a `(?=pattern)` assertion to succeed, the `pattern` must match the text after the current position, but the current position is not changed. The `(?!pattern)` form negates the assertion — it succeeds if the `pattern` does not match at the current position.

The `pattern` can contain capturing groups. See the capturing groups page for more information on the behavior in this case.

Unlike other regular expression operators, there's no backtracking into a lookahead — this behavior is inherited from Perl. This only matters when the `pattern` contains capturing groups and the pattern following the lookahead contains backreferences to those captures. For example:

js

```
/(?=(a+))a*b\1/.exec("baabac"); // ['aba', 'a']
// Not ['aaba', 'a']
```

The matching of the pattern above happens as follows:

1. The lookahead `(a+)` succeeds before the first `"a"` in `"baabac"`, and `"aa"` is captured because the quantifier is greedy.

2. `a*b` matches the `"aab"` in `"baabac"` because lookaheads don't consume their matched strings.
3. `\1` does not match the following string, because that requires 2 `"a"`s, but only 1 is available. So the matcher backtracks, but it doesn't go into the lookahead, so the capturing group cannot be reduced to 1 `"a"`, and the entire match fails at this point.
4. `exec()` re-attempts matching at the next position — before the second `"a"`. This time, the lookahead matches `"a"`, and `a*b` matches `"ab"`. The backreference `\1` matches the captured `"a"`, and the match succeeds.

If the regex is able to backtrack into the lookahead and revise the choice made in there, then the match would succeed at step 3 by `(a+)` matching the first `"a"` (instead of the first two `"a"`s) and `a*b` matching `"aab"`, without even re-attempting the next input position.

Negative lookaheads can contain capturing groups as well, but backreferences only make sense within the `pattern`, because if matching continues, `pattern` would necessarily be unmatched (otherwise the assertion fails). This means outside of the `pattern`, backreferences to those capturing groups in negative lookaheads always succeed. For example:

```js
/(.*?)a(?!(a+)b\2c)\2(.*)/.exec("baaabaac"); // ['baaabaac', 'ba', undefined, 'abaac']
```

The matching of the pattern above happens as follows:

1. The `(.*?)` pattern is non-greedy, so it starts by matching nothing. However, the next character is `a`, which fails to match `"b"` in the input.
2. The `(.*?)` pattern matches `"b"` so that the `a` in the pattern matches the first `"a"` in `"baaabaac"`.
3. At this position, the lookahead succeeds to match, because if `(a+)` matches `"aa"`, then `(a+)b\2c` matches `"aabaac"`. This causes the assertion to fail, so the matcher backtracks.
4. The `(.*?)` pattern matches the `"ba"` so that the `a` in the pattern matches the second `"a"` in `"baaabaac"`.
5. At this position, the lookahead fails to match, because the remaining input does not follow the pattern "any number of `"a"`s, a `"b"`, the same number of `"a"`s, a `c`". This causes the assertion to succeed.
6. However, because nothing was matched within the assertion, the `\2` backreference has no value, so it matches the empty string. This causes the rest of the input to be consumed by the `(.*)` at the end.

Normally, assertions cannot be quantified. However, in Unicode-unaware mode, lookahead assertions can be quantified. This is a deprecated syntax for web compatibility, and you should not rely on it.

```js
/(?=a)?b/.test("b"); // true; the lookahead is matched 0 times
```

Examples

Matching strings without consuming them

Sometimes it's useful to validate that the matched string is followed by something without returning that as the result. The following example matches a string that is followed by a comma/period, but the punctuation is not included in the result:

```js
function getFirstSubsentence(str) {
  return /^.*?(?=[,.])/.exec(str)?.[0];
}

getFirstSubsentence("Hello, world!"); // "Hello"
getFirstSubsentence("Thank you."); // "Thank you"
```

A similar effect can be achieved by capturing the submatch you are interested in.

Pattern subtraction and intersection

Using lookahead, you can match a string multiple times with different patterns, which allows you to express complex relationships like subtraction (is X but not Y) and intersection (is both X and Y).

The following example matches any identifier that's not a reserved word (only showing three reserved words here for brevity; more reserved words can be added to this disjunction). The `[$_\p{ID_Start}][$\p{ID_Continue}]*` syntax describes exactly the set of identifier strings in the language spec; you can read more about identifiers in lexical grammar and the `\p` escape in Unicode character class escape.

```js
function isValidIdentifierName(str) {
  const re = /^(?!(?:break|case|catch)$)[$_\p{ID_Start}][$\p{ID_Continue}]*$/u;
  return re.test(str);
}

isValidIdentifierName("break"); // false
isValidIdentifierName("foo"); // true
isValidIdentifierName("cases"); // true
```

The following example matches a string that's both ASCII and can be used as an identifier part:

```js
function isASCIIIDPart(char) {
  return /^(?=\p{ASCII}$)[$\p{ID_Continue}]$/u.test(char);
}

isASCIIIDPart("a"); // true
isASCIIIDPart("α"); // false
isASCIIIDPart(":"); // false
```

If you are doing intersection and subtraction with finitely many characters, you may want to use the character set intersection syntax enabled with the `v` flag.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Assertion`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Lookahead assertion: `(?=...)`, `(?!...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.lookahead_assertion`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Capturing group: `(...)`

JavaScript › Reference › Regular expressions

Lookbehind assertion: (?<=...), (?<!...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Lookbehind_assertion · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2023.

A **lookbehind assertion** "looks behind": it attempts to match the previous input with the given pattern, but it does not consume any of the input — if the match is successful, the current position in the input stays the same. It matches each atom in its pattern in the reverse order.

Syntax

```regex
(?<=pattern)
(?<!pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A regular expression generally matches from left to right. This is why lookahead and lookbehind assertions are called as such — lookahead asserts what's on the right, and lookbehind asserts what's on the left.

In order for a `(?<=pattern)` assertion to succeed, the `pattern` must match the input immediately to the left of the current position, but the current position is not changed before matching the subsequent

input. The `(?<!pattern)` form negates the assertion — it succeeds if the `pattern` does not match the input immediately to the left of the current position.

Lookbehind generally has the same semantics as lookahead — however, within a lookbehind assertion, the regular expression matches *backwards*. For example,

```js
/(?<=([ab]+)([bc]+))$/.exec("abc"); // ['', 'a', 'bc']
// Not ['', 'ab', 'c']
```

If the lookbehind matches from left to right, it should first greedily match `[ab]+`, which makes the first group capture `"ab"`, and the remaining `"c"` is captured by `[bc]+`. However, because `[bc]+` is matched first, it greedily grabs `"bc"`, leaving only `"a"` for `[ab]+`.

This behavior is reasonable — the matcher does not know where to *start* the match (because the lookbehind may not be fixed-length), but it does know where to *end* (at the current position). Therefore, it starts from the current position and works backwards. (Regexes in some other languages forbid non-fixed-length lookbehind to avoid this issue.)

For quantified capturing groups inside the lookbehind, the match furthest to the left of the input string — instead of the one on the right — is captured because of backward matching. See the capturing groups page for more information. Backreferences inside the lookbehind must appear on the *left* of the group it's referring to, also due to backward matching. However, disjunctions are still attempted left-to-right.

Examples

Matching strings without consuming them

Similar to lookaheads, lookbehinds can be used to match strings without consuming them so that only useful information is extracted. For example, the following regex matches the number in a price label:

```js
function getPrice(label) {
  return /(?<=\$)\d+(?:\.\d*)?/.exec(label)?.[0];
}

getPrice("$10.53"); // "10.53"
getPrice("10.53"); // undefined
```

A similar effect can be achieved by capturing the submatch you are interested in.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Assertion

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Lookbehind assertion: `(?<=...)`, `(?<!...)`	62	79	78	49	16.4	62	79	46	16.4	8.0	62	16.4	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.lookbehind_assertion`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Capturing group: `(...)`

JavaScript › Reference › Regular expressions

Modifier: (?ims-ims:...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Modifier · MDN page last modified 2026-11-06

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

A **modifier** overrides flag settings in a specific part of a regular expression. It can be used to enable or disable flags that change the meanings of certain regex syntax elements. These flags are `i`, `m`, and `s`.

Syntax

regex

```
(?flags1:pattern)
(?flags1-flags2:pattern)
```

> **Note:** JavaScript only has the "bounded" modifier form, where the pattern is placed inside the modifier group. Most other languages that support modifiers have an "unbounded" form, where the modifier is applied until the end of the closest containing group.

Parameters

`flags1` Optional

A string of flags to enable. Can contain any combination of `i`, `m`, and `s`.

`flags2` Optional

A string of flags to disable. Can contain any combination of `i`, `m`, and `s`, but must not contain any flags included in `flags1`.

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

Some flags change the meanings of regex syntax elements:

- The `i` flag makes the regex case-insensitive by normalizing the case of literal characters and character classes.
- The `m` flag changes the behavior of input boundary assertions `^` and `$` to match the start and end of each line, in addition to the start and end of the input string.
- The `s` flag changes the behavior of the wildcard `.` character to match any character, including line terminators.

Sometimes you may want these changes to only take effect in a specific part of a regex pattern. You can do so by wrapping that part in a modifier. For example:

```js
/(?i:Hello) world/;
```

In this regex, the `i` flag is only enabled for the `Hello` part of the pattern. The `world` part is case-sensitive. Therefore, it matches `Hello world`, `hello world`, and `HELLO world`, but not `HELLO WORLD`. The following is equivalent, by enabling the `i` flag globally, and then disabling it for the `world` part:

```js
/Hello (?-i:world)/i;
```

The `flags1` and `flags2` parameters can contain any combination of `i`, `m`, and `s`. However, the flags must all be unique between `flags1` and `flags2`—you cannot enable or disable a flag twice, or enable a flag and then immediately disable it.

The `flags1` and `flags2` parameters are optional, but at least one must be non-empty. `(?flags1-:pattern)` is a modifier that only enables flags (equivalent to `(?flags1:pattern)`). `(?-flags2:pattern)` is a modifier that only disables flags. `(?:pattern)` is just a non-capturing group, and `(?-:pattern)` is a syntax error.

Other flags don't make sense in a modifier and are thus syntax errors if included:

- The `g` and `y` flags determine how multiple calls to `exec()` behave and affect matching behavior of the whole regex.
- The `d` flag enables additional information in the `exec()` result and affects matching behavior of the whole regex.
- The `u` and `v` flags change the behavior of the regex engine in a way that's too complex to be locally modified. They also have global effects on the regex, such as how the `lastIndex` is advanced.

Examples

Matching a multiline format only at the start of the string

The following regex defines a format for a multiline string. The first `^` represents the start of the whole input string, by being inside a `(?-m:)` modifier, while all other `^` characters represent the start of a line:

```js
const pattern = /(?-m:^)---\n^title:.*^slug:.*^---/ms;

const input = `---
title: "Modifier: (?ims-ims:...)"
slug: Web/JavaScript/Reference/Regular_expressions/Modifier
---`;

pattern.test(input); // true

// Extra line break at the start of string
const input2 = `\n${input}`;

pattern.test(input2); // false
```

Matching certain words case-insensitively

Imagine you are finding all variable declarations called `foo` or `bar` (because they are bad names). The word may appear in any case, but you know the keyword is always lowercase, so you can do this:

```js
const pattern = /(?:var|let|const) (?i:foo|bar)\b/;

pattern.test("let foo;"); // true
pattern.test("const BAR = 1;"); // true
pattern.test("Let foo be a number"); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-RegularExpressionModifiers`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Modifier: `(?ims-ims:...)`	125	125	132	111	26	125	132	83	26	27.0	125	26	1.2.3	1.44	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.modifier`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Groups and backreferences guide
- Regular expressions
- Non-capturing group: `(?:...)`

JavaScript › Reference › Regular expressions

Named backreference: \k<name>

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Named_backreference · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2020.

A **named backreference** refers to the submatch of a previous named capturing group and matches the same text as that group. For unnamed capturing groups, you need to use the normal backreference syntax.

Syntax

regex

```
\k<name>
```

Parameters

`name`

The name of the group. Must be a valid identifier and refer to an existent named capturing group.

Description

Named backreferences are very similar to normal backreferences: they refer to the text matched by a capturing group and match the same text. The difference is that you refer to the capturing group by name instead of by number. This makes the regular expression more readable and easier to refactor and maintain.

In Unicode-unaware mode, the sequence `\k` only starts a named backreference if the regex contains at least one named capturing group. Otherwise, it is an identity escape and is the same as the literal character `k`. This is a deprecated syntax for web compatibility, and you should not rely on it.

js

```
/\k/.test("k"); // true
```

Examples

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

js

```
function parseTitle(metastring) {
  return metastring.match(/title=(?<quote>["'])(.*?)\k<quote>/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups'
advantages"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-AtomEscape`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Named backreference: `\k<name>`	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.named_backreference`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`
- Backreference: `\1`, `\2`

Named capturing group: (?<name>...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Named_capturing_group · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2020.

A **named capturing group** is a particular kind of capturing group that allows you to give a name to the group. The group's matching result can later be identified by this name instead of by its index in the pattern.

Syntax

regex

```
(?<name>pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

`name`

The name of the group. Must be a valid identifier.

Description

Named capturing groups can be used just like capturing groups — they also have their match index in the result array, and they can be referenced through `\1`, `\2`, etc. The only difference is that they can be *additionally* referenced by their name. The information of the capturing group's match can be accessed through:

- The `groups` property of the return value of `RegExp.prototype.exec()`, or `String.prototype.match()` when the regex does not have the `g` flag
- The `groups` property of each match array yielded by the iterator returned by `String.prototype.matchAll()`
- The `groups` parameter of the `String.prototype.replace()` and `String.prototype.replaceAll()` methods' `replacement` callback function
- Named backreferences within the same pattern

All names must be unique within the same pattern. Multiple named capturing groups with the same name result in a syntax error.

js · ✗ bad example

```
/(?<name>)(?<name>)/; // SyntaxError: Invalid regular expression: Duplicate capture group
name
```

This restriction is relaxed if the duplicate named capturing groups are not in the same disjunction alternative, so for any string input, only one named capturing group can actually be matched. This is a much newer feature, so check browser compatibility before using it.

js

```
/(?<year>\d{4})-\d{2}|\d{2}-(?<year>\d{4})/;
// Works; "year" can either come before or after the hyphen
```

Named capturing groups will all be present in the result. If a named capturing group is not matched (for example, it belongs to an unmatched alternative in a disjunction), the corresponding property on the `groups` object has value `undefined`.

js

```
/(?<ab>ab)|(?<cd>cd)/.exec("cd").groups; // [Object: null prototype] { ab: undefined, cd:
'cd' }
```

You can get the start and end indices of each named capturing group in the input string by using the `d` flag. In addition to accessing them on the `indices` property on the array returned by `exec()`, you can also access them by their names on `indices.groups`.

Compared to unnamed capturing groups, named capturing groups have the following advantages:

- They allow you to provide a descriptive name for each submatch result.
- They allow you to access submatch results without having to remember the order in which they appear in the pattern.
- When refactoring code, you can change the order of capturing groups without worrying about breaking other references.

Examples

Using named capturing groups

The following example parses a timestamp and an author name from a Git log entry (output with `git log --format=%ct,%an -- filename`):

```js
function parseLog(entry) {
  const { author, timestamp } = /^(?<timestamp>\d+),(?<author>.+)$/.exec(
    entry,
  ).groups;
  return `${author} committed on ${new Date(
    parseInt(timestamp, 10) * 1000,
  ).toLocaleString()}`;
}

parseLog("1560979912,Caroline"); // "Caroline committed on 6/19/2019, 5:31:52 PM"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Atom

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Named capture group: `(?<name>...)`	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0
Duplicate names in different disjunction alternatives are allowed	125	125	129	111	17	125	129	83	17	27.0	125	17	1.0.0	1.44	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.named_capturing_group`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of named capturing groups in `core-js`
- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Non-capturing group: `(?:...)`
- Named backreference: `\k<name>`
- ESLint rule: `prefer-named-capture-group`

JavaScript › Reference › Regular expressions

Non-capturing group: (?:...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Non-capturing_group · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **non-capturing group** groups a subpattern, allowing you to apply a quantifier to the entire group or use disjunctions within it. It acts like the grouping operator in JavaScript expressions, and unlike capturing groups, it does not memorize the matched text, allowing for better performance and avoiding confusion when the pattern also contains useful capturing groups.

Syntax

regex

```
(?:pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Examples

Grouping a subpattern and applying a quantifier

In the following example, we test if a file path ends with `styles.css` or `styles.[a hex hash].css`. Because the entire `\.[\da-f]+` part is optional, in order to apply the `?` quantifier to it, we need to group it into a new atom. Using a non-capturing group improves performance by not creating the extra match information that we don't need.

js

```
function isStylesheet(path) {
  return /styles(?:\.[\da-f]+)?\.css$/.test(path);
}

isStylesheet("styles.css"); // true
isStylesheet("styles.1234.css"); // true
isStylesheet("styles.cafe.css"); // true
isStylesheet("styles.1234.min.css"); // false
```

Grouping a disjunction

A disjunction has the lowest precedence in a regular expression. If you want to use a disjunction as a part of a bigger pattern, you must group it. You are advised to use a non-capturing group unless you rely on the matched text of the disjunction. The following example matches file extensions, using the same code as the input boundary assertion article:

```js
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

Avoiding refactoring hazards

Capturing groups are accessed by their position in the pattern. If you add or remove a capturing group, you must also update the positions of the other capturing groups, if you are accessing them through match results or backreferences. This can be a source of bugs, especially if most groups are purely for syntactic purposes (to apply quantifiers or to group disjunctions). Using non-capturing groups avoids this problem, and allows the indices of actual capturing groups to be easily tracked.

For example, suppose we have a function that matches the `title='xxx'` pattern in a string (example taken from capturing group). To ensure the quotes match, we use a backreference to refer to the first quote.

```js
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
```

If we later decided to add `name='xxx'` as an alias for `title=`, we would need to group the disjunction in another group:

js · ✗ bad example

```js
function parseTitle(metastring) {
  // Oops — the backreference and index access are now off by one!
  return metastring.match(/(title|name)=(["'])(.*?)\1/)[2];
}

parseTitle('name="foo"'); // Cannot read properties of null (reading '2')
// Because \1 now refers to the "name" string, which isn't found at the end.
```

Instead of locating all places where we are referring to the capturing groups' indices and updating them one-by-one, it's better to avoid using a capturing group:

js · ✓ good example

```js
function parseTitle(metastring) {
  // Do not capture the title|name disjunction
  // because we don't use its value
  return metastring.match(/(?:title|name)=(["'])(.*?)\1/)[2];
}

parseTitle('name="foo"'); // 'foo'
```

Named capturing groups are another way to avoid refactoring hazards. They allow capturing groups to be accessed by a custom name, which is unaffected when other capturing groups are added or removed.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Atom`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Non-capturing group: `(?:...)`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.non_capturing_group`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`

JavaScript › Reference › Regular expressions

Quantifier: *, +, ?, {n}, {n,}, {n,m}

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Quantifier · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2015.

A **quantifier** repeats an atom a certain number of times. The quantifier is placed after the atom it applies to.

Syntax

regex

```
// Greedy
atom?
atom*
atom+
atom{count}
atom{min,}
atom{min,max}

// Non-greedy
atom??
atom*?
atom+?
atom{count}?
atom{min,}?
atom{min,max}?
```

> **Note:** Adding `?` after `{count}` is syntactically valid but practically useless. Since `{count}` always matches exactly `count` times, `atom{count}?` behaves the same as `atom{count}`.

Parameters

`atom`

A single atom.

`count`

A non-negative integer. The number of times the atom should be repeated.

`min`

A non-negative integer. The minimum number of times the atom can be repeated.

`max` Optional

A non-negative integer. The maximum number of times the atom can be repeated. If omitted, the atom can be repeated as many times as needed.

Description

A quantifier is placed after an atom to repeat it a certain number of times. It cannot appear on its own. Each quantifier is able to specify a minimum and maximum number that a pattern must be repeated for.

Quantifier	Minimum	Maximum
`?`	0	1
`*`	0	Infinity
`+`	1	Infinity
`{count}`	`count`	`count`
`{min,}`	`min`	Infinity
`{min,max}`	`min`	`max`

For the `{count}`, `{min,}`, and `{min,max}` syntaxes, there cannot be white spaces around the numbers — otherwise, it becomes a literal pattern.

js · ✗ bad example

```
const re = /a{1, 3}/;
re.test("aa"); // false
re.test("a{1, 3}"); // true
```

This behavior is fixed in Unicode-aware mode, where braces cannot appear literally without escaping. The ability to use { and } literally without escaping is a deprecated syntax for web compatibility, and you should not rely on it.

js · ✗ bad example

```
/a{1, 3}/u; // SyntaxError: Invalid regular expression: Incomplete quantifier
```

It is a syntax error if the minimum is greater than the maximum.

js · ✗ bad example

```
/a{3,2}/; // SyntaxError: Invalid regular expression: numbers out of order in {} quantifier
```

Quantifiers can cause capturing groups to match multiple times. See the capturing groups page for more information on the behavior in this case.

Each repeated match doesn't have to be the same string.

```js
/[ab]*/.exec("aba"); // ['aba']
```

Quantifiers are *greedy* by default, which means they try to match as many times as possible until the maximum is reached, or until it's not possible to match further. You can make a quantifier *non-greedy* by adding a `?` after it. In this case, the quantifier will try to match as few times as possible, only matching more times if it's impossible to match the rest of the pattern with this many repetitions.

```js
/a*/.exec("aaa"); // ['aaa']; the entire input is consumed
/a*?/.exec("aaa"); // ['']; it's possible to consume no characters and still match
successfully
/^a*?$/.exec("aaa"); // ['aaa']; it's not possible to consume fewer characters and still
match successfully
```

However, as soon as the regex successfully matches the string at some index, it will not try subsequent indices, although that may result in fewer characters being consumed.

```js
/a*?$/.exec("aaa"); // ['aaa']; the match already succeeds at the first character, so the
regex never attempts to start matching at the second character
```

Greedy quantifiers may try fewer repetitions if it's otherwise impossible to match the rest of the pattern.

```js
/[ab]+[abc]c/.exec("abbc"); // ['abbc']
```

In this example, `[ab]+` first greedily matches `"abb"`, but `[abc]c` is not able to match the rest of the pattern (`"c"`), so the quantifier is reduced to match only `"ab"`.

Greedy quantifiers avoid matching infinitely many empty strings. If the minimum number of matches is reached and no more characters are being consumed by the atom at this position, the quantifier stops matching. This is why `/(a*)*/.exec("b")` does not result in an infinite loop.

Greedy quantifiers try to match as many *times* as possible; they do not maximize the *length* of the match. For example, `/(aa|aabaac|ba)*/.exec("aabaac")` matches `"aa"` and then `"ba"` instead of `"aabaac"`.

Quantifiers apply to a single atom. If you want to quantify a longer pattern or a disjunction, you must group it. Quantifiers cannot be applied to assertions.

js · ✗ bad example

```
/^*/; // SyntaxError: Invalid regular expression: nothing to repeat
```

In Unicode-unaware mode, lookahead assertions can be quantified. This is a deprecated syntax for web compatibility, and you should not rely on it.

```js
/(?=a)?b/.test("b"); // true; the lookahead is matched 0 times
```

Examples

Removing HTML tags

The following example removes HTML tags enclosed in angle brackets. Note the use of `?` to avoid consuming too many characters at once.

```js
function stripTags(str) {
  return str.replace(/<.+?>/gs, "");
}

stripTags("<p><em>lorem</em> <strong>ipsum</strong></p>"); // 'lorem ipsum'
```

The same effect can be achieved with a greedy match, but not allowing the repeated pattern to match `>`.

```js
function stripTags(str) {
  return str.replace(/<[^>]+>/g, "");
}

stripTags("<p><em>lorem</em> <strong>ipsum</strong></p>"); // 'lorem ipsum'
```

> **Warning:** This is for demonstration only — it doesn't handle `>` in attribute values. Use a proper HTML sanitizer like the HTML sanitizer API instead.

Locating Markdown paragraphs

In Markdown, paragraphs are separated by one or more blank lines. The following example counts all paragraphs in a string by matching two or more line breaks.

```js
function countParagraphs(str) {
  return (str.match(/(?:\r?\n){2,}/g)?.length ?? 0) + 1;
}

countParagraphs(`
Paragraph 1

Paragraph 2
Containing some line breaks, but still the same paragraph

Another paragraph
`); // 3
```

> **Warning:** This is for demonstration only — it doesn't handle line breaks in code blocks or other Markdown block elements like headings. Use a proper Markdown parser instead.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-Quantifier

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Quantifier: *, +, ?, {n}, {n,}, {n,m}	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.quantifier`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Quantifiers guide
- Regular expressions
- Disjunction: |
- Character class: `[...]`, `[^...]`

JavaScript › Reference › Regular expressions

Unicode character class escape: \p{...}, \P{...}

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Unicode_character_class_escape · MDN page last modified 2026-11-06

Baseline: Widely available — available across major browsers since July 2020.

A **Unicode character class escape** is a kind of character class escape that matches a set of characters specified by a Unicode property. It's only supported in Unicode-aware mode. When the `v` flag is enabled, it can also be used to match finite-length strings.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Regex Unicode character class escape". It runs live in the browser there; its source code follows.

js · interactive example source

```
const sentence = "A ticket to 大阪 costs ¥2000 🔌.";

const regexpEmojiPresentation = /\p{Emoji_Presentation}/gu;
console.log(sentence.match(regexpEmojiPresentation));
// Expected output: Array ["🔌"]

const regexpNonLatin = /\P{Script_Extensions=Latin}+/gu;
console.log(sentence.match(regexpNonLatin));
// Expected output: Array [" ", " ", " 大阪 ", " ¥2000 🔌."]

const regexpCurrencyOrPunctuation = /\p{Sc}|\p{P}/gu;
console.log(sentence.match(regexpCurrencyOrPunctuation));
// Expected output: Array ["¥", "."]
```

Syntax

regex

```
\p{loneProperty}
\P{loneProperty}

\p{property=value}
\P{property=value}
```

Parameters

`loneProperty`

A lone Unicode property name or value, following the same syntax as `value`. It specifies the value for the `General_Category` property, or a binary property name. In `v` mode, it can also be a binary Unicode property of strings.

> **Note:** ICU syntax allows omitting the `Script` property name as well, but JavaScript does not support this, because most of the time `Script_Extensions` is more useful than `Script`.

`property`

A Unicode property name. Must be made of ASCII letters (`A–Z`, `a–z`) and underscores (`_`), and must be one of the non-binary property names.

`value`

A Unicode property value. Must be made of ASCII letters (`A–Z`, `a–z`), underscores (`_`), and digits (`0–9`), and must be one of the supported values listed in `PropertyValueAliases.txt`.

Description

`\p` and `\P` are only supported in Unicode-aware mode. In Unicode-unaware mode, they are identity escapes for the `p` or `P` character.

Every Unicode character has a set of properties that describe it. For example, the character `a` has the `General_Category` property with value `Lowercase_Letter`, and the `Script` property with value `Latn`. The `\p` and `\P` escape sequences allow you to match a character based on its properties. For example, `a` can be matched by `\p{Lowercase_Letter}` (the `General_Category` property name is optional) as well as `\p{Script=Latn}`. `\P` creates a *complement class* that consists of code points without the specified property.

When the `i` flag is set, `\P` character classes are handled slightly differently in `u` and `v` modes. In `u` mode, case-folding happens after subtraction; in `v` mode, case-folding happens before subtraction. More concretely, in `u` mode, `\P{property}` matches `caseFold(allCharacters - charactersWithProperty)`. This means `/\P{Lowercase_Letter}/iu` still matches `"a"`, because `A` is not a `Lowercase_Letter`. In `v` mode, `\P{property}` matches `caseFold(allCharacters) - caseFold(charactersWithProperty)`. This means `/\P{Lowercase_Letter}/iv` does not match `"a"`, because `A` is not even in the set of all case-folded Unicode characters. See also complement classes and case-insensitive matching.

To compose multiple properties, use the character set intersection syntax enabled with the `v` flag, or see pattern subtraction and intersection.

In `v` mode, `\p` may match a sequence of code points, defined in Unicode as "properties of strings". This is most useful for emojis, which are often composed of multiple code points. However, `\P` can only complement character properties.

> **Note:** There are plans to port the properties of strings feature to `u` mode as well.

Examples

General categories

General categories are used to classify Unicode characters and subcategories are available to define a more precise categorization. It is possible to use both short and long forms in Unicode property escapes.

They can be used to match letters, numbers, symbols, punctuation, spaces, etc. For a more exhaustive list of general categories, please refer to the Unicode specification.

```js
// finding all the letters of a text
const story = "It's the Cheshire Cat: now I shall have somebody to talk to.";

// Most explicit form
story.match(/\p{General_Category=Letter}/gu);

// It is not mandatory to use the property name for General categories
story.match(/\p{Letter}/gu);

// This is equivalent (short alias):
story.match(/\p{L}/gu);

// This is also equivalent (disjunction of all the subcategories using short aliases)
story.match(/\p{Lu}|\p{Ll}|\p{Lt}|\p{Lm}|\p{Lo}/gu);
```

Scripts and script extensions

Some languages use different scripts for their writing system. For instance, English and Spanish are written using the Latin script while Arabic and Russian are written with other scripts (respectively Arabic and Cyrillic). The `Script` and `Script_Extensions` Unicode properties allow regular expressions to match characters according to the script they are mainly used with (`Script`) or according to the set of scripts they belong to (`Script_Extensions`).

For example, `A` belongs to the `Latin` script and `ε` to the `Greek` script.

```js
const mixedCharacters = "aεЛ";

// Using the canonical "long" name of the script
mixedCharacters.match(/\p{Script=Latin}/u); // a

// Using a short alias (ISO 15924 code) for the script
mixedCharacters.match(/\p{Script=Grek}/u); // ε

// Using the short name sc for the Script property
mixedCharacters.match(/\p{sc=Cyrillic}/u); // Л
```

For more details, refer to the Unicode specification, the Scripts table in the ECMAScript specification, and the ISO 15924 list of script codes.

If a character is used in a limited set of scripts, the `Script` property will only match for the "predominant" used script. If we want to match characters based on a "non-predominant" script, we

could use the `Script_Extensions` property (`scx` for short).

```js
// ٢ is the digit 2 in Arabic-Indic notation
// while it is predominantly written within the Arabic script
// it can also be written in the Thaana script

"٢".match(/\p{Script=Thaana}/u);
// null as Thaana is not the predominant script

"٢".match(/\p{Script_Extensions=Thaana}/u);
// ["٢", index: 0, input: "٢", groups: undefined]
```

Unicode property escapes vs. character classes

With JavaScript regular expressions, it is also possible to use character classes and especially `\w` or `\d` to match letters or digits. However, such forms only match characters from the *Latin* script (in other words, ASCII letters, numbers, and the underscore character for `\w`, and ASCII numbers for `\d`). As shown in this example, character classes may be clumsy when used with non-Latin text.

Unicode property escape categories cover many more characters, and `\p{Letter}` and `\p{Number}` work for any script.

```js
// Trying to use ranges to avoid \w limitations:

const nonEnglishText = "Приключения Алисы в Стране чудес";
const regexpBMPWord = /([\u0000-\u001F\u0021-\uFFFF])+/gu;
// BMP goes through U+0000 to U+FFFF but space is U+0020

console.table(nonEnglishText.match(regexpBMPWord));

// Using Unicode property escapes instead
const regexpUPE = /\p{L}+/gu;
console.table(nonEnglishText.match(regexpUPE));
```

Matching prices

The following example matches prices in a string:

```js
function getPrices(str) {
  // Sc stands for "currency symbol"
  return [...str.matchAll(/\p{Sc}\s*[\d.,]+/gu)].map((match) => match[0]);
}

const str = `California rolls $6.99
Crunchy rolls $8.49
Shrimp tempura $10.99`;
console.log(getPrices(str)); // ["$6.99", "$8.49", "$10.99"]

const str2 = `US store $19.99
Europe store €18.99
Japan store ¥2000`;
console.log(getPrices(str2)); // ["$19.99", "€18.99", "¥2000"]
```

Matching strings

With the v flag, \p{...} can match strings that are potentially longer than one character by using a property of strings:

```js
const flag = "🇺🇳";
console.log(flag.length); // 4 (two code points, each a surrogate pair)
console.log([...flag].length); // 2
console.log(/\p{RGI_Emoji_Flag_Sequence}/v.exec(flag)); // [ '🇺🇳' ]
```

However, you can't use \P to match "a string that does not have a property", because it's unclear how many characters should be consumed.

js · ✗ bad example

```
/\P{RGI_Emoji_Flag_Sequence}/v; // SyntaxError: Invalid regular expression: Invalid property name
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-CharacterClassEscape

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Unicode character class escape: \p{...}, \P{...}	64	79	78	51	11.1	64	79	47	11.3	9.0	64	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.regular_expressions.unicode_character_class_escape. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Character classes guide
- Regular expressions
- Character class: [...], [^...]
- Character class escape: \d, \D, \w, \W, \s, \S
- Character escape: \n, \u{...}
- Disjunction: |
- Unicode character property on Wikipedia
- ES2018: RegExp Unicode property escapes by Dr. Axel Rauschmayer (2017)
- Unicode regular expressions § Properties
- Unicode Utilities: UnicodeSet
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

JavaScript › Reference › Regular expressions

Wildcard: .

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Wildcard · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

A **wildcard** matches all characters except line terminators. It also matches line terminators if the `s` flag is set.

Syntax

regex

```
.
```

Description

`.` matches any character except line terminators. If the `s` flag is set, `.` also matches line terminators.

The exact character set matched by `.` depends on whether the regex is Unicode-aware. If it is Unicode-aware, `.` matches any Unicode code point; otherwise, it matches any UTF-16 code unit. For example:

js

```
/../.test("😄"); // true; matches two UTF-16 code units as a surrogate pair
/../u.test("😄"); // false; input only has one Unicode character
```

Examples

Usage with quantifiers

Wildcards are often used with quantifiers to match any character sequence, until the next character of interest is found. For example, the following example extracts the title of a Markdown page in the form `# Title`:

js

```
function parseTitle(entry) {
  // Use multiline mode because the title may not be at the start of
  // the file. Note that the m flag does not make . match line
  // terminators, so the title must be on a single line
  // Return text matched by the first capturing group.
  return /^#[ \t]+(.+)$/m.exec(entry)?.[1];
}

parseTitle("# Hello world"); // "Hello world"
parseTitle("## Subsection"); // undefined
parseTitle(`
---
slug: Web/JavaScript/Reference/Regular_expressions/Wildcard
---

# Wildcard: .

A **wildcard** matches all characters except line terminators.
`); // "Wildcard: ."
```

Matching code block content

The following example matches the content of a code block enclosed by three backticks in Markdown. It uses the `s` flag to make `.` match line terminators, because the content of a code block may span multiple lines:

```js
function parseCodeBlock(entry) {
  return /^```.*?^(.+?)\n```/ms.exec(entry)?.[1];
}

parseCodeBlock(`
\`\`\`js
console.log("Hello world");
\`\`\`
`); // "console.log("Hello world");"

parseCodeBlock(`
A \`try...catch\` statement must have the blocks enclosed in curly braces.

\`\`\`js example-bad
try
  doSomething();
catch (e)
  console.log(e);
\`\`\`
`); // "try\n  doSomething();\ncatch (e)\n  console.log(e);"
```

> **Warning:** These examples are for demonstration only. If you want to parse Markdown, use a dedicated Markdown parser because there are many edge cases to consider.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Atom`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Wildcard: `.`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.wildcard`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`

JavaScript › Reference › Regular expressions

Word boundary assertion: \b, \B

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Regular_expressions/Word_boundary_assertion · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

A **word boundary assertion** checks if the current position in the string is a word boundary. A word boundary is where the next character is a word character and the previous character is not a word character, or vice versa.

Syntax

regex

```
\b
\B
```

Description

`\b` asserts that the current position in the string is a word boundary. `\B` negates the assertion: it asserts that the current position is not a word boundary. Both are *assertions*, so unlike other character escapes or character class escapes, `\b` and `\B` don't consume any characters.

A word character includes the following:

- Letters (A–Z, a–z), numbers (0–9), and underscore (_).
- If the regex is Unicode-aware and the `i` flag is set, other Unicode characters that get canonicalized to one of the characters above through case folding.

Word characters are also matched by the `\w` character class escape.

Out-of-bounds input positions are considered non-word characters. For example, the following are successful matches:

js

```
/\ba/.exec("abc");
/c\b/.exec("abc");

/\B /.exec(" abc");
/ \B/.exec("abc ");
```

Examples

Detecting words

The following example detects if a string contains the word "thanks" or "thank you":

```js
function hasThanks(str) {
  return /\b(thanks|thank you)\b/i.test(str);
}

hasThanks("Thanks! You helped me a lot."); // true
hasThanks("Just want to say thank you for all your work."); // true
hasThanks("Thanksgiving is around the corner."); // false
```

> **Warning:** Not all languages have clearly defined word boundaries. If you are working with languages like Chinese or Thai, where there are no whitespace separators, use a more advanced library like `Intl.Segmenter` to search for words instead.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#prod-Assertion`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Word boundary assertion: \b, \B	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.regular_expressions.word_boundary_assertion`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Character escape: `\n`, `\u{...}`

JavaScript › Reference

JavaScript error reference

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors · MDN page last modified 2025-07-08

Below, you'll find a list of errors which are thrown by JavaScript. These errors can be a helpful debugging aid, but the reported problem isn't always immediately clear. The pages below will provide additional details about these errors. Each error is an object based upon the `Error` object, and has a `name` and a `message`.

Errors displayed in the Web console may include a link to the corresponding page below to help you quickly comprehend the problem in your code.

For a beginner's introductory tutorial on fixing JavaScript errors, see What went wrong? Troubleshooting JavaScript.

List of errors

In this list, each page is listed by name (the type of error) and message (a more detailed human-readable error message). Together, these two properties provide a starting point toward understanding and resolving the error. For more information, follow the links below!

- AggregateError: No Promise in Promise.any was resolved
- Error: Permission denied to access property "x"
- InternalError: too much recursion
- RangeError: argument is not a valid code point
- RangeError: BigInt division by zero
- RangeError: BigInt negative exponent
- RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'
- RangeError: invalid array length
- RangeError: invalid date
- RangeError: precision is out of range
- RangeError: radix must be an integer
- RangeError: repeat count must be less than infinity
- RangeError: repeat count must be non-negative
- RangeError: x can't be converted to BigInt because it isn't an integer
- ReferenceError: "x" is not defined
- ReferenceError: assignment to undeclared variable "x"
- ReferenceError: can't access lexical declaration 'X' before initialization
- ReferenceError: must call super constructor before using 'this' in derived class constructor
- ReferenceError: super() called twice in derived class constructor
- SyntaxError: 'arguments'/'eval' can't be defined or assigned to in strict mode code
- SyntaxError: "0"-prefixed octal literals are deprecated
- SyntaxError: "use strict" not allowed in function with non-simple parameters
- SyntaxError: "x" is a reserved identifier
- SyntaxError: \ at end of pattern
- SyntaxError: a declaration in the head of a for-of loop can't have an initializer
- SyntaxError: applying the 'delete' operator to an unqualified name is deprecated

- SyntaxError: arguments is not valid in fields
- SyntaxError: await is only valid in async functions, async generators and modules
- SyntaxError: await/yield expression can't be used in parameter
- SyntaxError: cannot use ?? unparenthesized within || and && expressions
- SyntaxError: character class escape cannot be used in class range in regular expression
- SyntaxError: continue must be inside loop
- SyntaxError: duplicate capture group name in regular expression
- SyntaxError: duplicate formal argument x
- SyntaxError: for-in loop head declarations may not have initializers
- SyntaxError: function statement requires a name
- SyntaxError: functions cannot be labelled
- SyntaxError: getter and setter for private name #x should either be both static or non-static
- SyntaxError: getter functions must have no arguments
- SyntaxError: identifier starts immediately after numeric literal
- SyntaxError: illegal character
- SyntaxError: import declarations may only appear at top level of a module
- SyntaxError: incomplete quantifier in regular expression
- SyntaxError: invalid assignment left-hand side
- SyntaxError: invalid BigInt syntax
- SyntaxError: invalid capture group name in regular expression
- SyntaxError: invalid character in class in regular expression
- SyntaxError: invalid class set operation in regular expression
- SyntaxError: invalid decimal escape in regular expression
- SyntaxError: invalid identity escape in regular expression
- SyntaxError: invalid named capture reference in regular expression
- SyntaxError: invalid property name in regular expression
- SyntaxError: invalid range in character class
- SyntaxError: invalid regexp group
- SyntaxError: invalid regular expression flag "x"
- SyntaxError: invalid unicode escape in regular expression
- SyntaxError: JSON.parse: bad parsing
- SyntaxError: label not found
- SyntaxError: missing : after property id
- SyntaxError: missing) after argument list
- SyntaxError: missing) after condition
- SyntaxError: missing] after element list
- SyntaxError: missing } after function body
- SyntaxError: missing } after property list
- SyntaxError: missing = in const declaration
- SyntaxError: missing formal parameter
- SyntaxError: missing name after . operator
- SyntaxError: missing variable name

- SyntaxError: negated character class with strings in regular expression
- SyntaxError: new keyword cannot be used with an optional chain
- SyntaxError: nothing to repeat
- SyntaxError: numbers out of order in {} quantifier.
- SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code
- SyntaxError: parameter after rest parameter
- SyntaxError: private fields can't be deleted
- SyntaxError: property name __proto__ appears more than once in object literal
- SyntaxError: raw bracket is not allowed in regular expression with unicode flag
- SyntaxError: redeclaration of formal parameter "x"
- SyntaxError: reference to undeclared private field or method #x
- SyntaxError: rest parameter may not have a default
- SyntaxError: return not in function
- SyntaxError: setter functions must have one argument
- SyntaxError: string literal contains an unescaped line break
- SyntaxError: super() is only valid in derived class constructors
- SyntaxError: tagged template cannot be used with optional chain
- SyntaxError: Unexpected '#' used outside of class body
- SyntaxError: Unexpected token
- SyntaxError: unlabeled break must be inside loop or switch
- SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'
- SyntaxError: use of super property/member accesses only valid within methods or eval code within methods
- SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead
- TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed
- TypeError: 'x' is not iterable
- TypeError: "x" is (not) "y"
- TypeError: "x" is not a constructor
- TypeError: "x" is not a function
- TypeError: "x" is not a non-null object
- TypeError: "x" is read-only
- TypeError: already executing generator
- TypeError: BigInt value can't be serialized in JSON
- TypeError: calling a builtin X constructor without new is forbidden
- TypeError: can't access/set private field or method: object is not the right class
- TypeError: can't assign to property "x" on "y": not an object
- TypeError: can't convert BigInt to number
- TypeError: can't convert x to BigInt
- TypeError: can't define property "x": "obj" is not extensible
- TypeError: can't delete non-configurable array element
- TypeError: can't redefine non-configurable property "x"

- TypeError: can't set prototype of this object
- TypeError: can't set prototype: it would cause a prototype chain cycle
- TypeError: cannot use 'in' operator to search for 'x' in 'y'
- TypeError: class constructors must be invoked with 'new'
- TypeError: cyclic object value
- TypeError: derived class constructor returned invalid value x
- TypeError: getting private setter-only property
- TypeError: Initializing an object twice is an error with private fields/methods
- TypeError: invalid 'instanceof' operand 'x'
- TypeError: invalid Array.prototype.sort argument
- TypeError: invalid assignment to const "x"
- TypeError: Iterator/AsyncIterator constructor can't be used directly
- TypeError: matchAll/replaceAll must be called with a global RegExp
- TypeError: More arguments needed
- TypeError: null/undefined has no properties
- TypeError: property "x" is non-configurable and can't be deleted
- TypeError: Reduce of empty array with no initial value
- TypeError: setting getter-only property "x"
- TypeError: WeakSet key/WeakMap value 'x' must be an object or an unregistered symbol
- TypeError: X.prototype.y called on incompatible type
- URIError: malformed URI sequence
- Warning: -file- is being assigned a //# sourceMappingURL, but already has one
- Warning: unreachable code after return statement

AggregateError: No Promise in Promise.any was resolved

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Promise_any_all_rejected · MDN page last modified 2025-07-08

The JavaScript exception "No Promise in Promise.any was resolved" occurs when all promises passed to `Promise.any()` are rejected. It is the only built-in usage of `AggregateError`.

Message

```
AggregateError: All promises were rejected (V8-based)
AggregateError: No Promise in Promise.any was resolved (Firefox)
AggregateError (Safari)
```

Error type

`AggregateError`

What went wrong?

`Promise.any()` only rejects when all promises passed to it are rejected. You should access `errors` to get the array of rejection reasons. See Using promises for more information on how to handle

asynchronously rejected promises. This error is also raised when `Promise.any()` receives an empty iterable.

Examples

Empty iterable

```js
Promise.any([]).catch((error) => {
  console.error(error); // AggregateError: No Promise in Promise.any was resolved
});
```

Handling all rejections

```js
const promises = [
  fetch("/data-location1"),
  fetch("/data-location1"),
  fetch("/data-location1"),
];

Promise.any(promises)
  .then((value) => console.log(value))
  .catch((error) => {
    // None of the fetches were successful
    for (const e of error.errors) {
      console.error(e);
    }
  });

// Using await
async function fetchFirstSuccessful() {
  try {
    const value = await Promise.any(promises);
    console.log(value);
  } catch (error) {
    for (const e of error.errors) {
      console.error(e);
    }
  }
}
```

See also

- AggregateError
- Promise.any()

Error: Permission denied to access property "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Property_access_denied · MDN page last modified 2025-11-30

The JavaScript exception "Permission denied to access property" occurs when there was an attempt to access an object for which you have no permission.

Message

```
DOMException: Blocked a frame with origin "x" from accessing a cross-origin frame.
(Chromium-based)
DOMException: Permission denied to access property "x" on cross-origin object (Firefox)
SecurityError: Blocked a frame with origin "x" from accessing a cross-origin frame.
Protocols, domains, and ports must match. (Safari)
```

Error type

`DOMException`.

What went wrong?

There was attempt to access an object for which you have no permission. This is likely an `<iframe>` element loaded from a different domain for which you violated the same-origin policy.

Examples

No permission to access document

html

```
<iframe id="myframe" src="http://www1.w3c-test.org/common/blank.html"></iframe>
```

js

```
console.log(frames[0].document);
// Error: Permission denied to access property "document"
```

See also

- `<iframe>`
- Same-origin policy

JavaScript › Reference › JavaScript error reference

InternalError: too much recursion

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Too_much_recursion · MDN page last modified 2025-07-08

The JavaScript exception "too much recursion" or "Maximum call stack size exceeded" occurs when there are too many function calls, or a function is missing a base case.

Message

```
RangeError: Maximum call stack size exceeded (Chrome)
InternalError: too much recursion (Firefox)
RangeError: Maximum call stack size exceeded. (Safari)
```

Error type

`InternalError` in Firefox; `RangeError` in Chrome and Safari.

What went wrong?

A function that calls itself is called a *recursive function*. Once a condition is met, the function stops calling itself. This is called a *base case*.

In some ways, recursion is analogous to a loop. Both execute the same code multiple times, and both require a condition (to avoid an infinite loop, or rather, infinite recursion in this case). When there are too many function calls, or a function is missing a base case, JavaScript will throw this error.

Examples

This recursive function runs 10 times, as per the exit condition.

js

```
function loop(x) {
  if (x >= 10)
    // "x >= 10" is the exit condition
    return;
  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

Setting this condition to an extremely high value, won't work:

js · ✗ bad example

```
function loop(x) {
  if (x >= 1000000000000) return;
  // do stuff
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

This recursive function is missing a base case. As there is no exit condition, the function will call itself infinitely.

js · ✗ bad example

```
function loop(x) {
  // The base case is missing
  loop(x + 1); // Recursive call
}

loop(0);

// InternalError: too much recursion
```

Class error: too much recursion

js · ✗ bad example

```
class Person {
  constructor() {}
  set name(name) {
    this.name = name; // Recursive call
  }
}

const tony = new Person();
tony.name = "Tonisha"; // InternalError: too much recursion
```

When a value is assigned to the property name (this.name = name;) JavaScript needs to set that property. When this happens, the setter function is triggered.

In this example when the setter is triggered, it is told to do the same thing again: *to set the same property that it is meant to handle.* This causes the function to call itself, again and again, making it infinitely recursive.

This issue also appears if the same variable is used in the getter.

js · ✗ bad example

```
class Person {
  get name() {
    return this.name; // Recursive call
  }
}
```

To avoid this problem, make sure that the property being assigned to inside the setter function is different from the one that initially triggered the setter. The same goes for the getter.

js

```
class Person {
  constructor() {}
  set name(name) {
    this._name = name;
  }
  get name() {
    return this._name;
  }
}
const tony = new Person();
tony.name = "Tonisha";
console.log(tony);
```

See also

- Recursion
- Recursive functions

JavaScript › Reference › JavaScript error reference

RangeError: argument is not a valid code point

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Not_a_valid_code_point · MDN page last modified 2025-07-08

The JavaScript exception "Invalid code point" occurs when `NaN` values, negative Integers (-1), non-Integers (5.4), or values larger than 0x10FFFF (1114111) are used with `String.fromCodePoint()`.

Message

```
RangeError: Invalid code point -1 (V8-based)
RangeError: -1 is not a valid code point (Firefox)
RangeError: Arguments contain a value that is out of range of code points (Safari)
```

Error type

`RangeError`

What went wrong?

`String.fromCodePoint()` throws this error when passed `NaN` values, negative Integers (-1), non-Integers (5.4), or values larger than 0x10FFFF (1114111).

A code point is a value in the Unicode codespace; that is, the range of integers from `0` to `0x10FFFF`.

Examples

Invalid cases

js · ✗ bad example

```
String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

Valid cases

js · ✓ good example

```
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // 'Є' (U+0404)
String.fromCodePoint(0x2f804); // '你' (U+2F804)
String.fromCodePoint(194564); // '你'
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // '𝌆a𝌇'
```

See also

- `String.fromCodePoint()`

JavaScript › Reference › JavaScript error reference

RangeError: BigInt division by zero

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/BigInt_division_by_zero · MDN page last modified 2026-08-31

The JavaScript exception "BigInt division by zero" occurs when a `BigInt` is divided by `0n`.

Message

```
RangeError: Division by zero (V8-based)
RangeError: BigInt division by zero (Firefox)
RangeError: 0 is an invalid divisor value. (Safari)
```

Error type

`RangeError`.

What went wrong?

The divisor of a division or remainder operator is `0n`. In `Number` arithmetic, this produces `Infinity`, but there's no "infinity value" in BigInts, so an error is issued. Check if the divisor is `0n` before doing the division.

Examples

Division by 0n

js · ✗ bad example

```
const a = 1n;
const b = 0n;
const quotient = a / b;
// RangeError: BigInt division by zero
```

Instead, check if the divisor is `0n` first, and either issue an error with a better message, or fall back to a different value, like `Infinity` or `undefined`.

js · ✓ good example

```
const a = 1n;
const b = 0n;
const quotient = b === 0n ? undefined : a / b;
```

See also

- `BigInt`
- Division (`/`)
- Remainder (`%`)

JavaScript › Reference › JavaScript error reference

RangeError: BigInt negative exponent

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/BigInt_negative_exponent · MDN page last modified 2026-08-31

The JavaScript exception "BigInt negative exponent" occurs when a `BigInt` is raised to the power of a negative BigInt value.

Message

```
RangeError: Exponent must be positive (V8-based)
RangeError: BigInt negative exponent (Firefox)
RangeError: Negative exponent is not allowed (Safari)
```

Error type

`RangeError`.

What went wrong?

The exponent of an exponentiation operation must be positive. Since negative exponents would take the reciprocal of the base, the result will be between -1 and 1 in almost all cases, which gets rounded to `0n`. To catch mistakes, negative exponents are not allowed. Check if the exponent is non-negative before doing exponentiation.

Examples

Using a negative BigInt as exponent

js · ✗ bad example

```
const a = 1n;
const b = -1n;
const c = a ** b;
// RangeError: BigInt negative exponent
```

Instead, check if the exponent is negative first, and either issue an error with a better message, or fall back to a different value, like `0n` or `undefined`.

js · ✓ good example

```
const a = 1n;
const b = -1n;
const quotient = b >= 0n ? a ** b : 0n;
```

See also

- `BigInt`
- Exponentiation (**)

JavaScript › Reference › JavaScript error reference

RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Form_must_be_one_of · MDN page last modified 2025-07-08

The JavaScript exception "form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'" occurs when an unrecognized string is passed to the `String.prototype.normalize()` method.

Message

```
RangeError: The normalization form should be one of NFC, NFD, NFKC, NFKD. (V8-based)
RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD' (Firefox)
RangeError: argument does not match any normalization form (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.normalize()` method only accepts the following four values as its `form` argument: `"NFC"`, `"NFD"`, `"NFKC"`, or `"NFKD"`. If you pass any other value, an error will be thrown. Read the reference of `normalize()` to learn about different normalization forms.

Examples

Invalid cases

js · ✗ bad example

```
"foo".normalize("nfc"); // RangeError
"foo".normalize(" NFC "); // RangeError
```

Valid cases

js · ✓ good example

```
"foo".normalize("NFC"); // 'foo'
```

See also

- `String.prototype.normalize()`

JavaScript › Reference › JavaScript error reference

RangeError: invalid array length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_array_length · MDN page last modified 2026-08-21

The JavaScript exception "Invalid array length" occurs when specifying an array length that is either negative, a floating number or exceeds the maximum supported by the platform (i.e., when creating an `Array` or `ArrayBuffer`, or when setting the `length` property).

The maximum allowed array length depends on the platform, browser and browser version. For `Array` the maximum length is 2^{32}-1. For `ArrayBuffer` the maximum is 2^{31}-1 (2GiB-1) on 32-bit systems. From

Firefox version 89 the maximum value of ArrayBuffer is 2^{33} (8GiB) on 64-bit systems.

> **Note:** Array and ArrayBuffer are independent data structures (the implementation of one does not affect the other).

Message

```
RangeError: invalid array length (V8-based & Firefox)
RangeError: Array size is not a small enough positive integer. (Safari)

RangeError: Invalid array buffer length (V8-based)
RangeError: length too large (Safari)
```

Error type

RangeError

What went wrong?

The error might appear when attempting to produce an Array or ArrayBuffer with an invalid length, which includes:

- Negative length, via the constructor or setting the length property.
- Non-integer length, via the constructor or setting the length property. (The ArrayBuffer constructor coerces the length to an integer, but the Array constructor does not.)
- Exceeding the maximum length supported by the platform. For arrays, the maximum length is 2^{32}-1. For ArrayBuffer, the maximum length is 2^{31}-1 (2GiB-1) on 32-bit systems, or 2^{33} (8GiB) on 64-bit systems. This can happen via the constructor, setting the length property, or array methods that implicitly set the length property (such as push and concat).

If you are creating an Array using the constructor, you probably want to use the literal notation instead, as the first argument is interpreted as the length of the Array. Otherwise, you might want to clamp the length before setting the length property, or using it as an argument of the constructor.

Examples

Invalid cases

js · ✗ bad example

```
new Array(2 ** 40);
new Array(-1);
new ArrayBuffer(2 ** 32); // 32-bit system
new ArrayBuffer(-1);

const a = [];
a.length -= 1; // set the length property to -1

const b = new Array(2 ** 32 - 1);
b.length += 1; // set the length property to 2^32
b.length = 2.5; // set the length property to a floating-point number

const c = new Array(2.5); // pass a floating-point number
```

(code continues on the next page)

(code continued from the previous page)

```
// Concurrent modification that accidentally grows the array infinitely
const arr = [1, 2, 3];
for (const e of arr) {
  arr.push(e * 10);
}
```

Valid cases

js · ✓ good example

```
[2 ** 40]; // [ 1099511627776 ]
[-1]; // [ -1 ]
new ArrayBuffer(2 ** 31 - 1);
new ArrayBuffer(2 ** 33); // 64-bit systems after Firefox 89
new ArrayBuffer(0);

const a = [];
a.length = Math.max(0, a.length - 1);

const b = new Array(2 ** 32 - 1);
b.length = Math.min(0xffffffff, b.length + 1);
// 0xffffffff is the hexadecimal notation for 2^32 - 1
// which can also be written as (-1 >>> 0)

b.length = 3;

const c = new Array(3);

// Because array methods save the length before iterating, it is safe to grow
// the array during iteration
const arr = [1, 2, 3];
arr.forEach((e) => arr.push(e * 10));
```

See also

- Array
- length
- ArrayBuffer

JavaScript › Reference › JavaScript error reference

RangeError: invalid date

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_date · MDN page last modified 2026-09-02

The JavaScript exception "invalid date" occurs when an invalid date is attempted to be converted to an ISO date string.

Message

```
RangeError: Invalid time value (V8-based)
RangeError: invalid date (Firefox)
RangeError: Invalid Date (Safari)
```

Error type

RangeError

What went wrong?

You are converting an invalid date value to an ISO date string by calling the toISOString() method.

An *invalid date* is produced when you attempt to parse an invalid date string, or set the timestamp to an out-of-bounds value. Invalid dates usually cause all date methods to return NaN or other special values. However, such dates do not have valid ISO string representations, so an error is thrown when you attempt to do so.

> **Note:** toJSON() does not throw this error. It checks whether the date is finite before formatting it, and returns null for an invalid date instead of calling toISOString(). JSON.stringify(), which calls toJSON(), therefore serializes an invalid date as null rather than throwing.

Examples

Invalid cases

js · ✗ bad example

```
const invalid = new Date("nothing");
invalid.toISOString(); // RangeError: invalid date
```

However, most other methods return special values:

js · ✗ bad example

```
invalid.toString(); // "Invalid Date"
invalid.getDate(); // NaN
invalid.toJSON(); // null
JSON.stringify({ date: invalid }); // '{"date":null}'
```

For more details, see the Date.parse() documentation.

Valid cases

js · ✓ good example

```
new Date("05 October 2011 14:48 UTC").toISOString(); // "2011-10-05T14:48:00.000Z"
new Date(1317826080).toISOString(); // "2011-10-05T14:48:00.000Z"
```

See also

- Date
- Date.prototype.parse()
- Date.prototype.toISOString()

JavaScript › Reference › JavaScript error reference

RangeError: precision is out of range

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Precision_range · MDN page last modified 2025-07-08

The JavaScript exception "precision is out of range" occurs when a number that's outside of the allowed range was passed into `toExponential`, `toFixed`, or `toPrecision`.

Message

```
RangeError: toExponential() argument must be between 0 and 100 (V8-based & Safari)
RangeError: toFixed() digits argument must be between 0 and 100 (V8-based & Safari)
RangeError: toPrecision() argument must be between 1 and 100 (V8-based & Safari)
RangeError: precision -1 out of range (Firefox)
```

Error type

`RangeError`

What went wrong?

There was an out of range precision argument in one of these methods:

- `Number.prototype.toExponential()`, which requires the arguments to be between 0 and 100, inclusive.
- `Number.prototype.toFixed()`, which requires the arguments to be between 0 and 100, inclusive.
- `Number.prototype.toPrecision()`, which requires the arguments to be between 1 and 100, inclusive.

Examples

Invalid cases

js · ✗ bad example

```
(77.1234).toExponential(-1); // RangeError
(77.1234).toExponential(101); // RangeError

(2.34).toFixed(-100); // RangeError
(2.34).toFixed(1001); // RangeError

(1234.5).toPrecision(-1); // RangeError
(1234.5).toPrecision(101); // RangeError
```

Valid cases

js · ✓ good example

```
(77.1234).toExponential(4); // 7.7123e+1
(77.1234).toExponential(2); // 7.71e+1

(2.34).toFixed(1); // 2.3
(2.35).toFixed(1); // 2.4 (note that it rounds up in this case)

(5.123456).toPrecision(5); // 5.1235
(5.123456).toPrecision(2); // 5.1
(5.123456).toPrecision(1); // 5
```

See also

- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`

JavaScript › Reference › JavaScript error reference

RangeError: radix must be an integer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_radix · MDN page last modified 2025-07-08

The JavaScript exception "radix must be an integer at least 2 and no greater than 36" occurs when the optional `radix` parameter of the `Number.prototype.toString()` or the `BigInt.prototype.toString()` method was specified and is not between 2 and 36.

Message

```
RangeError: toString() radix argument must be between 2 and 36 (V8-based & Safari)
RangeError: radix must be an integer at least 2 and no greater than 36 (Firefox)
```

Error type

`RangeError`

What went wrong?

The optional `radix` parameter of the `Number.prototype.toString()` or the `BigInt.prototype.toString()` method was specified. Its value must be an integer (a number) between 2 and 36, specifying the base of the number system to be used for representing numeric values. For example, the decimal (base 10) number 169 is represented in hexadecimal (base 16) as A9.

Why is this parameter's value limited to 36? A radix that is larger than 10 uses alphabetical characters as digits; therefore, the radix can't be larger than 36, since the Latin alphabet (used by English and many other languages) only has 26 characters.

The most common radixes:

- 2 for binary numbers,
- 8 for octal numbers,
- 10 for decimal numbers,

- 16 for hexadecimal numbers.

Examples

Invalid cases

js · ✗ bad example

```
(42).toString(0);
(42).toString(1);
(42).toString(37);
(42).toString(150);
// You cannot use a string like this for formatting:
(12071989).toString("MM-dd-yyyy");
```

Valid cases

js · ✓ good example

```
(42).toString(2); // "101010" (binary)
(13).toString(8); // "15" (octal)
(0x42).toString(10); // "66" (decimal)
(100000).toString(16); // "186a0" (hexadecimal)
```

See also

- `Number.prototype.toString()`
- `BigInt.prototype.toString()`

JavaScript › Reference › JavaScript error reference

RangeError: repeat count must be less than infinity

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Resulting_string_too_large · MDN page last modified 2025-07-08

The JavaScript exception "repeat count must be less than infinity" occurs when the `String.prototype.repeat()` method is used with a `count` argument that is infinity.

Message

```
RangeError: Invalid string length (V8-based)
RangeError: Invalid count value: Infinity (V8-based)
RangeError: repeat count must be less than infinity and not overflow maximum string size
(Firefox)
RangeError: Out of memory (Safari)
RangeError: String.prototype.repeat argument must be greater than or equal to 0 and not be
Infinity (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.repeat()` method has been used. It has a `count` parameter indicating the number of times to repeat the string. It must be between 0 and less than positive `Infinity` and cannot be a negative number. The range of allowed values can be described like this: [0, +∞).

The resulting string can also not be larger than the maximum string size, which can differ in JavaScript engines. In Firefox (SpiderMonkey) the maximum string size is 2^{30} - 2 (~2GiB).

Examples

Invalid cases

js · ✗ bad example

```
"abc".repeat(Infinity); // RangeError
"a".repeat(2 ** 30); // RangeError
```

Valid cases

js · ✓ good example

```
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
```

See also

- `String.prototype.repeat()`

JavaScript › Reference › JavaScript error reference

RangeError: repeat count must be non-negative

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Negative_repetition_count · MDN page last modified 2025-07-08

The JavaScript exception "repeat count must be non-negative" occurs when the `String.prototype.repeat()` method is used with a `count` argument that is a negative number.

Message

```
RangeError: Invalid count value: -1 (V8-based)
RangeError: repeat count must be non-negative (Firefox)
RangeError: String.prototype.repeat argument must be greater than or equal to 0 and not be
Infinity (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.repeat()` method has been used. It has a `count` parameter indicating the number of times to repeat the string. It must be between 0 and less than positive `Infinity` and cannot be a negative number. The range of allowed values can be described like this: [0, +∞).

Examples

Invalid cases

js · ✗ bad example

```
"abc".repeat(-1); // RangeError
```

Valid cases

js · ✓ good example

```
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
```

See also

- `String.prototype.repeat()`

JavaScript › Reference › JavaScript error reference

RangeError: x can't be converted to BigInt because it isn't an integer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_be_converted_to_BigInt_because_it_isnt_an_integer · MDN page last modified 2025-07-08

The JavaScript exception "x can't be converted to BigInt because it isn't an integer" occurs when the `BigInt()` function is used on a number that isn't an integer.

Message

```
RangeError: The number 1.5 cannot be converted to a BigInt because it is not an integer (V8-based & Firefox)
RangeError: Not an integer (Safari)
```

Error type

`RangeError`.

What went wrong?

When using the `BigInt()` function to convert a number to a BigInt, the number must be an integer (such that `Number.isInteger` returns true).

Examples

Invalid cases

js · ✗ bad example

```
const a = BigInt(1.5);
// RangeError: The number 1.5 cannot be converted to a BigInt because it is not an integer
const b = BigInt(NaN);
// RangeError: NaN cannot be converted to a BigInt because it is not an integer
```

Valid cases

js · ✓ good example

```
const a = BigInt(1);
```

See also

- `BigInt()` constructor
- `Number.isInteger()`

ReferenceError: "x" is not defined

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Not_defined · MDN page last modified 2025-07-08

The JavaScript exception "*variable* is not defined" occurs when there is a non-existent variable referenced somewhere.

Message

```
ReferenceError: "x" is not defined (V8-based & Firefox)
ReferenceError: Can't find variable: x (Safari)
```

Error type

`ReferenceError`.

What went wrong?

There is a non-existent variable referenced somewhere. This variable needs to be declared, or you need to make sure it is available in your current script or scope.

> **Note:** When loading a library (such as jQuery), make sure it is loaded before you access library variables, such as "$". Put the `<script>` element that loads the library before your code that uses it.

Examples

Variable not declared

js · ✗ bad example

```
foo.substring(1); // ReferenceError: foo is not defined
```

The "foo" variable isn't defined anywhere. It needs to be some string, so that the `String.prototype.substring()` method will work.

js · ✓ good example

```
const foo = "bar";
foo.substring(1); // "ar"
```

Wrong scope

A variable needs to be available in the current context of execution. Variables defined inside a function cannot be accessed from anywhere outside the function, because the variable is defined only in the scope of the function

js · ✗ bad example

```
function numbers() {
  const num1 = 2;
  const num2 = 3;
  return num1 + num2;
}

console.log(num1); // ReferenceError num1 is not defined.
```

However, a function can access all variables and functions defined inside the scope in which it is defined. In other words, a function defined in the global scope can access all variables defined in the global scope.

js · ✓ good example

```
const num1 = 2;
const num2 = 3;

function numbers() {
  return num1 + num2;
}

console.log(numbers()); // 5
```

See also

- Scope
- Declaring variables in the JavaScript Guide
- Function scope in the JavaScript Guide

JavaScript › Reference › JavaScript error reference

ReferenceError: assignment to undeclared variable "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Undeclared_var · MDN page last modified 2025-12-18

The JavaScript strict mode-only exception "Assignment to undeclared variable" occurs when the value has been assigned to an undeclared variable.

Message

```
ReferenceError: x is not defined (V8-based)
ReferenceError: assignment to undeclared variable x (Firefox)
ReferenceError: Can't find variable: x (Safari)
```

Error type

`ReferenceError` in strict mode only.

What went wrong?

You have an assignment of the form `x = ...`, but `x` has not been previously declared with the `var`, `let`, or `const` keyword. This error occurs in strict mode code only. In non-strict code, assignment to an undeclared variable implicitly creates a property on the global scope.

Examples

Invalid cases

In this case, the variable "bar" is an undeclared variable.

js · ✗ bad example

```
function foo() {
  "use strict";
  bar = true;
}
foo(); // ReferenceError: assignment to undeclared variable bar
```

Valid cases

To make "bar" a declared variable, you can add a `let`, `const`, or `var` keyword in front of it.

js · ✓ good example

```
function foo() {
  "use strict";
  const bar = true;
}
foo();
```

See also

- Strict mode
- `var`
- `let`
- `const`

JavaScript › Reference › JavaScript error reference

ReferenceError: can't access lexical declaration 'X' before initialization

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_access_lexical_declaration_before_init · MDN page last modified 2026-11-05

The JavaScript exception "can't access lexical declaration 'X' before initialization" occurs when a lexical variable was accessed before it was initialized. This happens within any scope (global, module, function, or block) when `let` or `const` variables are accessed before the place where they are declared is executed.

Message

```
ReferenceError: Cannot access 'X' before initialization (V8-based)
ReferenceError: can't access lexical declaration 'X' before initialization (Firefox)
ReferenceError: Cannot access uninitialized variable. (Safari)
```

Error type

`ReferenceError`

What went wrong?

A lexical variable was accessed before it was initialized. This happens within any scope (global, module, function, or block) when variables declared with `let` or `const` are accessed before the place where they are declared has been executed.

Note that it is the execution order of access and variable declaration that matters, not the order in which the statements appear in the code. For more information, see the description of Temporal Dead Zone.

This issue does not occur for variables declared using `var`, because they are initialized with a default value of `undefined` when they are hoisted.

This error can also occur in cyclic imports when a module uses a variable that depends on the module itself being evaluated.

Examples

Invalid cases

In this case, the variable `foo` is accessed before it is declared. At this point foo has not been initialized with a value, so accessing the variable throws a reference error.

js · ✗ bad example

```
function test() {
  // Accessing the 'const' variable foo before it's declared
  console.log(foo); // ReferenceError: foo is not initialized
  const foo = 33; // 'foo' is declared and initialized here using the 'const' keyword
}

test();
```

In this example, the imported variable `a` is accessed but is uninitialized, because the evaluation of `a.js` is blocked by the evaluation of the current module `b.js`.

js · ✗ bad example

```
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

console.log(a); // ReferenceError: Cannot access 'a' before initialization
export const b = 1;
```

Valid cases

In the following example, we correctly declare a variable using the `const` keyword before accessing it.

js · ✓ good example

```
function test() {
  // Declaring variable foo
  const foo = 33;
  console.log(foo); // 33
}
test();
```

In this example, the imported variable `a` is asynchronously accessed, so both modules are evaluated before the access to `a` occurs.

js · ✓ good example

```
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);
export const b = 1;
```

See also

- let
- const
- var
- class

ReferenceError: must call super constructor before using 'this' in derived class constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Super_not_called · MDN page last modified 2026-05-22

The JavaScript exception "must call super constructor before using 'this' in derived class constructor" occurs when the `super()` is not called for a given derived class constructor, and the derived constructor tries to access the value of `this`, or the derived constructor has already returned and the return value is not an object.

Message

```
ReferenceError: Must call super constructor in derived class before accessing 'this' or
returning from derived constructor (V8-based)
ReferenceError: must call super constructor before using 'this' in derived class constructor
(Firefox)
ReferenceError: 'super()' must be called in derived constructor before accessing |this| or
returning non-object. (Safari)
```

Error type

`ReferenceError`

What went wrong?

The `super()` call can only be called at most once for each `new` call to a derived class constructor. Often, you need to call it exactly once, because if you don't call it, the parent constructor cannot initialize the value of `this`, so you cannot access `this` in the derived constructor and the `this` is not considered a valid constructed object (and throws if the derived constructor completes in this state). The way around it is to return an object from the derived class constructor, in which case the object returned will be used as the constructed object instead of `this`, allowing you to not call `super()`. This is rarely done though.

Examples

Invalid cases

js · ✗ bad example

```
class Base {
  constructor() {
    this.x = 1;
  }
}

class Derived extends Base {
  constructor() {
    console.log(this.x);
    // The Base constructor is not called yet, so this.x is undefined
    // ReferenceError: must call super constructor before using 'this' in derived class
constructor
  }
}
```

Valid cases

js · ✓ good example

```
class Base {
  constructor() {
    this.x = 1;
  }
}

class Derived extends Base {
  constructor() {
    super();
    console.log(this.x); // 1
  }
}
```

See also

- Classes
- `super`

JavaScript › Reference › JavaScript error reference

ReferenceError: super() called twice in derived class constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Super_called_twice · MDN page last modified 2025-07-08

The JavaScript exception "super() called twice in derived class constructor" occurs when the `super()` is called a second time for a given derived class constructor.

Message

```
ReferenceError: Super constructor may only be called once (V8-based)
ReferenceError: super() called twice in derived class constructor (Firefox)
ReferenceError: 'super()' can't be called more than once in a constructor. (Safari)
```

Error type

`ReferenceError`

What went wrong?

The `super()` call can only be called at most once for each `new` call to a derived class constructor. This is because `super()` is responsible for initializing the parent class, and calling it more than once would result in the parent constructor being called multiple times.

The best way to prevent this is to ensure that `super()` is placed outside of any control flow structure. Otherwise, make sure that all code paths in the constructor lead to only one `super()` call.

The `super()` call can be "saved" in an arrow function nested within the constructor. Then, when you call the arrow function, you will also call `super()`, and the same rule applies: the arrow function can only be called at most once.

Examples

Invalid cases

js · ✗ bad example

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    super();
  }
}
```

Sometimes the bug may be more subtle.

js · ✗ bad example

```
class Base {
  constructor(flavor) {
    // Do something with the flavor
  }
}

class Derived extends Base {
  constructor(flavors) {
    if (flavors.includes("chocolate")) {
      super("chocolate");
    }
    if (flavors.includes("vanilla")) {
      super("vanilla");
    }
  }
}
```

Originally, `flavors` may never simultaneously include both "chocolate" and "vanilla", but if that ever happens, the constructor will call `super()` twice. You need to rethink about how your class should be structured to avoid this issue.

Valid cases

js · ✓ good example

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    // More initialization logic
  }
}
```

See also

- Classes
- `super`

JavaScript › Reference › JavaScript error reference

SyntaxError: 'arguments'/'eval' can't be defined or assigned to in strict mode code

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_strict_arguments_eval · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "'arguments' can't be defined or assigned to in strict mode code" or "'eval' can't be defined or assigned to in strict mode code" occurs when attempting to create a binding called `arguments` or `eval`, or assign to such a name.

Message

```
SyntaxError: Unexpected eval or arguments in strict mode (V8-based)
SyntaxError: 'arguments' can't be defined or assigned to in strict mode code (Firefox)
SyntaxError: Cannot modify 'arguments' in strict mode. (Safari)
SyntaxError: Cannot destructure to a parameter name 'arguments' in strict mode. (Safari)
SyntaxError: Cannot declare a variable named arguments in strict mode. (Safari)
SyntaxError: Cannot declare a catch variable named 'arguments' in strict mode. (Safari)
SyntaxError: 'arguments' is not a valid function name in strict mode. (Safari)
```

Error type

`SyntaxError`

What went wrong?

In strict mode, the names `arguments` and `eval` behave as if they are reserved words: you cannot make they refer to anything other than the `arguments` object in functions or the global `eval` function.

Examples

Invalid cases

js · ✗ bad example

```
"use strict";

const arguments = [1, 2, 3];
console.log(Math.max(...arguments));

function foo(...arguments) {
  console.log(arguments);
}
```

Valid cases

js · ✓ good example

```
"use strict";

const args = [1, 2, 3];
console.log(Math.max(...args));

function foo(...args) {
  console.log(args);
}
```

See also

- Strict mode
- `arguments`
- `eval()`

JavaScript › Reference › JavaScript error reference

SyntaxError: "0"-prefixed octal literals are deprecated

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_octal_literal · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "0-prefixed octal literals are deprecated; use the "0o" prefix instead" occurs when deprecated octal literals (`0` followed by digits) are used.

Message

```
SyntaxError: Octal literals are not allowed in strict mode. (V8-based)
SyntaxError: Decimals with leading zeros are not allowed in strict mode. (V8-based)
SyntaxError: Unexpected number (V8-based)
SyntaxError: "0"-prefixed octal literals are deprecated; use the "0o" prefix instead
(Firefox)
SyntaxError: Decimal integer literals with a leading zero are forbidden in strict mode
(Safari)
```

Error type

`SyntaxError` in strict mode only.

What went wrong?

Octal literals are deprecated. When you prefix a decimal integer with `0`, you actually change it to an octal literal, which may be surprising. The standardized syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `0O`).

Leading zeros are always forbidden, even when the literal is not valid octal literal syntax (such as when the literal contains the digits `8` or `9`, or it has a decimal point). A number literal may only start with `0` if that 0 is its units place.

Examples

"0"-prefixed octal literals

js · ✗ bad example

```
"use strict";

03;

// SyntaxError: "0"-prefixed octal literals are deprecated; use the "0o" prefix instead
```

Valid octal numbers

Use a leading zero followed by the letter "o" or "O":

js · ✓ good example

```
0o3;
```

See also

- Lexical grammar

JavaScript › Reference › JavaScript error reference

SyntaxError: "use strict" not allowed in function with non-simple parameters

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Strict_non_simple_params · MDN page last modified 2025-07-08

The JavaScript exception ""use strict" not allowed in function" occurs when a "use strict" directive is used at the top of a function with default parameters, rest parameters, or destructuring parameters.

Message

```
SyntaxError: Illegal 'use strict' directive in function with non-simple parameter list (V8-
based)
SyntaxError: "use strict" not allowed in function with default parameter (Firefox)
SyntaxError: "use strict" not allowed in function with rest parameter (Firefox)
SyntaxError: "use strict" not allowed in function with destructuring parameter (Firefox)
SyntaxError: 'use strict' directive not allowed inside a function with a non-simple
parameter list. (Safari)
```

Error type

SyntaxError.

What went wrong?

A "use strict" directive is written at the top of a function that has one of the following parameters:

- Default parameters
- Rest parameters
- Destructuring parameters

A "use strict" directive is not allowed at the top of such functions per the ECMAScript specification.

Examples

Function statement

In this case, the function sum has default parameters a=1 and b=2:

js · ✗ bad example

```
function sum(a = 1, b = 2) {
  // SyntaxError: "use strict" not allowed in function with default parameter
  "use strict";
  return a + b;
}
```

If the function should be in strict mode, and the entire script or enclosing function is also okay to be in strict mode, you can move the "use strict" directive outside of the function:

js · ✓ good example

```
"use strict";
function sum(a = 1, b = 2) {
  return a + b;
}
```

Function expression

A function expression can use yet another workaround:

js · ✗ bad example

```
const sum = function sum([a, b]) {
  // SyntaxError: "use strict" not allowed in function with destructuring parameter
  "use strict";
  return a + b;
};
```

This can be converted to the following expression:

js · ✓ good example

```
const sum = (function () {
  "use strict";
  return function sum([a, b]) {
    return a + b;
  };
})();
```

Arrow function

If an arrow function needs to access the this variable, you can use the arrow function as the enclosing function:

js · ✗ bad example

```
const callback = (...args) => {
  // SyntaxError: "use strict" not allowed in function with rest parameter
  "use strict";
  return this.run(args);
};
```

This can be converted to the following expression:

js · ✓ good example

```
const callback = (() => {
  "use strict";
  return (...args) => this.run(args);
})();
```

See also

- Strict mode
- function statement
- function expression
- Default parameters
- Rest parameters
- Destructuring parameters

JavaScript › Reference › JavaScript error reference

SyntaxError: "x" is a reserved identifier

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Reserved_identifier · MDN page last modified 2025-07-08

The JavaScript exception "*variable* is a reserved identifier" occurs when reserved keywords are used as identifiers.

Message

```
SyntaxError: Unexpected reserved word (V8-based)
SyntaxError: implements is a reserved identifier (Firefox)
SyntaxError: Cannot use the reserved word 'implements' as a variable name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Reserved keywords will throw in if they are used as identifiers. These are reserved in strict mode and sloppy mode:

- `enum`

The following are only reserved when they are found in strict mode code:

- `implements`
- `interface`
- `let`
- `package`
- `private`
- `protected`
- `public`
- `static`

Examples

Strict and non-strict reserved keywords

The `enum` identifier is generally reserved.

js · ✗ bad example

```
const enum = { RED: 0, GREEN: 1, BLUE: 2 };
// SyntaxError: enum is a reserved identifier
```

In strict mode code, more identifiers are reserved.

js · ✗ bad example

```
"use strict";
const package = ["potatoes", "rice", "fries"];
// SyntaxError: package is a reserved identifier
```

You'll need to rename these variables.

js · ✓ good example

```
const colorEnum = { RED: 0, GREEN: 1, BLUE: 2 };
const list = ["potatoes", "rice", "fries"];
```

Update older browsers

If you are using an older browser that does not yet implement `let` or `class`, for example, you should update to a more recent browser version that does support these new language features.

js

```
"use strict";
class DocArchiver {}

// SyntaxError: class is a reserved identifier
// (throws in older browsers only, e.g. Firefox 44 and older)
```

See also

- Lexical grammar

JavaScript › Reference › JavaScript error reference

SyntaxError: \ at end of pattern

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_backslash_at_end_of_pattern · MDN page last modified 2025-07-08

The JavaScript exception "\ at end of pattern" occurs when a regular expression pattern ends with an unescaped backslash (\). In a regex literal, the backslash would cause the closing slash / to be a literal character, so this can only appear when using the `RegExp()` constructor.

Message

```
SyntaxError: Invalid regular expression: /\/: \ at end of pattern (V8-based)
SyntaxError: \ at end of pattern (Firefox)
SyntaxError: Invalid regular expression: \ at end of pattern (Safari)
```

Error type

SyntaxError

What went wrong?

A backslash cannot appear literally in a regular expression. It either precedes another character to escape it, or is itself escaped by another backslash. A backslash at the end of a regular expression pattern is invalid because it is not escaping anything.

Examples

Double-escaping backslashes

This error can only happen when using the `RegExp()` constructor. Consider the following code, which intends to match a single backslash character:

js · ✗ bad example

```
const pattern = new RegExp("\\");
```

In JavaScript strings, backslashes are also escape sequences. Therefore, the two backslashes in the string literal `"\\"` are interpreted as a single backslash. The `RegExp()` constructor then only sees a single backslash character in the regex source. To fix this, you need to double-escape the backslash:

js · ✓ good example

```
const pattern = new RegExp("\\\\");
```

The four backslashes in the string literal represent two backslashes in the regex source, which then becomes a character escape for a single backslash literal character.

See also

- Regular expressions
- Escape sequences

SyntaxError: a declaration in the head of a for-of loop can't have an initializer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_for-of_initializer · MDN page last modified 2025-07-08

The JavaScript exception "a declaration in the head of a for-of loop can't have an initializer" occurs when the head of a for...of loop contains an initializer expression such as `for (const i = 0 of iterable)`. This is not allowed in for-of loops.

Message

```
SyntaxError: for-of loop variable declaration may not have an initializer. (V8-based)
SyntaxError: a declaration in the head of a for-of loop can't have an initializer (Firefox)
SyntaxError: Cannot assign to the loop variable inside a for-of loop header. (Safari)
```

Error type

SyntaxError

What went wrong?

The head of a for...of loop contains an initializer expression. That is, a variable is declared and assigned a value `for (const i = 0 of iterable)`. This is not allowed in for-of loops. You might want a `for` loop that does allow an initializer.

Examples

Invalid for-of loop

js · ✗ bad example

```
const iterable = [10, 20, 30];

for (const value = 50 of iterable) {
  console.log(value);
}

// SyntaxError: a declaration in the head of a for-of loop can't
// have an initializer
```

Valid for-of loop

You need to remove the initializer (`value = 50`) in the head of the `for-of` loop. Maybe you intended to make 50 an offset value, in that case you could add it to the loop body, for example.

js · ✓ good example

```
const iterable = [10, 20, 30];

for (let value of iterable) {
  value += 50;
  console.log(value);
}
// 60
// 70
// 80
```

See also

- for...of
- for...in
- for

JavaScript › Reference › JavaScript error reference

SyntaxError: applying the 'delete' operator to an unqualified name is deprecated

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Delete_in_strict_mode · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "applying the 'delete' operator to an unqualified name is deprecated" occurs when variables are attempted to be deleted using the `delete` operator.

Message

```
SyntaxError: Delete of an unqualified identifier in strict mode. (V8-based)
SyntaxError: applying the 'delete' operator to an unqualified name is deprecated (Firefox)
SyntaxError: Cannot delete unqualified property 'a' in strict mode. (Safari)
```

Error type

`SyntaxError` in strict mode only.

What went wrong?

Normal variables in JavaScript can't be deleted using the `delete` operator. In strict mode, an attempt to delete a variable will throw an error and is not allowed.

The `delete` operator can only delete properties on an object. Object properties are "qualified" if they are configurable.

Unlike what common belief suggests, the `delete` operator has **nothing** to do with directly freeing memory. Memory management is done indirectly via breaking references, see the memory management page and the `delete` operator page for more details.

This error only happens in strict mode code. In non-strict code, the operation just returns `false`.

Examples

Freeing the contents of a variable

Attempting to delete a plain variable throws an error in strict mode:

js · ✗ bad example

```
"use strict";

var x;

// …

delete x;

// SyntaxError: applying the 'delete' operator to an unqualified name
// is deprecated
```

To free the contents of a variable, you can set it to `null`:

js · ✓ good example

```
"use strict";

var x;

// …

x = null;

// x can be garbage collected
```

See also

- delete
- Memory management
- TypeError: property "x" is non-configurable and can't be deleted

---

JavaScript › Reference › JavaScript error reference

SyntaxError: arguments is not valid in fields

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Arguments_not_allowed · MDN page last modified 2025-07-08

The JavaScript exception "SyntaxError: arguments is not valid in fields" occurs when the arguments identifier is read in a class field initializer or in a static initialization block, outside of a non-arrow function.

Message

```
SyntaxError: 'arguments' is not allowed in class field initializer or static initialization
block (V8-based)
SyntaxError: arguments is not valid in fields (Firefox)
SyntaxError: Unexpected identifier 'arguments'. Cannot reference 'arguments' in class field
initializer. (Safari)
```

Error type

SyntaxError

What went wrong?

A class field initializer expression or a class static initialization block does not have `arguments` in its scope. Trying to access it is a syntax error.

- This is true even if `arguments` is bound in a parent scope (such as when the class is nested in a non-arrow function).
- A non-arrow function declared within this scope will still bind its own `arguments` and read it normally.

Examples

js · ✗ bad example

```
function makeOne() {
  class C {
    args = { ...arguments }; // SyntaxError: arguments is not valid in fields
  }
  return new C();
}
```

js · ✗ bad example

```
let CArgs;
class C {
  static {
    CArgs = arguments; // SyntaxError: arguments is not valid in fields
  }
}
```

js · ✓ good example

```
class C {
  args = {};
  constructor() {
    this.args = arguments; // You can use arguments in constructors
  }
  myMethod() {
    this.args = arguments; // You can also use it in methods
  }
}
```

js · ✓ good example

```
function makeOne() {
  const _arguments = arguments;
  class C {
    args = { ..._arguments }; // Only the identifier is forbidden
  }
  return new C();
}
```

See also

- Classes
- Public class fields
- Static initialization blocks

JavaScript › Reference › JavaScript error reference

SyntaxError: await is only valid in async functions, async generators and modules

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_await · MDN page last modified 2026-11-05

The JavaScript exception "await is only valid in async functions, async generators and modules" occurs when an `await` expression is used outside of async functions or modules or other async contexts.

Message

```
SyntaxError: await is only valid in async functions and the top level bodies of modules (V8-based)
SyntaxError: await is only valid in async functions, async generators and modules (Firefox)
SyntaxError: Unexpected identifier (Safari)
```

Error type

SyntaxError.

What went wrong?

JavaScript execution is never blocking: an `await` can never block the execution of the program. Instead, it pauses the execution of the surrounding async task, while allowing other tasks to continue running. Therefore, `await` cannot be used in sync tasks, such as functions, generator functions, or top level of scripts. It is not always apparent whether the current file is a script or a module — see the Modules guide for more information.

Examples

Top-level await

You cannot use `await` at the top level of a script:

html · ✗ bad example

```html
<script>
  await fetch("https://example.com");
  // SyntaxError: await is only valid in async functions, async generators and modules
</script>
```

Instead, make the script a module:

html · ✓ good example

```html
<script type="module">
  await fetch("https://example.com");
</script>
```

Async callbacks

You cannot use `await` in a sync callback:

js · ✗ bad example

```js
urls.forEach((url) => {
  await fetch(url);
  // SyntaxError: await is only valid in async functions, async generators and modules
});
```

Instead, make the callback async. See more explanation in the Using promises guide.

js · ✓ good example

```
Promise.all(
  urls.map(async (url) => {
    await fetch(url);
  }),
);
```

See also

- await

JavaScript › Reference › JavaScript error reference

SyntaxError: await/yield expression can't be used in parameter

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/await_yield_in_parameter · MDN page last modified 2025-07-08

The JavaScript exception "await expression can't be used in parameter" or "yield expression can't be used in parameter" occurs when the default parameter expression contains the `await` or `yield` keyword and has the effect of pausing default parameter evaluation.

Message

```
SyntaxError: Illegal await-expression in formal parameters of async function (V8-based)
SyntaxError: await expression can't be used in parameter (Firefox)
SyntaxError: Cannot use 'await' within a parameter default expression. (Safari)

SyntaxError: Yield expression not allowed in formal parameter (V8-based)
SyntaxError: yield expression can't be used in parameter (Firefox)
SyntaxError: Unexpected keyword 'yield'. Cannot use yield expression within parameters.
(Safari)
```

Error type

SyntaxError

What went wrong?

The default expression must be able to evaluate *synchronously*. If it contains an `await` or `yield` expression, it will pause the evaluation of the default expression, which is not allowed.

> **Note:** This error is only generated when `await` or `yield` are valid operators in this function context. Otherwise, `await` or `yield` would be parsed as an identifier, and either not cause an error, or cause an error like "reserved identifier", or "unexpected token" if there's an expression following it.

Examples

Invalid cases

js · ✗ bad example

```
function *gen(a = yield 1) {}

async function f(a = await Promise.resolve(1)) {}
```

Valid cases

You can use the nullish coalescing assignment to provide a default value instead. If you want to treat `null` and `undefined` differently, you would need to use a condition.

js · ✓ good example

```
function* gen(a) {
  a ??= yield 1;
}

async function f(a) {
  a ??= await Promise.resolve(1);
}
```

You are also allowed to use `await` or `yield` if the expression is contained in a function expression of the initializer and would not pause the evaluation of the default expression.

js · ✓ good example

```
async function f(a = (async () => await Promise.resolve(1))()) {}
```

See also

- Default parameters
- `await`
- `yield`

SyntaxError: cannot use ?? unparenthesized within || and && expressions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_use_nullish_coalescing_unparenthesized · MDN page last modified 2026-03-20

The JavaScript exception "cannot use `??` unparenthesized within `||` and `&&` expressions" occurs when a nullish coalescing operator is used with a logical OR or logical AND in the same expression without parentheses.

Message

```
SyntaxError: Unexpected token '??' (V8-based)
SyntaxError: cannot use `??` unparenthesized within `||` and `&&` expressions (Firefox)
SyntaxError: Unexpected token '??'. Coalescing and logical operators used together in the
same expression; parentheses must be used to disambiguate. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The operator precedence chain looks like this:

```
|   >   &&   >   ||   >   =
|   >   ??   >   =
```

However, the precedence *between* `??` and `&&`/`||` is intentionally undefined, because the short circuiting behavior of logical operators can make the expression's evaluation counter-intuitive. Therefore, the following combinations are all syntax errors, because the language doesn't know how to parenthesize the operands:

js · ✗ bad example

```
a ?? b || c;
a || b ?? c;
a ?? b && c;
a && b ?? c;
```

Instead, make your intent clear by parenthesizing either side explicitly:

js · ✓ good example

```
(a ?? b) || c;
a ?? (b && c);
```

Examples

When migrating legacy code that uses `||` and `&&` for guarding against `null` or `undefined`, you may often convert it partially:

js · ✗ bad example

```
function getId(user, fallback) {
  // Previously: user && user.id || fallback
  return user && user.id ?? fallback; // SyntaxError: cannot use `??` unparenthesized within
`||` and `&&` expressions
}
```

Instead, consider parenthesizing the `&&`:

js

```
function getId(user, fallback) {
  return (user && user.id) ?? fallback;
}
```

Even better, consider using optional chaining instead of `&&`:

js · ✓ good example

```
function getId(user, fallback) {
  return user?.id ?? fallback;
}
```

See also

- Issue about nullish coalescing precedence in the TC39 nullish-coalescing proposal
- Nullish coalescing operator (`??`)
- Operator precedence

SyntaxError: character class escape cannot be used in class range in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_character_class_escape_in_class_range · MDN page last modified 2026-09-01

The JavaScript exception "character class escape cannot be used in class range in regular expression" occurs when a Unicode-aware regular expression pattern contains a character class where a boundary of a character range is another character class, such as a character class escape.

Message

```
SyntaxError: Invalid regular expression: /[\s-1]/u: Invalid character class (V8-based)
SyntaxError: character class escape cannot be used in class range in regular expression
(Firefox)
SyntaxError: Invalid regular expression: invalid range in character class for Unicode
pattern (Safari)
```

Error type

SyntaxError

What went wrong?

A character class can specify a range of characters by using a hyphen (`-`) between two characters. For example, `[a-z]` matches any lowercase letter from `a` to `z`. The two bounds of the range must represent single characters in order for the range to make sense. If one of the bounds actually represents multiple characters, an error is generated. In non-v-mode character classes, only character class escapes are allowed inside character classes; in v-mode character classes, this can also happen if one the bounds is another `[...]` character class.

In Unicode-unaware mode, this syntax causes the `-` to become a literal character instead of generating an error, but this is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

js · ✗ bad example

```
/[\s-_]/u; // \s is a character class escape for whitespace
/[A-\D]/u; // \D is a character class escape for non-digits
/[\p{L}-\p{N}]/u; // \p{L} is a character class escape for Unicode letters
/[[A-z]-_]/v; // In unicodeSets mode, character classes can be nested
```

Valid cases

js · ✓ good example

```
// Put the hyphen at the start of the character class,
// so it matches the literal character
/[-\s_]/u;
// Escape the hyphen so it also matches the literal character
/[\s\-_]/u;
// Remove the backslash so the bound is a literal character
/[A-D]/u;
// Remove the hyphen so the two bounds represent two alternatives
/[\p{L}\p{N}]/u;
// Use -- in unicodeSets mode, which represents set subtraction
/[[A-z]--_]/v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

JavaScript › Reference › JavaScript error reference

SyntaxError: continue must be inside loop

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_continue · MDN page last modified 2025-07-08

The JavaScript exception "continue must be inside loop" occurs when a `continue` statement is not inside a loop statement.

Message

```
SyntaxError: Illegal continue statement: no surrounding iteration statement (V8-based)
SyntaxError: Illegal continue statement: 'label' does not denote an iteration statement (V8-based)
SyntaxError: continue must be inside loop (Firefox)
SyntaxError: 'continue' is only valid inside a loop statement. (Safari)
SyntaxError: Cannot continue to the label 'label' as it is not targeting a loop. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

`continue` statements can be used to continue a loop, and using them elsewhere is a syntax error. Alternatively, you can provide a label to the `continue` statement to continue any loop with that label — however, if the label does not reference a containing statement, another error SyntaxError: label not found will be thrown, and if the label references a statement that is not a loop, a syntax error is still thrown.

Examples

Using continue in callbacks

If you want to proceed with the next iteration in a `forEach()` loop, use `return` instead, or convert it to a `for...of` loop.

js · ✗ bad example

```
array.forEach((value) => {
  if (value === 5) {
    continue; // SyntaxError: continue must be inside loop
  }
  // do something with value
});
```

js · ✓ good example

```
array.forEach((value) => {
  if (value === 5) {
    return;
  }
  // do something with value
});
```

js · ✓ good example

```
for (const value of array) {
  if (value === 5) {
    continue;
  }
  // do something with value
}
```

See also

- `continue`

SyntaxError: duplicate capture group name in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_duplicate_capture_group_name · MDN page last modified 2025-07-08

The JavaScript exception "duplicate capture group name in regular expression" occurs when a regular expression pattern contains two or more named capturing groups with the same name, and these capture groups could be matched at the same time.

Message

```
SyntaxError: Invalid regular expression: /(?<a>)(?<a>)/: Duplicate capture group name (V8-based)
SyntaxError: duplicate capture group name in regular expression (Firefox)
SyntaxError: Invalid regular expression: duplicate group specifier name (Safari)
```

Error type

SyntaxError

What went wrong?

All named capturing groups in a regular expression pattern must have unique names. A more recent feature allows named capturing groups to share names, as long as they belong to different disjunction alternatives and cannot be matched at the same time (see browser compatibility for this). However, it is still an error if the named capturing groups with the same name could be matched at the same time, as that would make other features, such as named backreferences, ambiguous.

Examples

Invalid cases

js · ✗ bad example

```
/(?<name>\w+) (?<name>\w+)/;
```

Valid cases

js · ✓ good example

```
/(?<firstName>\w+) (?<lastName>\w+)/;
/(?<year>\d{4})-\d{2}|\d{2}-(?<year>\d{4})/;
```

See also

- Regular expressions
- Named capturing group: (?<name>...)

JavaScript › Reference › JavaScript error reference

SyntaxError: duplicate formal argument x

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Duplicate_parameter · MDN page last modified 2025-07-08

The JavaScript exception "duplicate formal argument x" or "duplicate argument names not allowed in this context" occurs when a function creates two or more parameter bindings with the same name, and the function is not a non-strict function with only simple parameters.

Message

```
SyntaxError: Duplicate parameter name not allowed in this context (V8-based)
SyntaxError: duplicate formal argument x (Firefox)
SyntaxError: duplicate argument names not allowed in this context (Firefox)
SyntaxError: Cannot declare a parameter named 'x' in strict mode as it has already been
declared. (Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with default parameter values.
(Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with a rest parameter. (Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with destructuring parameters.
(Safari)
```

Error type

SyntaxError

What went wrong?

Having two formal parameters of the same name is likely a mistake—the second occurrence would cause the first occurrence to be inaccessible through the parameter name. In legacy JavaScript, this was allowed. Therefore, to not break existing code, this is only an error if the code is guaranteed to not be legacy—either because it is in strict mode or it uses modern parameter syntax (rest, default, or destructured parameters).

Examples

Invalid cases

js · ✗ bad example

```
"use strict";

function add(x, x) {
  // How can you access both "x" parameters?
  // SyntaxError: duplicate formal argument x
}
```

js · ✗ bad example

```
function doSomething(name, { name }) {
  // How can you access both "name" parameters?
  // SyntaxError: duplicate argument names not allowed in this context
}
```

Valid cases

js · ✓ good example

```
function doSomething(operationName, { name: userName }) {
  // You can access both "operationName" and "userName" parameters.
}

function doSomething(name, user) {
  // You can access both "name" and "user.name" parameters.
}
```

See also

- Functions
- Strict mode

JavaScript › Reference › JavaScript error reference

SyntaxError: for-in loop head declarations may not have initializers

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_for-in_initializer · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "for-in loop head declarations may not have initializers" occurs when the head of a for...in contains an initializer expression, such as `for (var i = 0 in obj)`. This is not allowed in for-in loops in strict mode. In addition, lexical declarations with initializers like `for (const i = 0 in obj)` are not allowed outside strict mode either.

Message

```
SyntaxError: for-in loop variable declaration may not have an initializer. (V8-based)
SyntaxError: for-in loop head declarations may not have initializers (Firefox)
SyntaxError: a lexical declaration in the head of a for-in loop can't have an initializer
(Firefox)
SyntaxError: Cannot assign to the loop variable inside a for-in loop header. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

The head of a for...in loop contains an initializer expression. That is, a variable is declared and assigned a value `for (var i = 0 in obj)`. In non-strict mode, this head declaration is silently ignored and behaves like `for (var i in obj)`. In strict mode, however, a `SyntaxError` is thrown. In addition, lexical declarations with initializers like `for (const i = 0 in obj)` are not allowed outside strict mode either, and will always produce a `SyntaxError`.

Examples

This example throws a `SyntaxError`:

js · ✗ bad example

```
const obj = { a: 1, b: 2, c: 3 };

for (const i = 0 in obj) {
  console.log(obj[i]);
}

// SyntaxError: for-in loop head declarations may not have initializers
```

Valid for-in loop

You can remove the initializer (`i = 0`) in the head of the for-in loop.

js · ✓ good example

```
const obj = { a: 1, b: 2, c: 3 };

for (const i in obj) {
  console.log(obj[i]);
}
```

Array iteration

The for...in loop shouldn't be used for Array iteration. Did you intend to use a `for` loop instead of a `for-in` loop to iterate an `Array`? The `for` loop allows you to set an initializer then as well:

js · ✓ good example

```
const arr = ["a", "b", "c"];

for (let i = 2; i < arr.length; i++) {
  console.log(arr[i]);
}

// "c"
```

See also

- `for...in`
- `for...of`
- `for`

JavaScript › Reference › JavaScript error reference

SyntaxError: function statement requires a name

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Unnamed_function_statement · MDN page last modified 2025-07-08

The JavaScript exception "function statement requires a name" occurs when there is a function statement in the code that requires a name.

Message

```
SyntaxError: Function statements require a function name (V8-based)
SyntaxError: function statement requires a name (Firefox)
SyntaxError: Function statements must have a name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is a function statement in the code that requires a name. You'll need to check how functions are defined and if you need to provide a name for it, or if the function in question needs to be a function expression, an IIFE, or if the function code is placed correctly in this context at all.

Examples

Statements vs. expressions

A *function statement* (or *function declaration*) requires a name. This won't work:

js · ✗ bad example

```
function () {
  return "Hello world";
}
// SyntaxError: function statement requires a name
```

You can use a function expression (assignment) instead:

js · ✓ good example

```
const greet = function () {
  return "Hello world";
};
```

If your function is intended to be an IIFE (Immediately Invoked Function Expression, which is a function that runs as soon as it is defined) you will need to add a few more braces:

js · ✓ good example

```
(function () {
  // …
})();
```

Labeled functions

Labels are an entirely different feature from function names. You can't use a label as a function name.

js · ✗ bad example

```
function Greeter() {
  german: function () {
    return "Moin";
  }
}
// SyntaxError: function statement requires a name
```

In addition, labeled function declarations themselves are a deprecated feature. Use regular function declarations instead.

js · ✓ good example

```
function Greeter() {
  function german() {
    return "Moin";
  }
}
```

Object methods

If you intended to create a method of an object, you will need to create an object. The following syntax without a name after the `function` keyword is valid then.

js · ✓ good example

```
const greeter = {
  german: function () {
    return "Moin";
  },
};
```

You can also use the method syntax.

js

```
const greeter = {
  german() {
    return "Moin";
  },
};
```

Callback syntax

Also, check your syntax when using callbacks. Braces and commas can quickly get confusing.

js · ✗ bad example

```
promise.then(
  function () {
    console.log("success");
  });
  function () {
    console.log("error");
}
// SyntaxError: function statement requires a name
```

Correct would be:

js · ✓ good example

```
promise.then(
  function () {
    console.log("success");
  },
  function () {
    console.log("error");
  },
);
```

See also

- Functions guide
- `function`
- `function` expression
- IIFE
- Labeled statement

JavaScript › Reference › JavaScript error reference

SyntaxError: functions cannot be labelled

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Function_label · MDN page last modified 2026-08-27

The JavaScript exception "functions cannot be labelled" occurs when a `function` declaration has a label before it.

Message

```
SyntaxError: In non-strict mode code, functions can only be declared at top level, inside a
block, or as the body of an if statement. (V8-based)
SyntaxError: In strict mode code, functions can only be declared at top level or inside a
block. (V8-based)
```

(code continues on the next page)

(code continued from the previous page)

```
SyntaxError: Generators can only be declared at the top level or inside a block. (V8-based)
SyntaxError: Async functions can only be declared at the top level or inside a block. (V8-
based)

SyntaxError: functions can only be labelled inside blocks (Firefox)
SyntaxError: functions cannot be labelled (Firefox)
SyntaxError: generator functions cannot be labelled (Firefox)
SyntaxError: async function declarations can't appear in single-statement context (Firefox)

SyntaxError: Unexpected keyword 'function'. Function declarations are only allowed inside
block statements or at the top level of a program. (Safari)
SyntaxError: Function declarations are only allowed inside blocks or switch statements in
strict mode. (Safari)
SyntaxError: Unexpected token '*'. Cannot use generator function declaration in single-
statement context. (Safari)
SyntaxError: Unexpected keyword 'function'. Cannot use async function declaration in single-
statement context. (Safari)
```

Error type

SyntaxError

What went wrong?

Function declarations are never supposed to be labeled, because labels should only apply to statements, not declarations. There's no way to actually jump to this label. However, due to some legacy JavaScript syntax rules, the error condition is a bit more complicated than necessary:

- In strict mode, function declarations are never allowed to be labeled.
- In non-strict mode, function declarations are allowed to be labeled, but not when the function is the sole statement of an `if` statement (which itself is a deprecated feature).
- Async functions, generator functions, and async generator functions are never allowed to be labeled.

The error message may say something along the lines of "invalid place for a function declaration to appear", because when the parser sees a label, it expects a statement to follow, and a function declaration is not a statement. It depends on whether the error's perspective is that a label cannot be followed by a function, or that a function cannot be preceded by a label.

Examples

Misparsed object literal

While it's possible that you actually expect the label to do *something* along the lines of being a jump target, usually you didn't intend for it to be a label. The most common case is you actually want it to be a property key in an object literal:

js · ✗ bad example

```
const createObj = () => {
  greet: function greet() { // SyntaxError: functions cannot be labelled
    console.log("Hello");
  }
};
```

Here, `{...}` is actually not an object literal, but is instead the block body of the arrow function, so `greet:` becomes a label. To fix this, you need to wrap the object literal in parentheses:

js · ✓ good example

```
const createObj = () => ({
  greet: function greet() {
    console.log("Hello");
  },
});
```

You may also want to use the method syntax for object literals, which avoids this pitfall:

js · ✓ good example

```
const createObj = () => ({
  greet() {
    console.log("Hello");
  },
});
```

See also

- Labeled statement
- `function`
- Strict mode
- Deprecated and obsolete features

JavaScript › Reference › JavaScript error reference

SyntaxError: getter and setter for private name #x should either be both static or non-static

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Either_be_both_static_or_non-static · MDN page last modified 2025-07-08

The JavaScript exception "mismatched placement" occurs when a private getter and setter are mismatched in whether or not they are `static`.

Message

```
SyntaxError: Identifier '#x' has already been declared (V8-based)
SyntaxError: getter and setter for private name #x should either be both static or non-
static (Firefox)
SyntaxError: Cannot declare a private non-static getter if there is a static private setter
with used name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Private getters and setters for the same name must either be both `static`, or both non-static. This limitation does not exist for public methods.

Examples

Mismatched placement

js · ✗ bad example

```
class Test {
  static set #foo(_) {}
  get #foo() {}
}

// SyntaxError: getter and setter for private name #foo should either be both static or non-
static
```

Since `foo` is private, the methods must be either both `static`:

js · ✓ good example

```
class Test {
  static set #foo(_) {}
  static get #foo() {}
}
```

or non-static:

js · ✓ good example

```
class Test {
  set #foo(_) {}
  get #foo() {}
}
```

See also

- get
- set
- static
- Private elements

JavaScript › Reference › JavaScript error reference

SyntaxError: getter functions must have no arguments

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Getter_no_arguments · MDN page last modified 2025-07-08

The JavaScript exception "getter functions must have no arguments" occurs when a getter is declared and the parameter list is non-empty.

Message

```
SyntaxError: Getter must not have any formal parameters. (V8-based)
SyntaxError: getter functions must have no arguments (Firefox)
SyntaxError: Unexpected identifier 'x'. getter functions must have no parameters. (Safari)
```

Error type

SyntaxError

What went wrong?

The get property syntax looks like a function, but it is stricter and not all function syntax is allowed. A getter is always invoked with no arguments, so defining it with any parameter is likely an error.

Note that this error only applies to property getters using the get syntax. If you define the getter using Object.defineProperty(), etc., the getter is defined as a normal function, although it's likely still an error if the getter expects any arguments, as it will be called without any.

Examples

Invalid cases

js · ✗ bad example

```
const obj = {
  get value(type) {
    return type === "string" ? String(Math.random()) : Math.random();
  },
};
```

Valid cases

js · ✓ good example

```
// Remove the parameter
const obj = {
  get value() {
    return Math.random();
  },
};

// Use a normal method, if you need a parameter
const obj = {
  getValue(type) {
    return type === "string" ? String(Math.random()) : Math.random();
  },
};
```

See also

- Functions
- get

SyntaxError: identifier starts immediately after numeric literal

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Identifier_after_number · MDN page last modified 2025-07-08

The JavaScript exception "identifier starts immediately after numeric literal" occurs when an identifier started with a digit. Identifiers can only start with a letter, underscore (_), or dollar sign ($).

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: identifier starts immediately after numeric literal (Firefox)
SyntaxError: No identifiers allowed directly after numeric literal (Safari)
```

Error type

SyntaxError

What went wrong?

The names of variables, called identifiers, conform to certain rules, which your code must adhere to!

A JavaScript identifier must start with a letter, underscore (_), or dollar sign ($). They can't start with a digit! Only subsequent characters can be digits (0-9).

Examples

Variable names starting with numeric literals

Variable names can't start with numbers in JavaScript. The following fails:

js · ✗ bad example

```
const 1life = "foo";
// SyntaxError: identifier starts immediately after numeric literal

const foo = 1life;
// SyntaxError: identifier starts immediately after numeric literal
```

You will need to rename your variable to avoid the leading number.

js · ✓ good example

```
const life1 = "foo";
const foo = life1;
```

In JavaScript, there is a syntactic peculiarity when calling properties or methods on numbers. If you want to call a method on an integer, you cannot immediately use a dot after the number because the dot is interpreted as the start of a decimal fraction, causing the parser to see the method's name as an identifier immediately after a number literal. To avoid this, you need to either wrap the number in parentheses or use a double dot, where the first dot is a decimal point for the number literal and the second dot is the property accessor.

js · ✗ bad example

```
alert(typeof 1.toString())
// SyntaxError: identifier starts immediately after numeric literal
```

Correct ways to call methods on numbers:

js · ✓ good example

```
// Wrap the number in parentheses
alert(typeof (1).toString());

// Add an extra dot for the number literal
alert(typeof 2..toString());

// Use square brackets
alert(typeof 3["toString"]());
```

See also

- Lexical grammar
- Grammar and types guide

JavaScript › Reference › JavaScript error reference

SyntaxError: illegal character

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Illegal_character · MDN page last modified 2025-07-08

The JavaScript exception "illegal character" occurs when the lexer reads a character that's not part of a string literal, and the character cannot constitute a valid token in the language.

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: illegal character U+201C (Firefox)
SyntaxError: Invalid character '\u201c' (Safari)
```

Error type

SyntaxError

What went wrong?

There is an invalid character that the interpreter doesn't understand. You should either put it in a string literal or replace it with another character. Use an editor that supports syntax highlighting and carefully check your code against mismatches like a minus sign (-) versus a dash (–) or simple quotes (") versus non-standard quotation marks (“).

Examples

Mismatched characters

Some characters look similar, but will cause the parser to fail interpreting your code. Famous examples of this are quotes, the minus or semicolon (greek question mark (U+37e) looks same).

js · ✗ bad example

```
“This looks like a string”; // SyntaxError: illegal character
// “ and ” are not " but look like it

42 – 13; // SyntaxError: illegal character
// – (en-dash) is not - but looks like it

const foo = "bar"; // SyntaxError: illegal character
// <37e> is not ; but looks like it
```

This should work:

js · ✓ good example

```
"This is actually a string";
42 - 13;
const foo = "bar";
```

Some editors and IDEs will notify you or at least use a slightly different highlighting for it, but not all. When something like this happens to your code and you're not able to find the source of the problem, it's often best to just delete the problematic line and retype it.

Forgotten characters

It's easy to forget a character here or there.

js · ✗ bad example

```
const operators = ["+", "-", ×", "÷"];
// SyntaxError: illegal character U+00D7
```

Add the missing quote for "×".

js · ✓ good example

```
const operators = ["+", "-", "×", "÷"];
```

Hidden characters

When copy pasting code from external sources, there might be invalid characters. Watch out!

js · ✗ bad example

```
const foo = "bar";
// SyntaxError: illegal character
```

When inspecting this code in an editor like VIM, you can see that there is actually a zero-width space (ZWSP) (U+200B) character.

js

```
const foo = "bar";<200b>
```

See also

- Lexical grammar

JavaScript › Reference › JavaScript error reference

SyntaxError: import declarations may only appear at top level of a module

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/import_decl_module_top_level · MDN page last modified 2025-07-08

The JavaScript exception "import declarations may only appear at top level of a module" occurs when an import declaration is not at the top level of a module. This might be because the import declaration is nested in other constructs (functions, blocks, etc.), or more often because the current file is not treated as a module.

Message

```
SyntaxError: Cannot use import statement outside a module (V8-based)
SyntaxError: import declarations may only appear at top level of a module (Firefox)
SyntaxError: Unexpected identifier 'x'. import call expects one or two arguments. (Safari)
```

Error type

SyntaxError

What went wrong?

You may have an `import` declaration that's nested in another construct, such as a function or a block. The `import` declaration must be at the top level of the module. If you want to conditionally import a module, or lazily import it on demand, use dynamic import instead.

If the `import` is already at the top level of the code, it may be because the file is not interpreted as a module. Runtimes require external hints to determine whether a file is a module or not, and here are several ways to provide such hints:

- If the file is directly loaded from HTML, make sure the `<script>` tag has the `type="module"` attribute.
- If the file is run in Node, make sure either the file has the `.mjs` extension, or the closest `package.json` file has the `"type": "module"` field.
- If the file is run as a worker, make sure you call the `Worker()` constructor with the `type: "module"` option.
- Import this file from another module.

One other cause might be when you are writing `import` with a compiler (such as TypeScript) and you accidentally ran the source file. Since `import` declarations usually appear at the very start of the program, they are the first thing the parser sees and then complains about. Make sure to compile the source file and execute the compiled file instead.

Examples

Conditional imports

You cannot use `import` inside other constructs, like you might do in Python.

js · ✗ bad example

```
if (writeOutput) {
  import fs from "fs"; // SyntaxError
}
```

Either move the `import` to the top level, or use dynamic import.

js · ✓ good example

```
if (writeOutput) {
  import("fs").then((fs) => {
    // use fs
  });
}
```

Importing in a non-module script

If you are loading the script from HTML, make sure to add the `type="module"` attribute to the `<script>` tag.

html

```
<script type="module" src="main.js"></script>
```

If for some reason you cannot migrate the script to a module, you can use dynamic import.

js · ✓ good example

```
async function main() {
  const myModule = await import("./my-module.js");
  // use myModule
}

main();
```

See also

- Using modules
- `import`
- `import()`

SyntaxError: incomplete quantifier in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_incomplete_quantifier · MDN page last modified 2025-07-08

The JavaScript exception "incomplete quantifier in regular expression" occurs when a regular expression pattern contains a {, but it does not start a valid quantifier.

Message

```
SyntaxError: Invalid regular expression: /1{/u: Incomplete quantifier (V8-based)
SyntaxError: incomplete quantifier in regular expression (Firefox)
SyntaxError: Invalid regular expression: incomplete {} quantifier for Unicode pattern
(Safari)
```

Error type

SyntaxError

What went wrong?

A { character in a regular expression pattern starts a quantifier. A valid quantifier is in the form {n}, {n,}, or {n,m}, where n and m are non-negative integers and m is not less than n. If the { character does not start a valid quantifier, a SyntaxError occurs.

In Unicode-unaware mode, this syntax causes the { to become a literal character instead of generating an error, but this is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

js · ✗ bad example

```
/1{/u;
/1{a}/u;
/1{}/u;
/1{1,2,3}/u;
/1{1, 2}/u;
```

Valid cases

js · ✓ good example

```
/1{1}/u;
/1{1,}/u;
/1{1,2}/u;
```

See also

- Regular expressions
- Quantifier: *, +, ?, {n}, {n,}, {n,m}

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid assignment left-hand side

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_assignment_left-hand_side · MDN page last modified 2026-09-01

The JavaScript exception "invalid assignment left-hand side" occurs when there was an unexpected assignment somewhere. It may be triggered when a single = sign was used instead of == or ===.

Message

```
SyntaxError: Invalid left-hand side in assignment (V8-based)
SyntaxError: invalid assignment left-hand side (Firefox)
SyntaxError: Left side of assignment is not a reference. (Safari)

ReferenceError: Invalid left-hand side in assignment (V8-based)
ReferenceError: cannot assign to function call (Firefox)
ReferenceError: Left side of assignment is not a reference. (Safari)
```

Error type

SyntaxError or ReferenceError, depending on the syntax.

What went wrong?

There was an unexpected assignment somewhere. This might be due to a mismatch of an assignment operator and an equality operator, for example. While a single = sign assigns a value to a variable, the == or === operators compare a value.

Examples

Typical invalid assignments

js · ✗ bad example

```
if (Math.PI + 1 = 3 || Math.PI + 1 = 4) {
  console.log("no way!");
}
// SyntaxError: invalid assignment left-hand side

const str = "Hello, "
+= "is it me "
+= "you're looking for?";
// SyntaxError: invalid assignment left-hand side
```

In the if statement, you want to use an equality operator (===), and for the string concatenation, the plus (+) operator is needed.

js · ✓ good example

```
if (Math.PI + 1 === 3 || Math.PI + 1 === 4) {
  console.log("no way!");
}

const str = "Hello, "
  + "from the "
  + "other side!";
```

Assignments producing ReferenceErrors

Invalid assignments don't always produce syntax errors. Sometimes the syntax is almost correct, but at runtime, the left-hand side expression evaluates to a *value* instead of a *reference*, so the assignment is still invalid. Such errors occur later in execution, when the statement is actually executed.

js · ✗ bad example

```
function foo() {
  return { a: 1 };
}
foo() = 1; // ReferenceError: invalid assignment left-hand side
```

Function calls, new calls, super(), and this are all values instead of references. If you want to use them on the left-hand side, the assignment target needs to be a property of their produced values instead.

js · ✓ good example

```
function foo() {
  return { a: 1 };
}
foo().a = 1;
```

> **Note:** In Firefox and Safari, the first example produces a `ReferenceError` in non-strict mode, and a `SyntaxError` in strict mode. Chrome throws a runtime `ReferenceError` for both strict and non-strict modes.

Using optional chaining as assignment target

Optional chaining is not a valid target of assignment.

js · ✗ bad example

```
obj?.foo = 1; // SyntaxError: invalid assignment left-hand side
```

Instead, you have to first guard the nullish case.

js · ✓ good example

```
if (obj) {
  obj.foo = 1;
}
```

See also

- Assignment operators
- Equality operators

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid BigInt syntax

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_BigInt_syntax · MDN page last modified 2025-07-08

The JavaScript exception "invalid BigInt syntax" occurs when a string value is being coerced to a `BigInt` but it failed to be parsed as an integer.

Message

```
SyntaxError: Cannot convert x to a BigInt (V8-based)
SyntaxError: invalid BigInt syntax (Firefox)
SyntaxError: Failed to parse String to BigInt (Safari)
```

Error type

`SyntaxError`.

What went wrong?

When using the `BigInt()` function to convert a string to a BigInt, the string will be parsed in the same way as source code, and the resulting value must be an integer value.

Examples

Invalid cases

js · ✗ bad example

```
const a = BigInt("1.5");
const b = BigInt("1n");
const c = BigInt.asIntN(4, "8n");
// SyntaxError: invalid BigInt syntax
```

Valid cases

js · ✓ good example

```
const a = BigInt("1");
const b = BigInt("  1   ");
const c = BigInt.asIntN(4, "8");
```

See also

- BigInt

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid capture group name in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_capture_group_name · MDN page last modified 2025-07-08

The JavaScript exception "invalid capture group name in regular expression" occurs when a named capturing group or named backreference contains an invalid identifier.

Message

```
SyntaxError: Invalid regular expression: /(?<1>)/: Invalid capture group name (V8-based)
SyntaxError: invalid capture group name in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid group specifier name (Safari)
```

Error type

SyntaxError

What went wrong?

Each named capturing group must have a name that is a valid identifier. You cannot use arbitrary strings as the group identifier.

Examples

Invalid cases

js · ✗ bad example

```
/(?<1>\d+) (?<2>\d+)/;
```

Or you might be building the regex dynamically:

js · ✗ bad example

```
const tokenTypes = {
  "number literal": /\d+/,
  "string literal": /".+?"/,
  identifier: /[a-zA-Z_]\w*/,
};

const tokenPattern = new RegExp(
  Object.entries(tokenTypes)
    .map(([name, pattern]) => `(?<${name}>${pattern.source})`)
    .join("|"),
);
```

Valid cases

js · ✓ good example

```
/(?<group1>\d+) (?<group2>\d+)/;
```

If the regex is built dynamically, make sure the names are all valid identifiers. For example:

js · ✓ good example

```
const tokenTypes = {
  numberLiteral: /\d+/,
  stringLiteral: /".+?"/,
  identifier: /[a-zA-Z_]\w*/,
};

const tokenPattern = new RegExp(
  Object.entries(tokenTypes)
    .map(([name, pattern]) => `(?<${name}>${pattern.source})`)
    .join("|"),
);
```

See also

- Regular expressions
- Named capturing group: `(?<name>...)`

SyntaxError: invalid character in class in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_char_in_class · MDN page last modified 2025-07-08

The JavaScript exception "invalid character in class in regular expression" occurs when a character appears in a v-mode character class but it's not allowed to appear literally.

Message

```
SyntaxError: Invalid regular expression: /[|]/v: Invalid character in character class (V8-based)
SyntaxError: invalid character in class in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid class set character (Safari)
```

Error type

SyntaxError

What went wrong?

Normally, character classes can contain almost all characters literally. However, the v mode made the character class syntax more sophisticated, and in order to leave room for future syntax extensions, some syntax characters are forbidden from appearing literally in a character class. They include: (,), [,], {, }, /, -, |. If you want to match these literal characters, escape them; for example: `/[\|]/v`.

Examples

Invalid cases

js · ✗ bad example

```
/[(){}]/v;
```

Valid cases

js · ✓ good example

```
/[\(\)\{\}]/v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid class set operation in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_class_set_operation · MDN page last modified 2025-07-08

The JavaScript exception "invalid class set operation in regular expression" occurs when a double punctuator sequence appears in a v-mode character class but it is not recognized by the syntax.

Message

```
SyntaxError: Invalid regular expression: /[&&]/v: Invalid set operation in character class
(V8-based)
SyntaxError: invalid class set operation in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid operation in class set (Safari)
```

Error type

SyntaxError

What went wrong?

There are three possible ways this could happen:

- You are trying to use `&&` or `--`, but the syntax is wrong. Each of these operators must join two characters or character sets.
- You are mixing operators on the same level. For example, `[\w&&[A-z]--_]` is invalid because it uses `&&` and `--` on the same level. You need to use nested character classes to disambiguate, such as `[\w&&[[A-z]--_]]`. Note that the union operation uses an operator that has no text; for example, `[AB&&C]` is invalid because `A` and `B` are implicitly joined by the union operator. You need to use `[A[B&&C]]` instead.
- You are using a double punctuator sequence that's not `&&` or `--`. These sequences are reserved for future syntax extensions. They include: `&&`, `!!`, `##`, `$$`, `%%`, `**`, `++`, `,,`, `..`, `::`, `;;`, `<<`, `==`, `>>`, `??`, `@@`, `^^`, `` `` ``, `~~`. However, these sequences don't make much sense anyway, and can either be replaced with a single character or cause two adjacent ranges to be merged.

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

SyntaxError: invalid decimal escape in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_decimal_escape · MDN page last modified 2025-07-08

The JavaScript exception "invalid decimal escape in regular expression" occurs when a legacy octal escape sequence is used in a Unicode-aware regular expression pattern.

Message

```
SyntaxError: Invalid regular expression: /\00/u: Invalid decimal escape (V8-based)
SyntaxError: invalid decimal escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid octal escape for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

In a regular expression, `\0` followed by another digit is a *legacy octal escape sequence*. The same syntax is banned in template strings and strict-mode string literals. In regexes, this feature is disabled by the Unicode-aware modes (`u` and `v`). `\0` *not* followed by another digit is a valid escape sequence that represents the null character (U+0000).

`\` followed by a non-zero digit is a backreference, and is invalid in Unicode-aware mode if it doesn't refer to a capturing group; see invalid identity escape for more information.

Examples

Invalid cases

js · ✗ bad example

```
/\00/u;
/\01/u;
```

Valid cases

js · ✓ good example

```
// If you want to match NULL followed by a digit, use a character class
/[\0]0/u;
// If you want to match a character by its character value, use \x
/\x01/u;
```

See also

- Regular expressions
- Escape sequences
- Character escape: `\n`, `\u{...}`

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid identity escape in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_identity_escape · MDN page last modified 2025-07-08

The JavaScript exception "invalid identity escape in regular expression" occurs when a Unicode-aware regular expression pattern contains an escape sequence that does not represent a recognized escape sequence.

Message

```
SyntaxError: Invalid regular expression: /\q/u: Invalid escape (V8-based)
SyntaxError: invalid identity escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid escaped character for Unicode pattern
(Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-unaware mode, `\` could be used to escape any character, including those without a defined meaning. In these cases, the escaped character represents itself. For example, `\q` would match the character `q`. This severely limits the ability to add new escape sequences in the future, so in Unicode-aware mode, only recognized escape sequences are allowed. Do not add `\` redundantly.

This error is also thrown when the `\x` character escape is not followed by two hexadecimal digits.

For an exhaustive list of available escape sequences, see the regular expressions reference. For a table of which characters can appear literally escaped or unescaped in each context, see literal characters.

Examples

Invalid cases

js · ✗ bad example

```
/[\f\v\n\t\ ]/u;
```

Valid cases

js · ✓ good example

```
// There's no need to escape the space
/[\f\v\n\t ]/u;
```

See also

- Regular expressions
- Escape sequences
- Character escape: \n, \u{...}
- Literal character: a, b

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid named capture reference in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_named_capture_reference · MDN page last modified 2025-07-08

The JavaScript exception "invalid named capture reference in regular expression" occurs when a regular expression pattern contains a named backreference that does not refer to a named capture group before it. The similar error message "invalid named reference in regular expression" is thrown when the sequence \k is encountered but is not followed by <.

Message

```
SyntaxError: Invalid regular expression: /\k<x>/u: Invalid named capture referenced (V8-
based)
SyntaxError: invalid named capture reference in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid \k<> named backreference (Safari)

SyntaxError: Invalid regular expression: /\k/u: Invalid named reference (V8-based)
SyntaxError: invalid named reference in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid escaped character for Unicode pattern
(Safari)
```

Error type

SyntaxError

What went wrong?

You are using the escape sequence \k which is parsed as a named backreference, but the \k is either syntactically invalid or does not refer to a named capture group in the pattern.

`\k` only starts a named backreference when the pattern contains a named capturing group or when the pattern is Unicode-aware; otherwise, it is an identity escape for the character `k`.

Examples

Invalid cases

js · ✗ bad example

```
/\k<x>/u; // Does not refer to a named capture group
/(?<x>.+)\k<y>/u; // Does not refer to an existing named capture group
/(?<x>.+)\k{x}/u; // Invalid syntax
```

Valid cases

js · ✓ good example

```
/(?<x>.+)\k<x>/u; // Refers to an existing named capture group
```

See also

- Regular expressions
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid property name in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_property_name · MDN page last modified 2025-07-08

The JavaScript exception "invalid property name in regular expression" or "invalid class property name in regular expression" occurs when the `\p` and `\P` Unicode character class escapes are not followed by a valid Unicode property name and/or value.

Message

```
SyntaxError: Invalid regular expression: /\p{x}/u: Invalid property name (V8-based)
SyntaxError: Invalid regular expression: /[\p{x}]/u: Invalid property name in character
class (V8-based)
SyntaxError: invalid property name in regular expression (Firefox)
SyntaxError: invalid class property name in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid property expression (Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-aware mode, the `\p` and `\P` escape sequences are used to match characters or character sequences based on their Unicode properties. The `\p` escape sequence matches characters with the specified Unicode property, while the `\P` escape sequence matches characters without the specified Unicode property. The syntax is:

regex

```
\p{loneProperty}
\P{loneProperty}

\p{property=value}
\P{property=value}
```

If it's not followed by {, then a recognizable property name/value, then }, this syntax error is thrown. For more information on what lone properties and property names are allowed, see the Unicode character class escape reference.

Examples

Invalid cases

js · ✗ bad example

```
/\p{x}/u; // "x" is not a valid Unicode property name
/\p{Script=x}/u; // "x" is not a valid value for Script
/\property/u; // "\p" is not followed by the right syntax
/\p{RGI_Emoji_Flag_Sequence}/u;
// "RGI_Emoji_Flag_Sequence" is a property of strings, so the "v" flag must be used
/\P{RGI_Emoji_Flag_Sequence}/v; // \P cannot match properties of strings
```

Valid cases

js · ✓ good example

```
/\p{Script=Latin}/u; // "Script=Latin" is a valid Unicode property
/\p{Letter}/u; // "Letter" is valid value for General_Category
/\p{RGI_Emoji_Flag_Sequence}/v; // Property of strings can only be used in "v" mode
```

See also

- Regular expressions
- Unicode character class escape: `\p{...}`, `\P{...}`

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid range in character class

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_range_in_character_class · MDN page last modified 2025-07-08

The JavaScript exception "invalid range in character class" occurs when a character class in a regular expression uses a range, but the start of the range is greater than the end.

Message

```
SyntaxError: Invalid regular expression: /[2-1]/: Range out of order in character class (V8-based)
SyntaxError: invalid range in character class (Firefox)
SyntaxError: Invalid regular expression: range out of order in character class (Safari)
```

Error type

SyntaxError

What went wrong?

In character classes, you can join two characters with a hyphen - to represent an inclusive interval of characters based on their Unicode code points. For example, [a-z] matches any lowercase letter. However, if the end of the range is less than the start, the range cannot match anything and is likely a mistake.

Examples

Invalid cases

js · ✗ bad example

```
/[9-1]/; // The range is out of order
/[_-=]/; // _ has value 95, = has value 61
```

Valid cases

js · ✓ good example

```
/[1-9]/; // Swap the range
/[_\-=]/; // Escape the hyphen so it matches the literal character
```

See also

- Regular expressions
- Character class: [...], [^...]

SyntaxError: invalid regexp group

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_group · MDN page last modified 2026-09-01

The JavaScript exception "invalid regexp group" occurs when the sequence (? does not start a valid group syntax. Recognized group syntaxes that start with (? include:

- (?: for non-capturing groups
- (?= for positive lookahead
- (?! for negative lookahead
- (?<= for positive lookbehind
- (?<! for negative lookbehind
- (?< for named capturing groups
- (?-i:, (?i:, (?m:, (?ims-:, etc. for modifiers

(? followed by any other character would cause this error.

Message

```
SyntaxError: Invalid regular expression: /(?1)/: Invalid group (V8-based)
SyntaxError: invalid regexp group (Firefox)
SyntaxError: Invalid regular expression: unrecognized character after (? (Safari)
```

Error type

SyntaxError

What went wrong?

`?` is not an atom, so it does not make sense to appear at the start of a capturing group (`?` is a quantifier and should be placed after an atom). Maybe you want to match the `?` character literally, in which case you should escape it with a backslash (`\?`). Maybe you remembered the regex syntax wrong, and you intend to use one of the recognized group syntaxes listed above. Maybe you are using a feature that is not supported by the current JavaScript engine.

Examples

Invalid cases

js · ✗ bad example

```
/Hello(?|!)/;
// This is Perl syntax
/(?[\p{Thai}&\p{Digit}])/;
```

Valid cases

js · ✓ good example

```
/Hello(\?|!)/;
// This is JavaScript syntax for character set operations
/[\p{Script=Thai}&&\p{Nd}]/v;
```

See also

- Regular expressions
- Capturing group: `(...)`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Modifier: `(?ims-ims:...)`
- Named capturing group: `(?<name>...)`
- Non-capturing group: `(?:...)`

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid regular expression flag "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_regexp_flag · MDN page last modified 2025-07-08

The JavaScript exception "invalid regular expression flag" occurs when the flags in a regular expression contain any flag that is not one of: `d`, `g`, `i`, `m`, `s`, `u`, `v`, or `y`. It may also be raised if the expression contains more than one instance of a valid flag, or when the `u` and `v` flags are used together.

Message

```
SyntaxError: Invalid flags supplied to RegExp constructor 'x' (V8-based)
SyntaxError: Invalid regular expression flags (V8-based)
SyntaxError: invalid regular expression flag x (Firefox)
SyntaxError: Invalid flags supplied to RegExp constructor. (Safari)
SyntaxError: Invalid regular expression: invalid flags (Safari)
```

Error type

`SyntaxError`

What went wrong?

The regular expression contains invalid flags, or valid flags have been used more than once in the expression.

The valid (allowed) flags are `d`, `g`, `i`, `m`, `s`, `u`, `v`, and `y`. They are introduced in more detail in Regular expressions > Advanced searching with flags.

The `u` and `v` flags are mutually exclusive, so they cannot be used together. You can read the references to understand what their behavior differences are.

Examples

In a regular expression literal, which consists of a pattern enclosed between slashes, the flags are defined after the second slash. Regular expression flags can be used separately or together in any order. This syntax shows how to declare the flags using the regular expression literal:

js

```
const re = /pattern/flags;
```

They can also be defined in the constructor function of the `RegExp` object (second parameter):

js

```
const re = new RegExp("pattern", "flags");
```

Here is an example showing use of only correct flags.

js · ✓ good example

```
/foo/g;
/foo/gims;
/foo/uy;
```

Below is an example showing the use of some invalid flags `b`, `a` and `r`:

js · ✗ bad example

```
/foo/bar;

// SyntaxError: invalid regular expression flag "b"
```

The code below is incorrect, because `W`, `e` and `b` are not valid flags.

js · ✗ bad example

```
const obj = {
  url: /docs/Web,
};

// SyntaxError: invalid regular expression flag "W"
```

An expression containing two slashes is interpreted as a regular expression literal. Most likely the intent was to create a string literal, using single or double quotes as shown below:

js · ✓ good example

```
const obj = {
  url: "/docs/Web",
};
```

See also

- Regular expressions guide

JavaScript › Reference › JavaScript error reference

SyntaxError: invalid unicode escape in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_invalid_unicode_escape · MDN page last modified 2026-10-08

The JavaScript exception "invalid unicode escape in regular expression" occurs when the `\c` and `\u` character escapes are not followed by valid characters.

Message

```
SyntaxError: Invalid regular expression: /\u{123456}/u: Invalid Unicode escape (V8-based)
SyntaxError: invalid unicode escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid Unicode code point \u{} escape (Safari)
```

Error type

SyntaxError

What went wrong?

In Unicode-aware mode, the `\c` escape sequence must be followed by a letter from `A` to `Z` or `a` to `z`, and the `\u` escape sequence must either be followed by 4 hexadecimal digits, or 1 to 6 hexadecimal digits enclosed in curly braces (`{}`). Furthermore, when using the `\u{xxx}` escape sequence, the digits must represent a valid Unicode code point, which means its value cannot exceed `10FFFF`.

Examples

Invalid cases

js · ✗ bad example

```
/\u{123456}/u; // Unicode code point is too large
/\u65/u; // Not enough digits
/\c1/u; // Not a letter
```

Valid cases

js · ✓ good example

```
/\u0065/u; // Lowercase "e"
/\u{1f600}/u; // Grinning face emoji
/\cA/u; // U+0001 (Start of Heading)
```

See also

- Regular expressions
- Character escape: \n, \u{...}

JavaScript › Reference › JavaScript error reference

SyntaxError: JSON.parse: bad parsing

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/JSON_bad_parse · MDN page last modified 2025-07-08

The JavaScript exceptions thrown by `JSON.parse()` occur when string failed to be parsed as JSON.

Message

```
SyntaxError: JSON.parse: unterminated string literal
SyntaxError: JSON.parse: bad control character in string literal
SyntaxError: JSON.parse: bad character in string literal
SyntaxError: JSON.parse: bad Unicode escape
SyntaxError: JSON.parse: bad escape character
SyntaxError: JSON.parse: unterminated string
SyntaxError: JSON.parse: no number after minus sign
SyntaxError: JSON.parse: unexpected non-digit
SyntaxError: JSON.parse: missing digits after decimal point
SyntaxError: JSON.parse: unterminated fractional number
SyntaxError: JSON.parse: missing digits after exponent indicator
SyntaxError: JSON.parse: missing digits after exponent sign
SyntaxError: JSON.parse: exponent part is missing a number
SyntaxError: JSON.parse: unexpected end of data
SyntaxError: JSON.parse: unexpected keyword
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: end of data while reading object contents
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: end of data when ',' or ']' was expected
SyntaxError: JSON.parse: expected ',' or ']' after array element
SyntaxError: JSON.parse: end of data when property name was expected
SyntaxError: JSON.parse: expected double-quoted property name
SyntaxError: JSON.parse: end of data after property name when ':' was expected
SyntaxError: JSON.parse: expected ':' after property name in object
SyntaxError: JSON.parse: end of data after property value in object
SyntaxError: JSON.parse: expected ',' or '}' after property value in object
```

(code continues on the next page)

(code continued from the previous page)

```
SyntaxError: JSON.parse: expected ',' or '}' after property-value pair in object literal
SyntaxError: JSON.parse: property names must be double-quoted strings
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: unexpected non-whitespace character after JSON data
```

Error type

SyntaxError

What went wrong?

JSON.parse() parses a string as JSON. This string has to be valid JSON and will throw this error if incorrect syntax was encountered.

Examples

JSON.parse() does not allow trailing commas

Both lines will throw a SyntaxError:

js · ✗ bad example

```
JSON.parse("[1, 2, 3, 4,]");
JSON.parse('{"foo": 1,}');
// SyntaxError JSON.parse: unexpected character
// at line 1 column 14 of the JSON data
```

Omit the trailing commas to parse the JSON correctly:

js · ✓ good example

```
JSON.parse("[1, 2, 3, 4]");
JSON.parse('{"foo": 1}');
```

Property names must be double-quoted strings

You cannot use single-quotes around properties, like 'foo'.

js · ✗ bad example

```
JSON.parse("{'foo': 1}");
// SyntaxError: JSON.parse: expected property name or '}'
// at line 1 column 2 of the JSON data
```

Instead write "foo":

js · ✓ good example

```
JSON.parse('{"foo": 1}');
```

Leading zeros and decimal points

You cannot use leading zeros, like 01, and decimal points must be followed by at least one digit.

js · ✗ bad example

```
JSON.parse('{"foo": 01}');
// SyntaxError: JSON.parse: expected ',' or '}' after property value
// in object at line 1 column 2 of the JSON data

JSON.parse('{"foo": 1.}');
// SyntaxError: JSON.parse: unterminated fractional number
// at line 1 column 2 of the JSON data
```

Instead write just 1 without a zero and use at least one digit after a decimal point:

js · ✓ good example

```
JSON.parse('{"foo": 1}');
JSON.parse('{"foo": 1.0}');
```

See also

- JSON
- JSON.parse()
- JSON.stringify()

JavaScript › Reference › JavaScript error reference

SyntaxError: label not found

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Label_not_found · MDN page last modified 2025-07-08

The JavaScript exception "label not found" occurs when a break or continue statement references a label that does not exist on any statement that contains the break or continue statement.

Message

```
SyntaxError: Undefined label 'label' (V8-based)
SyntaxError: label not found (Firefox)
SyntaxError: Cannot use the undeclared label 'label'. (Safari)
```

Error type

SyntaxError.

What went wrong?

In JavaScript, labels are very limited: you can only use them with break and continue statements, and you can only jump to them from a statement contained within the labeled statement. You cannot jump to this label from anywhere in the program.

Examples

Unsyntactic jump

You cannot use labels as if they are goto.

js · ✗ bad example

```
start: console.log("Hello, world!");
console.log("Do it again");
break start;
```

Instead, you can only use labels to enhance the normal semantics of `break` and `continue` statements.

js · ✓ good example

```
start: {
  console.log("Hello, world!");
  if (Math.random() > 0.5) {
    break start;
  }
  console.log("Maybe I'm logged");
}
```

See also

- Labeled statement

JavaScript › Reference › JavaScript error reference

SyntaxError: missing : after property id

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_colon_after_property_id · MDN page last modified 2025-07-08

The JavaScript exception "missing : after property id" occurs when objects are created using the object initializer syntax. A colon (`:`) separates keys and values for the object's properties. Somehow, this colon is missing or misplaced.

Message

```
SyntaxError: Invalid shorthand property initializer (V8-based)
SyntaxError: missing : after property id (Firefox)
SyntaxError: Unexpected token '='. Expected a ':' following the property name 'x'. (Safari)
SyntaxError: Unexpected token '+'. Expected an identifier as property name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

When creating objects with the object initializer syntax, a colon (`:`) separates keys and values for the object's properties.

js

```
const obj = { propertyKey: "value" };
```

Examples

Colons vs. equal signs

This code fails, as the equal sign can't be used this way in this object initializer syntax.

js · ✗ bad example

```
const obj = { propertyKey = "value" };
// SyntaxError: missing : after property id
```

Correct would be to use a colon, or to use square brackets to assign a new property after the object has been created already.

js · ✓ good example

```
const obj = { propertyKey: "value" };
```

Or alternatively:

js

```
const obj = {};
obj.propertyKey = "value";
```

Computed properties

If you create a property key from an expression, you need to use square brackets. Otherwise the property name can't be computed:

js · ✗ bad example

```
const obj = { "b"+"ar": "foo" };
// SyntaxError: missing : after property id
```

Put the expression in square brackets `[]`:

js · ✓ good example

```
const obj = { ["b" + "ar"]: "foo" };
```

See also

- Object initializer

SyntaxError: missing) after argument list

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_parenthesis_after_argument_list · MDN page last modified 2025-07-08

The JavaScript exception "missing) after argument list" occurs when there is an error with how a function is called. This might be a typo, a missing operator, or an unescaped string.

Message

```
SyntaxError: missing ) after argument list (V8-based & Firefox)
SyntaxError: Unexpected identifier 'x'. Expected ')' to end an argument list. (Safari)
```

Error type

SyntaxError.

What went wrong?

There is an error with how a function is called. This might be a typo, a missing operator, or an unescaped string, for example.

Examples

Because there is no "+" operator to concatenate the string, JavaScript expects the argument for the `log` function to be just `"PI: "`. In that case, it should be terminated by a closing parenthesis.

js · ✗ bad example

```
console.log("PI: " Math.PI);
// SyntaxError: missing ) after argument list
```

You can correct the `log` call by adding the `+` operator:

js · ✓ good example

```
console.log("PI: " + Math.PI);
// "PI: 3.141592653589793"
```

Alternatively, you can consider using a template literal, or take advantage of the fact that `console.log` accepts multiple parameters:

js · ✓ good example

```
console.log(`PI: ${Math.PI}`);
console.log("PI:", Math.PI);
```

Unterminated strings

js · ✗ bad example

```
console.log('"Java" + "Script" = \"' + "Java" + 'Script\");
// SyntaxError: missing ) after argument list
```

Here JavaScript thinks that you meant to have `);` inside the string and ignores it, and it ends up not knowing that you meant the `);` to end the function `console.log`. To fix this, we could put a `'` after the "Script" string:

js · ✓ good example

```
console.log('"Java" + "Script" = "' + "Java" + 'Script"');
// '"Java" + "Script" = "JavaScript"'
```

See also

- Functions guide

JavaScript › Reference › JavaScript error reference

SyntaxError: missing) after condition

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_parenthesis_after_condition · MDN page last modified 2025-07-08

The JavaScript exception "missing) after condition" occurs when there is an error with how an `if` condition is written. It must appear in parenthesis after the `if` keyword.

Message

```
SyntaxError: missing ) after condition (Firefox)
SyntaxError: Unexpected token '{'. Expected ')' to end an 'if' condition. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is an error with how an `if` condition is written. In any programming language, code needs to make decisions and carry out actions accordingly depending on different inputs. The if statement executes a statement if a specified condition is truthy. In JavaScript, this condition must appear in parenthesis after the `if` keyword, like this:

js

```
if (condition) {
  // do something if the condition is true
}
```

Examples

Missing parenthesis

It might just be an oversight, carefully check all you parenthesis in your code.

js · ✗ bad example

```
if (Math.PI < 3 {
  console.log("wait what?");
}

// SyntaxError: missing ) after condition
```

To fix this code, you would need to add a parenthesis that closes the condition.

js · ✓ good example

```
if (Math.PI < 3) {
  console.log("wait what?");
}
```

Misused is keyword

If you are coming from another programming language, it is also easy to add keywords that don't mean the same or have no meaning at all in JavaScript.

js · ✗ bad example

```
if (done is true) {
 console.log("we are done!");
}

// SyntaxError: missing ) after condition
```

Instead you need to use a correct comparison operator. For example:

js

```
if (done === true) {
  console.log("we are done!");
}
```

Or even better:

js · ✓ good example

```
if (done) {
  console.log("we are done!");
}
```

See also

- if...else
- Relational operators
- Learn: Making decisions in your code — conditionals

JavaScript › Reference › JavaScript error reference

SyntaxError: missing] after element list

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_bracket_after_list · MDN page last modified 2025-07-08

The JavaScript exception "missing] after element list" occurs when there is an error with the array initializer syntax somewhere. Likely there is a closing square bracket (`]`) or a comma (`,`) missing.

Message

```
SyntaxError: missing ] after element list (Firefox)
SyntaxError: Unexpected token ';'. Expected either a closing ']' or a ',' following an array
element. (Safari)
```

Error type

SyntaxError.

What went wrong?

There is an error with the array initializer syntax somewhere. Likely there is a closing square bracket (`]`) or a comma (`,`) missing.

Examples

Incomplete array initializer

js · ✗ bad example

```
const list = [1, 2,

const instruments = [
  "Ukulele",
  "Guitar",
  "Piano",
};

const data = [{ foo: "bar" } { bar: "foo" }];
```

Correct would be:

js · ✓ good example

```
const list = [1, 2];

const instruments = ["Ukulele", "Guitar", "Piano"];

const data = [{ foo: "bar" }, { bar: "foo" }];
```

See also

- Array

JavaScript › Reference › JavaScript error reference

SyntaxError: missing } after function body

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_curly_after_function_body · MDN page last modified 2025-07-08

The JavaScript exception "missing } after function body" occurs when there is a syntax mistake when creating a function somewhere. Check if any closing curly braces or parenthesis are in the correct order.

Message

```
SyntaxError: missing } after function body (Firefox)
```

Error type

SyntaxError

What went wrong?

There is a syntax mistake when creating a function somewhere. Also check if any closing curly braces or parenthesis are in the correct order. Indenting or formatting the code a bit nicer might also help you to see through the jungle.

Examples

Forgotten closing curly bracket

Oftentimes, there is a missing curly bracket in your function code:

js · ✗ bad example

```
function charge() {
  if (sunny) {
    useSolarCells();
  } else {
    promptBikeRide();
}
```

Correct would be:

js · ✓ good example

```
function charge() {
  if (sunny) {
    useSolarCells();
  } else {
    promptBikeRide();
  }
}
```

It can be more obscure when using IIFEs or other constructs that use a lot of different parenthesis and curly braces, for example.

js · ✗ bad example

```
(function () {
  if (Math.random() < 0.01) {
    doSomething();
  }
)();
```

Oftentimes, indenting differently or double checking indentation helps to spot these errors.

js · ✓ good example

```
(function () {
  if (Math.random() < 0.01) {
    doSomething();
  }
})();
```

See also

- Functions guide

JavaScript › Reference › JavaScript error reference

SyntaxError: missing } after property list

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_curly_after_property_list · MDN page last modified 2025-07-08

The JavaScript exception "missing } after property list" occurs when there is a mistake in the object initializer syntax somewhere. Might be in fact a missing curly bracket, but could also be a missing comma.

Message

```
SyntaxError: missing } after property list (Firefox)
SyntaxError: Unexpected identifier 'c'. Expected '}' to end an object literal. (Safari)
```

Error type

SyntaxError

What went wrong?

There is a mistake in the object initializer syntax somewhere. Might be in fact a missing curly bracket, but could also be a missing comma, for example. Also check if any closing curly braces or parenthesis are in the correct order. Indenting or formatting the code a bit nicer might also help you to see through the jungle.

Examples

Forgotten comma

Oftentimes, there is a missing comma in your object initializer code:

js · ✗ bad example

```
const obj = {
  a: 1,
  b: { myProp: 2 }
  c: 3
};
```

Correct would be:

js · ✓ good example

```
const obj = {
  a: 1,
  b: { myProp: 2 },
  c: 3,
};
```

See also

- Object initializer

JavaScript › Reference › JavaScript error reference

SyntaxError: missing = in const declaration

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_initializer_in_const · MDN page last modified 2025-08-08

The JavaScript exception "missing = in const declaration" occurs when a const declaration was not given a value in the same statement (like `const RED_FLAG;`). You need to provide one (`const RED_FLAG = true`).

Message

```
SyntaxError: Missing initializer in const declaration (V8-based)
SyntaxError: missing = in const declaration (Firefox)
SyntaxError: Unexpected token ';'. const declared variable 'x' must have an initializer.
(Safari)
```

Error type

SyntaxError

What went wrong?

A constant is a value that cannot be altered by the program during normal execution. It cannot change through re-assignment, and it can't be redeclared. In JavaScript, constants are declared using the `const` keyword. An initializer for a constant is required; that is, you must specify its value in the same statement in which it's declared (which makes sense, given that it can't be changed later).

Examples

Missing const initializer

Unlike `var` or `let`, you must specify a value for a `const` declaration. This throws:

js · ✗ bad example

```
const COLUMNS;
// SyntaxError: missing = in const declaration
```

Fixing the error

There are multiple options to fix this error. Check what was intended to be achieved with the constant in question.

Adding a constant value

Specify the constant value in the same statement in which it's declared:

js · ✓ good example

```
const COLUMNS = 80;
```

`const`, `let` or `var`?

Do not use `const` if you weren't meaning to declare a constant. Maybe you meant to declare a block-scoped variable with `let` or global variable with `var`. Both don't require an initial value.

js · ✓ good example

```
let columns;
```

See also

- const
- let
- var

JavaScript › Reference › JavaScript error reference

SyntaxError: missing formal parameter

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_formal_parameter · MDN page last modified 2025-07-08

The JavaScript exception "missing formal parameter" occurs when your function declaration is missing valid parameters.

Message

```
SyntaxError: missing formal parameter (Firefox)
SyntaxError: Unexpected number '3'. Expected a parameter pattern or a ')' in parameter list. (Safari)
SyntaxError: Unexpected string literal "x". Expected a parameter pattern or a ')' in parameter list. (Safari)
```

Error type

SyntaxError

What went wrong?

"Formal parameter" is a fancy way of saying "function parameter". Your function declaration is missing valid parameters. In the declaration of a function, the parameters must be identifiers, not any value like numbers, strings, or objects. Declaring functions and calling functions are two separate steps. Declarations require identifier as parameters, and only when calling (invoking) the function, you provide the values the function should use.

In JavaScript, identifiers can contain only alphanumeric characters (or "$" or "_"), and may not start with a digit. An identifier differs from a **string** in that a string is data, while an identifier is part of the code.

Examples

Provide proper function parameters

Function parameters must be identifiers when setting up a function. All these function declarations fail, as they are providing values for their parameters:

js · ✗ bad example

```
function square(3) {
  return number * number;
}
// SyntaxError: missing formal parameter

function greet("Howdy") {
  return greeting;
}
// SyntaxError: missing formal parameter

function log({ obj: "value"}) {
  console.log(arg)
}
// SyntaxError: missing formal parameter
```

You will need to use identifiers in function declarations:

js · ✓ good example

```
function square(number) {
  return number * number;
}

function greet(greeting) {
  return greeting;
}

function log(arg) {
  console.log(arg);
}
```

You can then call these functions with the arguments you like:

js

```
square(2); // 4

greet("Howdy"); // "Howdy"

log({ obj: "value" }); // { obj: "value" }
```

See also

- SyntaxError: redeclaration of formal parameter "x"

JavaScript › Reference › JavaScript error reference

SyntaxError: missing name after . operator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Missing_name_after_dot_operator · MDN page last modified 2025-07-08

The JavaScript exception "missing name after . operator" occurs when there is a problem with how the dot operator (`.`) is used for property access.

Message

```
SyntaxError: missing name after . operator (Firefox)
SyntaxError: Unexpected token '['. Expected a property name after '.'. (Safari)
```

Error type

SyntaxError

What went wrong?

The dot operator (`.`) is used for property access. You will have to specify the name of the property that you want to access. For computed property access, you might need to change your property access from using a dot to using square brackets. These will allow you to compute an expression. Maybe you intended to do concatenation instead? A plus operator (+) is needed in that case. Please see the examples below.

Examples

Property access

Property accessors in JavaScript use either the dot (.) or square brackets (`[]`), but not both. Square brackets allow computed property access.

js · ✗ bad example

```
const obj = { foo: { bar: "baz", bar2: "baz2" } };
const i = 2;

obj.[foo].[bar]
// SyntaxError: missing name after . operator

obj.foo."bar"+i;
// SyntaxError: missing name after . operator
```

To fix this code, you need to access the object like this:

js · ✓ good example

```
obj.foo.bar; // "baz"
// or alternatively
obj["foo"]["bar"]; // "baz"

// computed properties require square brackets
obj.foo["bar" + i]; // "baz2"
// or as template literal
obj.foo[`bar${i}`]; // "baz2"
```

Property access vs. concatenation

If you are coming from another programming language (like PHP), it is also easy to mix up the dot operator (`.`) and the concatenation operator (+).

js · ✗ bad example

```
console.log("Hello" . "world");

// SyntaxError: missing name after . operator
```

Instead you need to use a plus sign for concatenation:

js · ✓ good example

```
console.log("Hello" + "World");
```

See also

- Property accessors

JavaScript › Reference › JavaScript error reference

SyntaxError: missing variable name

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/No_variable_name · MDN page last modified 2025-07-08

The JavaScript exception "missing variable name" is a common error. It is usually caused by omitting a variable name or a typographic error.

Message

```
SyntaxError: missing variable name (Firefox)
SyntaxError: Unexpected token '='. Expected a parameter pattern or a ')' in parameter list.
(Safari)
```

Error type

SyntaxError

What went wrong?

A variable is missing a name. The cause is most likely a typo or a forgotten variable name. Make sure that you've provided the name of the variable before the = sign.

When declaring multiple variables at the same time, make sure that the previous lines/declaration does not end with a comma instead of a semicolon.

Examples

Missing a variable name

js · ✗ bad example

```
const = "foo";
```

It is easy to forget to assign a name for your variable!

js · ✓ good example

```
const description = "foo";
```

Reserved keywords can't be variable names

There are a few variable names that are reserved keywords. You can't use these. Sorry :(

js · ✗ bad example

```
const debugger = "whoop";
// SyntaxError: missing variable name
```

Declaring multiple variables

Pay special attention to commas when declaring multiple variables. Is there an excess comma, or did you use commas instead of semicolons? Did you remember to assign values for all your `const` variables?

js · ✗ bad example

```
let x, y = "foo",
const z, = "foo"

const first = document.getElementById("one"),
const second = document.getElementById("two"),

// SyntaxError: missing variable name
```

The fixed version:

js · ✓ good example

```
let x,
  y = "foo";
const z = "foo";

const first = document.getElementById("one");
const second = document.getElementById("two");
```

Arrays

`Array` literals in JavaScript need square brackets around the values. This won't work:

js · ✗ bad example

```
const arr = 1,2,3,4,5;
// SyntaxError: missing variable name
```

This would be correct:

js · ✓ good example

```
const arr = [1, 2, 3, 4, 5];
```

See also

- Lexical grammar
- `var`
- Grammar and types guide

JavaScript › Reference › JavaScript error reference

SyntaxError: negated character class with strings in regular expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_negated_char_class_with_strings · MDN page last modified 2025-07-08

The JavaScript exception "negated character class with strings in regular expression" occurs when a v-mode character class is negated and may be able to match a string (more than one character).

Message

```
SyntaxError: Invalid regular expression: /[^\p{RGI_Emoji_Flag_Sequence}]/v: Negated
character class may contain strings (V8-based)
SyntaxError: negated character class with strings in regular expression (Firefox)
SyntaxError: Invalid regular expression: negated class set may contain strings (Safari)
```

Error type

SyntaxError

What went wrong?

In v mode, character classes are able to match more than 1 character. For example, `/[\q{abc}]/v` would match the sequence `"abc"`, and `/[\p{RGI_Emoji_Flag_Sequence}]/v` would match any character sequence that represents an emoji flag. However, negated character classes `[^...]` are not allowed to match strings, so `/[^\p{RGI_Emoji_Flag_Sequence}]/v` is invalid, because it's unclear how many characters it should match. For more information, see the v-mode character class reference.

Examples

Invalid cases

js · ✗ bad example

```
/[^\p{RGI_Emoji_Flag_Sequence}]/v;
```

Valid cases

js · ✓ good example

```
// Matches two characters that are not an emoji flag sequence
/(?!\p{RGI_Emoji_Flag_Sequence})../v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`
- Unicode character class escape: `\p{...}`, `\P{...}`

JavaScript › Reference › JavaScript error reference

SyntaxError: new keyword cannot be used with an optional chain

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_new_optional · MDN page last modified 2026-05-22

The JavaScript exception "new keyword cannot be used with an optional chain" occurs when the constructor of a new expression is an optional chain, or if there's an optional chain between the constructor and the parenthesized list of arguments.

Message

```
SyntaxError: Invalid optional chain from new expression (V8-based)
SyntaxError: new keyword cannot be used with an optional chain (Firefox)
```

(code continues on the next page)

(code continued from the previous page)

```
SyntaxError: Cannot call constructor in an optional chain. (Safari)
```

Error type

SyntaxError

What went wrong?

There are two ways to get this error. The first one is if the constructor expression is an optional chain expression, like this:

js · ✗ bad example

```
new Intl?.DateTimeFormat();
Number?.[parseMethod]`Hello, world!`;
```

The second one is if `?.` occurs between the constructor and the arguments list, like this:

js

```
new Intl.DateTimeFormat?.();
```

Optional `new` is specifically forbidden because its syntax is complicated (`new` with and without arguments), and the result is unclear (it would be the only case where `new` does not evaluate to an object value). You need to translate the optional chaining to its underlying condition (see optional chaining for more information).

js

```
const result =
  Intl.DateTimeFormat === null || Intl.DateTimeFormat === undefined
    ? undefined
    : new Intl.DateTimeFormat();
```

Remember that optional chaining only short-circuits within a parenthesized unit. If you parenthesize your constructor expression, the optional chaining will not cause an error, because now the constructor does not short-circuit and the result is clear (the constructor will produce `undefined` and then cause the `new` expression to throw).

js

```
new (Intl?.DateTimeFormat)(); // Throws if Intl?.DateTimeFormat is undefined
```

However this is a bit nonsensical anyway because optional chaining prevents errors inside the property access chain, but is then guaranteed to generate an error when calling `new`. You would probably still want to use a conditional check.

Note that optional chaining is only forbidden as the constructor expression. You can use optional chaining inside the argument list, or use optional chaining on the `new` expression as a whole.

js · ✓ good example

```
new Intl.DateTimeFormat(navigator?.languages);
new Intl.DateTimeFormat().resolvedOptions?.();
```

Note that there's no needs to use `?.` on the `new` expression itself: `new a()?.b`, because `new` is guaranteed to produce a non-nullish object value.

See also

- new
- Optional chaining (?.)
- Original discussion on whether "optional new" should be allowed

---

JavaScript › Reference › JavaScript error reference

SyntaxError: nothing to repeat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_nothing_to_repeat · MDN page last modified 2025-07-08

The JavaScript exception "nothing to repeat" or "invalid quantifier in regular expression" occurs when a quantifier in a regular expression is applied to nothing or applied to an assertion.

Message

```
SyntaxError: Invalid regular expression: /\b+/: Nothing to repeat (V8-based)
SyntaxError: Invalid regular expression: /(?=)+/u: Invalid quantifier (V8-based)
SyntaxError: nothing to repeat (Firefox)
SyntaxError: invalid quantifier in regular expression (Firefox)
SyntaxError: Invalid regular expression: nothing to repeat (Safari)
```

Error type

SyntaxError

What went wrong?

Quantifiers are used to specify how many times a character or group of characters can appear in a regular expression. For example, a{3} matches the character a exactly three times. Therefore, if the thing preceding the quantifier is not something that matches characters, the quantifier is invalid. For example: quantifiers at the start of a capturing group, at the start of a disjunction alternative, etc., cannot repeat anything. Assertions don't consume characters, so it also doesn't make sense to repeat them.

In Unicode-unaware mode, there's a deprecated syntax that allows the lookahead assertions to be quantified. This is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

js · ✗ bad example

```
/\b+/; // \b is a word boundary assertion, it doesn't consume characters
/(*hello*)/;
```

Valid cases

js · ✓ good example

```
/b+/; // b is a character, it can be repeated
/(\*hello\*)/; // Escape the asterisks to match them literally
```

See also

- Regular expressions
- Quantifier: *, +, ?, {n}, {n,}, {n,m}

JavaScript › Reference › JavaScript error reference

SyntaxError: numbers out of order in {} quantifier.

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_numbers_out_of_order_in_quantifier · MDN page last modified 2025-07-08

The JavaScript exception "numbers out of order in {} quantifier" occurs when a quantifier in a regular expression uses the `{n,m}` syntax but `m` is less than `n`.

Message

```
SyntaxError: Invalid regular expression: /1{2,1}/: numbers out of order in {} quantifier
(V8-based)
SyntaxError: numbers out of order in {} quantifier. (Firefox)
SyntaxError: Invalid regular expression: numbers out of order in {} quantifier (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `{n,m}` syntax in a regular expression is used to specify that the preceding item is to be matched at least `n` times, but not more than `m` times. If `m` is less than `n`, the quantifier is nonsensical because, for example, a character cannot appear at least 2 times but not more than 1 time. Therefore, an error is thrown.

Examples

Invalid examples

js · ✗ bad example

```
/1{2,1}/;
```

Valid examples

js · ✓ good example

```
/1{1,2}/;
```

See also

- Regular expressions
- Quantifier: *, +, ?, {n}, {n,}, {n,m}

SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_octal_escape_sequence · MDN page last modified 2025-07-08

The JavaScript exception "octal escape sequences can't be used in untagged template literals or in strict mode code" occurs when octal escape sequences are used in strict mode string literals or untagged template literals.

Message

```
SyntaxError: Octal escape sequences are not allowed in strict mode. (V8-based)
SyntaxError: \8 and \9 are not allowed in strict mode. (V8-based)
SyntaxError: Octal escape sequences are not allowed in template strings. (V8-based)
SyntaxError: \8 and \9 are not allowed in template strings. (V8-based)
SyntaxError: octal escape sequences can't be used in untagged template literals or in strict
mode code (Firefox)
SyntaxError: the escapes \8 and \9 can't be used in untagged template literals or in strict
mode code (Firefox)
SyntaxError: The only valid numeric escape in strict mode is '\0' (Safari)
```

Error type

`SyntaxError`

What went wrong?

The string escape sequence of the form `\` followed by any number of digits, except a single `0`, is deprecated. If you want to represent a character by its code point value, you should use the `\x` or `\u` escape sequence instead, such as `\x01` or `\u0001` instead of `\1`.

Untagged template literals are never allowed to contain octal escape sequences, whether in strict mode or not. However, *tagged* template literals can contain any form of escape sequence, and will cause the template array received by the tag function to contain `undefined`.

Examples

Octal escape sequences

js · ✗ bad example

```
"use strict";

"\251";

// SyntaxError: octal escape sequences can't be used in untagged template literals or in
strict mode code
```

Valid octal numbers

For octal escape sequences, you can use hexadecimal escape sequences instead:

js · ✓ good example

```
"\xA9";
```

If you want to represent some source text literally without interpreting any escape sequence, use `String.raw`:

js · ✓ good example

```
String.raw`\251`; // A string containing four characters
```

See also

- Lexical grammar

JavaScript › Reference › JavaScript error reference

SyntaxError: parameter after rest parameter

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Parameter_after_rest_parameter · MDN page last modified 2025-07-08

The JavaScript exception "parameter after rest parameter" occurs when a rest parameter is followed by anything else in a parameter list, including another rest parameter, a formal parameter, or a trailing comma.

Message

```
SyntaxError: Rest parameter must be last formal parameter (V8-based)
SyntaxError: parameter after rest parameter (Firefox)
SyntaxError: Unexpected token ','. Rest parameter should be the last parameter in a function
declaration. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A rest parameter must be the last parameter in a function definition. This is because the rest parameter collects all the remaining arguments passed to the function, so it doesn't make sense to have any parameters after it. The next non-whitespace character must be the closing parenthesis of the parameter list.

Examples

Invalid cases

js · ✗ bad example

```
function replacer(match, ...groups, offset, string) {}

function doSomething(
  arg1,
  arg2,
  ...otherArgs, // Accidental trailing comma
) {}
```

Valid cases

js · ✓ good example

```
function replacer(match, ...args) {
  const offset = args.at(-2);
  const string = args.at(-1);
}

function doSomething(arg1, arg2, ...otherArgs) {}
```

See also

- Functions
- Rest parameters

JavaScript › Reference › JavaScript error reference

SyntaxError: private fields can't be deleted

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_delete_private_fields · MDN page last modified 2025-07-08

The JavaScript exception "SyntaxError: private fields can't be deleted" occurs when `delete` is used on a private element of a class or an object.

Message

```
SyntaxError: Private fields can not be deleted (V8-based)
SyntaxError: private fields can't be deleted (Firefox)
SyntaxError: Cannot delete private field X (Safari)
```

Error type

`SyntaxError`

What went wrong?

There's code trying to `delete` a private element (field or method) of an object or a class. This is forbidden by JavaScript—private elements cannot be added or removed on the fly.

Examples

js · ✗ bad example

```
class MyClass {
  #myPrivateField;
  deleteIt() {
    delete this.#myPrivateField; // SyntaxError: private fields can't be deleted
  }
}
```

js · ✗ bad example

```
class MyClass {
  #myPrivateMethod() {
  }
  #deleteIt() {
    delete this.#myPrivateMethod; // SyntaxError: private fields can't be deleted
  }
}
```

See also

- Classes
- Private elements

JavaScript › Reference › JavaScript error reference

SyntaxError: property name __proto__ appears more than once in object literal

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Duplicate_proto · MDN page last modified 2025-07-08

The JavaScript exception "property name __proto__ appears more than once in object literal" occurs when an object literal contains multiple occurrences of the `__proto__` field, which is used to set the prototype of this new object.

Message

```
SyntaxError: Duplicate __proto__ fields are not allowed in object literals (V8-based)
SyntaxError: property name __proto__ appears more than once in object literal (Firefox)
SyntaxError: Attempted to redefine __proto__ property. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `__proto__` key, unlike other property keys, is a special syntax in an object literal. It is used to set the prototype of the object being created and is not allowed to appear more than once in an object literal. Note that this restriction only applies to the `__proto__` prototype setter syntax: if it actually has the effect of creating a property called `__proto__`, then it can appear multiple times. See prototype setter for the exact syntax restrictions.

Worth noting that the `__proto__` key in object literals is a special syntax and is not deprecated, unlike the `Object.prototype.__proto__` accessor property.

Examples

Invalid cases

js · ✗ bad example

```
const obj = { __proto__: {}, __proto__: { a: 1 } };
```

Valid cases

js · ✓ good example

```
// Only setting the prototype once
const obj = { __proto__: { a: 1 } };

// These syntaxes all create a property called "__proto__" and can coexist
// They would overwrite each other and the last one is actually used
const __proto__ = null;
const obj2 = {
  ["__proto__"]: {},
  __proto__,
  __proto__() {},
  get __proto__() {
    return 1;
  },
};
```

See also

- Object initializer
- Inheritance and the prototype chain

JavaScript › Reference › JavaScript error reference

SyntaxError: raw bracket is not allowed in regular expression with unicode flag

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Regex_raw_bracket · MDN page last modified 2025-07-08

The JavaScript exception "raw bracket is not allowed in regular expression with unicode flag" occurs when a Unicode-aware regular expression pattern contains a raw bracket ({, },]) that is not part of a quantifier or character class.

Message

```
SyntaxError: Invalid regular expression: /{/u: Lone quantifier brackets (V8-based)
SyntaxError: raw bracket is not allowed in regular expression with unicode flag (Firefox)
SyntaxError: Invalid regular expression: incomplete {} quantifier for Unicode pattern
(Safari)
SyntaxError: Invalid regular expression: unmatched ] or } bracket for Unicode pattern
(Safari)
```

Error type

SyntaxError

What went wrong?

In Unicode-unaware mode, {, }, and] that are not part of a quantifier or character class are treated as literal characters. This may hide errors in your code and is therefore deprecated and disallowed in Unicode-aware mode. You should either check whether you have invalid syntax or you should escape the characters to match them literally.

If { appears in a context that accepts a quantifier, it is treated as the start of a quantifier. If what follows it is not a valid quantifier, another syntax error, incomplete quantifier, is thrown.

Examples

Invalid cases

js · ✗ bad example

```
/{{MDN_Macro}}/u;
/\[sic]/u;
```

Valid cases

js · ✓ good example

```
// All { and } need to be escaped
/\{\{MDN_Macro\}\}/u;
// The ] needs to be escaped
/\[sic\]/u;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`
- Character escape: `\n`, `\u{...}`
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

JavaScript › Reference › JavaScript error reference

SyntaxError: redeclaration of formal parameter "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Redeclared_parameter · MDN page last modified 2025-07-08

The JavaScript exception "redeclaration of formal parameter" occurs when the same variable name occurs as a function parameter and is then redeclared using a `let` assignment in a function body again.

Message

```
SyntaxError: Identifier "x" has already been declared (V8-based)
SyntaxError: redeclaration of formal parameter "x" (Firefox)
SyntaxError: Cannot declare a let variable twice: 'x'. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The same variable name occurs as a function parameter and is then redeclared using a `let` assignment in a function body again. Redeclaring the same variable within the same function or block scope using `let` is not allowed in JavaScript.

Examples

Redeclared argument

In this case, the variable "arg" redeclares the argument.

js · ✗ bad example

```
function f(arg) {
  let arg = "foo";
}

// SyntaxError: redeclaration of formal parameter "arg"
```

If you want to change the value of "arg" in the function body, you can do so, but you do not need to declare the same variable again. In other words: you can omit the `let` keyword. If you want to create a new variable, you need to rename it as conflicts with the function parameter already.

js · ✓ good example

```
function f(arg) {
  arg = "foo";
}

function g(arg) {
  let bar = "foo";
}
```

See also

- `let`
- `const`
- `var`
- Grammar and types guide

JavaScript › Reference › JavaScript error reference

SyntaxError: reference to undeclared private field or method #x

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Undeclared_private_field_or_method · MDN page last modified 2025-07-08

The JavaScript exception "reference to undeclared private field or method #x" occurs when a private name is used, but this private name is not declared in the class scope.

Message

```
SyntaxError: Private field '#x' must be declared in an enclosing class (V8-based)
SyntaxError: reference to undeclared private field or method #x (Firefox)
SyntaxError: Cannot reference undeclared private names: "#x" (Safari)
```

Error type

`SyntaxError`

What went wrong?

Unlike normal string or symbol properties, which return `undefined` if the property does not exist, private names are very strict and can only be legally accessed if they actually exist. Accessing an undeclared private name will result in a syntax error, while accessing a private name that is declared but doesn't exist on the object will result in a type error.

Examples

Undeclared private field

You cannot access a private field that is not declared in the class scope.

js · ✗ bad example

```
class MyClass {
  doSomething() {
    console.log(this.#x);
  }
}
```

The same error occurs if you use the `in` operator to perform a check on an undeclared private field.

js · ✗ bad example

```
class MyClass {
  doSomething() {
    console.log(#x in this);
  }
}
```

These code are probably mistakes because it's impossible for `#x` to exist on `this` if it's not declared in the class scope. Note that you cannot dynamically *add* private elements to unrelated objects. You should either remove this code or declare the private field in the class scope.

js · ✓ good example

```
class MyClass {
  #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}
```

See also

- Classes
- Private elements

SyntaxError: rest parameter may not have a default

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Rest_with_default · MDN page last modified 2025-07-08

The JavaScript exception "rest parameter may not have a default" occurs when a rest parameter has a default value. Because the rest parameter always creates an array, the default value would never apply.

Message

```
SyntaxError: Rest parameter may not have a default initializer (V8-based)
SyntaxError: rest parameter may not have a default (Firefox)
SyntaxError: Unexpected token '='. Expected a ')' or a ',' after a parameter declaration.
(Safari)
```

Error type

`SyntaxError`

What went wrong?

A default parameter gives a parameter a default value if the argument is not passed or passed as `undefined`. A rest parameter collects all the remaining arguments passed to the function and always creates an array. Therefore, it doesn't make sense to have a default value for a rest parameter.

Examples

Invalid cases

js · ✗ bad example

```
function doSomething(...args = []) {}
```

Valid cases

js · ✓ good example

```
function doSomething(...args) {
  // args is always an array
}
```

See also

- Functions
- Default parameters
- Rest parameters

JavaScript › Reference › JavaScript error reference

SyntaxError: return not in function

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_return · MDN page last modified 2025-07-08

The JavaScript exception "return not in function" occurs when a `return` statement is called outside of a function.

Message

```
SyntaxError: Illegal return statement (V8-based)
SyntaxError: return not in function (Firefox)
SyntaxError: Return statements are only valid inside functions. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

A `return` statement is called outside of a function. Maybe there are missing curly braces somewhere? The `return` statement must be in a function, because it ends function execution and specifies a value to be returned to the function caller.

Examples

Missing curly braces

js · ✗ bad example

```
function cheer(score) {
  if (score === 147)
    return "Maximum!";
  }
  if (score > 100) {
    return "Century!";
  }
}

// SyntaxError: return not in function
```

The curly braces look correct at a first glance, but this code snippet is missing a `{` after the first `if` statement. Correct would be:

js · ✓ good example

```
function cheer(score) {
  if (score === 147) {
    return "Maximum!";
  }
  if (score > 100) {
    return "Century!";
  }
}
```

See also

- return

JavaScript › Reference › JavaScript error reference

SyntaxError: setter functions must have one argument

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Setter_one_argument · MDN page last modified 2025-07-08

The JavaScript exception "setter functions must have one argument" occurs when a setter is declared and the parameter list is not consisted of exactly one formal parameter.

Message

```
SyntaxError: Setter must have exactly one formal parameter. (V8-based)
SyntaxError: Setter function argument must not be a rest parameter (V8-based)
SyntaxError: setter functions must have one argument (Firefox)
SyntaxError: Unexpected token ','. setter functions must have one parameter. (Safari)
SyntaxError: Unexpected token '...'. Expected a parameter pattern or a ')' in parameter
list. (Safari)
```

Error type

SyntaxError

What went wrong?

The set property syntax looks like a function, but it is stricter and not all function syntax is allowed. A setter is always invoked with exactly one argument, so defining it with any other number of parameters is likely an error. This parameter can be destructured or have a default value, but it cannot be a rest parameter.

Note that this error only applies to property setters using the `set` syntax. If you define the setter using `Object.defineProperty()`, etc., the setter is defined as a normal function, although it's likely still an error if the setter expects any other number of arguments, as it will be called with exactly one.

Examples

Invalid cases

js · ✗ bad example

```
const obj = {
  set value() {
    this._value = Math.random();
  },
};
```

Valid cases

js · ✓ good example

```
// You must declare one parameter, even if you don't use it
const obj = {
  set value(_ignored) {
    this._value = Math.random();
  },
};

// You can also declare a normal method instead
const obj = {
  setValue() {
    this._value = Math.random();
  },
};
```

See also

- Functions
- set

JavaScript › Reference › JavaScript error reference

SyntaxError: string literal contains an unescaped line break

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/String_literal_EOL · MDN page last modified 2025-07-08

The JavaScript error "string literal contains an unescaped line break" occurs when there is an unterminated string literal somewhere. String literals must be enclosed by single (') or double (") quotes and cannot split across multiple lines.

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: '' string literal contains an unescaped line break (Firefox)
SyntaxError: Unexpected EOF (Safari)
```

Error type

SyntaxError

What went wrong?

There is an unterminated string literal somewhere. String literals must be enclosed by single (') or double (") quotes. JavaScript makes no distinction between single-quoted strings and double-quoted strings. Escape sequences work in strings created with either single or double quotes. To fix this error, check if:

- you have opening and closing quotes (single or double) for your string literal,
- you have escaped your string literal correctly,
- your string literal isn't split across multiple lines.

Examples

Multiple lines

You can't split a string across multiple lines like this in JavaScript:

js · ✗ bad example

```
const longString = "This is a very long string which needs
                    to wrap across multiple lines because
                    otherwise my code is unreadable.";
// SyntaxError: unterminated string literal
```

Instead, use the + operator, a backslash, or template literals. The + operator variant looks like this:

js · ✓ good example

```
const longString =
  "This is a very long string which needs " +
  "to wrap across multiple lines because " +
  "otherwise my code is unreadable.";
```

Or you can use the backslash character ("\") at the end of each line to indicate that the string will continue on the next line. Make sure there is no space or any other character after the backslash (except for a line break), or as an indent; otherwise it will not work. That form looks like this:

js · ✓ good example

```
const longString =
  "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

Another possibility is to use template literals.

js · ✓ good example

```
const longString = `This is a very long string which needs
to wrap across multiple lines because
otherwise my code is unreadable.`;
```

See also

- string literal
- Template literals

JavaScript › Reference › JavaScript error reference

SyntaxError: super() is only valid in derived class constructors

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_super_call · MDN page last modified 2026-09-14

The JavaScript exception "super() is only valid in derived class constructors" occurs when the `super()` call is used somewhere that's not the body of a constructor in a class with `extends` keyword.

Message

```
SyntaxError: 'super' keyword unexpected here (V8-based)
SyntaxError: super() is only valid in derived class constructors (Firefox)
SyntaxError: super is not valid in this context. (Safari)
```

Error type

SyntaxError

What went wrong?

The `super()` call is used to invoke the base constructor of a derived class, so the base class can initialize the `this` object. Using it anywhere else doesn't make sense.

`super()` can also be defined in an arrow function that's nested within the constructor. However, it cannot be defined in any other kind of function.

Examples

Invalid cases

You cannot call `super()` if the class has no `extends`, because there's no base class to call:

js · ✗ bad example

```
class Base {
  constructor() {
    super();
  }
}
```

You cannot call `super()` in a class method, even if that method is called from the constructor:

js · ✗ bad example

```
class Base {}

class Derived extends Base {
  constructor() {
    this.init();
  }

  init() {
    super();
  }
}
```

You cannot call `super()` in a function, even if the function is used as a constructor:

js · ✗ bad example

```
function Base(x) {
  this.x = x;
}

function Derived() {
  super(1);
}

Object.setPrototypeOf(Derived.prototype, Base.prototype);
Object.setPrototypeOf(Derived, Base);
```

Valid cases

You can call `super()` before calling any other method in the constructor:

js · ✓ good example

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    this.init();
  }

  init() {
    // …
  }
}
```

You can call `super()` in an arrow function that's nested within the constructor:

js · ✓ good example

```
class Base {}

class Derived extends Base {
  constructor() {
    const init = () => {
      super();
    };

    init();
  }
}
```

See also

- Classes
- `super`

SyntaxError: tagged template cannot be used with optional chain

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_optional_template · MDN page last modified 2025-07-08

The JavaScript exception "tagged template cannot be used with optional chain" occurs when the tag expression of a tagged template literal is an optional chain, or if there's an optional chain between the tag and the template.

Message

```
SyntaxError: Invalid tagged template on optional chain (V8-based)
SyntaxError: tagged template cannot be used with optional chain (Firefox)
SyntaxError: Cannot use tagged templates in an optional chain. (Safari)
```

Error type

SyntaxError

What went wrong?

There are two ways to get this error. The first one is if the tag expression is an optional chain expression, like this:

js · ✗ bad example

```
String?.raw`Hello, world!`;
console.log?.()`Hello, world!`;
Number?.[parseMethod]`Hello, world!`;
```

The second one is if `?.` occurs between the tag and the template, like this:

js · ✗ bad example

```
String.raw?.`Hello, world!`;
```

Optional chaining in the tag is specifically forbidden because there's no great use case for it, and what the result is expected to be is unclear (should it be `undefined` or the template's value as if it's untagged?). You need to translate the optional chaining to its underlying condition (see optional chaining for more information).

js · ✓ good example

```
const result =
  String.raw === null || String.raw === undefined
    ? undefined
    : String.raw`Hello, world!`;
```

Remember that optional chaining only short-circuits within a parenthesized unit. If you parenthesize your tag expression, the optional chaining will not cause an error, because now the tag does not short-circuit and the result is clear (the tag will produce `undefined` and then cause the tagged template to throw).

js

```
(console?.log)`Hello, world!`; // Throws if console?.log is undefined
```

However this is a bit nonsensical anyway because optional chaining prevents errors inside the property access chain, but is then guaranteed to generate an error when calling the template tag. You would probably still want to use a conditional check.

Note that optional chaining is only forbidden as the tag expression. You can use optional chaining inside the embedded expressions, or use optional chaining on the tagged template expression as a whole.

js · ✓ good example

```
console.log`Hello, ${true.constructor?.name}!`; // ['Hello, ', '!', raw: Array(2)] 'Boolean'
console.log`Hello`?.toString(); // undefined
```

See also

- Template literals
- Optional chaining (`?.`)
- Original discussion on whether optional chaining should be allowed in tagged template literals

SyntaxError: Unexpected '#' used outside of class body

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Hash_outside_class · MDN page last modified 2025-07-08

The JavaScript exception "Unexpected '#' used outside of class body" occurs when a hash ("#") is encountered in an unexpected context, most notably outside of a class declaration. Hashes are valid at the beginning of a file as a hashbang comment, or inside of a class as part of a private field. You may encounter this error if you forget the quotation marks when trying to access a DOM identifier as well.

Message

```
SyntaxError: Unexpected '#' used outside of class body.
```

Error type

SyntaxError

What went wrong?

We encountered a # somewhere unexpected. This may be due to code moving around and no longer being part of a class, a hashbang comment found on a line other than the first line of a file, or accidentally forgetting the quotation marks around a DOM identifier.

Examples

Missing quotation marks

For each case, there might be something slightly wrong. For example

js · ✗ bad example

```
document.querySelector(#some-element)
```

This can be fixed via

js · ✓ good example

```
document.querySelector("#some-element");
```

Outside of a class

js · ✗ bad example

```
class ClassWithPrivateField {
  #privateField;

  constructor() {}
}

this.#privateField = 42;
```

This can be fixed by moving the private field back into the class

js · ✓ good example

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    this.#privateField = 42;
  }
}
```

See also

- SyntaxError

JavaScript › Reference › JavaScript error reference

SyntaxError: Unexpected token

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Unexpected_token · MDN page last modified 2026-08-21

The JavaScript exceptions "unexpected token" occur when the parser does not see a token it recognizes at the given position, so it cannot make sense of the structure of the program. This might be a simple typo.

Message

```
SyntaxError: Unexpected token ';' (V8-based)
SyntaxError: Unexpected identifier 'x' (V8-based)
SyntaxError: Unexpected number (V8-based)
SyntaxError: Unexpected string (V8-based)
SyntaxError: Unexpected regular expression (V8-based)
SyntaxError: Unexpected template string (V8-based)

SyntaxError: unexpected token: identifier (Firefox)
SyntaxError: expected expression, got "x" (Firefox)
SyntaxError: expected property name, got "x" (Firefox)
SyntaxError: expected target, got "x" (Firefox)
SyntaxError: expected meta, got "x" (Firefox)
SyntaxError: expected rest argument name, got "x" (Firefox)
SyntaxError: expected closing parenthesis, got "x" (Firefox)
```

Error type

SyntaxError

What went wrong?

A specific language construct was expected, but something else was provided. This might be a simple typo.

Examples

Expression expected

For example, when chaining expressions, trailing commas are not allowed.

js · ✗ bad example

```
for (let i = 0; i < 5,; ++i) {
  console.log(i);
}
// Uncaught SyntaxError: expected expression, got ';'
```

Correct would be omitting the comma or adding another expression:

js · ✓ good example

```
for (let i = 0; i < 5; ++i) {
  console.log(i);
}
```

Not enough parentheses

Sometimes, you leave out parentheses around `if` statements:

js · ✗ bad example

```
function round(n, upperBound, lowerBound) {
if (n > upperBound) || (n < lowerBound) { // Missing parentheses here!
    throw new Error(`Number ${n} is more than ${upperBound} or less than ${lowerBound}`);
  } else if (n < (upperBound + lowerBound) / 2) {
    return lowerBound;
  } else {
    return upperBound;
  }
} // SyntaxError: expected expression, got '||'
```

The parentheses may look correct at first, but note how the `||` is outside the parentheses. Correct would be putting parentheses around the `||`:

js · ✓ good example

```
function round(n, upperBound, lowerBound) {
  if ((n > upperBound) || (n < lowerBound)) {
    throw new Error(
      `Number ${n} is more than ${upperBound} or less than ${lowerBound}`,
    );
  } else if (n < (upperBound + lowerBound) / 2) {
    return lowerBound;
  } else {
    return upperBound;
  }
}
```

A structure error further up confused the meaning

Sometimes, the error is caused by some structure issues not directly next to the error location, so you need to look around for potential errors. For example, you intended to declare a method of an object, but you declared it as a property instead:

js · ✗ bad example

```
const MyComponent = {
  mounted: {
    document.getElementById("app").classList.add("loaded");
  }
}
```

The `.` after `document` is unexpected, because JavaScript is parsing the `{}` as an object literal instead of a function body, so it expects a `:`. The problem is solved by declaring `mounted` as a function.

js · ✓ good example

```
const MyComponent = {
  mounted() {
    document.getElementById("app").classList.add("loaded");
  }
}
```

See also

- SyntaxError

SyntaxError: unlabeled break must be inside loop or switch

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_break · MDN page last modified 2025-07-08

The JavaScript exception "unlabeled break must be inside loop or switch" occurs when a break statement is not inside a loop or a switch statement.

Message

```
SyntaxError: Illegal break statement (V8-based)
SyntaxError: unlabeled break must be inside loop or switch (Firefox)
SyntaxError: 'break' is only valid inside a switch or loop statement. (Safari)
```

Error type

SyntaxError.

What went wrong?

break statements can be used to exit a loop or a switch statement, and using them elsewhere is a syntax error. Alternatively, you can provide a label to the break statement to break out of any statement with that label — however, if the label does not reference a containing statement, another error SyntaxError: label not found will be thrown.

Examples

Unsyntactic break

break cannot be used outside switch or loops.

js · ✗ bad example

```
let score = 0;

function increment() {
  if (score === 100)
    break; // SyntaxError: unlabeled break must be inside loop or switch
  }
  score++;
}
```

Maybe instead of `break`, you intend to use `return` to early-terminate a function.

js · ✓ good example

```
let score = 0;

function increment() {
  if (score === 100) {
    return;
  }
  score++;
}
```

Using break in callbacks

`break` cannot be used in callbacks, even if the callback is called from a loop.

js · ✗ bad example

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

while (containingIndex < matrix.length) {
  matrix[containingIndex].forEach((value) => {
    if (value === 5) {
      break; // SyntaxError: unlabeled break must be inside loop or switch
    }
  });
  containingIndex++;
}
```

Instead, refactor the code so the `break` is used outside the callback.

js · ✓ good example

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

outer: while (containingIndex < matrix.length) {
  for (const value of matrix[containingIndex]) {
    if (value === 5) {
      break outer;
    }
  }
  containingIndex++;
}
```

js · ✓ good example

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

while (containingIndex < matrix.length) {
  if (matrix[containingIndex].includes(5)) {
    break;
  }
  containingIndex++;
}
```

There's no way to early-terminate a `forEach()` loop. You can use `some()` instead, or convert it to a `for...of` loop.

js · ✗ bad example

```
array.forEach((value) => {
  if (value === 5) {
    break; // SyntaxError: unlabeled break must be inside loop or switch
  }
  // do something with value
});
```

js · ✓ good example

```
array.some((value) => {
  if (value === 5) {
    return true;
  }
  // do something with value
  return false;
});
```

js · ✓ good example

```
for (const value of array) {
  if (value === 5) {
    break;
  }
  // do something with value
}
```

See also

- `break`

JavaScript › Reference › JavaScript error reference

SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Unparenthesized_unary_expr_lhs_exponentiation · MDN page last modified 2025-07-08

The JavaScript exception "unparenthesized unary expression can't appear on the left-hand side of '**'" occurs when a unary operator (one of `typeof`, `void`, `delete`, `await`, `!`, `~`, `+`, `-`) is used on the left operand of the exponentiation operator without parentheses.

Message

```
SyntaxError: Unary operator used immediately before exponentiation expression. Parenthesis
must be used to disambiguate operator precedence (V8-based)
SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'
(Firefox)
SyntaxError: Unexpected token '**'. Ambiguous unary expression in the left hand side of the
exponentiation expression; parentheses must be used to disambiguate the expression. (Safari)
```

Error type

`SyntaxError`

What went wrong?

You likely wrote something like this:

js · ✗ bad example

```
-a ** b
```

Whether it should be evaluated as `(-a) ** b` or `-(a ** b)` is ambiguous. In mathematics, $-x^2$ means `-(x ** 2)` — and that's how many languages, including Python, Haskell, and PHP, handle it. But making the unary minus operator take precedence over `**` breaks symmetry with `a ** -b`, which is unambiguously `a ** (-b)`. Therefore, the language forbids this syntax and requires you to parenthesize either side to resolve the ambiguity.

js · ✓ good example

```
(-a) ** b
-(a ** b)
```

Other unary operators cannot be the left-hand side of exponentiation either.

js · ✗ bad example

```
await a ** b
!a ** b
+a ** b
~a ** b
```

Examples

When writing complex math expressions involving exponentiation, you may write something like this:

js · ✗ bad example

```
function taylorSin(x) {
  return (n) => (-1 ** n * x ** (2 * n + 1)) / factorial(2 * n + 1);
  // SyntaxError: unparenthesized unary expression can't appear on the left-hand side of
'**'
}
```

However, the `-1 ** n` part is illegal in JavaScript. Instead, parenthesize the left operand:

js · ✓ good example

```
function taylorSin(x) {
  return (n) => ((-1) ** n * x ** (2 * n + 1)) / factorial(2 * n + 1);
}
```

This also makes the code's intent much clearer to other readers.

See also

- Exponentiation (**)
- Operator precedence
- Original discussion of exponentiation operator precedence on esdiscuss.org

---

JavaScript › Reference › JavaScript error reference

SyntaxError: use of super property/member accesses only valid within methods or eval code within methods

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Bad_super_prop · MDN page last modified 2025-07-08

The JavaScript exception "use of super property/member accesses only valid within methods or eval code within methods" occurs when the `super.x` or `super[x]` syntax is used outside of a method.

Message

```
SyntaxError: 'super' keyword unexpected here (V8-based)
SyntaxError: use of super property accesses only valid within methods or eval code within
methods (Firefox)
SyntaxError: super is not valid in this context. (Safari)
```

Error type

SyntaxError

What went wrong?

The `super.x` syntax is used to access properties on the prototype of the current object. It can be used in methods of both object literals and classes, field initializers, and static initialization blocks, but not in other contexts.

Examples

Invalid cases

You can't use `super.x` outside of a method in an object:

js · ✗ bad example

```
const obj = {
  __proto__: { x: 1 },
  x: super.x, // SyntaxError: use of super property accesses only valid within methods or
eval code within methods
};
```

You can't use `super.x` in a function, even if that function has the effect of being a method:

js · ✗ bad example

```
function getX() {
  return super.x; // SyntaxError: use of super property accesses only valid within methods
or eval code within methods
}

const obj = {
  getX,
  getX2: function () {
    return super.x; // SyntaxError: use of super property accesses only valid within methods
or eval code within methods
  },
};

class Derived extends Base {
  getX = () => super.x;
}
```

Valid cases

You can use `super.x` in a method:

js · ✓ good example

```
class Base {
  x = 1;
}

class Derived extends Base {
  getX() {
    return super.x;
  }
}
```

You can use `super.x` in a field initializer:

js · ✓ good example

```
class Derived extends Base {
  x = super.x;
}
```

You can use `super.x` in object methods too:

js · ✓ good example

```
const obj = {
  __proto__: { x: 1 },
  getX() {
    return super.x;
  },
};
```

See also

- Classes
- super

SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_source_map_pragma · MDN page last modified 2025-07-08

The JavaScript warning "Using `//@` to indicate sourceURL pragmas is deprecated. Use `//#` instead" occurs when there is a deprecated source map syntax in a JavaScript source.

Message

```
Warning: SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead

Warning: SyntaxError: Using //@ to indicate sourceMappingURL pragmas is deprecated. Use //#
instead
```

Error type

A warning that a `SyntaxError` occurred. JavaScript execution won't be halted.

What went wrong?

There is a deprecated source map syntax in a JavaScript source.

JavaScript sources are often combined and minified to make delivering them from the server more efficient. With source maps, the debugger can map the code being executed to the original source files.

The source map specification changed the syntax due to a conflict with IE whenever it was found in the page after `//@cc_on` was interpreted to turn on conditional compilation in the IE JScript engine. The conditional compilation comment in IE is a little known feature, but it broke source maps with jQuery and other libraries.

Examples

Deprecated syntax

Syntax with the "@" sign is deprecated.

js · ✗ bad example

```
//@ sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Standard syntax

Use the "#" sign instead.

js · ✓ good example

```
//# sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Or, alternatively, you can set a `SourceMap` header to your JavaScript file to avoid having a comment at all:

http · ✓ good example

```
SourceMap: /path/to/file.js.map
```

See also

- Use a source map in the Firefox source docs
- Introduction to JavaScript source maps on developer.chrome.com (2012)
- `SourceMap`

JavaScript › Reference › JavaScript error reference

TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Deprecated_caller_or_arguments_usage · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "'caller', 'callee', and 'arguments' properties may not be accessed on strict mode functions or the arguments objects for calls to them" occurs when the deprecated `arguments.callee`, `Function.prototype.caller`, or `Function.prototype.arguments` properties are used.

Message

```
TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict mode
functions or the arguments objects for calls to them (V8-based & Firefox)
TypeError: 'arguments', 'callee', and 'caller' cannot be accessed in this context. (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

In strict mode, the `arguments.callee`, `Function.prototype.caller`, or `Function.prototype.arguments` properties are used and shouldn't be. They are deprecated, because they leak the function caller, are non-standard, hard to optimize and potentially a performance-harmful feature.

Examples

Deprecated function.caller or arguments.callee

`Function.prototype.caller` and `arguments.callee` are deprecated (see the reference articles for more information).

js · ✗ bad example

```
"use strict";

function myFunc() {
  if (myFunc.caller === null) {
    return "The function was called from the top!";
  }
  return `This function's caller was ${myFunc.caller}`;
}

myFunc();
// TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict
mode functions or the arguments objects for calls to them
```

Function.prototype.arguments

`Function.prototype.arguments` is deprecated (see the reference article for more information).

js · ✗ bad example

```
"use strict";

function f(n) {
  g(n - 1);
}

function g(n) {
  console.log(`before: ${g.arguments[0]}`);
  if (n > 0) {
    f(n);
  }
  console.log(`after: ${g.arguments[0]}`);
}

f(2);

console.log(`returned: ${g.arguments}`);
// TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict
mode functions or the arguments objects for calls to them
```

See also

- Deprecated and obsolete features
- Strict mode
- `Function.prototype.arguments`
- `Function.prototype.caller`
- `arguments.callee`

JavaScript › Reference › JavaScript error reference

TypeError: 'x' is not iterable

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/is_not_iterable · MDN page last modified 2026-08-21

The JavaScript exception "is not iterable" occurs when the value which is spread into an array or function call, given as the right-hand side of `for...of`, as an argument of a function such as `Promise.all` or `Set()`, or as the right-hand side of an array destructuring, is not an iterable object. This error is also encountered when `Array.fromAsync()` or `for await...of` is used with a non-async iterable.

Message

```
TypeError: Spread syntax requires ...iterable[Symbol.iterator] to be a function (V8-based &
Safari)
TypeError: %Array%.from requires that the property of the first argument,
items[Symbol.iterator], when exists, be a function (V8-based & Safari)
TypeError: Array.fromAsync requires that the property of the first argument,
items[Symbol.asyncIterator], when exists, be a function (V8-based & Safari)
TypeError: object is not iterable (cannot read property Symbol(Symbol.iterator)) (V8-based)
TypeError: x is not async iterable (V8-based)
TypeError: x is not iterable (V8-based & Firefox)
TypeError: undefined is not a function (near '...y of x...') (Safari)
TypeError: Array.from: no function (Safari)
TypeError: Type error (Safari)
```

Error type

`TypeError`

What went wrong?

The value which is spread into an array or function call, given as the right-hand side of `for...of`, or as an argument of a function such as `Promise.all` or `Set()`, or as the source of an array destructuring pattern, is not an iterable object. An iterable can be a built-in iterable type such as `Array`, `String` or `Map`, a generator result, or an object implementing the iterable protocol.

js

```
const nonIterable1 = {};
const nonIterable2 = { [Symbol.iterator]: 1 };

[...nonIterable1];
Math.max(...nonIterable1);
for (const x of nonIterable1);
new Set(nonIterable1);
Array.from(nonIterable2);
new Int8Array(nonIterable2);
const [] = nonIterable1;
```

Examples

Array destructuring a non-iterable

js · ✗ bad example

```
const myObj = { arrayOrObjProp1: {}, arrayOrObjProp2: [42] };

const {
  arrayOrObjProp1: [value1],
  arrayOrObjProp2: [value2],
} = myObj; // TypeError: object is not iterable

console.log(value1, value2);
```

The non-iterable might turn to be `undefined` in some runtime environments.

Iterating over Object properties

In JavaScript, `Object`s are not iterable unless they implement the iterable protocol. Therefore, you cannot use `for...of` to iterate over the properties of an object.

js · ✗ bad example

```
const obj = { France: "Paris", England: "London" };
for (const p of obj) {
  // …
} // TypeError: obj is not iterable
```

Instead you have to use `Object.keys` or `Object.entries`, to iterate over the properties or entries of an object.

js · ✓ good example

```
const obj = { France: "Paris", England: "London" };
// Iterate over the property names:
for (const country of Object.keys(obj)) {
  const capital = obj[country];
  console.log(country, capital);
}

for (const [country, capital] of Object.entries(obj)) {
  console.log(country, capital);
}
```

Another option for this use case might be to use a `Map`:

js · ✓ good example

```
const map = new Map();
map.set("France", "Paris");
map.set("England", "London");
// Iterate over the property names:
for (const country of map.keys()) {
  const capital = map.get(country);
  console.log(country, capital);
}
```

(code continues on the next page)

(code continued from the previous page)

```
for (const capital of map.values()) {
  console.log(capital);
}

for (const [country, capital] of map.entries()) {
  console.log(country, capital);
}
```

Iterating over a generator

Generator functions are functions you call to produce an iterable object.

js · ✗ bad example

```
function* generate(a, b) {
  yield a;
  yield b;
}

for (const x of generate) {
  console.log(x);
} // TypeError: generate is not iterable
```

When they are not called, the `Function` object corresponding to the generator is callable, but not iterable. Calling a generator produces an iterable object which will iterate over the values yielded during the execution of the generator.

js · ✓ good example

```
function* generate(a, b) {
  yield a;
  yield b;
}

for (const x of generate(1, 2)) {
  console.log(x);
}
```

Iterating over a custom iterable

Custom iterables can be created by implementing the `Symbol.iterator` method. You must be certain that your iterator method returns an object which is an iterator, which is to say it must have a next method.

js · ✗ bad example

```
const myEmptyIterable = {
  [Symbol.iterator]() {
    return []; // [] is iterable, but it is not an iterator — it has no next method.
  },
};

Array.from(myEmptyIterable); // TypeError: myEmptyIterable is not iterable
```

Here is a correct implementation:

js · ✓ good example

```
const myEmptyIterable = {
  [Symbol.iterator]() {
    return [][Symbol.iterator]();
  },
};

Array.from(myEmptyIterable); // []
```

See also

- Iterable protocol
- `Object.keys`
- `Object.entries`
- `Map`
- Generator functions
- for...of

JavaScript › Reference › JavaScript error reference

TypeError: "x" is (not) "y"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Unexpected_type · MDN page last modified 2025-07-08

The JavaScript exception "*x* is (not) *y*" occurs when there was an unexpected type. Oftentimes, unexpected `undefined` or `null` values.

Message

```
TypeError: Cannot read properties of undefined (reading 'x') (V8-based)
TypeError: "x" is undefined (Firefox)
TypeError: "undefined" is not an object (Firefox)
TypeError: undefined is not an object (evaluating 'obj.x') (Safari)

TypeError: "x" is not a symbol (V8-based & Firefox)
TypeError: Symbol.keyFor requires that the first argument be a symbol (Safari)
```

Error type

`TypeError`.

What went wrong?

There was an unexpected type. This occurs oftentimes with `undefined` or `null` values.

Also, certain methods, such as `Object.create()` or `Symbol.keyFor()`, require a specific type, that must be provided.

Examples

Invalid cases

You cannot invoke a method on an `undefined` or `null` variable.

js · ✗ bad example

```
const foo = undefined;
foo.substring(1); // TypeError: foo is undefined

const foo2 = null;
foo2.substring(1); // TypeError: foo2 is null
```

Certain methods might require a specific type.

js · ✗ bad example

```
const foo = {};
Symbol.keyFor(foo); // TypeError: foo is not a symbol

const foo2 = "bar";
Object.create(foo2); // TypeError: "foo2" is not an object or null
```

Fixing the issue

To fix null pointer to `undefined` or `null` values, you can test if the value is `undefined` or `null` first.

js · ✓ good example

```
if (foo !== undefined && foo !== null) {
  // Now we know that foo is defined, we are good to go.
}
```

Or, if you are confident that `foo` will not be another falsy value like `""` or `0`, or if filtering those cases out is not an issue, you can simply test for its truthiness.

js · ✓ good example

```
if (foo) {
  // Now we know that foo is truthy, it will necessarily not be null/undefined.
}
```

See also

- `undefined`
- `null`

JavaScript › Reference › JavaScript error reference

TypeError: "x" is not a constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Not_a_constructor · MDN page last modified 2025-07-08

The JavaScript exception "is not a constructor" occurs when there was an attempt to use an object or a variable as a constructor, but that object or variable is not a constructor.

Message

```
TypeError: x is not a constructor (V8-based & Firefox & Safari)
```

Error type

TypeError

What went wrong?

There was an attempt to use an object or a variable as a constructor, but that object or variable is not a constructor. See constructor or the new operator for more information on what a constructor is.

There are many global objects, like String or Array, which are constructable using new. However, some global objects are not and their properties and methods are static. The following JavaScript standard built-in objects are not a constructor: Math, JSON, Symbol, Reflect, Intl, Atomics.

Generator functions cannot be used as constructors either.

Examples

Invalid cases

js · ✗ bad example

```
const Car = 1;
new Car();
// TypeError: Car is not a constructor

new Math();
// TypeError: Math is not a constructor

new Symbol();
// TypeError: Symbol is not a constructor

function* f() {}
const obj = new f();
// TypeError: f is not a constructor
```

A car constructor

Suppose you want to create an object type for cars. You want this type of object to be called Car, and you want it to have properties for make, model, and year. To do this, you would write the following function:

js

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
```

Now you can create an object called myCar as follows:

js

```
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

In Promises

When returning an immediately-resolved or immediately-rejected Promise, you do not need to create a `new Promise(...)` and act on it. Instead, use the `Promise.resolve()` or `Promise.reject()` static methods.

This is not legal (the `Promise` constructor is not being called correctly) and will throw a `TypeError: this is not a constructor` exception:

js · ✗ bad example

```
function fn() {
  return new Promise.resolve(true);
}
```

This is legal, but unnecessarily long:

js

```
function fn() {
  return new Promise((resolve, reject) => {
    resolve(true);
  });
}
```

Instead, return the static method:

js · ✓ good example

```
function resolveAlways() {
  return Promise.resolve(true);
}

function rejectAlways() {
  return Promise.reject(new Error());
}
```

See also

- constructor
- new

JavaScript › Reference › JavaScript error reference

TypeError: "x" is not a function

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Not_a_function · MDN page last modified 2025-07-08

The JavaScript exception "is not a function" occurs when there was an attempt to call a value from a function, but the value is not actually a function.

Message

```
TypeError: "x" is not a function. (V8-based & Firefox & Safari)
```

Error type

TypeError

What went wrong?

It attempted to call a value from a function, but the value is not actually a function. Some code expects you to provide a function, but that didn't happen.

Maybe there is a typo in the function name? Maybe the object you are calling the method on does not have this function? For example, JavaScript Objects have no map function, but the JavaScript Array object does.

There are many built-in functions in need of a (callback) function. You will have to provide a function in order to have these methods working properly:

- When working with Array or TypedArray objects:
  - Array.prototype.every(), Array.prototype.some(), Array.prototype.forEach(), Array.prototype.map(), Array.prototype.filter(), Array.prototype.reduce(), Array.prototype.reduceRight(), Array.prototype.find()
- When working with Map and Set objects:
  - Map.prototype.forEach() and Set.prototype.forEach()

Examples

A typo in the function name

In this case, which happens way too often, there is a typo in the method name:

js · ✗ bad example

```
const x = document.getElementByID("foo");
// TypeError: document.getElementByID is not a function
```

The correct function name is getElementById:

js · ✓ good example

```
const x = document.getElementById("foo");
```

Function called on the wrong object

For certain methods, you have to provide a (callback) function and it will work on specific objects only. In this example, Array.prototype.map() is used, which will work with Array objects only.

js · ✗ bad example

```
const obj = { a: 13, b: 37, c: 42 };

obj.map((num) => num * 2);

// TypeError: obj.map is not a function
```

Use an array instead:

js · ✓ good example

```
const numbers = [1, 4, 9];

numbers.map((num) => num * 2); // [2, 8, 18]
```

Function shares a name with a pre-existing property

Sometimes when making a class, you may have a property and a function with the same name. Upon calling the function, the compiler thinks that the function ceases to exist.

js · ✗ bad example

```
function Dog() {
  this.age = 11;
  this.color = "black";
  this.name = "Ralph";
  return this;
}

Dog.prototype.name = function (name) {
  this.name = name;
  return this;
};

const myNewDog = new Dog();
myNewDog.name("Cassidy"); // TypeError: myNewDog.name is not a function
```

Use a different property name instead:

js · ✓ good example

```
function Dog() {
  this.age = 11;
  this.color = "black";
  this.dogName = "Ralph"; // Using this.dogName instead of .name
  return this;
}

Dog.prototype.name = function (name) {
  this.dogName = name;
  return this;
};

const myNewDog = new Dog();
myNewDog.name("Cassidy"); // Dog { age: 11, color: 'black', dogName: 'Cassidy' }
```

Using parentheses for multiplication

In math, you can write 2 × (3 + 5) as 2*(3 + 5) or just 2(3 + 5).

Using the latter will throw an error:

js · ✗ bad example

```
const sixteen = 2(3 + 5);
console.log(`2 x (3 + 5) is ${sixteen}`);
// Uncaught TypeError: 2 is not a function
```

You can correct the code by adding a * operator:

js · ✓ good example

```
const sixteen = 2 * (3 + 5);
console.log(`2 x (3 + 5) is ${sixteen}`);
// 2 x (3 + 5) is 16
```

Import the exported module correctly

Ensure you are importing the module correctly.

An example helpers library (`helpers.js`)

js

```
function helpers() {}

helpers.groupBy = function (objectArray, property) {
  return objectArray.reduce((acc, obj) => {
    const key = obj[property];
    acc[key] ??= [];
    acc[key].push(obj);
    return acc;
  }, {});
};

export default helpers;
```

The correct import usage (`App.js`):

js

```
import helpers from "./helpers";
```

See also

- Functions

TypeError: "x" is not a non-null object

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/No_non-null_object · MDN page last modified 2025-07-08

The JavaScript exception "is not a non-null object" occurs when an object is expected somewhere and wasn't provided. `null` is not an object and won't work.

Message

```
TypeError: Property description must be an object: x (V8-based)
TypeError: Property descriptor must be an object, got "x" (Firefox)
TypeError: Property description must be an object. (Safari)
```

Error type

`TypeError`

What went wrong?

An object is expected somewhere and wasn't provided. `null` is not an object and won't work. You must provide a proper object in the given situation.

Examples

Property descriptor expected

When methods like `Object.create()` or `Object.defineProperty()` and `Object.defineProperties()` are used, the optional descriptor parameter expects a property descriptor object. Providing no object (like just a number), will throw an error:

js · ✗ bad example

```
Object.defineProperty({}, "key", 1);
// TypeError: 1 is not a non-null object

Object.defineProperty({}, "key", null);
// TypeError: null is not a non-null object
```

A valid property descriptor object might look like this:

js · ✓ good example

```
Object.defineProperty({}, "key", { value: "foo", writable: false });
```

See also

- `Object.create()`
- `Object.defineProperty()`
- `Object.defineProperties()`

JavaScript › Reference › JavaScript error reference

TypeError: "x" is read-only

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Read-only · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "is read-only" occurs when a global variable or object property that was assigned to is a read-only property.

Message

```
TypeError: Cannot assign to read only property 'x' of #<Object> (V8-based)
TypeError: "x" is read-only (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

The global variable or object property that was assigned to is a read-only property. (Technically, it is a non-writable data property.)

This error happens only in strict mode code. In non-strict code, the assignment is silently ignored.

Examples

Invalid cases

Read-only properties are not super common, but they can be created using `Object.defineProperty()` or `Object.freeze()`.

js · ✗ bad example

```
"use strict";
const obj = Object.freeze({ name: "Elsa", score: 157 });
obj.score = 0; // TypeError

("use strict");
Object.defineProperty(this, "LUNG_COUNT", { value: 2, writable: false });
LUNG_COUNT = 3; // TypeError

("use strict");
const frozenArray = Object.freeze([0, 1, 2]);
frozenArray[0]++; // TypeError
```

There are also a few read-only properties built into JavaScript. Maybe you tried to redefine a mathematical constant.

js · ✗ bad example

```
"use strict";
Math.PI = 4; // TypeError
```

Sorry, you can't do that.

The global variable `undefined` is also read-only, so you can't silence the infamous "undefined is not a function" error by doing this:

js · ✗ bad example

```
"use strict";
undefined = function () {}; // TypeError: "undefined" is read-only
```

Valid cases

js · ✓ good example

```
"use strict";
let obj = Object.freeze({ name: "Score", points: 157 });
obj = { name: obj.name, points: 0 }; // replacing it with a new object works
```

See also

- `Object.defineProperty()`
- `Object.freeze()`

JavaScript › Reference › JavaScript error reference

TypeError: already executing generator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Already_executing_generator · MDN page last modified 2025-07-08

The JavaScript exception "TypeError: already executing generator" occurs when a generator is continued using one of its methods (such as `next()`) while executing the generator function's body itself.

Message

```
TypeError: Generator is already running (V8-based)
TypeError: already executing generator (Firefox)
TypeError: Generator is executing (Safari)
```

Error type

`TypeError`

What went wrong?

The generator's methods, `next()`, `return()`, and `throw()`, are meant to continue the execution of a generator function when it's paused after a `yield` expression or before the first statement. If a call to one of these methods is made while executing the generator function, the error is thrown. If you want to return or throw within the generator function, use the `return` statement or the `throw` statement, respectively.

Examples

js · ✗ bad example

```
let it;
function* getNumbers(times) {
  if (times <= 0) {
    it.throw(new Error("times must be greater than 0"));
  }
  for (let i = 0; i < times; i++) {
    yield i;
  }
}
it = getNumbers(3);
it.next();
```

js · ✓ good example

```
let it;
function* getNumbers(times) {
  if (times <= 0) {
    throw new Error("times must be greater than 0");
  }
  for (let i = 0; i < times; i++) {
    yield i;
  }
}
it = getNumbers(3);
it.next(); // { value: 0, done: false }
```

See also

- Iterators and generators
- Iteration protocols
- `function*`
- `Generator`

JavaScript › Reference › JavaScript error reference

TypeError: BigInt value can't be serialized in JSON

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/BigInt_not_serializable · MDN page last modified 2025-09-10

The JavaScript exception "BigInt value can't be serialized in JSON" occurs when a `BigInt` is encountered in `JSON.stringify` with no custom serialization method provided.

Message

```
TypeError: Do not know how to serialize a BigInt (V8-based)
TypeError: BigInt value can't be serialized in JSON (Firefox)
TypeError: JSON.stringify cannot serialize BigInt. (Safari)
```

Error type

`TypeError`

What went wrong?

You are trying to serialize a BigInt value using `JSON.stringify`, which does not support BigInt values by default. Sometimes, JSON stringification happens implicitly in libraries, as part of data serialization. For example, sending data to the server, storing it in external storage, or transferring it between threads would all require serialization, which is often done using JSON.

There are several ways to handle this:

- If you can alter the data source, avoid using BigInt values and cast it to a number first (which may lose precision for large numbers).
- If you can alter the stringification process, pass a replacer function to `JSON.stringify` that converts BigInt values to strings or numbers.
- You can also provide a `BigInt.prototype.toJSON` method globally that gets called whenever a BigInt value is stringified.

For more information on various tradeoffs, see BigInt reference.

Examples

Providing a custom serialization method

By default, BigInt values are not serializable in JSON:

js · ✗ bad example

```
const data = { a: 1n };
JSON.stringify(data);
// TypeError: BigInt value can't be serialized in JSON
```

Assuming you intend for the JSON to contain a number value, here are a few approaches that work:

- Convert the BigInt to a number before stringifying:

  js

  ```
  const data = { a: 1n };
  JSON.stringify({ ...data, a: Number(data.a) });
  // '{"a":1}'
  ```

- Provide a replacer function that converts BigInt values to numbers or raw JSON objects:

  js

  ```
  const data = { a: 1n };
  JSON.stringify(data, (key, value) =>
    typeof value === "bigint" ? Number(value) : value,
  );
  // '{"a":1}'
  ```

  js

  ```
  const data = { a: 1n };
  JSON.stringify(data, (key, value) =>
    typeof value === "bigint" ? JSON.rawJSON(value.toString()) : value,
  );
  // '{"a":1}'
  ```

- Provide a `BigInt.prototype.toJSON` method that gets called whenever a BigInt value is stringified:

  js

  ```
  BigInt.prototype.toJSON = function () {
    return Number(this);
  };
  const data = { a: 1n };
  JSON.stringify(data);
  // '{"a":1}'
  ```

  js

  ```
  BigInt.prototype.toJSON = function () {
    return JSON.rawJSON(this.toString());
  };
  const data = { a: 1n };
  JSON.stringify(data);
  // '{"a":1}'
  ```

See also

- BigInt
- JSON.stringify()
- JSON.rawJSON()

JavaScript › Reference › JavaScript error reference

TypeError: calling a builtin X constructor without new is forbidden

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Builtin_ctor_no_new · MDN page last modified 2026-05-22

The JavaScript exception "calling a builtin X constructor without new is forbidden" occurs when you try to call a builtin constructor without using the new keyword. All modern constructors, such as Promise and Map, must be called with new.

Message

```
TypeError: Constructor X requires 'new' (V8-based)
TypeError: Promise constructor cannot be invoked without 'new' (V8-based)
TypeError: calling a builtin X constructor without new is forbidden (Firefox)
TypeError: calling X constructor without new is invalid (Safari)
```

Error type

TypeError

What went wrong?

In JavaScript, *calling* a function without new and *constructing* a function with new are two distinct operations, and functions can behave differently depending on how they are called.

Apart from the following legacy constructors, all modern constructors must be called with new:

- Object()
- Function() (and its subclasses)
- Error() (and its subclasses)
- RegExp()
- Array()

Some other constructors, such as Date(), and primitive wrappers, such as String(), Number(), and Boolean(), can also be called with or without new, but the return types differ in the two cases.

On every constructor page, you can find information about whether the constructor must be called with new.

Examples

Invalid cases

js · ✗ bad example

```
const m = Map(); // TypeError: calling a builtin Map constructor without new is forbidden
```

Valid cases

js · ✓ good example

```
const m = new Map();
```

See also

- new

JavaScript › Reference › JavaScript error reference

TypeError: can't access/set private field or method: object is not the right class

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Get_set_missing_private · MDN page last modified 2026-01-25

The JavaScript exception "can't access private field or method: object is not the right class" or "can't set private field: object is not the right class" occurs when a private field or method is get or set on an object that does not have this private element defined.

Message

```
TypeError: Cannot read private member #x from an object whose class did not declare it (V8-based)
TypeError: Cannot write private member #x to an object whose class did not declare it (V8-based)
TypeError: can't access private field or method: object is not the right class (Firefox)
TypeError: can't set private field: object is not the right class (Firefox)
TypeError: Cannot access invalid private field (evaluating 'this.#x') (Safari)
```

Error type

TypeError

What went wrong?

You are trying to get or set a private field or method on an object, but that object does not contain this private element. Private instance properties can only be accessed on instances of the class (including its subclasses) that declares them; private static properties can only be accessed on the class itself that declares them, and not on subclasses.

This error occurs when the private name exists in the class scope but the object it's accessed on is invalid. If the private name does not exist, you will get a syntax error instead.

Examples

Mismatched static/instance fields

You may have declared the field as a static field, but are trying to access it on an instance, or vice versa.

js · ✗ bad example

```
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

const obj = new MyClass();
obj.doSomething();
// TypeError: can't access private field: object is not the right class
```

To fix this, either change the field to be an instance field, or access the field on the class itself, or declare another field on the instance. Note that the private namespace is shared between static and instance properties, so you cannot have a static and instance private element with the same name.

js · ✓ good example

```
class MyClass {
  #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

class MyClass2 {
  static #x = 0;
  doSomething() {
    console.log(MyClass2.#x);
  }
}
```

Wrong object used

Perhaps you have a method that access `this.#x`, but it is called with another `this` value.

js · ✗ bad example

```
class JSONReplacer {
  #count = 0;
  func(key, value) {
    if (typeof value === "object") {
      this.#count++;
    }
    return value;
  }
}

JSON.stringify({ a: 1, b: { c: 2 } }, new JSONReplacer().func);
// TypeError: can't access private field: object is not the right class
```

This is because `JSON.stringify()` calls the replacer function with the object containing `value` as `this`, so the private field is not accessible. To fix this, you can bind the method to the object, or use an arrow function, to ensure that `replacer.func` is called with the correct `this` value.

js · ✓ good example

```
const replacer = new JSONReplacer();
JSON.stringify({ a: 1, b: { c: 2 } }, replacer.func.bind(replacer));
JSON.stringify({ a: 1, b: { c: 2 } }, (...args) => replacer.func(...args));
```

Most of the time, if you accidentally unbound a method, the method would be called with `undefined` as `this`, which would result in a different error (TypeError: can't convert undefined to object). This error only occurs when the method is called with a different object as `this`, either by using `call()` or `apply()`, or by passing the method as a callback to a function that calls it with a different `this` value.

If you don't know for sure that the object will contain the private element, as in the following code:

```js
class MyClass {
  #x = 0;
  static doSomething(obj) {
    console.log(obj.#x); // Throws if obj is not an instance of MyClass
  }
}
```

You can use the `in` operator to perform a *branded check* first.

```js
class MyClass {
  #x = 0;
  static doSomething(obj) {
    if (!(#x in obj)) {
      return;
    }
    console.log(obj.#x);
  }
}
```

Accessing static elements on subclasses

If you have a private static property, you can only access it on the class that declares it, not on subclasses.

js · ✗ bad example

```js
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

class MySubClass extends MyClass {}

MySubClass.doSomething();
// TypeError: can't access private field: object is not the right class
```

To fix this, never access private static properties through `this`. Instead, always explicitly specify the class's name.

js · ✓ good example

```js
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(MyClass.#x);
  }
}
```

Accessing same-name private elements on another class

Unlike normal string or symbol properties, private names are not shared between classes. If you have a private element with the same name in two classes, they are still not the same element, and you cannot access one class's private element from another class.

js · ✗ bad example

```
class MyClass {
  #x = 0;
}

class MyOtherClass {
  #x = 1;
  doSomething(o) {
    console.log(o.#x);
  }
}

const obj = new MyClass();
new MyOtherClass().doSomething(obj);
// TypeError: can't access private field: object is not the right class
```

Adding private elements to unrelated objects

You cannot dynamically *add* private elements to unrelated objects.

js · ✗ bad example

```
class MyClass {
  #x = 0;
  static stamp(obj) {
    obj.#x = 1;
  }
}

MyClass.stamp({});
// TypeError: can't set private field: object is not the right class
```

If you really want to do this, consider the return override trick. However, in general, you would probably want to use a `WeakMap` instead.

js · ✓ good example

```
class MyClass {
  static #objToX = new WeakMap();
  static stamp(obj) {
    MyClass.#objToX.set(obj, 1);
  }
}

MyClass.stamp({});
```

See also

- Classes
- Private elements

JavaScript › Reference › JavaScript error reference

TypeError: can't assign to property "x" on "y": not an object

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_assign_to_property · MDN page last modified 2025-07-08

The JavaScript strict mode exception "can't assign to property" occurs when attempting to create a property on primitive value such as a symbol, a string, a number or a boolean. Primitive values cannot hold any property.

Message

```
TypeError: Cannot create property 'x' on number '1' (V8-based)
TypeError: can't assign to property "x" on 1: not an object (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`.

What went wrong?

In strict mode, a `TypeError` is raised when attempting to create a property on primitive value such as a symbol, a string, a number or a boolean. Primitive values cannot hold any property.

The problem might be that an unexpected value is flowing at an unexpected place, or that an object variant of a `String` or a `Number` is expected.

Examples

Invalid cases

js · ✗ bad example

```
"use strict";

const foo = "my string";
// The following line does nothing if not in strict mode.
foo.bar = {}; // TypeError: can't assign to property "bar" on "my string": not an object
```

Fixing the issue

Either fix the code to prevent the primitive from being used in such places, or fix the issue by creating the object equivalent `Object`.

js · ✓ good example

```
"use strict";

const foo = new String("my string");
foo.bar = {};
```

See also

- Strict mode
- primitive

JavaScript › Reference › JavaScript error reference

TypeError: can't convert BigInt to number

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_convert_BigInt_to_number · MDN page last modified 2025-07-08

The JavaScript exception "can't convert BigInt to number" occurs when an arithmetic operation involves a mix of `BigInt` and `Number` values.

Message

```
TypeError: Cannot convert a BigInt value to a number (V8-based)
TypeError: Cannot mix BigInt and other types, use explicit conversions (V8-based)
TypeError: BigInts have no unsigned right shift, use >> instead (V8-based)
TypeError: can't convert BigInt to number (Firefox)
TypeError: Conversion from 'BigInt' to 'number' is not allowed. (Safari)
TypeError: Invalid mix of BigInt and other type in addition/multiplication/…. (Safari)
TypeError: BigInt does not support >>> operator (Safari)
```

Error type

`TypeError`.

What went wrong?

The two sides of an arithmetic operator must both be BigInts or both not. If an operation involves a mix of BigInts and numbers, it's ambiguous whether the result should be a BigInt or number, since there may be loss of precision in both cases.

The error also happens when a BigInt is implicitly converted to a number via the number coercion process. For example, if a BigInt is passed to a built-in method that expects a number.

The error can also happen if the unsigned right shift operator (`>>>`) is used between two BigInts. In Firefox, the message is the same: "can't convert BigInt to number".

Examples

Mixing numbers and BigInts in operations

js · ✗ bad example

```
const sum = 1n + 1;
// TypeError: can't convert BigInt to number
```

Instead, explicitly coerce one side to a BigInt or number.

js · ✓ good example

```
const sum = 1n + BigInt(1);
const sum2 = Number(1n) + 1;
```

Using unsigned right shift on BigInts

js · ✗ bad example

```
const a = 4n >>> 2n;
// TypeError: can't convert BigInt to number
```

Use normal right shift instead.

js · ✓ good example

```
const a = 4n >> 2n;
```

See also

- BigInt

JavaScript › Reference › JavaScript error reference

TypeError: can't convert x to BigInt

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_convert_x_to_BigInt · MDN page last modified 2025-07-08

The JavaScript exception "x can't be converted to BigInt" occurs when attempting to convert a Symbol, null, or undefined value to a BigInt, or if an operation expecting a BigInt parameter receives a number.

Message

```
TypeError: Cannot convert null to a BigInt (V8-based)
TypeError: can't convert null to BigInt (Firefox)
TypeError: Invalid argument type in ToBigInt operation (Safari)
```

Error type

TypeError.

What went wrong?

When using the BigInt() function to convert a value to a BigInt, the value would first be converted to a primitive. Then, if it's not one of BigInt, string, number, and boolean, the error is thrown.

Some operations, like BigInt.asIntN, require the parameter to be a BigInt. Passing in a number in this case will also throw this error.

Examples

Using BigInt() on invalid values

js · ✗ bad example

```
const a = BigInt(null);
// TypeError: can't convert null to BigInt
const b = BigInt(undefined);
// TypeError: can't convert undefined to BigInt
const c = BigInt(Symbol("1"));
// TypeError: can't convert Symbol("1") to BigInt
```

js · ✓ good example

```
const a = BigInt(1);
const b = BigInt(true);
const c = BigInt("1");
const d = BigInt(Symbol("1").description);
```

Note: Simply coercing the value to a string or number using `String()` or `Number()` before passing it to `BigInt()` is usually not sufficient to avoid all errors. If the string is not a valid integer number string, a SyntaxError is thrown; if the number is not an integer (most notably, `NaN`), a RangeError is thrown. If the range of input is unknown, properly validate it before using `BigInt()`.

Passing a number to a function expecting a BigInt

js · ✗ bad example

```
const a = BigInt.asIntN(4, 8);
// TypeError: can't convert 8 to BigInt
const b = new BigInt64Array(3).fill(3);
// TypeError: can't convert 3 to BigInt
```

js · ✓ good example

```
const a = BigInt.asIntN(4, 8n);
const b = new BigInt64Array(3).fill(3n);
```

See also

- `BigInt()` constructor

TypeError: can't define property "x": "obj" is not extensible

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_define_property_object_not_extensible · MDN page last modified 2025-11-09

The JavaScript exception "can't define property "x": "obj" is not extensible" occurs when an object is marked as non-extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. Objects can be made non-extensible by calling `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`.

Message

```
TypeError: Cannot add property x, object is not extensible (V8-based)
TypeError: Cannot define property x, object is not extensible (V8-based)
TypeError: can't define property "x": Object is not extensible (Firefox)
TypeError: Attempting to define property on object that is not extensible. (Safari)
```

Error type

`TypeError`

What went wrong?

Usually, an object is extensible and new properties can be added to it. However, in this case the object isn't extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. You could have marked the object as non-extensible by calling `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`, or a library you are using could have done that for you.

Examples

Adding new properties to a non-extensible objects

In strict mode, attempting to add new properties to a non-extensible object via assignment throws a `TypeError`. In sloppy mode, the addition of the "x" property is silently ignored.

js · ✗ bad example

```
"use strict";

const obj = {};
Object.preventExtensions(obj);

obj.x = "foo";
// TypeError: can't define property "x": Object is not extensible
```

In both strict mode and sloppy mode, a call to `Object.defineProperty()` throws when adding a new property to a non-extensible object.

js · ✗ bad example

```
const obj = {};
Object.preventExtensions(obj);

Object.defineProperty(obj, "x", { value: "foo" });
// TypeError: can't define property "x": Object is not extensible
```

Non-extensible objects created by other means

The `Object.seal()` and `Object.freeze()` methods also create non-extensible objects—they just have additional restrictions on modifying existing properties as well.

js · ✗ bad example

```
"use strict";

const obj = { y: "bar" };
Object.seal(obj);
obj.x = "foo";
// TypeError: can't define property "x": Object is not extensible
```

Fixing the error

There are three ways to fix this error: you can remove the property addition entirely if you don't need it, you can copy the existing properties to a new extensible object, or you can add the property before making the object non-extensible.

js · ✓ good example

```
"use strict";

const obj = {};
obj.x = "foo"; // add property first and only then prevent extensions

Object.preventExtensions(obj);
```

See also

- Object.preventExtensions()
- Object.seal()
- Object.freeze()

JavaScript › Reference › JavaScript error reference

TypeError: can't delete non-configurable array element

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Non_configurable_array_element · MDN page last modified 2025-07-08

The JavaScript exception "can't delete non-configurable array element" occurs when it was attempted to shorten the length of an array, but one of the array's elements is non-configurable.

Message

```
TypeError: Cannot delete property '1' of [object Array] (V8-based)
TypeError: can't delete non-configurable array element (Firefox)
TypeError: Unable to delete property. (Safari)
```

Error type

TypeError

What went wrong?

It was attempted to shorten the length of an array, but one of the array's elements is non-configurable. When shortening an array, the elements beyond the new array length will be deleted, which failed in this situation.

The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `writable`) can be changed.

Usually, properties in an object created by an array initializer are configurable. However, for example, when using `Object.defineProperty()`, the property isn't configurable by default.

Examples

Non-configurable properties created by Object.defineProperty

The `Object.defineProperty()` creates non-configurable properties by default if you haven't specified them as configurable.

js · ✗ bad example

```
"use strict";
const arr = [];
Object.defineProperty(arr, 0, { value: 0 });
Object.defineProperty(arr, 1, { value: "1" });

arr.length = 1;
// TypeError: can't delete non-configurable array element
```

You will need to set the elements as configurable, if you intend to shorten the array.

js · ✓ good example

```
"use strict";
const arr = [];
Object.defineProperty(arr, 0, { value: 0, configurable: true });
Object.defineProperty(arr, 1, { value: "1", configurable: true });

arr.length = 1;
```

Sealed Arrays

The `Object.seal()` function marks all existing elements as non-configurable.

js · ✗ bad example

```
"use strict";
const arr = [1, 2, 3];
Object.seal(arr);

arr.length = 1;
// TypeError: can't delete non-configurable array element
```

You either need to remove the `Object.seal()` call, or make a copy of it. In case of a copy, shortening the copy of the array does not modify the original array length.

js · ✓ good example

```
"use strict";
const arr = [1, 2, 3];
Object.seal(arr);

// Copy the initial array to shorten the copy
const copy = Array.from(arr);
copy.length = 1;
// arr.length === 3
```

See also

- [[Configurable]]
- `length`
- `Object.defineProperty()`
- `Object.seal()`

JavaScript › Reference › JavaScript error reference

TypeError: can't redefine non-configurable property "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_redefine_property · MDN page last modified 2025-07-08

The JavaScript exception "can't redefine non-configurable property" occurs when it was attempted to redefine a property, but that property is non-configurable.

Message

```
TypeError: Cannot redefine property: "x" (V8-based)
TypeError: can't redefine non-configurable property "x" (Firefox)
TypeError: Attempting to change value of a readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

It was attempted to redefine a property, but that property is non-configurable. The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `writable`) can be changed. Usually, properties in an object created by an object initializer are configurable. However, for example, when using `Object.defineProperty()`, the property isn't configurable by default.

Examples

Non-configurable properties created by Object.defineProperty

The `Object.defineProperty()` creates non-configurable properties if you haven't specified them as configurable.

js · ✗ bad example

```
const obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar" });

Object.defineProperty(obj, "foo", { value: "baz" });
// TypeError: can't redefine non-configurable property "foo"
```

You will need to set the "foo" property to configurable, if you intend to redefine it later in the code.

js · ✓ good example

```
const obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar", configurable: true });
Object.defineProperty(obj, "foo", { value: "baz", configurable: true });
```

See also

- [[Configurable]]
- `Object.defineProperty()`

JavaScript › Reference › JavaScript error reference

TypeError: can't set prototype of this object

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_set_prototype · MDN page last modified 2025-07-08

The JavaScript exception "can't set prototype of this object" occurs when attempting to set the prototype of an object, but the object's prototype is frozen, either by being a built-in immutable prototype object, or by being non-extensible.

Message

```
TypeError: Immutable prototype object 'Object.prototype' cannot have their prototype set
(V8-based)
TypeError: #<Object> is not extensible (V8-based)
TypeError: can't set prototype of this object (Firefox)
TypeError: Cannot set prototype of immutable prototype object (Safari)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

You are using one of the prototype-mutating methods—most notably, `Object.setPrototypeOf()`—on an object whose prototype is immutable. Some built-in objects have immutable prototypes, such as `Object.prototype` and `window`, for security reasons. User objects can also prevent prototype changes by using `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`.

Examples

Changing the prototype of a built-in object

A selected few built-in objects have immutable prototypes. For example, you cannot change the prototype of `Object.prototype`:

js · ✗ bad example

```
Object.setPrototypeOf(Object.prototype, {});
```

This prevents you from being able to arbitrarily change the behavior of all objects in the system. The prototype of `Object.prototype` is always `null`. However, other built-in prototype objects, such as `Function.prototype` and `Array.prototype`, are not protected by default in this regard.

Changing the prototype of a non-extensible object

If you make an object non-extensible, you cannot change its prototype either:

js · ✗ bad example

```
const obj = {};
Object.preventExtensions(obj);
Object.setPrototypeOf(obj, {});
// TypeError: can't set prototype of this object
```

See also

- `Object.setPrototypeOf()`
- `Object.preventExtensions()`

JavaScript › Reference › JavaScript error reference

TypeError: can't set prototype: it would cause a prototype chain cycle

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cyclic_prototype · MDN page last modified 2025-07-08

The JavaScript exception "TypeError: can't set prototype: it would cause a prototype chain cycle" occurs when an object's prototype is set to an object such that the prototype chain becomes circular (`a` and `b` both have each other in their prototype chains).

Message

```
TypeError: Cyclic __proto__ value (V8-based)
TypeError: can't set prototype: it would cause a prototype chain cycle (Firefox)
TypeError: cyclic __proto__ value (Safari)
```

Error type

`TypeError`

What went wrong?

A loop, also called a cycle, was introduced in a prototype chain. That means that when walking this prototype chain, the same place would be accessed over and over again, instead of eventually reaching `null`.

This error is thrown at the time of setting the prototype. In an operation like `Object.setPrototypeOf(a, b)`, if `a` already exists in the prototype chain of `b`, this error will be thrown.

Examples

js · ✗ bad example

```
const a = {};
Object.setPrototypeOf(a, a);
// TypeError: can't set prototype: it would cause a prototype chain cycle
```

js · ✗ bad example

```
const a = {};
const b = {};
const c = {};
Object.setPrototypeOf(a, b);
Object.setPrototypeOf(b, c);
Object.setPrototypeOf(c, a);
// TypeError: can't set prototype: it would cause a prototype chain cycle
```

See also

- Object Prototypes
- Inheritance and the prototype chain

TypeError: cannot use 'in' operator to search for 'x' in 'y'

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/in_operator_no_object · MDN page last modified 2025-07-08

The JavaScript exception "right-hand side of 'in' should be an object" occurs when the `in` operator was used to search in strings, or in numbers, or other primitive types. It can only be used to check if a property is in an object.

Message

```
TypeError: Cannot use 'in' operator to search for 'x' in 'y' (V8-based & Firefox)
TypeError: right-hand side of 'in' should be an object, got null (Firefox)
TypeError: "y" is not an Object. (evaluating '"x" in "y"') (Safari)
```

Error type

`TypeError`

What went wrong?

The `in` operator can only be used to check if a property is in an object. You can't search in strings, or in numbers, or other primitive types.

Examples

Searching in strings

Unlike in other programming languages (e.g., Python), you can't search in strings using the `in` operator.

js · ✗ bad example

```
"Hello" in "Hello World";
// TypeError: cannot use 'in' operator to search for 'Hello' in 'Hello World'
```

Instead you will need to use `String.prototype.includes()`, for example.

js · ✓ good example

```
"Hello World".includes("Hello");
// true
```

The operand can't be null or undefined

Make sure the object you are inspecting isn't actually `null` or `undefined`.

js · ✗ bad example

```
const foo = null;
"bar" in foo;
// TypeError: cannot use 'in' operator to search for 'bar' in 'foo' (Chrome)
// TypeError: right-hand side of 'in' should be an object, got null (Firefox)
```

The `in` operator always expects an object.

js · ✓ good example

```
const foo = { baz: "bar" };
"bar" in foo; // false

"PI" in Math; // true
"pi" in Math; // false
```

Searching in arrays

Be careful when using the `in` operator to search in `Array` objects. The `in` operator checks the index number, not the value at that index.

js

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
3 in trees; // true
"oak" in trees; // false
```

See also

- `in`

JavaScript › Reference › JavaScript error reference

TypeError: class constructors must be invoked with 'new'

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Class_ctor_no_new · MDN page last modified 2026-05-22

The JavaScript exception "class constructors must be invoked with 'new'" occurs when a class constructor is called without the `new` keyword. All class constructors must be called with `new`.

Message

```
TypeError: Class constructor X cannot be invoked without 'new' (V8-based)
TypeError: Class constructors cannot be invoked without 'new' (V8-based)
TypeError: class constructors must be invoked with 'new' (Firefox)
TypeError: Cannot call a class constructor without |new| (Safari)
```

Error type

`TypeError`

What went wrong?

In JavaScript, *calling* a function without `new` and *constructing* a function with `new` are two distinct operations, and functions can behave differently depending on how they are called.

Traditionally, JavaScript functions have been used as both constructors and normal functions, and can detect how they were called using `new.target`. However, class constructors are always constructors and cannot be called as normal functions.

Examples

Invalid cases

js · ✗ bad example

```
class X {}

X(); // TypeError: class constructors must be invoked with 'new'
```

Valid cases

js · ✓ good example

```
class X {}

new X();
```

See also

- Classes
- new

JavaScript › Reference › JavaScript error reference

TypeError: cyclic object value

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cyclic_object_value · MDN page last modified 2025-07-08

The JavaScript exception "cyclic object value" occurs when object references were found in JSON. `JSON.stringify()` doesn't try to solve them and fails accordingly.

Message

```
TypeError: Converting circular structure to JSON (V8-based)
TypeError: cyclic object value (Firefox)
TypeError: JSON.stringify cannot serialize cyclic structures. (Safari)
```

Error type

`TypeError`

What went wrong?

The JSON format per se doesn't support object references (although an IETF draft exists), hence `JSON.stringify()` doesn't try to solve them and fails accordingly.

Examples

Circular references

In a circular structure like the following:

js

```
const circularReference = { otherData: 123 };
circularReference.myself = circularReference;
```

`JSON.stringify()` will fail

js · ✗ bad example

```
JSON.stringify(circularReference);
// TypeError: cyclic object value
```

To serialize circular references you can use a library that supports them (e.g., cycle.js) or implement a solution by yourself, which will require finding and replacing (or removing) the cyclic references by serializable values.

The snippet below illustrates how to find and filter (thus causing data loss) a cyclic reference by using the `replacer` parameter of `JSON.stringify()`:

js

```
function getCircularReplacer() {
  const ancestors = [];
  return function (key, value) {
    if (typeof value !== "object" || value === null) {
      return value;
    }
    // `this` is the object that value is contained in,
    // i.e., its direct parent.
    while (ancestors.length > 0 && ancestors.at(-1) !== this) {
      ancestors.pop();
    }
    if (ancestors.includes(value)) {
      return "[Circular]";
    }
    ancestors.push(value);
    return value;
  };
}

JSON.stringify(circularReference, getCircularReplacer());
// {"otherData":123,"myself":"[Circular]"}

const o = {};
const notCircularReference = [o, o];
JSON.stringify(notCircularReference, getCircularReplacer());
// [{},{}]
```

See also

- `JSON.stringify()`
- cycle.js on GitHub

TypeError: derived class constructor returned invalid value x

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_derived_return · MDN page last modified 2026-05-22

The JavaScript exception "derived class constructor returned invalid value x" occurs when a derived class constructor returns a value that is not an object or `undefined`.

Message

```
TypeError: Derived constructors may only return object or undefined (V8-based)
TypeError: derived class constructor returned invalid value 1 (Firefox)
TypeError: Cannot return a non-object type in the constructor of a derived class. (Safari)
```

Error type

`TypeError`

What went wrong?

Typically, a constructor does not need to return anything—the value of `this` is automatically returned when the class is constructed. A constructor can also return an object, and this object will override `this` as the newly constructed instance. However, returning something that's neither an object nor `undefined` is usually a mistake, because that value is ignored. In base classes and function constructors (using the `function` syntax), returning such a value is silently ignored, while in derived classes, it throws an error.

Examples

Invalid cases

js · ✗ bad example

```
class Base {
  constructor() {}
}

class Derived extends Base {
  constructor() {
    return 2;
  }
}

new Derived(); // TypeError: derived class constructor returned invalid value 2
```

Valid cases

js · ✓ good example

```
class Base {
  constructor() {}
}

class Derived extends Base {
  constructor() {
    return { x: 1 };
  }
}

new Derived(); // { x: 1 }
```

See also

- Classes
- extends
- new

JavaScript › Reference › JavaScript error reference

TypeError: getting private setter-only property

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Private_setter_only · MDN page last modified 2025-07-08

The JavaScript exception "getting private setter-only property" occurs when reading the value of a private element for which only a setter is defined.

Message

```
TypeError: '#x' was defined without a getter (V8-based)
TypeError: getting private setter-only property (Firefox)
TypeError: Trying to access an undefined private getter (Safari)
```

Error type

TypeError

What went wrong?

There is an attempt to get the value of a private element for which only a setter is specified. Unlike normal objects, where an undefined getter just means the property would always return undefined, for private elements, this is an error.

Examples

Private element with no getter

Here, `#name` has no getter, so trying to read it using `this.#name` will throw an error.

js · ✗ bad example

```
class Person {
  set #name(value) {}

  get name() {
    return this.#name;
  }
}

const person = new Person();
console.log(person.name);
```

It's not usual for a private element to have a setter without a getter. Either add a getter or refactor your program so the setter can be removed too.

See also

- Private elements
- set

JavaScript › Reference › JavaScript error reference

TypeError: Initializing an object twice is an error with private fields/methods

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Private_double_initialization · MDN page last modified 2025-07-08

The JavaScript exception "Initializing an object twice is an error with private fields/methods" occurs when an object that was created via a class constructor goes through the class construction again, and the class contains a private element. This is usually caused by the return override trick.

Message

```
TypeError: Cannot initialize #x twice on the same object (V8-based)
TypeError: Initializing an object twice is an error with private fields (Firefox)
TypeError: Cannot redefine existing private field (evaluating 'super(o)') (Safari)

TypeError: Cannot initialize private methods of class X twice on the same object (V8-based)
TypeError: Initializing an object twice is an error with private methods (Firefox)
TypeError: Cannot install same private methods on object more than once (evaluating
'super(o)') (Safari)
```

Error type

TypeError

What went wrong?

For any object, if it already contains a private field or method, it would be an error to install the same field again. Private elements are installed on the value of `this` when the class constructor is called, so this error could happen if the `this` value is an already-constructed instance of this class.

Usually, `this` in a constructor is a newly created object which doesn't have any preexisting properties. However, it can be overridden by the return value of the base class. If the base class returns another

object, that object would replace the current object as the value of `this`:

```js
class Base {
  constructor(o) {
    // This object will become the this value of any subclass
    return o;
  }
}

class Derived extends Base {
  #x = 0;
}
```

If you call `new Derived(anyObject)`, where `anyObject` is not an instance of `Derived`, the `Derived` constructor will be called with `anyObject` as the `this` value, and therefore install the `#x` private field on `anyObject`. This is the "return override" trick, which allows you to define arbitrary information on unrelated objects. However, if you call `new Derived(new Derived())`, or call `new Derived(anyObject)` again, the `Derived` constructor will try to install the `#x` private field again on an object which already has the `#x` private field, causing this error.

See also

- Classes
- Private elements

JavaScript › Reference › JavaScript error reference

TypeError: invalid 'instanceof' operand 'x'

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/invalid_right_hand_side_instanceof_operand · MDN page last modified 2025-07-08

The JavaScript exception "invalid 'instanceof' operand" occurs when the right-hand side operands of the `instanceof` operator isn't used with a constructor object, i.e., an object which has a `prototype` property and is callable.

Message

```
TypeError: Right-hand side of 'instanceof' is not an object (V8-based)
TypeError: invalid 'instanceof' operand "x" (Firefox)
TypeError: Right hand side of instanceof is not an object (Safari)

TypeError: Right-hand side of 'instanceof' is not callable (V8-based)
TypeError: x is not a function (Firefox)
TypeError: x is not a function. (evaluating 'x instanceof y') (Safari)

TypeError: Function has non-object prototype 'undefined' in instanceof check (V8-based)
TypeError: 'prototype' property of x is not an object (Firefox)
TypeError: instanceof called on an object with an invalid prototype property. (Safari)
```

Error type

TypeError

What went wrong?

The instanceof operator expects the right-hand-side operands to be a constructor object, i.e., an object which has a prototype property and is callable. It can also be an object with a Symbol.hasInstance method. This error can occur if:

- The right-hand side operand is not an object.
- The right-hand side operand is not a callable and it has no Symbol.hasInstance method.
- The right-hand side operand is a callable, but its prototype property is not an object. (For example, arrow functions do not have a prototype property.)

Examples

instanceof vs. typeof

js · ✗ bad example

```
"test" instanceof ""; // TypeError: invalid 'instanceof' operand ""
42 instanceof 0; // TypeError: invalid 'instanceof' operand 0

function Foo() {}
const f = Foo(); // Foo() is called and returns undefined
const x = new Foo();

x instanceof f; // TypeError: invalid 'instanceof' operand f
x instanceof x; // TypeError: x is not a function
```

To fix these errors, you will either need to replace the instanceof operator with the typeof operator, or to make sure you use the function name, instead of the result of its evaluation.

js · ✓ good example

```
typeof "test" === "string"; // true
typeof 42 === "number"; // true

function Foo() {}
const f = Foo; // Do not call Foo.
const x = new Foo();

x instanceof f; // true
x instanceof Foo; // true
```

See also

- instanceof
- typeof

JavaScript › Reference › JavaScript error reference

TypeError: invalid Array.prototype.sort argument

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Array_sort_argument · MDN page last modified 2026-02-15

The JavaScript exception "invalid Array.prototype.sort argument" occurs when the argument of `Array.prototype.sort()` (and its related methods: `Array.prototype.toSorted()`, `TypedArray.prototype.sort()`, `TypedArray.prototype.toSorted()`) isn't either `undefined` or a function that compares its operands.

Message

```
TypeError: The comparison function must be either a function or undefined (V8-based)

TypeError: invalid Array.prototype.sort argument (Firefox)
TypeError: non-function passed to Array.prototype.toSorted (Firefox)
TypeError: invalid %TypedArray%.prototype.sort argument (Firefox)

TypeError: Array.prototype.sort requires the comparator argument to be a function or
undefined (Safari)
TypeError: Array.prototype.toSorted requires the comparator argument to be a function or
undefined (Safari)
TypeError: TypedArray.prototype.sort requires the comparator argument to be a function or
undefined (Safari)
TypeError: TypedArray.prototype.toSorted requires the comparator argument to be a function
or undefined (Safari)
```

Error type

`TypeError`

What went wrong?

The argument of `Array.prototype.sort()` (and its related methods: `Array.prototype.toSorted()`, `TypedArray.prototype.sort()`, `TypedArray.prototype.toSorted()`) is expected to be either `undefined` or a function which compares its operands.

Examples

Invalid cases

js · ✗ bad example

```
[1, 3, 2].sort(5); // TypeError
students.toSorted("name"); // TypeError
```

Valid cases

js · ✓ good example

```
[1, 3, 2].sort(); // [1, 2, 3]
[1, 3, 2].sort((a, b) => a - b); // [1, 2, 3]
students.toSorted((a, b) => a.name.localeCompare(b.name));
```

See also

- `Array.prototype.sort()`
- `Array.prototype.toSorted()`
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toSorted()`

JavaScript › Reference › JavaScript error reference

TypeError: invalid assignment to const "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Invalid_const_assignment · MDN page last modified 2025-07-08

The JavaScript exception "invalid assignment to const" occurs when it was attempted to alter a constant value. JavaScript `const` declarations can't be re-assigned or redeclared.

Message

```
TypeError: Assignment to constant variable. (V8-based)
TypeError: invalid assignment to const 'x' (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

A constant is a value that cannot be altered by the program during normal execution. It cannot change through re-assignment, and it can't be redeclared. In JavaScript, constants are declared using the `const` keyword.

Examples

Invalid redeclaration

Assigning a value to the same constant name in the same block-scope will throw.

js · ✗ bad example

```
const COLUMNS = 80;

// …

COLUMNS = 120; // TypeError: invalid assignment to const `COLUMNS'
```

Fixing the error

There are multiple options to fix this error. Check what was intended to be achieved with the constant in question.

Rename

If you meant to declare another constant, pick another name and re-name. This constant name is already taken in this scope.

js · ✓ good example

```
const COLUMNS = 80;
const WIDE_COLUMNS = 120;
```

const, let or var?

Do not use const if you weren't meaning to declare a constant. Maybe you meant to declare a block-scoped variable with `let` or global variable with `var`.

js · ✓ good example

```
let columns = 80;

// …

columns = 120;
```

Scoping

Check if you are in the correct scope. Should this constant appear in this scope or was it meant to appear in a function, for example?

js · ✓ good example

```
const COLUMNS = 80;

function setupBigScreenEnvironment() {
  const COLUMNS = 120;
}
```

const and immutability

The `const` declaration creates a read-only reference to a value. It does **not** mean the value it holds is immutable, just that the variable identifier cannot be reassigned. For instance, in case the content is an object, this means the object itself can still be altered. This means that you can't mutate the value stored in a variable:

js · ✗ bad example

```
const obj = { foo: "bar" };
obj = { foo: "baz" }; // TypeError: invalid assignment to const `obj'
```

But you can mutate the properties in a variable:

js · ✓ good example

```
obj.foo = "baz";
obj; // { foo: "baz" }
```

See also

- `const`
- `let`
- `var`

JavaScript › Reference › JavaScript error reference

TypeError: Iterator/AsyncIterator constructor can't be used directly

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Constructor_cant_be_used_directly · MDN page last modified 2026-05-11

The JavaScript exception "Iterator constructor can't be used directly" or "AsyncIterator constructor can't be used directly" occurs when you try to use the `Iterator()` or `AsyncIterator` constructors directly to create instances. These constructors are *abstract classes* and should only be inherited from.

Message

```
TypeError: Abstract class Iterator not directly constructable (V8-based)
TypeError: Iterator constructor can't be used directly (Firefox)
TypeError: Iterator cannot be constructed directly (Safari)

TypeError: Abstract class AsyncIterator not directly constructable (V8-based)
TypeError: AsyncIterator constructor can't be used directly (Firefox)
TypeError: AsyncIterator cannot be constructed directly (Safari)
```

Error type

`TypeError`

What went wrong?

The `Iterator` and `AsyncIterator` constructors are abstract classes and should not be used directly. They check the value of `new.target` and throw if it is the same as the constructor itself. The only way to use these constructors is to inherit from them in a subclass and call `super()` in the subclass constructor. The subclass must also define a `next()` method to be useful.

Examples

Invalid cases

js · ✗ bad example

```
new Iterator();
```

Valid cases

js · ✓ good example

```
class MyIterator extends Iterator {
  #step;
  #end;
  constructor(start, end) {
    // Implicitly calls new Iterator(), but with a different `new.target`
    super();
    this.#step = start;
    this.#end = end;
  }
  next() {
    if (this.#step >= this.#end) {
      return { done: true };
    }
    return { value: this.#step++, done: false };
```

(code continues on the next page)

(code continued from the previous page)

```
  }
}

new MyIterator();
```

See also

- `AsyncIterator`
- `Iterator`

TypeError: matchAll/replaceAll must be called with a global RegExp

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Requires_global_RegExp · MDN page last modified 2025-07-08

The JavaScript exception "TypeError: matchAll/replaceAll must be called with a global RegExp" occurs when the `String.prototype.matchAll()` or `String.prototype.replaceAll()` method is used with a `RegExp` object that does not have the `global` flag set.

Message

```
TypeError: String.prototype.matchAll called with a non-global RegExp argument (V8-based)
TypeError: String.prototype.replaceAll called with a non-global RegExp argument (V8-based)
TypeError: matchAll must be called with a global RegExp (Firefox)
TypeError: replaceAll must be called with a global RegExp (Firefox)
TypeError: String.prototype.matchAll argument must not be a non-global regular expression
(Safari)
TypeError: String.prototype.replaceAll argument must not be a non-global regular expression
(Safari)
```

Error type

`TypeError`

What went wrong?

The `String.prototype.matchAll()` and `String.prototype.replaceAll()` methods require a `RegExp` object with the `global` flag set. This flag indicates that the regular expression can match all locations of the input string, instead of stopping at the first match. Although the g flag is redundant when using these methods (because these methods always do a global replacement), they are still required to make the intention clear.

It's worth noting that the g flag validation is done in the `matchAll` and `replaceAll` methods. If you use the `[Symbol.matchAll]()` method of `RegExp` instead, you won't get this error, but there will only be a single match.

Examples

Invalid cases

js · ✗ bad example

```
"abc".matchAll(/./); // TypeError
"abc".replaceAll(/./, "f"); // TypeError
```

Valid cases

If you intend to do global matching/replacement: either add the g flag, or construct a new `RegExp` object with the g flag, if you want to keep the original regex unchanged.

js · ✓ good example

```
[..."abc".matchAll(/./g)]; // [[ "a" ], [ "b" ], [ "c" ]]
"abc".replaceAll(/./g, "f"); // "fff"

const existingPattern = /./;
const newPattern = new RegExp(
  existingPattern.source,
  `${existingPattern.flags}g`,
);
"abc".replaceAll(newPattern, "f"); // "fff"
```

If you only intend to do a single matching/replacement: use `String.prototype.match()` or `String.prototype.replace()` instead. You can also use the `[Symbol.matchAll]()` method if you want an iterator like `matchAll` returns that only contains one match, but doing so will be very confusing.

js · ✓ good example

```
"abc".match(/./); // [ "a" ]
"abc".replace(/./, "f"); // "fbc"

[..././[Symbol.matchAll]("abc")]; // [[ "a" ]]
```

See also

- `String.prototype.matchAll()`
- `String.prototype.replaceAll()`
- `RegExp.prototype.global`

JavaScript › Reference › JavaScript error reference

TypeError: More arguments needed

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/More_arguments_needed · MDN page last modified 2025-07-08

The JavaScript exception "more arguments needed" occurs when there is an error with how a function is called. More arguments need to be provided.

Message

```
TypeError: Object prototype may only be an Object or null: undefined (V8-based)
TypeError: Object.create requires at least 1 argument, but only 0 were passed (Firefox)
TypeError: Object.setPrototypeOf requires at least 2 arguments, but only 0 were passed
(Firefox)
TypeError: Object.defineProperties requires at least 1 argument, but only 0 were passed
(Firefox)
TypeError: Object prototype may only be an Object or null. (Safari)
```

Error type

`TypeError`.

What went wrong?

There is an error with how a function is called. More arguments need to be provided.

Examples

Required arguments not provided

The `Object.create()` method requires at least one argument and the `Object.setPrototypeOf()` method requires at least two arguments:

js · ✗ bad example

```
const obj = Object.create();
// TypeError: Object.create requires at least 1 argument, but only 0 were passed

const obj2 = Object.setPrototypeOf({});
// TypeError: Object.setPrototypeOf requires at least 2 arguments, but only 1 were passed
```

You can fix this by setting `null` as the prototype, for example:

js · ✓ good example

```
const obj = Object.create(null);

const obj2 = Object.setPrototypeOf({}, null);
```

See also

- Functions guide

JavaScript › Reference › JavaScript error reference

TypeError: null/undefined has no properties

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/No_properties · MDN page last modified 2025-07-08

The JavaScript exception "null (or undefined) has no properties" occurs when you attempt to access properties of `null` and `undefined`. They don't have any.

Message

```
TypeError: Cannot read properties of undefined (reading 'x') (V8-based)
TypeError: Cannot destructure 'x' as it is undefined. (V8-based)
TypeError: Cannot destructure property 'x' of 'y' as it is undefined. (V8-based)
TypeError: null has no properties (Firefox)
TypeError: undefined has no properties (Firefox)
TypeError: undefined is not an object (evaluating 'undefined.x') (Safari)
TypeError: Right side of assignment cannot be destructured (Safari)
```

Error type

TypeError.

What went wrong?

Both null and undefined have no properties you could access. Therefore, you cannot use property accessors on them, or destructure them.

Examples

null and undefined have no properties

js · ✗ bad example

```
null.foo;
// TypeError: null has no properties

undefined.bar;
// TypeError: undefined has no properties
```

See also

- null
- undefined

JavaScript › Reference › JavaScript error reference

TypeError: property "x" is non-configurable and can't be deleted

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Cant_delete · MDN page last modified 2025-07-08

The JavaScript exception "property is non-configurable and can't be deleted" occurs when it was attempted to delete a property, but that property is non-configurable.

Message

```
TypeError: Cannot delete property 'x' of #<Object> (V8-based)
TypeError: property "x" is non-configurable and can't be deleted (Firefox)
TypeError: Unable to delete property. (Safari)
```

Error type

TypeError in strict mode only.

What went wrong?

It was attempted to delete a property, but that property is non-configurable. The configurable attribute controls whether the property can be deleted from the object and whether its attributes (other than writable) can be changed.

This error happens only in strict mode code. In non-strict code, the operation returns false.

Examples

Attempting to delete non-configurable properties

Non-configurable properties are not super common, but they can be created using `Object.defineProperty()` or `Object.freeze()`.

js · ✗ bad example

```
"use strict";
const obj = Object.freeze({ name: "Elsa", score: 157 });
delete obj.score; // TypeError
```

js · ✗ bad example

```
"use strict";
const obj = {};
Object.defineProperty(obj, "foo", { value: 2, configurable: false });
delete obj.foo; // TypeError
```

js · ✗ bad example

```
"use strict";
const frozenArray = Object.freeze([0, 1, 2]);
frozenArray.pop(); // TypeError
```

There are also a few non-configurable properties built into JavaScript. Maybe you tried to delete a mathematical constant.

js · ✗ bad example

```
"use strict";
delete Math.PI; // TypeError
```

See also

- `delete`
- `Object.defineProperty()`
- `Object.freeze()`

JavaScript › Reference › JavaScript error reference

TypeError: Reduce of empty array with no initial value

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Reduce_of_empty_array_with_no_initial_value · MDN page last modified 2025-11-03

The JavaScript exception "reduce of empty array with no initial value" occurs when a reduce function is used.

Message

```
TypeError: Reduce of empty array with no initial value (V8-based & Firefox & Safari)
```

Error type

`TypeError`

What went wrong?

In JavaScript, there are several reduce functions:

- `Array.prototype.reduce()`, `Array.prototype.reduceRight()` and
- `TypedArray.prototype.reduce()`, `TypedArray.prototype.reduceRight()`).

These functions optionally take an `initialValue` (which will be used as the first argument to the first call of the `callback`). However, if no initial value is provided, it will use the first element of the `Array` or `TypedArray` as the initial value. This error is raised when an empty array is provided because no initial value can be returned in that case.

Examples

Invalid cases

This problem appears frequently when combined with a filter (`Array.prototype.filter()`, `TypedArray.prototype.filter()`) which will remove all elements of the list. Thus leaving none to be used as the initial value.

js · ✗ bad example

```
const ints = [0, -1, -2, -3, -4, -5];
ints
  .filter((x) => x > 0) // removes all elements
  .reduce((x, y) => x + y); // no more elements to use for the initial value.
```

Similarly, the same issue can happen if there is a typo in a selector, or an unexpected number of elements in a list:

js · ✗ bad example

```
const names = document.getElementsByClassName("names");
const nameList = Array.prototype.reduce.call(
  names,
  (acc, name) => `${acc}, ${name}`,
);
```

Valid cases

These problems can be solved in two different ways.

One way is to actually provide an `initialValue` as the neutral element of the operator, such as 0 for the addition, 1 for a multiplication, or an empty string for a concatenation.

js · ✓ good example

```
const ints = [0, -1, -2, -3, -4, -5];
ints
  .filter((x) => x > 0) // removes all elements
  .reduce((x, y) => x + y, 0); // the initial value is the neutral element of the addition
```

Another way would be to handle the empty case, either before calling `reduce`, or in the callback after adding an unexpected dummy initial value.

js · ✓ good example

```
const names = document.getElementsByClassName("names");

let nameList1 = "";
if (names.length >= 1) {
  nameList1 = Array.prototype.reduce.call(
    names,
    (acc, name) => `${acc}, ${name}`,
  );
}
// nameList1 === "" when names is empty.

const nameList2 = Array.prototype.reduce.call(
  names,
  (acc, name) => {
    if (acc === "")
      // initial value
      return name;
    return `${acc}, ${name}`;
  },
  "",
);
// nameList2 === "" when names is empty.
```

See also

- Array.prototype.reduce()
- Array.prototype.reduceRight()
- TypedArray.prototype.reduce()
- TypedArray.prototype.reduceRight()
- Array
- TypedArray
- Array.prototype.filter()
- TypedArray.prototype.filter()

JavaScript › Reference › JavaScript error reference

TypeError: setting getter-only property "x"

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Getter_only · MDN page last modified 2025-07-08

The JavaScript strict mode-only exception "setting getter-only property" occurs when there is an attempt to set a new value to a property for which only a getter is specified, or when setting a private accessor property that similarly only has a getter defined.

Message

```
TypeError: Cannot set property x of #<Object> which has only a getter (V8-based)
TypeError: '#x' was defined without a setter (V8-based)
TypeError: setting getter-only property "x" (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
TypeError: Trying to access an undefined private setter (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

There is an attempt to set a new value to a property for which only a getter is specified. While this will be silently ignored in non-strict mode, it will throw a `TypeError` in strict mode. Classes are always in strict mode, so assigning to a getter-only private element always throws this error.

Examples

Property with no setter

The example below shows how to set a getter for a property. It doesn't specify a setter, so a `TypeError` will be thrown upon trying to set the `temperature` property to `30`. For more details see also the `Object.defineProperty()` page.

js · ✗ bad example

```
"use strict";

function Archiver() {
  const temperature = null;
  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
  });
}

const arc = new Archiver();
arc.temperature; // 'get!'

arc.temperature = 30;
// TypeError: setting getter-only property "temperature"
```

To fix this error, you will either need to remove the `arc.temperature = 30` line, which attempts to set the temperature property, or you will need to implement a setter for it, for example like this:

js · ✓ good example

```
"use strict";

function Archiver() {
  let temperature = null;
  const archive = [];

  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
    set(value) {
      temperature = value;
      archive.push({ val: temperature });
```

(code continues on the next page)

(code continued from the previous page)

```
    },
  });

  this.getArchive = function () {
    return archive;
  };
}

const arc = new Archiver();
arc.temperature; // 'get!'
arc.temperature = 11;
arc.temperature = 13;
arc.getArchive(); // [{ val: 11 }, { val: 13 }]
```

See also

- `Object.defineProperty()`
- `Object.defineProperties()`

TypeError: WeakSet key/WeakMap value 'x' must be an object or an unregistered symbol

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Key_not_weakly_held · MDN page last modified 2025-08-15

The JavaScript exception "WeakSet key (or WeakMap value) 'x' must be an object or an unregistered symbol" occurs when a value of invalid type is used as a key in a `WeakSet` or as a value in a `WeakMap`.

Message

```
TypeError: Invalid value used as weak map key (V8-based)
TypeError: WeakMap key 1 must be an object or an unregistered symbol (Firefox)
TypeError: WeakMap keys must be objects or non-registered symbols (Safari)

TypeError: Invalid value used in weak set (V8-based)
TypeError: WeakSet value 1 must be an object or an unregistered symbol (Firefox)
TypeError: WeakSet values must be objects or non-registered symbols (Safari)
```

Error type

`TypeError`

What went wrong?

`WeakSet` and `WeakMap` require the keys to be *garbage collectable*. Only objects and non-registered symbols (that is, symbols not returned by `Symbol.for()`) are valid. For more information, see Memory management. If you want to add keys that are strings, numbers, or other primitive values, you should store them in a regular `Set` or `Map` instead.

Examples

Invalid cases

js · ✗ bad example

```
new WeakSet().add(1); // TypeError
new WeakMap().set(1, {}); // TypeError
new WeakSet([1]); // TypeError
new WeakMap([[1, {}]]); // TypeError
```

Valid cases

js · ✓ good example

```
new WeakSet().add({}); // OK
new WeakMap().set({}, 1); // OK

new Set([1]); // OK
new Map([[1, {}]]); // OK
```

See also

- Memory management
- `WeakSet`
- `WeakMap`
- `Set`
- `Map`

JavaScript › Reference › JavaScript error reference

TypeError: X.prototype.y called on incompatible type

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Called_on_incompatible_type · MDN page last modified 2025-07-08

The JavaScript exception "called on incompatible target (or object)" occurs when a function (on a given object), is called with a `this` not corresponding to the type expected by the function.

Message

```
TypeError: Method Set.prototype.add called on incompatible receiver undefined (V8-based)
TypeError: Bind must be called on a function (V8-based)
TypeError: Illegal invocation (V8-based)
TypeError: Function.prototype.toString requires that 'this' be a Function (V8-based)
TypeError: this is not a Date object. (V8-based)
TypeError: this is not a typed array. (V8-based)
TypeError: Function.prototype.toString called on incompatible object (Firefox)
TypeError: Function.prototype.bind called on incompatible target (Firefox)
TypeError: 'addEventListener' called on an object that does not implement interface
EventTarget. (Firefox)
TypeError: Type error (Safari)
TypeError: undefined is not an object (Safari)
```

Error type

`TypeError`

What went wrong?

When this error is thrown, a function (on a given object), is called with a `this` not corresponding to the type expected by the function.

This issue can arise when using the `Function.prototype.call()` or `Function.prototype.apply()` methods, and providing a `this` argument which does not have the expected type.

This issue can also happen when providing a function that is stored as a property of an object as an argument to another function. In this case, the object that stores the function won't be the `this` target of that function when it is called by the other function. To work-around this issue, you will either need to wrap the callback function in another function, or use the `Function.prototype.bind()` method to force the `this` argument to the expected object.

Examples

Invalid cases

js · ✗ bad example

```
const mySet = new Set();
["bar", "baz"].forEach(mySet.add);
// mySet.add is a function, but "mySet" is not captured as this.

function myFun() {
  console.log(this);
}
["bar", "baz"].forEach(myFun.bind);
// myFun.bind is a function, but "myFun" is not captured as this.
```

Valid cases

js · ✓ good example

```
const mySet = new Set();
["bar", "baz"].forEach(mySet.add.bind(mySet));
// This works due to binding "mySet" as this.

function myFun() {
  console.log(this);
}
["bar", "baz"].forEach((x) => myFun.bind(x));
// This works using the "bind" function. It creates a new function forwarding the argument.
```

See also

- `Function.prototype.call()`
- `Function.prototype.apply()`
- `Function.prototype.bind()`

URIError: malformed URI sequence

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Malformed_URI · MDN page last modified 2025-07-08

The JavaScript exception "malformed URI sequence" occurs when URI encoding or decoding wasn't successful.

Message

```
URIError: URI malformed (V8-based)
URIError: malformed URI sequence (Firefox)
URIError: String contained an illegal UTF-16 sequence. (Safari)
```

Error type

URIError

What went wrong?

URI encoding or decoding wasn't successful. An argument given to either the decodeURI, encodeURI, encodeURIComponent, or decodeURIComponent function was not valid, so that the function was unable encode or decode properly.

Examples

Encoding

Encoding replaces each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character. An URIError will be thrown if there is an attempt to encode a surrogate which is not part of a high-low pair, for example:

js · ✗ bad example

```
encodeURI("\uD800");
// "URIError: malformed URI sequence"

encodeURI("\uDFFF");
// "URIError: malformed URI sequence"
```

A high-low pair is OK. For example:

js · ✓ good example

```
encodeURI("\uD800\uDFFF");
// "%F0%90%8F%BF"
```

Decoding

Decoding replaces each escape sequence in the encoded URI component with the character that it represents. If there isn't such a character, an error will be thrown:

js · ✗ bad example

```
decodeURIComponent("%E0%A4%A");
// "URIError: malformed URI sequence"
```

With proper input, this should usually look like something like this:

js · ✓ good example

```
decodeURIComponent("JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B");
// "JavaScript_шеллы"
```

See also

- URIError
- decodeURI
- encodeURI
- encodeURIComponent
- decodeURIComponent

JavaScript › Reference › JavaScript error reference

Warning: -file- is being assigned a //# sourceMappingURL, but already has one

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Already_has_pragma · MDN page last modified 2025-07-08

The JavaScript warning "-file- is being assigned a //# sourceMappingURL, but already has one." occurs when a source map has been specified more than once for a given JavaScript source.

Message

```
Warning: -file- is being assigned a //# sourceMappingURL, but already has one.
```

Error type

A warning. JavaScript execution won't be halted.

What went wrong?

A source map has been specified more than once for a given JavaScript source.

JavaScript sources are often combined and minified to make delivering them from the server more efficient. With source maps, the debugger can map the code being executed to the original source files. There are two ways to assign a source map, either by using a comment or by setting a header to the JavaScript file.

Examples

Setting source maps

Setting a source map by using a comment in the file:

js · ✓ good example

```
//# sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Or, alternatively, you can set a header to your JavaScript file:

http · ✓ good example

```
X-SourceMap: /path/to/file.js.map
```

See also

- Use a source map in the Firefox source docs
- Introduction to JavaScript source maps on developer.chrome.com (2012)

JavaScript › Reference › JavaScript error reference

Warning: unreachable code after return statement

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Stmt_after_return · MDN page last modified 2025-07-08

The JavaScript warning "unreachable code after return statement" occurs when using an expression after a `return` statement, or when using a semicolon-less return statement but including an expression directly after.

Message

```
Warning: unreachable code after return statement (Firefox)
```

Error type

Warning

What went wrong?

Unreachable code after a return statement might occur in these situations:

- When using an expression after a `return` statement, or
- when using a semicolon-less return statement but including an expression directly after.

When an expression exists after a valid `return` statement, a warning is given to indicate that the code after the `return` statement is unreachable, meaning it can never be run.

Why should I have semicolons after `return` statements? In the case of semicolon-less `return` statements, it can be unclear whether the developer intended to return the statement on the following line, or to stop execution and return. The warning indicates that there is ambiguity in the way the `return` statement is written.

Warnings will not be shown for semicolon-less returns if these statements follow it:

- `throw`
- `break`
- `var`
- `function`

Examples

Invalid cases

js · ✗ bad example

```
function f() {
  let x = 3;
  x += 4;
  return x;   // return exits the function immediately
  x -= 3;     // so this line will never run; it is unreachable
}

function g() {
  return     // this is treated like `return;`
    3 + 4;   // so the function returns, and this line is never reached
}
```

Valid cases

js · ✓ good example

```
function f() {
  let x = 3;
  x += 4;
  x -= 3;
  return x; // OK: return after all other statements
}

function g() {
  return 3 + 4 // OK: semicolon-less return with expression on the same line
}
```

See also

- Automatic semicolon insertion

JavaScript › Reference

JavaScript technologies overview

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview · MDN page last modified 2026-11-05

Whereas HTML defines a webpage's structure and content and CSS sets the formatting and appearance, JavaScript adds interactivity to a webpage and creates rich web applications.

However, the umbrella term "JavaScript" as understood in a web browser context contains several very different elements. One of them is the core language (ECMAScript), another is the collection of the Web APIs, including the DOM (Document Object Model).

JavaScript, the core language (ECMAScript)

The core language of JavaScript is standardized by the ECMA TC39 committee as a language named ECMAScript. "ECMAScript" is the term for the language standard, but "ECMAScript" and "JavaScript" can be used interchangeably.

This core language is also used in non-browser environments, for example in Node.js.

What falls under the ECMAScript scope?

Among other things, ECMAScript defines:

- Language syntax (parsing rules, keywords, control flow, object literal initialization, ...)
- Error handling mechanisms (`throw`, `try...catch`, ability to create user-defined `Error` types)
- Types (boolean, number, string, function, object, ...)
- A prototype-based inheritance mechanism
- Built-in objects and functions, including `JSON`, `Math`, Array methods, `parseInt`, `decodeURI`, etc.
- Strict mode
- A module system
- Basic memory model

Standardization process

ECMAScript editions are approved and published as a standard by the ECMA General Assembly on a yearly basis. All development is public on the Ecma TC39 GitHub organization, which hosts proposals, the official specification text, and meeting notes.

Before the 6th edition of ECMAScript (known as ES6), specifications were published once every several years, and are commonly referred to by their major version numbers — ES3, ES5, etc. After ES6, the specification is named by the publishing year — ES2017, ES2018, etc. ES6 is synonymous with ES2015. *ESNext* is a dynamic name that refers to whatever the next version is at the time of writing. ESNext features are more correctly called proposals, because, by definition, the specification has not been finalized yet.

The current committee-approved snapshot of ECMA-262 is available in PDF and HTML format on Ecma International's ECMA-262 language specification page. ECMA-262 and ECMA-402 are continuously maintained and kept up to date by the specification editors; the TC39 website hosts the latest, up-to-date ECMA-262 and ECMA-402 versions.

New language features, including introduction of new syntaxes and APIs and revision of existing behaviors, are discussed in the form of proposals. Each proposal goes through a staged process, and is

typically implemented by JavaScript engines at stage 3 or stage 4 and thus available for public consumption.

See Wikipedia ECMAScript entry for more information on ECMAScript history.

Internationalization API

The ECMAScript Internationalization API Specification is an addition to the ECMAScript Language Specification, also standardized by Ecma TC39. The internationalization API provides collation (string comparison), number formatting, and date-and-time formatting for JavaScript applications, letting the applications choose the language and tailor the functionality to their needs. The initial standard was approved in December 2012; the status of implementations in browsers is tracked in the documentation of the `Intl` object. The Internationalization specification is nowadays also ratified on a yearly basis and browsers constantly improve their implementation.

Related resources

There are a variety of ways you can participate in or just track current work on the ECMAScript Language Specification and the ECMAScript Internationalization API Specification and related resources:

- ECMAScript Language Specification repo
- ECMAScript Internationalization API Specification repo
- ECMAScript proposals repo
- ECMAScript conformance test suite repo
- TC39 meeting notes
- ECMAScript spec discussion; current mailing list
- ECMAScript spec discussion; historical mailing-list archives (until March 2021)

DOM APIs

WebIDL

The WebIDL specification provides the glue between the DOM technologies and ECMAScript.

The Core of the DOM

The Document Object Model (DOM) is a cross-platform, **language-independent convention** for representing and interacting with objects in HTML, XHTML and XML documents. Objects in the **DOM tree** may be addressed and manipulated by using methods on the objects. Nowadays, the DOM core spec is maintained by WHATWG (superseding the W3C version). It defines language-agnostic interfaces that abstract HTML and XML documents as objects, and also defines mechanisms to manipulate this abstraction. This includes: `Node`, `Element`, `DocumentFragment`, `Document`, `DOMImplementation`, `Event`, `EventTarget`, and more.

From the ECMAScript point of view, objects defined in the DOM specification are called "host objects".

HTML DOM

HTML, the Web's markup language, is specified in terms of the DOM. Layered above the abstract concepts defined in DOM Core, HTML also defines the *meaning* of elements. The HTML DOM includes APIs such as `Document.body` and HTML element interfaces.

The HTML specification also defines restrictions on documents; for example, it requires all children of a `<ul>` element, which represents an unordered list, to be `<li>` elements, as those represent list items. In general, it also forbids using elements and attributes that aren't defined in a standard.

Looking for the `Document` object, `Window` object, and the other DOM elements? Read the DOM documentation.

Other notable APIs

- The `setTimeout()` and `setInterval()` functions were first specified on the `Window` interface in HTML Standard.
- XMLHttpRequest makes it possible to send asynchronous HTTP requests.
- The Fetch API provides a more ergonomic abstraction for network requests.
- The CSS Object Model abstracts CSS rules as objects.
- WebWorkers allow parallel computation.
- WebSockets allow low-level bidirectional communication.
- Canvas 2D Context is a drawing API for `<canvas>`.
- The WebAssembly interface provides utilities for communication between JavaScript code and WebAssembly modules.

Non-browser environments (like Node.js) often do not have DOM APIs — because they don't interact with a document — but they still usually implement many web APIs, such as `fetch()` and `setTimeout()`.

JavaScript implementations

JavaScript engines used in current web browsers include:

- Mozilla's SpiderMonkey, used in Firefox, Servo, and Flow. Other non-browser usage includes MongoDB, CouchDB, and more. This was the first *ever* JavaScript engine, created by Brendan Eich at Netscape.
- Google's V8, used in Chrome and Chromium-based browsers such as Opera, Edge, and Brave. Other non-browser usage includes Node.js, Deno, Electron, and more.
- Apple's JavaScriptCore (also known as SquirrelFish/Nitro), used in Safari and other WebKit-based browsers. Other non-browser usage includes Bun.
- LibJS, used in Ladybird.

Some JavaScript engines used in earlier browsers include:

- Carakan, used in Opera before it became a Chromium-based browser.
- Microsoft's Chakra, used in Internet Explorer (although the language it implements is formally called "JScript" to avoid trademark issues). Earlier versions of Edge used another JavaScript engine, confusingly also called Chakra, before it became a Chromium-based browser.

Some JavaScript engines specifically tailored for non-browser purposes include:

- Engine262, written in JavaScript and intended essentially as a reference implementation of the language.
- Meta's Hermes, optimized for React Native.
- Mozilla's Rhino, written in Java.
- Oracle's GraalJS, written in Java and built on top of GraalVM.

- Moddable XS, intended for IoT/embedded systems.
- QuickJS, intended to be small and lightweight.

JavaScript engines expose a public API which application developers can use to integrate JavaScript into their software. By far, the most common host environment for JavaScript is web browsers. Web browsers typically use the public API to create **host objects** responsible for reflecting the DOM into JavaScript.

Another common application for JavaScript is as a (Web) server-side scripting language. A JavaScript web server exposes host objects representing an HTTP request and response objects, which can then be manipulated by a JavaScript program to dynamically generate web pages. Node.js is a popular example of this.

Shells

A JavaScript shell allows you to quickly test snippets of JavaScript code without having to reload a web page. They are extremely useful for developing and debugging code.

Standalone JavaScript shells

The following JavaScript shells are stand-alone environments, like Perl or Python.

- Node.js - A V8-based JavaScript runtime.
- Bun - A JavaScriptCore-based JavaScript runtime.

Browser-based JavaScript shells

The following JavaScript shells run code through the browser's JavaScript engine.

- Firefox has a built-in JavaScript console, which supports multi-line editing.
- Babel REPL - A browser-based REPL for experimenting with future JavaScript.
- TypeScript playground — A browser-based playground for experimenting with both new JavaScript features (via the tsc compiler) and TypeScript syntax.

Tools & resources

Helpful tools for writing and debugging your JavaScript code.

Firefox Developer Tools
: Web Console, JavaScript Profiler, Debugger, and more.

Learn JavaScript
: An excellent resource for aspiring web developers — Learn JavaScript in an interactive environment, with short lessons and interactive tests, guided by automated assessment. The first 40 lessons are free, and the complete course is available for a small one-time payment.

TogetherJS
: Collaboration made easy. By adding TogetherJS to your site, your users can help each other out on a website in real-time!

Stack Overflow
: Stack Overflow questions tagged with "JavaScript".

JSFiddle

Edit JavaScript, CSS, and HTML and get live results. Use external resources and collaborate with your team online.

Plunker

Plunker is an online community for creating, collaborating on, and sharing your web development ideas. Edit your JavaScript, CSS, and HTML files and get live results and file structure.

JS Bin

JS Bin is an open-source collaborative web development debugging tool.

CodePen

CodePen is another collaborative web development tool used as a live result playground.

StackBlitz

StackBlitz is another online playground/debugging tool, which can host and deploy full-stack applications using React, Angular, etc.

RunJS

RunJS is a desktop playground/scratchpad tool, which provides live results and access to both Node and Browser APIs.

JavaScript › Reference

JavaScript execution model

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model · MDN page last modified 2026-11-02

This page introduces the basic infrastructure of the JavaScript runtime environment. The model is largely theoretical and abstract, without any platform-specific or implementation-specific details. Modern JavaScript engines heavily optimize the described semantics.

This page is a reference. It assumes you are already familiar with the execution model of other programming languages, such as C and Java. It makes heavy references to existing concepts in operating systems and programming languages.

The engine and the host

JavaScript execution requires the cooperation of two pieces of software: the **JavaScript engine** and the **host environment**.

The JavaScript engine implements the ECMAScript (JavaScript) language, providing the core functionality. It takes source code, parses it, and executes it. However, in order to interact with the outside world, such as to produce any meaningful output, to interface with external resources, or to implement security- or performance-related mechanisms, we need additional environment-specific mechanisms provided by the host environment. For example, the HTML DOM is the host environment when JavaScript is executed in a web browser. Node.js is another host environment that allows JavaScript to be run on the server side.

While we focus primarily on the mechanisms defined in ECMAScript in this reference, we will occasionally talk about mechanisms defined in the HTML spec, which is often mimicked by other host environments like Node.js or Deno. This way, we can give a coherent picture of the JavaScript execution model as used on the web and beyond.

Agent execution model

In the JavaScript specification, each autonomous executor of JavaScript is called an **agent**, which maintains its facilities for code execution:

- **Heap** (of objects): this is just a name to denote a large (mostly unstructured) region of memory. It gets populated as objects get created in the program. Note that in the case of shared memory, each agent has its own heap with its own version of a `SharedArrayBuffer` object, but the underlying memory represented by the buffer is shared.
- **Queue** (of jobs): this is known in HTML (and also commonly) as the *event loop* which enables asynchronous programming in JavaScript while being single-threaded. It's called a queue because it's generally first-in-first-out: earlier jobs are executed before later ones.
- **Stack** (of execution contexts): this is what's known as a *call stack* and allows transferring control flow by entering and exiting execution contexts like functions. It's called a stack because it's last-in-first-out. Every job enters by pushing a new frame onto the (empty) stack, and exits by emptying the stack.

These are three distinct data structures that keep track of different data. We will introduce the queue and the stack in more detail in the following sections. To read more about how heap memory is allocated and freed, see memory management.

Each agent is analogous to a thread (note that the underlying implementation may or may not be an actual operating system thread). Each agent can own multiple realms (which 1-to-1 correlate with global objects) that can synchronously access each other, and thus needs to run in a single execution thread. An agent also has a single memory model, indicating whether it's little-endian, whether it can be synchronously blocked, whether atomic operations are lock-free, etc.

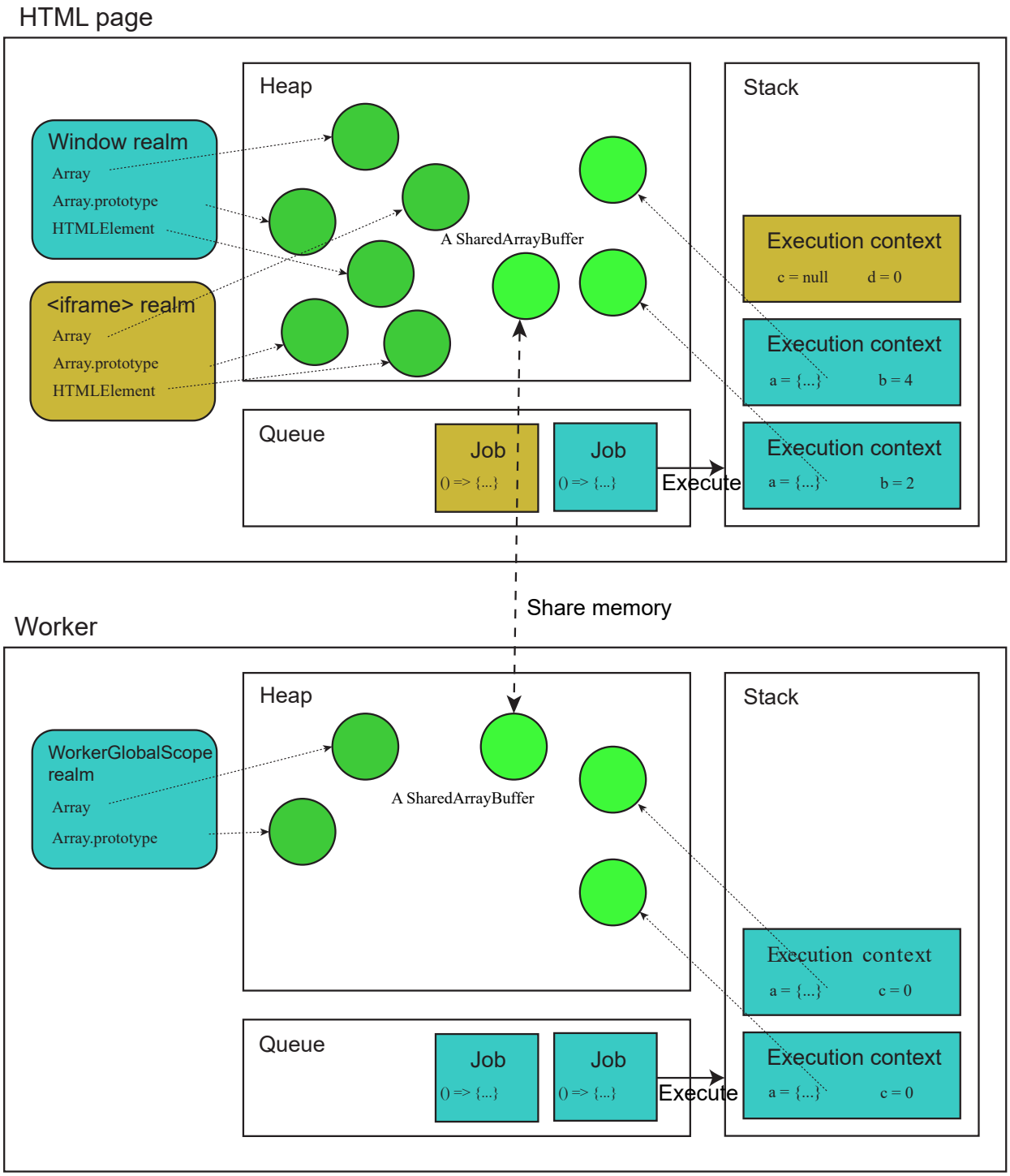
An agent on the web can be one of the following:

- A *Similar-origin window agent*, which contains various `Window` objects which can potentially reach each other, either directly or by using `document.domain`. If the window is origin-keyed, then only same-origin windows can reach each other.
- A *Dedicated worker agent* containing a single `DedicatedWorkerGlobalScope`.
- A *Shared worker agent* containing a single `SharedWorkerGlobalScope`.
- A *Service worker agent* containing a single `ServiceWorkerGlobalScope`.
- A *Worklet agent* containing a single `WorkletGlobalScope`.

In other words, each worker creates its own agent, while one or more windows may be within the same agent—usually a main document and its similar-origin iframes. In Node.js, a similar concept called worker threads is available.

The diagram below illustrates the execution model of agents:

Agent cluster



Realms

Each agent owns one or more **realms**. Each piece of JavaScript code is associated with a realm when it's loaded, which remains the same even when called from another realm. A realm consists of the following information:

- A list of intrinsic objects like `Array`, `Array.prototype`, etc.
- Globally declared variables, the value of `globalThis`, and the global object
- A cache of template literal arrays, because evaluation of the same tagged template literal expression always causes the tag to receive the same array object

On the web, the realm and the global object are 1-to-1 corresponded. The global object is either a `Window`, a `WorkerGlobalScope`, or a `WorkletGlobalScope`. So for example, every `iframe` executes in a different realm, though it may be in the same agent as the parent window.

Realms are usually mentioned when talking about the identities of global objects. For example, we need methods such as `Array.isArray()` or `Error.isError()`, because an array constructed in another realm will have a different prototype object than the `Array.prototype` object in the current realm, so `instanceof Array` will wrongly return `false`.

Stack and execution contexts

We first consider synchronous code execution. Each job enters by calling its associated callback. Code inside this callback may create variables, call functions, or exit. Each function needs to keep track of its own variable environments and where to return to. To handle this, the agent needs a stack to keep track of the execution contexts. An **execution context**, also known generally as a *stack frame*, is the smallest unit of execution. It tracks the following information:

- Code evaluation state
- The module or script, the function (if applicable), and the currently executing generator that contains this code
- The current realm
- Bindings, including:
  - Variables defined with `var`, `let`, `const`, `function`, `class`, etc.
  - Private identifiers like `#foo` which are only valid in the current context
  - `this` reference

Imagine a program consisting of a single job defined by the following code:

```js
function foo(b) {
  const a = 10;
  return a + b + 11;
}

function bar(x) {
  const y = 3;
  return foo(x * y);
}

const baz = bar(7); // assigns 42 to baz
```

1. When the job starts, the first frame is created, where the variables `foo`, `bar`, and `baz` are defined. It calls `bar` with the argument `7`.
2. A second frame is created for the `bar` call, containing bindings for the parameter `x` and the local variable `y`. It first performs the multiplication `x * y`, then calls `foo` with the result.
3. A third frame is created for the `foo` call, containing bindings for the parameter `b` and the local variable `a`. It first performs the addition `a + b + 11`, then returns the result.
4. When `foo` returns, the top frame element is popped out of the stack, and the call expression `foo(x * y)` resolves to the return value. It then continues execution, which is just to return this result.
5. When `bar` returns, the top frame element is popped out of the stack, and the call expression `bar(7)` resolves to the return value. This initializes `baz` with the return value.

6. We reach the end of the job's source code, so the stack frame for the entrypoint is popped out of the stack. The stack is empty, so the job is considered completed.

Generators and reentry

When a frame is popped, it's not necessarily gone forever, because sometimes we need to come back to it. For example, consider a generator function:

```js
function* gen() {
  console.log(1);
  yield;
  console.log(2);
}

const g = gen();
g.next(); // logs 1
g.next(); // logs 2
```

In this case, calling `gen()` first creates an execution context which is suspended—no code inside `gen` gets executed yet. The generator `g` saves this execution context internally. The current running execution context remains to be the entrypoint. When `g.next()` is called, the execution context for `gen` is pushed onto the stack, and the code inside `gen` is executed until the `yield` expression. Then, the generator execution context gets suspended and removed from the stack, which returns control back to the entrypoint. When `g.next()` is called again, the generator execution context is pushed back onto the stack, and the code inside `gen` resumes from where it left off.

Tail calls

One mechanism defined in the specification is *proper tail call* (PTC). A function call is a tail call if the caller does nothing after the call except return the value:

```js
function f() {
  return g();
}
```

In this case, the call to `g` is a tail call. In strict mode only, if a function call is in tail position, the engine is required to discard the current execution context and replace it with the context of the tail call, instead of pushing a new frame for the `g()` call. This means that tail recursion is not subject to the stack size limits:

```js
function factorial(n, acc = 1) {
  if (n <= 1) return acc;
  return factorial(n - 1, n * acc);
}
```

In reality, discarding the current frame causes debugging problems, because if `g()` throws an error, `f` is no longer on the stack and won't appear in the stack trace. Currently, only Safari (JavaScriptCore) implements PTC, and they have invented some specific infrastructure to address the debuggability issue.

Closures

Another interesting phenomenon related to variable scoping and function calls is closures. Whenever a function is created, it also memorizes internally the variable bindings of the current running execution context. Then, these variable bindings can outlive the execution context.

```js
let f;
{
  let x = 10;
  f = () => x;
}
console.log(f()); // logs 10
```

Job queue and event loop

An agent is a thread, which means the interpreter can only process one statement at a time. When the code is all synchronous, this is fine because we can always make progress. But if the code needs to perform asynchronous action, then we cannot progress unless that action is completed. However, it would be detrimental to user experience if that halts the whole program—the nature of JavaScript as a web scripting language requires it to be never blocking. Therefore, the code that handles the completion of that asynchronous action is defined as a callback. This callback defines a **job**, which gets placed into a **job queue**—or, in HTML terminology, an event loop—once the action is completed.

Every time, the agent pulls a job from the queue and executes it. When the job is executed, it may create more jobs, which are added to the end of the queue. Jobs can also be added via the completion of asynchronous platform mechanisms, such as timers, I/O, and events. A job is considered completed when the stack is empty; then, the next job is pulled from the queue. Jobs might not be pulled with uniform priority—for example, HTML event loops split jobs into two categories: *tasks* and *microtasks*. Microtasks have higher priority and the microtask queue is drained first before the task queue is pulled. For more information, check the HTML microtask guide. If the job queue is empty, the agent waits for more jobs to be added.

"Run-to-completion"

Each job is processed completely before any other job is processed. This offers some nice properties when reasoning about your program, including the fact that whenever a function runs, it cannot be preempted and will run entirely before any other code runs (and can modify data the function manipulates). This differs from C, for instance, where if a function runs in a thread, it may be stopped at any point by the runtime system to run some other code in another thread.

For example, consider this example:

```js
const promise = Promise.resolve();
let i = 0;
promise.then(() => {
  i += 1;
  console.log(i);
});
promise.then(() => {
  i += 1;
  console.log(i);
});
```

In this example, we create an already-resolved promise, which means any callback attached to it will be immediately scheduled as a job. The two callbacks seem to cause a race condition, but actually, the output is fully predictable: `1` and `2` will be logged in order. This is because each job runs to completion before the next one is executed, so the overall order is always `i += 1; console.log(i); i += 1; console.log(i);` and never `i += 1; i += 1; console.log(i); console.log(i);`.

A downside of this model is that if a job takes too long to complete, the web application is unable to process user interactions like click or scroll. The browser mitigates this with the "a script is taking too long to run" dialog. A good practice to follow is to make job processing short and, if possible, cut down one job into several jobs.

Never blocking

Another important guarantee offered by the event loop model is that JavaScript execution is never blocking. Handling I/O is typically performed via events and callbacks, so when the application is waiting for an IndexedDB query to return or a `fetch()` request to return, it can still process other things like user input. The code that executes after the completion of an asynchronous action is always provided as a callback function (for example, the promise `then()` handler, the callback function in `setTimeout()`, or the event handler), which defines a job to be added to the job queue once the action completes.

Of course, the guarantee of "never-blocking" requires the platform API to be inherently asynchronous, but some legacy exceptions exist like `alert()` or synchronous XHR. It is considered good practice to avoid them to ensure the responsiveness of the application.

Agent clusters and memory sharing

Multiple agents can communicate via memory sharing, forming an **agent cluster**. Agents are within the same cluster if and only if they can share memory. Agents not in the same cluster can still exchange information with other protocols, such as `postMessage()`.

When creating an agent (such as by spawning a worker), there are some criteria for whether it's in the same cluster as the current agent, or a new cluster is created. For example, the following pairs of global objects are each within the same agent cluster, and thus can share memory with each other:

- A `Window` object and a dedicated worker that it created.
- A worker (of any type) and a dedicated worker it created.
- A `Window` object A and the `Window` object of a same-origin `iframe` element that A created.
- A `Window` object and a same-origin `Window` object that opened it.
- A `Window` object and a worklet that it created.

The following pairs of global objects are not within the same agent cluster, and thus cannot share memory:

- A `Window` object and a shared worker it created.
- A worker (of any type) and a shared worker it created.
- A `Window` object and a service worker it created.
- A `Window` object A and the `Window` object of an `iframe` element that A created and is cross-origin with A when A's agent cluster is origin-keyed.
- Any two `Window` objects with no opener or ancestor relationship. This holds even if the two `Window` objects are same origin.

For the exact algorithm, check the HTML specification.

Cross-agent communication and memory model

As aforementioned, agents communicate via memory sharing. On the web, memory is shared via the `postMessage()` method. The using web workers guide provides an overview of this. Typically, data is passed by value only (via structured cloning), and therefore does not involve any concurrency complications. To share memory, one must post a `SharedArrayBuffer` object, which can be simultaneously accessed by multiple agents. Once two agents share access to the same memory via a `SharedArrayBuffer`, they can synchronize executions via the `Atomics` object.

There are two ways to access shared memory: via normal memory access (which is not atomic) and via atomic memory access. The latter is sequentially consistent (which means there is a strict total ordering of events agreed upon by all agents in the cluster), while the former is unordered (which means no ordering exists); JavaScript does not provide operations with other ordering guarantees.

The spec provides the following guidelines for programmers working with shared memory:

> We recommend programs be kept data race free, i.e., make it so that it is impossible for there to be concurrent non-atomic operations on the same memory location. Data race free programs have interleaving semantics where each step in the evaluation semantics of each agent are interleaved with each other. For data race free programs, it is not necessary to understand the details of the memory model. The details are unlikely to build intuition that will help one to better write ECMAScript.
>
> More generally, even if a program is not data race free it may have predictable behavior, so long as atomic operations are not involved in any data races and the operations that race all have the same access size. The simplest way to arrange for atomics not to be involved in races is to ensure that different memory cells are used by atomic and non-atomic operations and that atomic accesses of different sizes are not used to access the same cells at the same time. Effectively, the program should treat shared memory as strongly typed as much as possible. One still cannot depend on the ordering and timing of non-atomic accesses that race, but if memory is treated as strongly typed the racing accesses will not "tear" (bits of their values will not be mixed).

Concurrency and ensuring forward progress

When multiple agents cooperate, the never-blocking guarantee does not always hold. An agent can become *blocked*, or paused, while waiting for another agent to perform some action. This is different from waiting on a promise in the same agent, because it halts the entire agent and does not allow any other code to run in the meantime—in other words, it cannot make *forward progress*.

To prevent deadlocks, there are some strong restrictions on when and which agents can become blocked.

- Every unblocked agent with a dedicated executing thread eventually makes forward progress.
- In a set of agents that share an executing thread, one agent eventually makes forward progress.
- An agent does not cause another agent to become blocked except via explicit APIs that provide blocking.
- Only certain agents can be blocked. On the web, this includes dedicated workers and shared workers, but not similar-origin windows or service workers.

The agent cluster ensures some level of integrity over the activeness of its agents, in the case of external pauses or terminations:

- An agent may be paused or resumed without its knowledge or cooperation. For example, navigating away from a window may suspend code execution but preserve its state. However, an agent cluster is not allowed to be partially deactivated, to avoid an agent starving because another agent has been deactivated. For example, shared workers are never in the same agent cluster as the creator window or other dedicated workers. This is because a shared worker's lifetime is independent of documents: if a document is deactivated while its dedicated worker holds a lock, the shared worker is blocked from acquiring the lock until the dedicated worker is reactivated, if ever. Meanwhile other workers trying to access the shared worker from other windows will starve.
- Similarly, an agent may be terminated by factors external to the cluster. For example, operating systems or users killing a browser process, or the browser force-terminating one agent because it's using too many resources. In this case, all the agents in the cluster get terminated. (The spec also allows a second strategy, which is an API that allows at least one remaining member of the cluster to identify the termination and the agent that was terminated, but this is not implemented on the web.)

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/executable-code-and-execution-contexts.html`
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/memory-model.html`
HTML `https://html.spec.whatwg.org/multipage/webappapis.html`

See also

- Agent communication and coordination
- Event loops in the HTML standard
- What is the Event Loop? in the Node.js docs

JavaScript › Reference

Lexical grammar

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Lexical_grammar · MDN page last modified 2026-11-02

This page describes JavaScript's lexical grammar. JavaScript source text is just a sequence of characters — in order for the interpreter to understand it, the string has to be *parsed* to a more structured representation. The initial step of parsing is called lexical analysis, in which the text gets scanned from left to right and is converted into a sequence of individual, atomic input elements. Some input elements are insignificant to the interpreter, and will be stripped after this step — they include white space and comments. The others, including identifiers, keywords, literals, and punctuators (mostly operators), will be used for further syntax analysis. Line terminators and multiline comments are also syntactically insignificant, but they guide the process for automatic semicolon insertion to make certain invalid token sequences become valid.

Format-control characters

Format-control characters have no visual representation but are used to control the interpretation of the text.

Code point	Name	Abbreviation	Description
U+200C	Zero width non-joiner	<ZWNJ>	Placed between characters to prevent being connected into ligatures in certain languages (Wikipedia).
U+200D	Zero width joiner	<ZWJ>	Placed between characters that would not normally be connected in order to cause the characters to be rendered using their connected form in certain languages (Wikipedia).
U+FEFF	Byte order mark	<BOM>	Used at the start of the script to mark it as Unicode and to allow detection of the text's encoding and byte order (Wikipedia).

In JavaScript source text, <ZWNJ> and <ZWJ> are treated as identifier parts, while <BOM> (also called a zero-width no-break space <ZWNBSP> when not at the start of text) is treated as white space.

White space

White space characters improve the readability of source text and separate tokens from each other. These characters are usually unnecessary for the functionality of the code. Minification tools are often used to remove whitespace in order to reduce the amount of data that needs to be transferred.

Code point	Name	Abbreviation	Description	Escape sequence
U+0009	Character tabulation	<TAB>	Horizontal tabulation	\t
U+000B	Line tabulation	<VT>	Vertical tabulation	\v
U+000C	Form feed	<FF>	Page breaking control character (Wikipedia).	\f
U+0020	Space	<SP>	Normal space	
U+00A0	No-break space	<NBSP>	Normal space, but no point at which a line may break	
U+FEFF	Zero-width no-break space	<ZWNBSP>	When not at the start of a script, the BOM marker is a normal whitespace character.	
Others	Other Unicode space characters	<USP>	Characters in the "Space_Separator" general category	

> **Note:** Of those characters with the "White_Space" property but are not in the "Space_Separator" general category, U+0009, U+000B, and U+000C are still treated as white space in JavaScript; U+0085 NEXT LINE has no special role; others become the set of line terminators.

> **Note:** Changes to the Unicode standard used by the JavaScript engine may affect programs' behavior. For example, ES2016 upgraded the reference Unicode standard from 5.1 to 8.0.0, which caused U+180E MONGOLIAN VOWEL SEPARATOR to be moved from the "Space_Separator" category to the "Format (Cf)" category, and made it a non-whitespace. Subsequently, the result of `"\u180E".trim().length` changed from `0` to `1`.

Line terminators

In addition to white space characters, line terminator characters are used to improve the readability of the source text. However, in some cases, line terminators can influence the execution of JavaScript code as there are a few places where they are forbidden. Line terminators also affect the process of automatic semicolon insertion.

Outside the context of lexical grammar, white space and line terminators are often conflated. For example, `String.prototype.trim()` removes all white space and line terminators from the beginning and end of a string. The `\s` character class escape in regular expressions matches all white space and line terminators.

Only the following Unicode code points are treated as line terminators in ECMAScript. Other line breaking characters are treated as white space (for example, Line Tabulation, VT, U+000B).

Code point	Name	Abbreviation	Description	Escape sequence
U+000A	Line Feed	<LF>	New line character in UNIX systems.	\n
U+000D	Carriage Return	<CR>	New line character in Commodore and early Mac systems.	\r
U+2028	Line Separator	<LS>	Wikipedia	
U+2029	Paragraph Separator	<PS>	Wikipedia	

Comments

Comments are used to add hints, notes, suggestions, or warnings to JavaScript code. This can make it easier to read and understand. They can also be used to disable code to prevent it from being executed; this can be a valuable debugging tool.

JavaScript has two long-standing ways to add comments to code: line comments and block comments. In addition, there's a special hashbang comment syntax.

Line comments

The first way is the `//` comment; this makes all text following it on the same line into a comment. For example:

```js
function comment() {
  // This is a one line JavaScript comment
  console.log("Hello world!");
}
comment();
```

Block comments

The second way is the `/* */` style, which is much more flexible.

For example, you can use it on a single line:

```js
function comment() {
  /* This is a one line JavaScript comment */
  console.log("Hello world!");
}
comment();
```

You can also make multiple-line comments, like this:

```js
function comment() {
  /* This comment spans multiple lines. Notice
     that we don't need to end the comment until we're done. */
  console.log("Hello world!");
}
comment();
```

You can also use it in the middle of a line, if you wish, although this can make your code harder to read so it should be used with caution:

```js
function comment(x) {
  console.log("Hello " + x /* insert the value of x */ + " !");
}
comment("world");
```

In addition, you can use it to disable code to prevent it from running, by wrapping code in a comment, like this:

```js
function comment() {
  /* console.log("Hello world!"); */
}
comment();
```

In this case, the `console.log()` call is never issued, since it's inside a comment. Any number of lines of code can be disabled this way.

Block comments that contain at least one line terminator behave like line terminators in automatic semicolon insertion.

Hashbang comments

There's a special third comment syntax, the **hashbang comment**. A hashbang comment behaves exactly like a single line-only (`//`) comment, except that it begins with `#!` and **is only valid at the absolute**

start of a script or module. Note also that no whitespace of any kind is permitted before the `#!`. The comment consists of all the characters after `#!` up to the end of the first line; only one such comment is permitted.

Hashbang comments in JavaScript resemble shebangs in Unix which provide the path to a specific JavaScript interpreter that you want to use to execute the script. Before the hashbang comment became standardized, it had already been de-facto implemented in non-browser hosts like Node.js, where it was stripped from the source text before being passed to the engine. An example is as follows:

```js
#!/usr/bin/env node

console.log("Hello world");
```

The JavaScript interpreter will treat it as a normal comment — it only has semantic meaning to the shell if the script is directly run in a shell.

> **Warning:** If you want scripts to be runnable directly in a shell environment, encode them in UTF-8 without a BOM. Although a BOM will not cause any problems for code running in a browser — because it's stripped during UTF-8 decoding, before the source text is analyzed — a Unix/Linux shell will not recognize the hashbang if it's preceded by a BOM character.

You must only use the `#!` comment style to specify a JavaScript interpreter. In all other cases just use a `//` comment (or multiline comment).

Identifiers

An *identifier* is used to link a value with a name. Identifiers can be used in various places:

```js
const decl = 1; // Variable declaration (may also be `let` or `var`)
function fn() {} // Function declaration
const obj = { key: "value" }; // Object keys
// Class declaration
class C {
  #priv = "value"; // Private field
}
lbl: console.log(1); // Label
```

In JavaScript, identifiers are commonly made of alphanumeric characters, underscores (_), and dollar signs ($). Identifiers are not allowed to start with numbers. However, JavaScript identifiers are not only limited to ASCII — many Unicode code points are allowed as well. Namely:

- Start characters can be any character in the ID_Start category plus _ and $.
- After the first character, you can use any character in the ID_Continue category, which contains U+200C (ZWNJ) and U+200D (ZWJ).

> **Note:** If, for some reason, you need to parse some JavaScript source yourself, do not assume all identifiers follow the pattern `/[A-Za-z_$][\w$]*/` (i.e., ASCII-only)! The range of identifiers can be described by the regex `/[$_\p{ID_Start}][$\p{ID_Continue}]*/u` (excluding unicode escape sequences).

In addition, JavaScript allows using Unicode escape sequences in the form of `\u0000` or `\u{000000}` in identifiers, which encode the same string value as the actual Unicode characters. For example, 你好 and

`\u4f60\u597d` are the same identifiers:

js

```
const 你好 = "Hello";
console.log(\u4f60\u597d); // Hello
```

Not all places accept the full range of identifiers. Certain syntaxes, such as function declarations, function expressions, and variable declarations require using identifier names that are not reserved words.

js · ✗ bad example

```
function import() {} // Illegal: import is a reserved word.
```

Most notably, private elements and object properties allow reserved words.

js

```
const obj = { import: "value" }; // Legal despite `import` being reserved
class C {
  #import = "value";
}
```

Keywords

Keywords are tokens that look like identifiers but have special meanings in JavaScript. For example, the keyword `async` before a function declaration indicates that the function is asynchronous.

Some keywords are *reserved*, meaning that they cannot be used as an identifier for variable declarations, function declarations, etc. They are often called *reserved words*. A list of these reserved words is provided below. Not all keywords are reserved — for example, `async` can be used as an identifier anywhere. Some keywords are only *contextually reserved* — for example, `await` is only reserved within the body of an async function, and `let` is only reserved in strict mode code, or `const` and `let` declarations.

Identifiers are always compared by *string value*, so escape sequences are interpreted. For example, this is still a syntax error:

js · ✗ bad example

```
const els\u{65} = 1;
// `els\u{65}` encodes the same identifier as `else`
```

Reserved words

These keywords cannot be used as identifiers for variables, functions, classes, etc. anywhere in JavaScript source.

- `break`
- `case`
- `catch`
- `class`
- `const`
- `continue`
- `debugger`
- `default`

- `delete`
- `do`
- `else`
- `export`
- `extends`
- `false`
- `finally`
- `for`
- `function`
- `if`
- `import`
- `in`
- `instanceof`
- `new`
- `null`
- `return`
- `super`
- `switch`
- `this`
- `throw`
- `true`
- `try`
- `typeof`
- `var`
- `void`
- `while`
- `with`

The following are only reserved when they are found in strict mode code:

- `let` (also reserved in `const`, `let`, and class declarations)
- `static`
- `yield` (also reserved in generator function bodies)

The following are only reserved when they are found in module code or async function bodies:

- `await`

Future reserved words

The following are reserved as future keywords by the ECMAScript specification. They have no special functionality at present, but they might at some future time, so they cannot be used as identifiers.

These are always reserved:

- `enum`

The following are only reserved when they are found in strict mode code:

- `implements`

- `interface`
- `package`
- `private`
- `protected`
- `public`

Future reserved words in older standards

The following are reserved as future keywords by older ECMAScript specifications (ECMAScript 1 till 3).

- `abstract`
- `boolean`
- `byte`
- `char`
- `double`
- `final`
- `float`
- `goto`
- `int`
- `long`
- `native`
- `short`
- `synchronized`
- `throws`
- `transient`
- `volatile`

Identifiers with special meanings

A few identifiers have a special meaning in some contexts without being reserved words of any kind. They include:

- `arguments` (not a keyword, but cannot be declared as an identifier in strict mode)
- `as` (`import * as ns from "mod"`)
- `async`
- `eval` (not a keyword, but cannot be declared as an identifier in strict mode)
- `from` (`import x from "mod"`)
- `get`
- `of`
- `set`

Literals

> **Note:** This section discusses literals that are atomic tokens. Object literals and array literals are expressions that consist of a series of tokens.

Null literal

See also null for more information.

```js
null
```

Boolean literal

See also boolean type for more information.

```js
true
false
```

Numeric literals

The Number and BigInt types use numeric literals.

Decimal

```js
1234567890
42
```

Decimal literals can start with a zero (0) followed by another decimal digit, but if all digits after the leading 0 are smaller than 8, the number is interpreted as an octal number. This is considered a legacy syntax, and number literals prefixed with 0, whether interpreted as octal or decimal, cause a syntax error in strict mode — so, use the 0o prefix instead.

```
js · ✗ bad example
0888 // 888 parsed as decimal
0777 // parsed as octal, 511 in decimal
```

Exponential

The decimal exponential literal is specified by the following format: beN; where b is a base number (integer or floating), followed by an E or e character (which serves as separator or *exponent indicator*) and N, which is *exponent* or *power* number – a signed integer.

```js
0e-5   // 0
0e+5   // 0
5e1    // 50
175e-2 // 1.75
1e3    // 1000
1e-3   // 0.001
1E3    // 1000
```

Binary

Binary number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "B" (0b or 0B). The digits after the prefix must be 0 or 1.

```js
0b10000000000000000000000000000000 // 2147483648
0b01111111100000000000000000000000 // 2139095040
0B00000000011111111111111111111111 // 8388607
```

Octal

Octal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `0O`). The digits after the prefix must be in the range `0–7`.

```js
0O755 // 493
0o644 // 420
```

Hexadecimal

Hexadecimal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "X" (`0x` or `0X`). The digits after the prefix must be in the ranges `0–9`, `A–F`, or `a–f`.

```js
0xFFFFFFFFFFFFF // 4503599627370495
0xabcdef123456  // 188900967593046
0XA             // 10
```

BigInt literal

The BigInt type is a numeric primitive in JavaScript that can represent integers with arbitrary precision. BigInt literals are created by appending `n` to the end of an integer.

```js
123456789123456789n     // 123456789123456789
0o777777777777n         // 68719476735
0x123456789ABCDEFn      // 81985529216486895
0b11101001010101010101n // 955733
```

Decimal BigInt literals cannot have leading zeros to avoid confusion with legacy octal literals.

js · ✗ bad example

```
0755n; // SyntaxError: invalid BigInt syntax
```

For octal `BigInt` numbers, always use zero followed by the letter "o" (uppercase or lowercase):

js · ✓ good example

```
0o755n;
```

For more information about `BigInt`, see also JavaScript data structures.

Numeric separators

To improve readability for numeric literals, underscores (`_`, `U+005F`) can be used as separators:

```js
1_000_000_000_000
1_050.95
0b1010_0001_1000_0101
0o2_2_5_6
0xA0_B0_C0
1_000_000_000_000_000_000_000n
```

Note these limitations:

js · ✗ bad example

```
// More than one underscore in a row is not allowed
100__000; // SyntaxError

// Not allowed at the end of numeric literals
100_; // SyntaxError

// Can not be used after leading 0
0_1; // SyntaxError
```

String literals

A string literal is zero or more Unicode code points enclosed in single or double quotes. Unicode code points may also be represented by an escape sequence. All code points may appear literally in a string literal except for these code points:

- U+005C \ (backslash)
- U+000D <CR>
- U+000A <LF>
- The same kind of quote that begins the string literal

Any code points may appear in the form of an escape sequence. String literals evaluate to ECMAScript String values. When generating these String values Unicode code points are UTF-16 encoded.

js

```
'foo'
"bar"
```

The following subsections describe various escape sequences (\ followed by one or more characters) available in string literals. Any escape sequence not listed below becomes an "identity escape" that becomes the code point itself. For example, `\z` is the same as `z`. There's a deprecated octal escape sequence syntax described in the Deprecated and obsolete features page. Many of these escape sequences are also valid in regular expressions — see Character escape.

Escape sequences

Special characters can be encoded using escape sequences:

Escape sequence	Unicode code point
`\0`	null character (U+0000 NULL)
`\'`	single quote (U+0027 APOSTROPHE)
`\"`	double quote (U+0022 QUOTATION MARK)
`\\`	backslash (U+005C REVERSE SOLIDUS)
`\n`	newline (U+000A LINE FEED; LF)
`\r`	carriage return (U+000D CARRIAGE RETURN; CR)
`\v`	vertical tab (U+000B LINE TABULATION)
`\t`	tab (U+0009 CHARACTER TABULATION)
`\b`	backspace (U+0008 BACKSPACE)
`\f`	form feed (U+000C FORM FEED)
\ followed by a line terminator	empty string

The last escape sequence, \ followed by a line terminator, is useful for splitting a string literal across multiple lines without changing its meaning.

```js
const longString =
  "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

Make sure there is no space or any other character after the backslash (except for a line break), otherwise it will not work. If the next line is indented, the extra spaces will also be present in the string's value.

You can also use the + operator to append multiple strings together, like this:

```js
const longString =
  "This is a very long string which needs " +
  "to wrap across multiple lines because " +
  "otherwise my code is unreadable.";
```

Both of the above methods result in identical strings.

Hexadecimal escape sequences

Hexadecimal escape sequences consist of \x followed by exactly two hexadecimal digits representing a code unit or code point in the range 0x0000 to 0x00FF.

```js
"\xA9"; // "©"
```

Unicode escape sequences

A Unicode escape sequence consists of exactly four hexadecimal digits following \u. It represents a code unit in the UTF-16 encoding. For code points U+0000 to U+FFFF, the code unit is equal to the code point. Code points U+10000 to U+10FFFF require two escape sequences representing the two code units (a surrogate pair) used to encode the character; the surrogate pair is distinct from the code point.

See also `String.fromCharCode()` and `String.prototype.charCodeAt()`.

```js
"\u00A9"; // "©" (U+A9)
```

Unicode code point escapes

A Unicode code point escape consists of \u{, followed by a code point in hexadecimal base, followed by }. The value of the hexadecimal digits must be in the range 0 and 0x10FFFF inclusive. Code points in the range U+10000 to U+10FFFF do not need to be represented as a surrogate pair.

See also `String.fromCodePoint()` and `String.prototype.codePointAt()`.

```js
"\u{2F804}"; // CJK COMPATIBILITY IDEOGRAPH-2F804 (U+2F804)

// the same character represented as a surrogate pair
"\uD87E\uDC04";
```

Regular expression literals

Regular expression literals are enclosed by two forward slashes (`/`). The lexer consumes all characters up to the next unescaped forward slash or the end of the line, unless the forward slash appears within a character class (`[]`). Some characters (namely, those that are identifier parts) can appear after the closing slash, denoting flags.

The lexical grammar is very lenient: not all regular expression literals that get identified as one token are valid regular expressions.

See also `RegExp` for more information.

```js
/ab+c/g;
/[/]/;
```

A regular expression literal cannot start with two forward slashes (`//`), because that would be a line comment. To specify an empty regular expression, use `/(?:)/`.

Template literals

One template literal consists of several tokens: `` `xxx${ `` (template head), `}xxx${` (template middle), and `}xxx` ` (template tail) are individual tokens, while any expression may come between them.

See also template literals for more information.

```js
`string text`;

`string text line 1
 string text line 2`;

`string text ${expression} string text`;

tag`string text ${expression} string text`;
```

Automatic semicolon insertion

Some JavaScript statements' syntax definitions require semicolons (`;`) at the end. They include:

- `var`, `let`, `const`, `using`, `await using`
- Expression statements
- `do...while`
- `continue`, `break`, `return`, `throw`
- `debugger`
- Class field declarations (public or private)
- `import`, `export`

However, to make the language more approachable and convenient, JavaScript is able to automatically insert semicolons when consuming the token stream, so that some invalid token sequences can be "fixed" to valid syntax. This step happens after the program text has been parsed to tokens according to the lexical grammar. There are three cases when semicolons are automatically inserted:

1. When a token not allowed by the grammar is encountered, and it's separated from the previous token by at least one line terminator (including a block comment that includes at least one line terminator),

or the token is "}", then a semicolon is inserted before the token.

```js
{ 1
2 } 3

// is transformed by ASI into:

{ 1
;2 ;} 3;

// Which is valid grammar encoding three statements,
// each consisting of a number literal
```

The ending ")" of `do...while` is taken care of as a special case by this rule as well.

```js
do {
  // …
} while (condition) /* ; */ // ASI here
const a = 1
```

However, semicolons are not inserted if the semicolon would then become the separator in the `for` statement's head.

js · ✗ bad example

```
for (
  let a = 1 // No ASI here
  a < 10 // No ASI here
  a++
) {}
```

Semicolons are also never inserted as empty statements. For example, in the code below, if a semicolon is inserted after ")", then the code would be valid, with an empty statement as the `if` body and the `const` declaration being a separate statement. However, because automatically inserted semicolons cannot become empty statements, this causes a declaration to become the body of the `if` statement, which is not valid.

js · ✗ bad example

```
if (Math.random() > 0.5)
const x = 1 // SyntaxError: Unexpected token 'const'
```

2. When the end of the input stream of tokens is reached, and the parser is unable to parse the single input stream as a complete program, a semicolon is inserted at the end.

```js
const a = 1 /* ; */ // ASI here
```

This rule is a complement to the previous rule, specifically for the case where there's no "offending token" but the end of input stream.

3. When the grammar forbids line terminators in some place but a line terminator is found, a semicolon is inserted. These places include:

- `expr <here> ++`, `expr <here> --`
- `continue <here> lbl`
- `break <here> lbl`

- `return <here> expr`
- `throw <here> expr`
- `yield <here> expr`
- `yield <here> * expr`
- `(param) <here> => {}`
- `async <here> function`, `async <here> prop()`, `async <here> function*`, `async <here> *prop()`, `async <here> (param) <here> => {}`
- `using <here> id`, `await <here> using <here> id`

Here `++` is not treated as a postfix operator applying to variable `b`, because a line terminator occurs between `b` and `++`.

js

```
a = b
++c

// is transformed by ASI into

a = b;
++c;
```

Here, the `return` statement returns `undefined`, and the `a + b` becomes an unreachable statement.

js

```
return
a + b

// is transformed by ASI into

return;
a + b;
```

Note that ASI would only be triggered if a line break separates tokens that would otherwise produce invalid syntax. If the next token can be parsed as part of a valid structure, semicolons would not be inserted. For example:

js · ✗ bad example

```
const a = 1
(1).toString()

const b = 1
[1, 2, 3].forEach(console.log)
```

Because `()` can be seen as a function call, it would usually not trigger ASI. Similarly, `[]` may be a member access. The code above is equivalent to:

js · ✗ bad example

```
const a = 1(1).toString();

const b = 1[1, 2, 3].forEach(console.log);
```

This happens to be valid syntax. `1[1, 2, 3]` is a property accessor with a comma-joined expression. Therefore, you would get errors like "1 is not a function" and "Cannot read properties of undefined (reading 'forEach')" when running the code.

Within classes, class fields and generator methods can be a pitfall as well.

js · ✗ bad example

```
class A {
  a = 1
  *gen() {}
}
```

It is seen as:

js · ✗ bad example

```
class A {
  a = 1 * gen() {}
}
```

And therefore will be a syntax error around `{`.

There are the following rules-of-thumb for dealing with ASI, if you want to enforce semicolon-less style:

- Write postfix `++` and `--` on the same line as their operands.

  js · ✗ bad example

  ```
  const a = b
  ++
  console.log(a) // ReferenceError: Invalid left-hand side expression in prefix operation
  ```

  js · ✓ good example

  ```
  const a = b++
  console.log(a)
  ```

- The expressions after `return`, `throw`, or `yield` should be on the same line as the keyword.

  js · ✗ bad example

  ```
  function foo() {
    return
      1 + 1 // Returns undefined; 1 + 1 is ignored
  }
  ```

  js · ✓ good example

  ```
  function foo() {
    return 1 + 1
  }

  function foo() {
    return (
      1 + 1
    )
  }
  ```

- Similarly, the label identifier after `break` or `continue` should be on the same line as the keyword.

  js · ✗ bad example

  ```
  outerBlock: {
    innerBlock: {
      break
        outerBlock // SyntaxError: Illegal break statement
    }
  }
  ```

js · ✓ good example

```
outerBlock: {
  innerBlock: {
    break outerBlock
  }
}
```

- The `=>` of an arrow function should be on the same line as the end of its parameters.

js · ✗ bad example

```
const foo = (a, b)
  => a + b
```

js · ✓ good example

```
const foo = (a, b) =>
  a + b
```

- The `async` of async functions, methods, etc. cannot be directly followed by a line terminator.

js · ✗ bad example

```
async
function foo() {}
```

js · ✓ good example

```
async function
foo() {}
```

- The `using` keyword in `using` and `await using` statements should be on the same line as the first identifier it declares.

js · ✗ bad example

```
using
resource = acquireResource()
```

js · ✓ good example

```
using resource
  = acquireResource()
```

- If a line starts with one of `(`, `[`, `` ` ``, `+`, `-`, `/` (as in regex literals), prefix it with a semicolon, or end the previous line with a semicolon.

js · ✗ bad example

```
// The () may be merged with the previous line as a function call
(() => {
  // …
})()

// The [ may be merged with the previous line as a property access
[1, 2, 3].forEach(console.log)

// The ` may be merged with the previous line as a tagged template literal
`string text ${data}`.match(pattern).forEach(console.log)

// The + may be merged with the previous line as a binary + expression
+a.toString()
```

(code continues on the next page)

(code continued from the previous page)

```
// The - may be merged with the previous line as a binary - expression
-a.toString()

// The / may be merged with the previous line as a division expression
/pattern/.exec(str).forEach(console.log)
```

js · ✓ good example

```
;(() => {
  // …
})()
;[1, 2, 3].forEach(console.log)
;`string text ${data}`.match(pattern).forEach(console.log)
;+a.toString()
;-a.toString()
;/pattern/.exec(str).forEach(console.log)
```

- Class fields should preferably always be ended with semicolons — in addition to the previous rule (which includes a field declaration followed by a computed property, since the latter starts with `[`), semicolons are also required between a field declaration and a generator method.

js · ✗ bad example

```
class A {
  a = 1
  [b] = 2
  *gen() {} // Seen as a = 1[b] = 2 * gen() {}
}
```

js · ✓ good example

```
class A {
  a = 1;
  [b] = 2;
  *gen() {}
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-lexical-grammar.html`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Array literals (`[1, 2, 3]`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Binary numeric literals (`0b`)	41	12	25	28	9	41	25	28	9	4.0	41	9	1.0.0	1.0	4.0.0
Boolean literals (`true`/`false`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Decimal numeric literals (`1234567890`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Hashbang (`#!`) comment syntax	74	79	67	62	13.1	74	67	53	13.4	11.0	74	13.4	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Hexadecimal escape sequences (`'\xA9'`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Hexadecimal numeric literals (`0xAF`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Null literal (`null`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Numeric separators (`1_000_000_000_000`)	75	79	70	62	13	75	79	54	13	11.0	75	13	1.0.0	1.2	12.5.0
Octal numeric literals (`0o`)	41	12	25	28	9	41	25	28	9	4.0	41	9	1.0.0	1.0	4.0.0
Regular expression literals (`/ab+c/g`)	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Shorthand notation for object literals	43	12	33	30	9	43	33	30	9	4.0	43	9	1.0.0	1.0	4.0.0
String literals (`'Hello world'`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Template literals	41	12	34	28	9	41	34	28	9	4.0	41	9	1.0.0	1.0	4.0.0
Escape sequences allowed in tagged template literals	62	79	53	49	11	62	53	46	11	8.0	62	11	1.1.35	1.0	8.10.0
Trailing commas	1	12	1	9.5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in dynamic import	91	91	138	**No**	15	91	138	**No**	15	16.0	91	15	1.0.0	1.17	17.5.0 [1,2,3]
Trailing comma in function parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
Trailing comma in object literals	1	12	1	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unicode escape sequences (`'\u00A9'`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unicode point escapes (`\u{}`)	44	12	40	31	9	44	40	32	9	4.0	44	9	1.0.0	1.0	4.0.0

1. Also from version 16.15.0–17.0.0: supported.
2. Also from version 16.14.0–16.15.0: partial support.
3. Also from version 16.14.0–16.15.0: The second parameter no longer throws a parser error, but the `--experimental-json-modules` flag is still needed to load JSON modules.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.grammar`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Grammar and types guide
- Micro-feature from ES6, now in Firefox Aurora and Nightly: binary and octal numbers by Jeff Walden (2013)
- JavaScript character escape sequences by Mathias Bynens (2011)

JavaScript › Reference

Magic comments

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Magic_comments · MDN page last modified 2026-10-22

Magic comments (also called **comment directives**, **annotations**, etc.) are special types of comments that are recognized by certain engines, bundlers, type checkers, debuggers, etc. (collectively referred to as *consumers*) to enable opt-in functionality. Because they're comments, consumers that don't understand them ignore them. This article introduces the various types of magic comments commonly seen in JavaScript source code and their intended consumers.

Usually, the presence or absence of magic comments doesn't change the program's runtime behavior—for example, they may optimize execution, enable/disable static checking, or provide additional metadata. This contrasts with *directives*, which are strings that do alter runtime behavior. Examples include the standard `"use strict"` directive to opt into strict mode and React's `"use server"` and `"use client"` directives that determine if a component is executed server-side or client-side.

To avoid collision with regular comments, magic comments are usually marked by *sigils*, such as a leading `#` or `@` character: `//# my-setting-name` or `//@ my-setting-name`. The exact syntax varies between comment types.

> **Note:** There's no universal name for this type of feature, and the "alternatives" we listed may have subtle differences: *pragmas* are usually placed at the beginning of the file and inform compilers or engines about the entire script; *annotations* may be placed anywhere and apply to a specific construct; *directives* are generic terms but may be ambiguous with the JavaScript `"use strict"` style convention. In our writing, we use all these words interchangeably.

Compile hint pragmas

Compile hint pragmas give JavaScript engine hints about how to compile the code ahead-of-time. Their interpretation depends on the engine.

They are specified in the WICG proposal Explicit JavaScript Compile Hints.

Eager compilation

The following pragma enables eager compilation of all functions in the current script.

```js
//# allFunctionsCalledOnLoad
```

The syntax and placement requirements are as follows:

- This comment must be placed at the beginning of the script before any code or whitespace (only other comments, single-line or block, are allowed to precede it).
- The comment can be either a line comment or a block comment.
- There must be no space between `#` and `//` or `/*`.
- There can be any amount of whitespace before and after `allFunctionsCalledOnLoad`.

In the following example, the magic comment indicates that the functions in the file are likely to be called during page load:

```js
//# allFunctionsCalledOnLoad
function init() {
  console.log("init");
}

init();
```

Without the hint, the engine may defer compilation of a function until it is called.

Eager compilation offers the following benefits:

- It avoids double parsing: During initialization, the engine already does "light parsing" to know where the function starts and ends, and it has to do another full parse when the function is called. When the function can be compiled eagerly, the engine does the full parse up front, without a separate round of light parsing.
- It can be parallelized: During execution, if a function call triggers lazy compilation, the compilation must block the main thread to preserve synchronicity. Script loading is asynchronous and therefore parsing can be more efficiently scheduled, and even happen as the script gets fetched.

However, compiling functions that are never called can waste time and memory. Like everything about performance, you must do benchmarks and weigh the different tradeoffs. The good heuristic is, as the pragma itself says: only enable this mode if the functions are *called on load*.

See also Faster JavaScript Startup with Explicit Compile Hints on v8.dev.

Source map annotations

Source map annotations associate JavaScript code with source files, making generated, evaluated, or minified code easier to debug.

They are specified in the TC39 specification ECMA-426 Source map format.

Source URLs

The following annotation gives a piece of code a URL identifier.

```js
//# sourceURL=<url>
```

The syntax and placement requirements are as follows:

- This comment be must placed at the *end* of the script *after* any code (other single-line comments, whitespace, and line terminators are allowed to follow it).
- The comment must be a line comment.
- The sigil may be a `@` instead of `#`, but `#` is preferred (`//@` might conflict with Internet Explorer pragmas).
- There can be any amount of whitespace before and after `sourceURL=<url>`.
- The `<url>` must not contain whitespace characters (they should be percent-encoded as `%20`).

Source URLs are especially useful for code not originating from resources already associated with URLs, such as code executed with `eval()`:

```js
eval(
  'console.log("Hello"); throw new Error("error");\n//# sourceURL=generated-code.js',
);
```

The above may generate the following output in the browser console:

```
Hello        generated-code.js:1
Uncaught Error: error
    <anonymous> generated-code.js:1
    <anonymous> debugger eval code:1
```

This annotation is used by many features and is primarily associated with debugging:

- Console/debugger output, as demonstrated above.
- The `Error` `stack` property.
- Resolving the `sourceMapURL`.

See also naming evaluated code with `sourceURL` on developer.chrome.com and Give your eval a name with `//@ sourceURL` on Firebug.

Source map URLs

The following annotation links generated code to a source map.

```js
//# sourceMappingURL=<url>
```

The syntax and placement requirements are the same as those of `sourceURL`.

The source map URL can be a relative URL, in which case it can be resolved to the `sourceURL`, the `<script>` element's `src` attribute, the `<script>` element's containing document's origin, etc., as defined in ECMA 426. It can also be a `data:` URL containing an inline source map. The HTTP `SourceMap` header takes precedence over this comment.

A debugger can use the map to display the original source and map breakpoints to a minified bundle. This is also useful in editors for go-to-definition, finding references, etc.

Build tools (bundlers, transpilers, etc.) generate this comment in the compiled output along with the map. Regular developers usually do not need to hand-write them.

Bundler annotations

Bundlers, minifiers, and transpilers use magic comments to control code generation, optimization, and dependency handling. These annotations are consumed during the build, rather than by the JavaScript engine running the output.

There is no specification, and bundlers often invent their own annotations. This section only documents some annotations that are popular and somewhat widely supported. You must find if a bundler supports the particular annotation in its own documentation.

Tree shaking

The `/*#__PURE__*/` and `/*@__PURE__*/` annotations mark a particular function call or constructor invocation as safe to remove if the result is unused:

```js
function createPoint(x, y) {
  return { x, y };
}

const point = /*#__PURE__*/ createPoint(1, 2);
```

> **Note:** This doesn't imply purity in a functional programming sense. The result might be random, depend on external state, etc. But as long as removing it doesn't change the application behavior in a way that matters, it can be annotated as pure.

Without the annotation, the bundler may need to keep the function call even if the result variable, `point`, isn't used:

```js
// -- Compiler output --
function createPoint(x, y) {
  return { x, y };
}

createPoint(1, 2);
```

With the annotation, the bundler can remove the function call entirely, and if there's no other call to `createPoint()`, the function definition as well. An incorrect annotation can cause required behavior to be removed.

The annotation applies to the particular function call. The side-effects of evaluating the argument expressions are separately decided. For example:

```js
/*#__PURE__*/ createPoint(1, getY());
```

In this example, the `createPoint()` call is pure, and bundlers know that evaluating `1` has no side-effects. However, it cannot determine if `getY()` is pure, so the output contains the `getY()` call (and the `createPoint()` call may be retained as well). To elide everything, each function call must be separately annotated:

```js
/*#__PURE__*/ createPoint(1, /*#__PURE__*/ getY());
```

See also esbuild's pure annotations and Terser's annotations.

Some bundlers also recognize `/*#__NO_SIDE_EFFECTS__*/` and `/*@__NO_SIDE_EFFECTS__*/`, which annotate a function declaration, or a supported variable declaration containing a function, so calls to that function can be treated as side-effect-free:

```js
/*#__NO_SIDE_EFFECTS__*/
function createPoint(x, y) {
  return { x, y };
}
```

See Rollup's tree-shaking annotations.

Minimization

There are two important minimization techniques which may be unsafe to apply: *inlining* and *property mangling*. Inlining directly replaces a function call with the function body, while property mangling replaces property names with shorter strings.

You can use `/*@__INLINE__*/` and `/*@__NOINLINE__*/` on particular function calls to opt-in or opt-out of inlining. Things to consider:

- Inlining may improve call performance because it avoids pushing/popping stack frames. However, the engine itself is capable of inlining and source-level inlining may change its decisions in unpredictable ways.
- Inlining may decrease the bundle size if the function is used only once, and increase it if the function is used many times. Minimizers usually make decisions that optimize bundle size.

During property mangling, the minimizer globally replaces property names with shorter strings (while keeping distinct property names distinct). This is unsafe because the object may be accessible outside (passed to external functions, returned from exported functions, etc.), so they are usually opt-in. If you opt into mangling, you probably want to constrain mangling to name patterns known to be internal, such as names prefixed with underscores.

The `/*@__KEY__*/` annotation marks a string literal as a property name to rename. By default, the minimizer can only check for specific syntaxes, so strings passed to arbitrary functions must be explicitly marked.

```js
const record = { _internalValue: 42 };
Object.getOwnPropertyDescriptor(record, /*@__KEY__*/ "_internalValue");
```

The `/*@__MANGLE_PROP__*/` annotation can explicitly enable mangling for a particular property or class field.

See also Terser Annotations.

JSX transformation

JSX transpilers use file-level pragmas to select how JSX is converted into JavaScript. For example:

```jsx
/** @jsxRuntime automatic */
/** @jsxImportSource preact */

const heading = <h1>Hello</h1>;
```

This generates:

```js
import { jsx as _jsx } from "preact/jsx-runtime";

const heading = _jsx("h1", {
  children: "Hello",
});
```

These settings can also be globally configured in the transpiler; pragmas are only needed if a particular file uses a different setting.

See also Babel's JSX transform and esbuild's JSX configuration.

Static checker directives

Static checkers and related development tools also interpret comments as configuration or metadata. Some examples include:

- **JSDoc**: JSDoc annotations such as `@type` and `@param` supply type information. They also power editor features such as pop-up descriptions.
- **TypeScript (`tsc`)**: `// @ts-check` and `// @ts-nocheck` enable or disable type checking for a JavaScript file. `// @ts-ignore` suppresses diagnostics on the next line; `// @ts-expect-error` also reports an error if there was no error to suppress.
- **ESLint**: Configuration comments such as `/* eslint no-console: "warn" */` configure rules, while `// eslint-disable-next-line no-console` suppresses a rule for the next line.
- **Prettier**: `// prettier-ignore` excludes the next syntax node from formatting.
- **Flow**: `// @flow` enables type checking for a file. Comment types, such as `/*: number */`, embed type syntax inside JavaScript comments.

Legacy conditional compilation

> **Warning:** This section talks about an IE-specific mechanism. Like IE itself, it is now obsolete, but you may continue to see it in old code, and it continues to influence language design—such as why `//# sourceMapURL` is favored over `//@ sourceMapURL`. It is kept for historical interest.

Internet Explorer's JScript engine supported *conditional compilation*, an obsolete, non-standard mechanism that could interpret code inside specially marked comments. Unlike optimization hints or debugging metadata, these comments could change which code executed.

The `@cc_on` statement enabled conditional compilation. The `@set` statement defined conditional compilation variables, and `@if`, `@elif`, `@else`, and `@end` selected code based on those variables. Predefined variables included `@_jscript_version`, which identified the JScript engine version.

```js
var supportsConditionalCompilation = false;
/*@cc_on
  supportsConditionalCompilation = true;
@*/
```

In a supporting engine, the assignment inside the comment would execute. Other engines would treat the entire block as an ordinary comment, leaving the variable as `false`. A line-comment form, `//@cc_on`, also existed.

See also Microsoft's historical JScript Conditional Compilation draft.

JavaScript › Reference

Iteration protocols

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Iteration_protocols · MDN page last modified 2026-11-02

Iteration protocols aren't new built-ins or syntax, but *protocols*. These protocols can be implemented by any object by following some conventions.

There are two protocols: The iterable protocol and the iterator protocol.

The iterable protocol

The iterable protocol allows JavaScript objects to define or customize their iteration behavior, such as what values are looped over in a `for...of` construct. Some built-in types are built-in iterables with a default iteration behavior, such as `Array` or `Map`, while other types (such as `Object`) are not.

In order to be **iterable**, an object must implement the **`[Symbol.iterator]()`** method, meaning that the object (or one of the objects up its prototype chain) must have a property with a `[Symbol.iterator]` key which is available via constant `Symbol.iterator`:

`[Symbol.iterator]()`

A zero-argument function that returns an object, conforming to the iterator protocol.

Whenever an object needs to be iterated (such as at the beginning of a `for...of` loop), its `[Symbol.iterator]()` method is called with no arguments, and the returned **iterator** is used to obtain the values to be iterated.

Note that when this zero-argument function is called, it is invoked as a method on the iterable object. Therefore inside of the function, the `this` keyword can be used to access the properties of the iterable object, to decide what to provide during the iteration.

This function can be an ordinary function, or it can be a generator function, so that when invoked, an iterator object is returned. Inside of this generator function, each entry can be provided by using `yield`.

The iterator protocol

The iterator protocol defines a standard way to produce a sequence of values (either finite or infinite), and potentially a return value when all values have been generated.

An object is an iterator when it implements a **`next()`** method with the following semantics:

`next()`

A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface (see below). If a non-object value gets returned (such as `false` or `undefined`) when a built-in language feature (such as `for...of`) is using the iterator, a `TypeError` (`"iterator.next() returned a non-object value"`) will be thrown.

All iterator protocol methods (`next()`, `return()`, and `throw()`) are expected to return an object implementing the `IteratorResult` interface. It must have the following properties:

done Optional

A boolean that's `false` if the iterator was able to produce the next value in the sequence. (This is equivalent to not specifying the `done` property altogether.)

Has the value `true` if the iterator has completed its sequence. In this case, `value` optionally specifies the return value of the iterator.

`value` Optional

Any JavaScript value returned by the iterator. Can be omitted when `done` is `true`.

In practice, neither property is strictly required; if an object without either property is returned, it's effectively equivalent to `{ done: false, value: undefined }`.

If an iterator returns a result with `done: true`, any subsequent calls to `next()` are expected to return `done: true` as well, although this is not enforced on the language level.

The `next` method can receive a value which will be made available to the method body. The only language feature that passes a value is `yield*`. The value passed to the `next` method of generators will become the value of the corresponding `yield` expression.

Optionally, the iterator can also implement the **`return(value)`** and **`throw(exception)`** methods, which, when called, tell the iterator that the caller is done with iterating it and can perform any necessary cleanup (such as closing a database connection).

`return(value)` Optional

A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface, typically with `value` equal to the `value` passed in and `done` equal to `true`. Calling this method tells the iterator that the caller does not intend to make any more `next()` calls and can perform any cleanup actions. When built-in language features call `return()` for cleanup, `value` is always `undefined`.

`throw(exception)` Optional

A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface, typically with `done` equal to `true`. Calling this method tells the iterator that the caller detects an error condition, and `exception` is typically an `Error` instance. No built-in language feature calls `throw()` for cleanup purposes — it's a special feature of generators for the symmetry of `return`/`throw`.

> **Note:** It is not possible to know reflectively (i.e., without actually calling `next()` and validating the returned result) whether a particular object implements the iterator protocol.

It is very easy to make an iterator also iterable: just implement a `[Symbol.iterator]()` method that returns `this`.

```js
// Satisfies both the Iterator Protocol and Iterable
const myIterator = {
  next() {
    // …
  },
  [Symbol.iterator]() {
    return this;
  },
};
```

Such an object is called an *iterable iterator*. Doing so allows an iterator to be consumed by the various syntaxes expecting iterables — therefore, it is seldom useful to implement the Iterator Protocol without also implementing Iterable. (In fact, almost all syntaxes and APIs expect *iterables*, not *iterators*.) The generator object is an example:

```js
const generatorObject = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

console.log(typeof generatorObject.next);
// "function" — it has a next method (which returns the right result), so it's an iterator

console.log(typeof generatorObject[Symbol.iterator]);
// "function" — it has a [Symbol.iterator] method (which returns the right iterator), so
it's an iterable

console.log(generatorObject[Symbol.iterator]() === generatorObject);
// true — its [Symbol.iterator] method returns itself (an iterator), so it's an iterable
iterator
```

All built-in iterators inherit from `Iterator.prototype`, which implements the `[Symbol.iterator]()` method as returning `this`, so that built-in iterators are also iterable.

However, when possible, it's better for `iterable[Symbol.iterator]()` to return different iterators that always start from the beginning, like `Set.prototype[Symbol.iterator]()` does.

The async iterator and async iterable protocols

There is another pair of protocols used for async iteration, named **async iterator** and **async iterable** protocols. They have very similar interfaces compared to the iterable and iterator protocols, except that each return value from the calls to the iterator methods is wrapped in a promise.

An object implements the async iterable protocol when it implements the following methods:

`[Symbol.asyncIterator]()`
: A zero-argument function that returns an object, conforming to the async iterator protocol.

An object implements the async iterator protocol when it implements the following methods:

`next()`
: A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

`return(value)` Optional
: A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

`throw(exception)` Optional
: A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

Interactions between the language and iteration protocols

The language specifies APIs that either produce or consume iterables and iterators.

Built-in iterables

`String`, `Array`, `TypedArray`, `Map`, `Set`, and `Segments` (returned by `Intl.Segmenter.prototype.segment()`) are all built-in iterables, because each of their `prototype` objects implements a `[Symbol.iterator]()` method. In addition, the `arguments` object and some DOM collection types such as `NodeList` are also iterables. Other than `AsyncGenerator`, there is no object in the core JavaScript language that is async iterable. Some web APIs, such as `ReadableStream`, have the `Symbol.asyncIterator` method set by default.

Generator functions return generator objects, which are iterable iterators. Async generator functions return async generator objects, which are async iterable iterators.

The iterators returned from built-in iterables actually all inherit from a common class `Iterator`, which implements the aforementioned `[Symbol.iterator]() { return this; }` method, making them all iterable iterators. The `Iterator` class also provides additional helper methods in addition to the `next()` method required by the iterator protocol. You can inspect an iterator's prototype chain by logging it in a graphical console.

```
console.log([][Symbol.iterator]());

Array Iterator {}
  [[Prototype]]: Array Iterator     ==> This is the prototype shared by all array iterators
    next: ƒ next()
    Symbol(Symbol.toStringTag): "Array Iterator"
    [[Prototype]]: Object           ==> This is the prototype shared by all built-in
iterators
      Symbol(Symbol.iterator): ƒ [Symbol.iterator]()
      [[Prototype]]: Object         ==> This is Object.prototype
```

Built-in APIs accepting iterables

There are many APIs that accept iterables. Some examples include:

- `Map()`
- `WeakMap()`
- `Set()`
- `WeakSet()`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.race()`
- `Promise.any()`
- `Array.from()`
- `Object.groupBy()`
- `Map.groupBy()`

```js
const myObj = {};

new WeakSet(
  (function* () {
    yield {};
    yield myObj;
    yield {};
  })(),
).has(myObj); // true
```

Syntaxes expecting iterables

Some statements and expressions expect iterables, for example the `for...of` loops, array and parameter spreading, `yield*`, and array destructuring:

```js
for (const value of ["a", "b", "c"]) {
  console.log(value);
}
// "a"
// "b"
// "c"

console.log([..."abc"]); // ["a", "b", "c"]

function* gen() {
  yield* ["a", "b", "c"];
}

console.log(gen().next()); // { value: "a", done: false }

[a, b, c] = new Set(["a", "b", "c"]);
console.log(a); // "a"
```

When built-in syntaxes are iterating an iterator, and the last result's `done` is `false` (i.e., the iterator is able to produce more values) but no more values are needed, the `return` method will get called if present. This can happen, for example, if a `break` or `return` is encountered in a `for...of` loop, or if all identifiers are already bound in an array destructuring.

```js
const obj = {
  [Symbol.iterator]() {
    let i = 0;
    return {
      next() {
        i++;
        console.log("Returning", i);
        if (i === 3) return { done: true, value: i };
        return { done: false, value: i };
      },
      return() {
        console.log("Closing");
        return { done: true };
      },
    };
  },
};
```

(code continues on the next page)

(code continued from the previous page)

```
const [a] = obj;
// Returning 1
// Closing

const [b, c, d] = obj;
// Returning 1
// Returning 2
// Returning 3
// Already reached the end (the last call returned `done: true`),
// so `return` is not called
console.log([b, c, d]); // [1, 2, undefined]; the value associated with `done: true` is not
reachable

for (const b of obj) {
  break;
}
// Returning 1
// Closing
```

Using `for...of`, array spreading, etc. on an async iterable that's not also a sync iterable (i.e., it has `[Symbol.asyncIterator]()` but no `[Symbol.iterator]()`) will throw a TypeError: x is not iterable. You must use syntaxes designed for async iterables, including `for await...of`, `yield*` in async generator functions (but not sync generator functions), and `Array.fromAsync()`. On the other hand, these syntaxes can also take sync iterables and implicitly convert the results to promises.

Error handling

Because iteration involves transferring control back and forth between the iterator and the consumer, error handling happens in both ways: how the consumer handles errors thrown by the iterator, and how the iterator handles errors thrown by the consumer. When you are using one of the built-in ways of iteration, the language may also throw errors because the iterable breaks certain invariants. We will describe how built-in syntaxes generate and handle errors, which can be used as a guideline for your own code if you are manually stepping the iterator.

Non-well-formed iterables

Errors may happen when acquiring the iterator from the iterable. The language invariant enforced here is that the iterable must produce a valid iterator:

- It has a callable `[Symbol.iterator]()` method.
- The `[Symbol.iterator]()` method returns an object.
- The object returned by `[Symbol.iterator]()` has a callable `next()` method.

When using built-in syntax to initiate iteration on a non-well-formed iterable, a TypeError is thrown.

js · ✗ bad example

```
const nonWellFormedIterable = { [Symbol.iterator]: 1 };
[...nonWellFormedIterable]; // TypeError: nonWellFormedIterable is not iterable
nonWellFormedIterable[Symbol.iterator] = () => 1;
[...nonWellFormedIterable]; // TypeError: [Symbol.iterator]() returned a non-object value
nonWellFormedIterable[Symbol.iterator] = () => ({});
[...nonWellFormedIterable]; // TypeError: nonWellFormedIterable[Symbol.iterator]().next is
not a function
```

For async iterables, if its `[Symbol.asyncIterator]()` property has value `undefined` or `null`, JavaScript falls back to using the `[Symbol.iterator]` property instead (and wraps the resulting iterator into an

async iterator by forwarding the methods). Otherwise, the `[Symbol.asyncIterator]` property must conform to the above invariants too.

This type of error can be prevented by first validating the iterable before attempting to iterate it. However, it's fairly rare because usually you know the type of the object you are iterating over. If you are receiving this iterable from some other code, you should just let the error propagate to the caller so they know an invalid input was provided.

Errors during iteration

Most errors happen when stepping the iterator (calling `next()`). The language invariant enforced here is that the `next()` method must return an object (for async iterators, an object after awaiting). Otherwise, a TypeError is thrown.

If the invariant is broken or the `next()` method throws an error (for async iterators, it may also return a rejected promise), the error is propagated to the caller. For built-in syntaxes, the iteration in progress is aborted without retrying or cleanup (with the assumption that if the `next()` method threw the error, then it has cleaned up already). If you are manually calling `next()`, you may catch the error and retry calling `next()`, but in general you should assume the iterator is already closed.

If the caller decides to exit iteration for any reason other than the errors in the previous paragraph, such as when it enters an error state in its own code (for example, while handling an invalid value produced by the iterator), it should call the `return()` method on the iterator, if one exists. This allows the iterator to perform any cleanup. The `return()` method is only called for premature exits—if `next()` returns `done: true`, the `return()` method is not called, with the assumption that the iterator has already cleaned up.

The `return()` method might be invalid too! The language also enforces that the `return()` method must return an object and throws a TypeError otherwise. If the `return()` method throws an error, the error is propagated to the caller. However, if the `return()` method is called because the caller encountered an error in its own code, then this error overrides the error thrown by the `return()` method.

Usually, the caller implements error handling like this:

```js
try {
  for (const value of iterable) {
    // …
  }
} catch (e) {
  // Handle the error
}
```

The `catch` will be able to catch errors thrown when `iterable` is not a valid iterable, when `next()` throws an error, when `return()` throws an error (if the `for` loop exits early), and when the `for` loop body throws an error.

Most iterators are implemented with generator functions, so we will demonstrate how generator functions typically handle errors:

```js
function* gen() {
  try {
    yield doSomething();
    yield doSomethingElse();
  } finally {
    cleanup();
  }
}
```

The lack of `catch` here causes errors thrown by `doSomething()` or `doSomethingElse()` to propagate to the caller of `gen`. If these errors are caught within the generator function (which is equally advisable), the generator function can decide to continue yielding values or to exit early. However, the `finally` block is necessary for generators that keep open resources. The `finally` block is guaranteed to run, either when the last `next()` is called or when `return()` is called.

Forwarding errors

Some built-in syntaxes wrap an iterator into another iterator. They include the iterator produced by `Iterator.from()`, iterator helper methods (`map()`, `filter()`, `take()`, `drop()`, and `flatMap()`), `yield*`, and a hidden wrapper when you use async iteration (`for await...of`, `Array.fromAsync`) on sync iterators. The wrapped iterator is then responsible for forwarding errors between the inner iterator and the caller.

- Wrapper iterators generally directly forward the `next()` method of the inner iterator, including its return value and thrown errors. The async wrapper of a sync iterator awaits the returned object's `value` property and converts thrown errors into promise rejections. If a yielded promise rejects, the wrapper calls the inner iterator's `return()` method, if one exists, before rejecting its own promise. If the inner iterator's `next()` method itself throws, the wrapper rejects its promise without calling `return()`.
- Wrapper iterators generally directly forward the `return()` method of the inner iterator. If the `return()` method doesn't exist on the inner iterator, it returns `{ done: true, value: undefined }` instead. In the case of iterator helpers: if the iterator helper's `next()` method has not been called, after trying to call `return()` on the inner iterator, the current iterator always returns `{ done: true, value: undefined }`. This is consistent with generator functions where execution hasn't entered the `yield*` expression yet.
- `yield*` is the only built-in syntax that forwards the `throw()` method of the inner iterator. For information on how `yield*` forwards the `return()` and `throw()` methods, see its own reference.

Examples

User-defined iterables

You can make your own iterables like this:

```js
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};

console.log([...myIterable]); // [1, 2, 3]
```

Basic iterator

Iterators are stateful by nature. If you don't define it as a generator function (as the example above shows), you would likely want to encapsulate the state in a closure.

```js
function makeIterator(array) {
  let nextIndex = 0;
  return {
    next() {
      return nextIndex < array.length
        ? {
            value: array[nextIndex++],
            done: false,
          }
        : {
            done: true,
          };
    },
  };
}

const it = makeIterator(["yo", "ya"]);

console.log(it.next().value); // 'yo'
console.log(it.next().value); // 'ya'
console.log(it.next().done); // true
```

Infinite iterator

```js
function idMaker() {
  let index = 0;
  return {
    next() {
      return {
        value: index++,
        done: false,
      };
    },
  };
}
```

(code continues on the next page)

(code continued from the previous page)

```
const it = idMaker();

console.log(it.next().value); // 0
console.log(it.next().value); // 1
console.log(it.next().value); // 2
// …
```

Defining an iterable with a generator

js

```
function* makeGenerator(array) {
  let nextIndex = 0;
  while (nextIndex < array.length) {
    yield array[nextIndex++];
  }
}

const gen = makeGenerator(["yo", "ya"]);

console.log(gen.next().value); // 'yo'
console.log(gen.next().value); // 'ya'
console.log(gen.next().done); // true

function* idMaker() {
  let index = 0;
  while (true) {
    yield index++;
  }
}

const it = idMaker();

console.log(it.next().value); // 0
console.log(it.next().value); // 1
console.log(it.next().value); // 2
// …
```

Defining an iterable with a class

State encapsulation can be done with private fields as well.

js

```
class SimpleClass {
  #data;

  constructor(data) {
    this.#data = data;
  }

  [Symbol.iterator]() {
    // Use a new index for each iterator. This makes multiple
    // iterations over the iterable safe for non-trivial cases,
    // such as use of break or nested looping over the same iterable.
    let index = 0;

    return {
      // Note: using an arrow function allows `this` to point to the
      // one of `[Symbol.iterator]()` instead of `next()`
      next: () => {
        if (index >= this.#data.length) {
```

(code continues on the next page)

(code continued from the previous page)

```
          return { done: true };
        }
        return { value: this.#data[index++], done: false };
      },
    };
  }
}

const simple = new SimpleClass([1, 2, 3, 4, 5]);

for (const val of simple) {
  console.log(val); // 1 2 3 4 5
}
```

Overriding built-in iterables

For example, a `String` is a built-in iterable object:

js

```
const someString = "hi";
console.log(typeof someString[Symbol.iterator]); // "function"
```

`String`'s default iterator returns the string's code points one by one:

js

```
const iterator = someString[Symbol.iterator]();
console.log(`${iterator}`); // "[object String Iterator]"

console.log(iterator.next()); // { value: "h", done: false }
console.log(iterator.next()); // { value: "i", done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

You can redefine the iteration behavior by supplying your own `[Symbol.iterator]()`:

js

```
// need to construct a String object explicitly to avoid auto-boxing
const someString = new String("hi");

someString[Symbol.iterator] = function () {
  return {
    // this is the iterator object, returning a single element (the string "bye")
    next() {
      return this._first
        ? { value: "bye", done: (this._first = false) }
        : { done: true };
    },
    _first: true,
  };
};
```

Notice how redefining `[Symbol.iterator]()` affects the behavior of built-in constructs that use the iteration protocol:

js

```
console.log([...someString]); // ["bye"]
console.log(`${someString}`); // "hi"
```

Concurrent modifications when iterating

Almost all iterables have the same underlying semantic: they don't copy the data at the time when iteration starts. Rather, they keep a pointer and move it around. Therefore, if you add, delete, or modify elements in the collection while iterating over the collection, you may inadvertently change whether other *unchanged* elements in the collection are visited. This is very similar to how iterative array methods work.

Consider the following case using a `URLSearchParams`:

```js
const searchParams = new URLSearchParams(
  "deleteme1=value1&key2=value2&key3=value3",
);

// Delete unwanted keys
for (const [key, value] of searchParams) {
  console.log(key);
  if (key.startsWith("deleteme")) {
    searchParams.delete(key);
  }
}

// Output:
// deleteme1
// key3
```

Note how it never logs `key2`. This is because a `URLSearchParams` is underlyingly a list of key-value pairs. When `deleteme1` is visited and deleted, all other entries are shifted to the left by one, so `key2` occupies the position that `deleteme1` used to be in, and when the pointer moves to the next key, it lands on `key3`.

Certain iterable implementations avoid this problem by setting "tombstone" values to avoid shifting the remaining values. Consider the similar code using a `Map`:

```js
const myMap = new Map([
  ["deleteme1", "value1"],
  ["key2", "value2"],
  ["key3", "value3"],
]);

for (const [key, value] of myMap) {
  console.log(key);
  if (key.startsWith("deleteme")) {
    myMap.delete(key);
  }
}

// Output:
// deleteme1
// key2
// key3
```

Note how it logs all keys. This is because `Map` doesn't shift the remaining keys when one is deleted. If you want to implement something similar, here's how it may look:

```js
const tombstone = Symbol("tombstone");

class MyIterable {
  #data;
  constructor(data) {
    this.#data = data;
  }
  delete(deletedKey) {
    for (let i = 0; i < this.#data.length; i++) {
      if (this.#data[i][0] === deletedKey) {
        this.#data[i] = tombstone;
        return true;
      }
    }
    return false;
  }
  *[Symbol.iterator]() {
    for (const data of this.#data) {
      if (data !== tombstone) {
        yield data;
      }
    }
  }
}

const myIterable = new MyIterable([
  ["deleteme1", "value1"],
  ["key2", "value2"],
  ["key3", "value3"],
]);
for (const [key, value] of myIterable) {
  console.log(key);
  if (key.startsWith("deleteme")) {
    myIterable.delete(key);
  }
}
```

> **Warning:** Concurrent modifications, in general, are very bug-prone and confusing. Unless you know precisely how the iterable is implemented, it's best to avoid modifying the collection while iterating over it.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iteration

See also

- Iterators and generators guide
- `function*`
- `Symbol.iterator`
- `Iterator`

JavaScript › Reference

Strict mode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode · MDN page last modified 2026-11-02

> **Note:** Sometimes you'll see the default, non-strict mode referred to as *sloppy mode*. This isn't an official term, but be aware of it, just in case.

JavaScript's strict mode is a way to *opt in* to a restricted variant of JavaScript, thereby implicitly opting-out of "sloppy mode". Strict mode isn't just a subset: it *intentionally* has different semantics from normal code. Strict mode code and non-strict mode code can coexist, so scripts can opt into strict mode incrementally.

Strict mode makes several changes to normal JavaScript semantics:

1. Eliminates some JavaScript silent errors by changing them to throw errors.
2. Fixes mistakes that make it difficult for JavaScript engines to perform optimizations: strict mode code can sometimes be made to run faster than identical code that's not strict mode.
3. Prohibits some syntax likely to be defined in future versions of ECMAScript.

Invoking strict mode

Strict mode applies to *entire scripts* or to *individual functions*. It doesn't apply to block statements enclosed in `{}` braces; attempting to apply it to such contexts does nothing. `eval` code, `Function` code, event handler attributes, strings passed to `setTimeout()`, and related functions are either function bodies or entire scripts, and invoking strict mode in them works as expected.

Strict mode for scripts

To invoke strict mode for an entire script, put the *exact* statement `"use strict";` (or `'use strict';`) before any other statements.

```js
// Whole-script strict mode syntax
"use strict";
const v = "Hi! I'm a strict mode script!";
```

Strict mode for functions

Likewise, to invoke strict mode for a function, put the *exact* statement `"use strict";` (or `'use strict';`) in the function's body before any other statements.

```js
function myStrictFunction() {
  // Function-level strict mode syntax
  "use strict";
  function nested() {
    return "And so am I!";
  }
  return `Hi! I'm a strict mode function! ${nested()}`;
}
function myNotStrictFunction() {
  return "I'm not strict.";
}
```

The `"use strict"` directive can only be applied to the body of functions with simple parameters. Using `"use strict"` in functions with rest, default, or destructured parameters is a syntax error.

js · ✗ bad example

```
function sum(a = 1, b = 2) {
  // SyntaxError: "use strict" not allowed in function with default parameter
  "use strict";
  return a + b;
}
```

Strict mode for modules

The entire contents of JavaScript modules are automatically in strict mode, with no statement needed to initiate it.

js

```
function myStrictFunction() {
  // because this is a module, I'm strict by default
}
export default myStrictFunction;
```

Strict mode for classes

All parts of a class's body are strict mode code, including both class declarations and class expressions.

js

```
class C1 {
  // All code here is evaluated in strict mode
  test() {
    delete Object.prototype;
  }
}
new C1().test(); // TypeError, because test() is in strict mode

const C2 = class {
  // All code here is evaluated in strict mode
};

// Code here may not be in strict mode
delete Object.prototype; // Will not throw error
```

Changes in strict mode

Strict mode changes both syntax and runtime behavior. Changes generally fall into these categories:

- changes converting mistakes into errors (as syntax errors or at runtime)
- changes simplifying how variable references are resolved
- changes simplifying `eval` and `arguments`
- changes making it easier to write "secure" JavaScript
- changes anticipating future ECMAScript evolution.

Converting mistakes into errors

Strict mode changes some previously-accepted mistakes into errors. JavaScript was designed to be easy for novice developers, and sometimes it gives operations which should be errors non-error semantics.

Sometimes this fixes the immediate problem, but sometimes this creates worse problems in the future. Strict mode treats these mistakes as errors so that they're discovered and promptly fixed.

Assigning to undeclared variables

Strict mode makes it impossible to accidentally create global variables. In sloppy mode, mistyping a variable in an assignment creates a new property on the global object and continues to "work". Assignments which would accidentally create global variables throw an error in strict mode:

```js
"use strict";
let mistypeVariable;

// Assuming no global variable mistypeVarible exists
// this line throws a ReferenceError due to the
// misspelling of "mistypeVariable" (lack of an "a")
mistypeVarible = 17;
```

Failing to assign to object properties

In strict mode, certain assignments throw errors instead of failing silently. There are three ways to fail a property assignment:

- assignment to a non-writable data property
- assignment to a getter-only accessor property
- assignment to a new property on a non-extensible object

For example, `NaN` is a non-writable global variable. In sloppy mode, assigning to `NaN` does nothing; the developer receives no failure feedback. In strict mode, assigning to `NaN` throws an exception.

```js
"use strict";

// Assignment to a non-writable global
undefined = 5; // TypeError
Infinity = 5; // TypeError

// Assignment to a non-writable property
const obj1 = {};
Object.defineProperty(obj1, "x", { value: 42, writable: false });
obj1.x = 9; // TypeError

// Assignment to a getter-only property
const obj2 = {
  get x() {
    return 17;
  },
};
obj2.x = 5; // TypeError

// Assignment to a new property on a non-extensible object
const fixed = {};
Object.preventExtensions(fixed);
fixed.newProp = "ohai"; // TypeError
```

Failing to delete object properties

Attempts to delete a non-configurable or otherwise undeletable (e.g., it's intercepted by a proxy's `deleteProperty` handler which returns `false`) property throw in strict mode (where before the attempt would have no effect):

js

```
"use strict";
delete Object.prototype; // TypeError
delete [].length; // TypeError
```

Strict mode also forbids deleting plain names. `delete name` in strict mode is a syntax error:

js · ✗ bad example

```
"use strict";

var x;
delete x; // syntax error
```

If the name is a configurable global property, prefix it with `globalThis` to delete it.

js · ✓ good example

```
"use strict";

delete globalThis.x;
```

Duplicate parameter names

Strict mode requires that function parameter names be unique. In sloppy mode, the last duplicated argument hides previous identically-named arguments. Those previous arguments remain available through `arguments`, so they're not completely inaccessible. Still, this hiding makes little sense and is probably undesirable (it might hide a typo, for example), so in strict mode, duplicate argument names are a syntax error:

js · ✗ bad example

```
function sum(a, a, c) {
  // syntax error
  "use strict";
  return a + a + c; // wrong if this code ran
}
```

It is also a syntax error in non-strict mode to have duplicate parameter names, if the function has a default parameter, rest parameter, or destructured parameter.

Legacy octal literals

Strict mode forbids a `0`-prefixed octal literal. In sloppy mode, a number beginning with a `0`, such as `0644`, is interpreted as an octal number (`0644 === 420`), if all digits are smaller than 8. Novice developers sometimes believe a leading-zero prefix has no semantic meaning, so they might use it as an alignment device — but this changes the number's meaning! A leading-zero syntax for the octal is rarely useful and can be mistakenly used, so strict mode makes it a syntax error:

js · ✗ bad example

```
"use strict";
const sum =
  015 + // syntax error
  197 +
  142;
```

The standardized way to denote octal literals is via the `0o` prefix. For example:

js · ✓ good example

```
const sumWithOctal = 0o10 + 8;
console.log(sumWithOctal); // 16
```

Octal escape sequences, such as `"\45"`, which is equal to `"%"`, can be used to represent characters by extended-ASCII character code numbers in octal. In strict mode, this is a syntax error. More formally, it's disallowed to have `\` followed by any decimal digit other than `0`, or `\0` followed by a decimal digit; for example `\9` and `\07`.

Setting properties on primitive values

Strict mode forbids setting properties on primitive values. Accessing a property on a primitive implicitly creates a wrapper object that's unobservable, so in sloppy mode, setting properties is ignored (no-op). In strict mode, a `TypeError` is thrown.

js

```
"use strict";

false.true = ""; // TypeError
(14).sailing = "home"; // TypeError
"with".you = "far away"; // TypeError
```

Duplicate property names

Duplicate property names used to be considered a `SyntaxError` in strict mode. With the introduction of computed property names, making duplication possible at runtime, this restriction was removed in ES2015.

js

```
"use strict";
const o = { p: 1, p: 2 }; // syntax error prior to ECMAScript 2015
```

> **Note:** Making code that used to error become non-errors is always considered backwards-compatible. This is a good part of the language being strict about throwing errors: it leaves room for future semantic changes.

Simplifying scope management

Strict mode simplifies how variable names map to particular variable definitions in the code. Many compiler optimizations rely on the ability to say that variable *X* is stored in *that* location: this is critical to fully optimizing JavaScript code. JavaScript sometimes makes this basic mapping of name to variable definition in the code impossible to perform until runtime. Strict mode removes most cases where this happens, so the compiler can better optimize strict mode code.

Removal of the with statement

Strict mode prohibits `with`. The problem with `with` is that any name inside the block might map either to a property of the object passed to it, or to a variable in surrounding (or even global) scope, at runtime; it's impossible to know which beforehand. Strict mode makes `with` a syntax error, so there's no chance for a name in a `with` to refer to an unknown location at runtime:

js · ✗ bad example

```
"use strict";
const x = 17;
with (obj) {
  // Syntax error
  // If this weren't strict mode, would this be const x, or
  // would it instead be obj.x? It's impossible in general
  // to say without running the code, so the name can't be
  // optimized.
  x;
}
```

The alternative of assigning the object to a short name variable, then accessing the corresponding property on that variable, stands ready to replace `with`.

Non-leaking eval

In strict mode, eval does not introduce new variables into the surrounding scope. In sloppy mode, `eval("var x;")` introduces a variable `x` into the surrounding function or the global scope. This means that, in general, in a function containing a call to `eval`, every name not referring to an argument or local variable must be mapped to a particular definition at runtime (because that `eval` might have introduced a new variable that would hide the outer variable). In strict mode, `eval` creates variables only for the code being evaluated, so `eval` can't affect whether a name refers to an outer variable or some local variable:

js

```
var x = 17;
var evalX = eval("'use strict'; var x = 42; x;");
console.assert(x === 17);
console.assert(evalX === 42);
```

Whether the string passed to `eval()` is evaluated in strict mode depends on how `eval()` is invoked (direct eval or indirect eval).

Block-scoped function declarations

The JavaScript language specification, since its start, had not allowed function declarations nested in block statements. However, it was so intuitive that most browsers implemented it as an extension grammar. Unfortunately, the implementations' semantics diverged, and it became impossible for the language specification to reconcile all implementations. Therefore, block-scoped function declarations are only explicitly specified in strict mode (whereas they were once disallowed in strict mode), while sloppy mode behavior remains divergent among browsers.

Making eval and arguments simpler

Strict mode makes `arguments` and `eval` less bizarrely magical. Both involve a considerable amount of magical behavior in sloppy mode: `eval` to add or remove bindings and to change binding values, and `arguments` syncing named arguments with its indexed properties. Strict mode makes great strides toward treating `eval` and `arguments` as keywords.

Preventing binding or assigning eval and arguments

The names `eval` and `arguments` can't be bound or assigned in language syntax. All these attempts to do so are syntax errors:

js · ✗ bad example

```
"use strict";
eval = 17;
arguments++;
++eval;
const obj = { set p(arguments) {} };
let eval;
try {
} catch (arguments) {}
function x(eval) {}
function arguments() {}
const y = function eval() {};
const f = new Function("arguments", "'use strict'; return 17;");
```

No syncing between parameters and arguments indices

Strict mode code doesn't sync indices of the `arguments` object with each parameter binding. In a sloppy mode function whose first argument is `arg`, setting `arg` also sets `arguments[0]`, and vice versa (unless no arguments were provided or `arguments[0]` is deleted). `arguments` objects for strict mode functions store the original arguments when the function was invoked. `arguments[i]` does not track the value of the corresponding named argument, nor does a named argument track the value in the corresponding `arguments[i]`.

js

```
function f(a) {
  "use strict";
  a = 42;
  return [a, arguments[0]];
}
const pair = f(17);
console.assert(pair[0] === 42);
console.assert(pair[1] === 17);
```

"Securing" JavaScript

Strict mode makes it easier to write "secure" JavaScript. Some websites now provide ways for users to write JavaScript which will be run by the website *on behalf of other users*. JavaScript in browsers can access the user's private information, so such JavaScript must be partially transformed before it is run, to censor access to forbidden functionality. JavaScript's flexibility makes it effectively impossible to do this without many runtime checks. Certain language functions are so pervasive that performing runtime checks has a considerable performance cost. A few strict mode tweaks, plus requiring that user-submitted JavaScript be strict mode code and that it be invoked in a certain manner, substantially reduce the need for those runtime checks.

No this substitution

The value passed as `this` to a function in strict mode is not forced into being an object (a.k.a. "boxed"). For a sloppy mode function, `this` is always an object: either the provided object, if called with an object-valued `this`; or the boxed value of `this`, if called with a primitive as `this`; or the global object, if called with `undefined` or `null` as `this`. (Use `call`, `apply`, or `bind` to specify a particular `this`.) Not only is automatic boxing a performance cost, but exposing the global object in browsers is a security hazard because the global object provides access to functionality that "secure" JavaScript environments must restrict. Thus for a strict mode function, the specified `this` is not boxed into an object, and if unspecified, `this` is `undefined` instead of `globalThis`:

```js
"use strict";
function fun() {
  return this;
}
console.assert(fun() === undefined);
console.assert(fun.call(2) === 2);
console.assert(fun.apply(null) === null);
console.assert(fun.call(undefined) === undefined);
console.assert(fun.bind(true)() === true);
```

Removal of stack-walking properties

In strict mode it's no longer possible to "walk" the JavaScript stack. Many implementations used to implement some extension features that make it possible to detect the upstream caller of a function. When a function `fun` is in the middle of being called, `fun.caller` is the function that most recently called `fun`, and `fun.arguments` is the `arguments` for that invocation of `fun`. Both extensions are problematic for "secure" JavaScript because they allow "secured" code to access "privileged" functions and their (potentially unsecured) arguments. If `fun` is in strict mode, both `fun.caller` and `fun.arguments` are properties which throw when set or retrieved:

```js
function restricted() {
  "use strict";
  restricted.caller; // throws a TypeError
  restricted.arguments; // throws a TypeError
}
function privilegedInvoker() {
  return restricted();
}
privilegedInvoker();
```

Similarly, `arguments.callee` is no longer supported. In sloppy mode, `arguments.callee` refers to the enclosing function. This use case is weak: name the enclosing function! Moreover, `arguments.callee` substantially hinders optimizations like inlining functions, because it must be made possible to provide a reference to the un-inlined function if `arguments.callee` is accessed. `arguments.callee` for strict mode functions is a non-deletable property which throws an error when set or retrieved:

```js
"use strict";
function f() {
  return arguments.callee;
}
f(); // throws a TypeError
```

Future-proofing JavaScript

Extra reserved words

Reserved words are identifiers that can't be used as variable names. Strict mode reserves some more names than sloppy mode, some of which are already used in the language, and some of which are reserved for the future to make future syntax extensions easier to implement.

- `implements`
- `interface`
- `let`

- `package`
- `private`
- `protected`
- `public`
- `static`
- `yield`

Transitioning to strict mode

Strict mode has been designed so that the transition to it can be made gradually. It is possible to change each file individually and even to transition code to strict mode down to the function granularity.

You can migrate a codebase to strict mode by first adding `"use strict"` to a piece of source code, and then fixing all execution errors, while watching out for semantic differences.

Syntax errors

When adding `'use strict';`, the following cases will throw a `SyntaxError` before the script executes:

- Octal syntax `const n = 023;`
- `with` statement
- Using `delete` on a variable name `delete myVariable;`
- Using `eval` or `arguments` as variable or function argument name
- Using one of the newly reserved keywords (in anticipation of future language features): `implements`, `interface`, `let`, `package`, `private`, `protected`, `public`, `static`, and `yield`
- Declaring two function parameters with the same name `function f(a, b, b) {}`
- Declaring the same property name twice in an object literal `{a: 1, b: 3, a: 7}`. This constraint was later removed (bug 1041128).

These errors are good, because they reveal plain errors or bad practices. They occur before the code is running, so they are easily discoverable as long as the code gets parsed by the runtime.

New runtime errors

JavaScript used to silently fail in contexts where what was done should be an error. Strict mode throws in such cases. If your code base contains such cases, testing will be necessary to be sure nothing is broken. You can screen for such errors at the function granularity level.

- Assigning to an undeclared variable throws a `ReferenceError`. This used to set a property on the global object, which is rarely the expected effect. If you really want to set a value to the global object, explicitly assign it as a property on `globalThis`.
- Failing to assign to an object's property (e.g., it's read-only) throws a `TypeError`. In sloppy mode, this would silently fail.
- Deleting a non-deletable property throws a `TypeError`. In sloppy mode, this would silently fail.
- Accessing `arguments.callee`, `strictFunction.caller`, or `strictFunction.arguments` throws a `TypeError` if the function is in strict mode. If you are using `arguments.callee` to call the function recursively, you can use a named function expression instead.

Semantic differences

These differences are very subtle differences. It's possible that a test suite doesn't catch this kind of subtle difference. Careful review of your code base will probably be necessary to be sure these differences don't affect the semantics of your code. Fortunately, this careful review can be done gradually down the function granularity.

`this`

In sloppy mode, function calls like `f()` would pass the global object as the `this` value. In strict mode, it is now `undefined`. When a function was called with `call` or `apply`, if the value was a primitive value, this one was boxed into an object (or the global object for `undefined` and `null`). In strict mode, the value is passed directly without conversion or replacement.

`arguments`

In sloppy mode, modifying a value in the `arguments` object modifies the corresponding named argument. This made optimizations complicated for JavaScript engines and made code harder to read/understand. In strict mode, the `arguments` object is created and initialized with the same values as the named arguments, but changes to either the `arguments` object or the named arguments aren't reflected in one another.

`eval`

In strict mode code, `eval` doesn't create a new variable in the scope from which it was called. Also, of course, in strict mode, the string is evaluated with strict mode rules. Thorough testing will need to be performed to make sure nothing breaks. Not using eval if you don't really need it may be another pragmatic solution.

Block-scoped function declarations

In sloppy mode, a function declaration inside a block may be visible outside the block and even callable. In strict mode, a function declaration inside a block is only visible inside the block.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/strict-mode-of-ecmascript.html`

See also

- JavaScript modules guide
- Lexical grammar

JavaScript › Reference

Template literals (Template strings)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals · MDN page last modified 2026-11-02

Baseline: Widely available — available across major browsers since September 2015.

Template literals are literals delimited with backtick (`) characters, allowing for multi-line strings, string interpolation with embedded expressions, and special constructs called tagged templates.

Template literals are sometimes informally called *template strings*, because they are used most commonly for string interpolation (to create strings by doing substitution of placeholders). However, a tagged template literal may not result in a string; it can be used with a custom tag function to perform whatever operations you want on the different parts of the template literal.

Syntax

```js
`string text`

`string text line 1
 string text line 2`

`string text ${expression} string text`

tagFunction`string text ${expression} string text`
```

Parameters

`string text`

The string text that will become part of the template literal. Almost all characters are allowed literally, including line breaks and other whitespace characters. However, invalid escape sequences will cause a syntax error, unless a tag function is used.

`expression`

An expression to be inserted in the current position, whose value is converted to a string or passed to `tagFunction`.

`tagFunction`

If specified, it will be called with the template strings array and substitution expressions, and the return value becomes the value of the template literal. See tagged templates.

Description

Template literals are enclosed by backtick (`) characters instead of double or single quotes.

Along with having normal strings, template literals can also contain other parts called *placeholders*, which are embedded expressions delimited by a dollar sign and curly braces: `${expression}`. The strings and placeholders get passed to a function — either a default function, or a function you supply. The default function (when you don't supply your own) just performs string interpolation to do substitution of the placeholders and then concatenate the parts into a single string.

To supply a function of your own, precede the template literal with a function name; the result is called a **tagged template**. In that case, the template literal is passed to your tag function, where you can then perform whatever operations you want on the different parts of the template literal.

To escape a backtick in a template literal, put a backslash (\) before the backtick.

```js
`\`` === "`"; // true
```

Dollar signs can be escaped as well to prevent interpolation.

```js
`\${1}` === "${1}"; // true
```

Multi-line strings

Any newline characters inserted in the source are part of the template literal.

Using normal strings, you would have to use the following syntax in order to get multi-line strings:

```js
console.log("string text line 1\nstring text line 2");
// "string text line 1
// string text line 2"
```

Using template literals, you can do the same with this:

```js
console.log(`string text line 1
string text line 2`);
// "string text line 1
// string text line 2"
```

Like normal string literals, you can write a single-line string across multiple lines for source code readability, by escaping the newline with a backslash (\):

```js
console.log(`string text line 1 \
string text line 2`);
// "string text line 1 string text line 2"
```

String interpolation

Without template literals, when you want to combine output from expressions with strings, you'd concatenate them using the addition operator +:

```js
const a = 5;
const b = 10;
console.log("Fifteen is " + (a + b) + " and\nnot " + (2 * a + b) + ".");
// "Fifteen is 15 and
// not 20."
```

That can be hard to read – especially when you have multiple expressions.

With template literals, you can avoid the concatenation operator — and improve the readability of your code — by using placeholders of the form `${expression}` to perform substitutions for embedded expressions:

```js
const a = 5;
const b = 10;
console.log(`Fifteen is ${a + b} and
not ${2 * a + b}.`);
// "Fifteen is 15 and
// not 20."
```

Note that there's a mild difference between the two syntaxes. Template literals coerce their expressions directly to strings, while addition coerces its operands to primitives first. For more information, see the reference page for the + operator.

Nesting templates

In certain cases, nesting a template is the easiest (and perhaps more readable) way to have configurable strings. Within a backtick-delimited template, it is simple to allow inner backticks by using them inside an `${expression}` placeholder within the template.

For example, without template literals, if you wanted to return a certain value based on a particular condition, you could do something like the following:

js · ✗ bad example

```
let classes = "header";
classes += isLargeScreen()
  ? ""
  : item.isCollapsed
    ? " icon-expander"
    : " icon-collapser";
```

With a template literal but without nesting, you could do this:

js · ✗ bad example

```
const classes = `header ${
  isLargeScreen() ? "" : item.isCollapsed ? "icon-expander" : "icon-collapser"
}`;
```

With nesting of template literals, you can do this:

js · ✓ good example

```
const classes = `header ${
  isLargeScreen() ? "" : `icon-${item.isCollapsed ? "expander" : "collapser"}`
}`;
```

Tagged templates

Tagged templates are a more advanced form of template literals.

Tags allow you to parse template literals with a function. The first argument of a tag function contains an array of string values. The remaining arguments are related to the expressions.

The tag function can then perform whatever operations on these arguments you wish, and return the manipulated string. (Alternatively, it can return something completely different, as described in one of the following examples.)

The name of the function used for the tag can be whatever you want.

```js
const person = "Mike";
const age = 28;

function myTag(strings, personExp, ageExp) {
  const str0 = strings[0]; // "That "
  const str1 = strings[1]; // " is a "
  const str2 = strings[2]; // "."

  const ageStr = ageExp < 100 ? "youngster" : "centenarian";

  // We can even return a string built using a template literal
  return `${str0}${personExp}${str1}${ageStr}${str2}`;
}

const output = myTag`That ${person} is a ${age}.`;

console.log(output);
// That Mike is a youngster.
```

The tag does not have to be a plain identifier. You can use any expression with precedence greater than 16, which includes property access, function call, new expression, or even another tagged template literal.

```js
console.log`Hello`; // [ 'Hello' ]
console.log.bind(1, 2)`Hello`; // 2 [ 'Hello' ]
new Function("console.log(arguments)")`Hello`; // [Arguments] { '0': [ 'Hello' ] }

function recursive(strings, ...values) {
  console.log(strings, values);
  return recursive;
}
recursive`Hello``World`;
// [ 'Hello' ] []
// [ 'World' ] []
```

While technically permitted by the syntax, *untagged* template literals are strings and will throw a TypeError when chained.

```js
console.log(`Hello``World`); // TypeError: "Hello" is not a function
```

The only exception is optional chaining, which will throw a syntax error.

js · ✗ bad example

```
console.log?.`Hello`; // SyntaxError: Invalid tagged template on optional chain
console?.log`Hello`; // SyntaxError: Invalid tagged template on optional chain
```

Note that these two expressions are still parsable. This means they would not be subject to automatic semicolon insertion, which will only insert semicolons to fix code that's otherwise unparsable.

js · ✗ bad example

```
// Still a syntax error
const a = console?.log
`Hello`
```

Tag functions don't even need to return a string!

```js
function template(strings, ...keys) {
  return (...values) => {
    const dict = values[values.length - 1] || {};
    const result = [strings[0]];
    keys.forEach((key, i) => {
      const value = Number.isInteger(key) ? values[key] : dict[key];
      result.push(value, strings[i + 1]);
    });
    return result.join("");
  };
}

const t1Closure = template`${0}${1}${0}!`;
// const t1Closure = template(["","","","!"],0,1,0);
t1Closure("Y", "A"); // "YAY!"

const t2Closure = template`${0} ${"foo"}!`;
// const t2Closure = template(["","," ","!"],0,"foo");
t2Closure("Hello", { foo: "World" }); // "Hello World!"

const t3Closure = template`I'm ${"name"}. I'm almost ${"age"} years old.`;
// const t3Closure = template(["I'm ", ". I'm almost ", " years old."], "name", "age");
t3Closure("foo", { name: "MDN", age: 30 }); // "I'm MDN. I'm almost 30 years old."
t3Closure({ name: "MDN", age: 30 }); // "I'm MDN. I'm almost 30 years old."
```

The first argument received by the tag function is an array of strings. For any template literal, its length is equal to the number of substitutions (occurrences of `${…}`) plus one, and is therefore always non-empty.

For any particular tagged template literal expression, the tag function will always be called with the exact same literal array, no matter how many times the literal is evaluated.

```js
const callHistory = [];

function tag(strings, ...values) {
  callHistory.push(strings);
  // Return a freshly made object
  return {};
}

function evaluateLiteral() {
  return tag`Hello, ${"world"}!`;
}

console.log(evaluateLiteral() === evaluateLiteral()); // false; each time `tag` is called,
it returns a new object
console.log(callHistory[0] === callHistory[1]); // true; all evaluations of the same tagged
literal would pass in the same strings array
```

This allows the tag to cache the result based on the identity of its first argument. To further ensure the array value's stability, the first argument and its `raw` property are both frozen, so you can't mutate them in any way.

Raw strings

The special `raw` property, available on the first argument to the tag function, allows you to access the raw strings as they were entered, without processing escape sequences.

```js
function tag(strings) {
  console.log(strings.raw[0]);
}

tag`string text line 1 \n string text line 2`;
// Logs "string text line 1 \n string text line 2",
// including the two characters '\' and 'n'
```

> **Note:** The template literal syntax is still processed in the same way, which means unescaped backticks and `${` have special syntactic meaning, but escaping these characters creates extra backslashes in the raw string. See raw strings containing template literal syntax for more information.

In addition, the `String.raw()` method exists to create raw strings just like the default template function and string concatenation would create.

```js
const str = String.raw`Hi\n${2 + 3}!`;
// "Hi\\n5!"

str.length;
// 6

Array.from(str).join(",");
// "H,i,\\,n,5,!"
```

`String.raw` functions like an "identity" tag if the literal doesn't contain any escape sequences. In case you want an actual identity tag that always works as if the literal is untagged, you can make a custom function that passes the "cooked" (i.e., escape sequences are processed) literal array to `String.raw`, pretending they are raw strings.

```js
const identity = (strings, ...values) =>
  String.raw({ raw: strings }, ...values);
console.log(identity`Hi\n${2 + 3}!`);
// Hi
// 5!
```

This is useful for many tools which give special treatment to literals tagged by a particular name.

```js
const html = (strings, ...values) => String.raw({ raw: strings }, ...values);
// Some formatters will format this literal's content as HTML
const doc = html`<!doctype html>
  <html lang="en-US">
    <head>
      <title>Hello</title>
    </head>
    <body>
      <h1>Hello world!</h1>
    </body>
  </html>`;
```

Tagged templates and escape sequences

In normal template literals, the escape sequences in string literals are all allowed. Any other non-well-formed escape sequence is a syntax error. This includes:

- `\` followed by any decimal digit other than `0`, or `\0` followed by a decimal digit; for example `\9` and `\07` (which is a deprecated syntax)
- `\x` followed by fewer than two hex digits (including none); for example `\xz`
- `\u` not followed by `{` and followed by fewer than four hex digits (including none); for example `\uz`
- `\u{}` enclosing an invalid Unicode code point — it contains a non-hex digit, or its value is greater than `10FFFF`; for example `\u{110000}` and `\u{z}`

> **Note:** `\` followed by other characters, while they may be useless since nothing is escaped, are not syntax errors.

However, this is problematic for tagged templates, which, in addition to the "cooked" literal, also have access to the raw literals (escape sequences are preserved as-is).

Tagged templates enable the embedding of arbitrary string content, where escape sequences may follow a different syntax. Consider an example where we embed LaTeX source text in JavaScript via `String.raw`. We want to still be able to use LaTeX macros that start with `u` or `x` without following JavaScript syntax restrictions. Therefore, the syntax restriction of well-formed escape sequences is removed from tagged templates. The example below uses MathJax to render LaTeX in one element:

```js
const node = document.getElementById("formula");
MathJax.typesetClear([node]);
// Throws in older ECMAScript versions (ES2016 and earlier)
// SyntaxError: malformed Unicode character escape sequence
node.textContent = String.raw`$\underline{u}$`;
MathJax.typesetPromise([node]);
```

However, illegal escape sequences must still be represented in the "cooked" representation. They will show up as an `undefined` element in the "cooked" array:

```js
function log(str) {
  console.log("Cooked:", str[0]);
  console.log("Raw:", str.raw[0]);
}

log`\unicode`;
// Cooked: undefined
// Raw: \unicode
```

Note that the escape-sequence restriction is only dropped from *tagged* templates, but not from *untagged* template literals:

js · ✗ bad example

```
const bad = `bad escape sequence: \unicode`;
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-template-literals`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Template literals	41	12	34	28	9	41	34	28	9	4.0	41	9	1.0.0	1.0	4.0.0
Escape sequences allowed in tagged template literals	62	79	53	49	11	62	53	46	11	8.0	62	11	1.1.35	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.grammar.template_literals`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Numbers and strings guide
- `String`
- `String.raw()`
- Lexical grammar
- ES6 in Depth: Template strings on hacks.mozilla.org (2015)

JavaScript › Reference

Trailing commas

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Trailing_commas · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

Trailing commas (sometimes called "final commas") can be useful when adding new elements, parameters, or properties to JavaScript code. If you want to add a new property, you can add a new line without modifying the previously last line if that line already uses a trailing comma. This makes version-control diffs cleaner and editing code might be less troublesome.

JavaScript has allowed trailing commas in array literals since the beginning. Trailing commas are now also allowed in object literals, function parameters, named imports, named exports, and more.

JSON, however, disallows all trailing commas.

Description

JavaScript allows trailing commas wherever a comma-separated list of values is accepted and more values may be expected after the last item. This includes:

- Array literals
- Object literals
- Parameter definitions
- Function calls
- Named imports
- Named exports
- Dynamic import
- Array and object destructuring

In all these cases, the trailing comma is entirely optional and doesn't change the program's semantics in any way.

It is particularly useful when adding, removing, or reordering items in a list that spans multiple lines, because it reduces the number of lines that need to be changed, which helps with both editing and reviewing the diff.

diff

```
  [
    "foo",
+   "baz",
    "bar",
-   "baz",
  ]
```

Examples

Trailing commas in literals

Arrays

JavaScript ignores trailing commas in array literals:

```js
const arr = [
  1,
  2,
  3,
];

arr; // [1, 2, 3]
arr.length; // 3
```

If more than one trailing comma is used, an elision (or hole) is produced. An array with holes is called *sparse* (a *dense* array has no holes). When iterating arrays for example with `Array.prototype.forEach()` or `Array.prototype.map()`, array holes are skipped. Sparse arrays are generally unfavorable, so you should avoid having multiple trailing commas.

```js
const arr = [1, 2, 3, , ,];
arr.length; // 5
```

Objects

Trailing commas in object literals are legal as well:

```js
const object = {
  foo: "bar",
  baz: "qwerty",
  age: 42,
};
```

Trailing commas in functions

Trailing commas are also allowed in function parameter lists.

Parameter definitions

The following function definition pairs are legal and equivalent to each other. Trailing commas don't affect the `length` property of function declarations or their `arguments` object.

```js
function f(p) {}
function f(p,) {}

(p) => {};
(p,) => {};
```

The trailing comma also works with method definitions for classes or objects:

```js
class C {
  one(a,) {}
  two(a, b,) {}
}

const obj = {
  one(a,) {},
  two(a, b,) {},
};
```

Function calls

The following function invocation pairs are legal and equivalent to each other.

```js
f(p);
f(p,);

Math.max(10, 20);
Math.max(10, 20,);
```

Illegal trailing commas

Function parameter definitions or function invocations only containing a comma will throw a `SyntaxError`. Furthermore, when using rest parameters, trailing commas are not allowed:

```js · ✗ bad example
function f(,) {} // SyntaxError: missing formal parameter
(,) => {};       // SyntaxError: expected expression, got ','
f(,)             // SyntaxError: expected expression, got ','

function f(...p,) {} // SyntaxError: parameter after rest parameter
(...p,) => {}        // SyntaxError: expected closing parenthesis, got ','
```

Trailing commas in destructuring

A trailing comma is also allowed within a destructuring pattern:

```js
// array destructuring with trailing comma
[a, b,] = [1, 2];

// object destructuring with trailing comma
const o = {
  p: 42,
  q: true,
};
const { p, q, } = o;
```

However, a trailing comma is not allowed after the rest element, if present

```js · ✗ bad example
const [a, ...b,] = [1, 2, 3];
// SyntaxError: rest element may not have a trailing comma
```

Trailing commas in JSON

As JSON is based on a very restricted subset of JavaScript syntax, **trailing commas are not allowed in JSON**.

Both lines will throw a `SyntaxError`:

```js · ✗ bad example
JSON.parse("[1, 2, 3, 4, ]");
JSON.parse('{"foo" : 1, }');
// SyntaxError JSON.parse: unexpected character
// at line 1 column 14 of the JSON data
```

Omit the trailing commas to parse the JSON correctly:

js · ✓ good example

```
JSON.parse("[1, 2, 3, 4 ]");
JSON.parse('{"foo" : 1 }');
```

Trailing commas in named imports and named exports

Trailing commas are valid in named imports and named exports.

Named imports

js

```
import {
  A,
  B,
  C,
} from "D";

import { X, Y, Z, } from "W";

import { A as B, C as D, E as F, } from "Z";
```

Named exports

js

```
export {
  A,
  B,
  C,
};

export { A, B, C, };

export { A as B, C as D, E as F, };
```

Trailing commas in dynamic import

Trailing commas are only allowed in dynamic imports if the runtime also implements the second `options` parameter.

js

```
import("D",);
import(
  "D",
  { with: { type: "json" } },
);
```

Quantifier prefix

> **Note:** The trailing comma in a quantifier actually changes its semantics from matching "exactly `n`" to matching "at least `n`".

js

```
/x{2}/; // Exactly 2 occurrences of "x"; equivalent to /xx/
/x{2,}/; // At least 2 occurrences of "x"; equivalent to /xx+/
/x{2,4}/; // 2 to 4 occurrences of "x"; equivalent to /xxx?x?/
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-Elision
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ObjectLiteral
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ArrayLiteral
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-Arguments
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-FormalParameters
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-CoverParenthesizedExpressionAndArrowParameterList
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html#prod-NamedImports
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html#prod-NamedExports
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#prod-QuantifierPrefix
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#prod-annexB-InvalidBracedQuantifier

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Trailing commas	1	12	1	9.5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in dynamic import	91	91	138	**No**	15	91	138	**No**	15	16.0	91	15	1.0.0	1.17	17.5.0 [1,2,3]
Trailing comma in function parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
Trailing comma in object literals	1	12	1	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Also from version 16.15.0–17.0.0: supported.
2. Also from version 16.14.0–16.15.0: partial support.
3. Also from version 16.14.0–16.15.0: The second parameter no longer throws a parser error, but the `--experimental-json-modules` flag is still needed to load JSON modules.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.grammar.trailing_commas`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Grammar and types guide

JavaScript › Reference

Deprecated and obsolete features

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Deprecated_and_obsolete_features · MDN page last modified 2026-11-02

This page lists features of JavaScript that are deprecated (that is, still available but should not be used) and obsolete (that is, no longer usable).

Deprecated features

These deprecated features can still be used, but should be used with caution because they are not required to be implemented by every JavaScript engine. You should work to remove their use from your code.

Some of these deprecated features are listed in the Annex B section of the ECMAScript specification. This section is described as normative optional — that is, web browser hosts must implement these features, while non-web hosts may not. These features are likely stable because removing them will cause backward compatibility issues and break legacy websites. (JavaScript has the design goal of "don't break the web".) Still, they are not cross-platform portable and may not be supported by all analysis tools, so you are advised to not use them, as the introduction of Annex B states:

> … All of the language features and behaviors specified in this annex have one or more undesirable characteristics and in the absence of legacy usage would be removed from this specification. …
>
> … Programmers should not use or assume the existence of these features and behaviors when writing new ECMAScript code. …

Some others, albeit in the main spec body, are also marked as normative optional and should not be depended on.

HTML comments

JavaScript source, if parsed as scripts, allows HTML-like comments, as if the script is part of a `<script>` tag.

The following is valid JavaScript when running in a web browser (or Node.js, which uses the V8 engine powering Chrome):

```js
<!-- comment
console.log("a"); <!-- another comment
console.log("b");
--> More comment
// Logs "a" and "b"
```

`<!--` and `-->` both act like `//`, i.e., starting line comments. `-->` is only valid at the start of a line (to avoid ambiguity with a postfix decrement followed by a greater than operator), while `<!--` can occur anywhere in the line.

RegExp

The following properties are deprecated. This does not affect their use in replacement strings:

`$1–$9`

Parenthesized substring matches, if any.

`input`, `$_`
: The string against which a regular expression is matched.

`lastMatch`, `$&`
: The last matched substring.

`lastParen`, `$+`
: The last parenthesized substring match, if any.

**`leftContext`, `$`` **
: The substring preceding the most recent match.

`rightContext`, `$'`
: The substring following the most recent match.

> **Warning:** Avoid using these static properties, as they can cause issues when interacting with external code!

The `compile()` method is deprecated. Construct a new `RegExp` instance instead.

The following regex syntaxes are deprecated and only available in Unicode-unaware mode. In Unicode-aware mode, they are all syntax errors:

- Lookahead assertions can have quantifiers.
- Backreferences that do not refer to an existing capturing group become legacy octal escapes.
- In character classes, character ranges where one boundary is a character class make the `-` become a literal character.
- An escape sequence that's not recognized becomes an "identity escape".
- Escape sequences within character classes of the form `\cX` where `X` is a number or `_` are decoded in the same way as those with ASCII letters: `\c0` is the same as `\cP` when taken modulo 32. In addition, if the form `\cX` is encountered anywhere where `X` is not one of the recognized characters, then the backslash is treated as a literal character.
- The sequence `\k` within a regex that doesn't have any named capturing groups is treated as an identity escape.
- The syntax characters `]`, `{`, and `}` may appear literally without escaping if they cannot be interpreted as the end of a character class or quantifier delimiters.

Function

- The `caller` property of functions and the `arguments.callee` property are deprecated and unavailable in strict mode.
- Instead of accessing `arguments` as a property of a function, you should use the `arguments` object inside function closures.

Object

- The `Object.prototype.__proto__` accessors are deprecated. Use `Object.getPrototypeOf` and `Object.setPrototypeOf` instead. This does not apply to the `__proto__` literal key in object literals.
- The `Object.prototype.__defineGetter__`, `Object.prototype.__defineSetter__`, `Object.prototype.__lookupGetter__`, and `Object.prototype.__lookupSetter__` methods are deprecated. Use `Object.getOwnPropertyDescriptor` and `Object.defineProperty` instead.

String

- HTML wrapper methods like `String.prototype.fontsize` and `String.prototype.big`.
- `String.prototype.substr` probably won't be removed anytime soon, but it's defined in Annex B and hence normative optional.
- `String.prototype.trimLeft` and `String.prototype.trimRight` should be replaced with `String.prototype.trimStart` and `String.prototype.trimEnd`.

Date

- The `getYear()` and `setYear()` methods are affected by the Year-2000-Problem and have been subsumed by `getFullYear` and `setFullYear`.
- The `toGMTString()` method is deprecated. Use `toUTCString()` instead.

Escape sequences

- Octal escape sequences (\ followed by one, two, or three octal digits) are deprecated in string and regular expression literals.
- The `escape()` and `unescape()` functions are deprecated. Use `encodeURI()`, `encodeURIComponent()`, `decodeURI()`, or `decodeURIComponent()` to encode and decode escape sequences for special characters.

Statements

The `with` statement is deprecated and unavailable in strict mode.

Initializers in `var` declarations of `for...in` loop headers are deprecated and produce syntax errors in strict mode. The initializer expression is evaluated and assigned to the variable, but the value would be immediately reassigned on the first iteration of the loop.

Normally, the `catch` block of a `try...catch` statement cannot contain any variable declaration with the same name as the variables bound in the `catch()`. An extension grammar allows the `catch` block to contain a `var` declared variable with the same name as the `catch`-bound identifier, but only if the `catch` binding is a simple identifier, not a destructuring pattern. However, this variable's initialization and assignment would only act on the `catch`-bound identifier, instead of the upper scope variable, and the behavior could be confusing.

```js
var a = 2;
try {
  throw new Error();
} catch (a) {
  var a = 1; // This 1 is assigned to the caught `a`, not the outer `a`.
}
console.log(a); // 2

try {
  throw new Error();
  // Note: identifier changed to `err` to avoid conflict with
  // the inner declaration of `a`.
} catch (err) {
  var a = 1; // This 1 is assigned to the upper-scope `a`.
}
console.log(a); // 1
```

This is listed in Annex B of the spec and hence may not be implemented everywhere. Avoid any name conflicts between the `catch`-bound identifier and variables declared in the `catch` block.

Obsolete features

These obsolete features have been entirely removed from JavaScript and can no longer be used as of the indicated version of JavaScript.

RegExp

The following are now properties of `RegExp` instances, no longer of the `RegExp` constructor:

Property	Description
`global`	Whether or not to test the regular expression against all possible matches in a string, or only against the first.
`ignoreCase`	Whether or not to ignore case while attempting a match in a string.
`lastIndex`	The index at which to start the next match.
`multiline` (also via `RegExp.$*`)	Whether or not to search in strings across multiple lines.
`source`	The text of the pattern.

The `valueOf()` method is no longer specialized for `RegExp`. It uses `Object.prototype.valueOf()`, which returns itself.

Function

- Functions' `arity` property is obsolete. Use `length` instead.

Object

Property	Description	Alternative
`__count__`	Returns the number of enumerable properties directly available on a user-defined object.	`Object.keys()`
`__parent__`	Points to an object's context.	No direct replacement
`__iterator__`	Used with legacy iterators.	`Symbol.iterator` and the new iteration protocols
`__noSuchMethod__`	A method called when a non-existent property is called as a method.	`Proxy`
`Object.prototype.eval()`	Evaluates a string of JavaScript code in the context of the specified object.	No direct replacement
`Object.observe()`	Asynchronously observing the changes to an object.	`Proxy`
`Object.unobserve()`	Remove observers.	`Proxy`
`Object.getNotifier()`	Create a notifier object that allows you to synthetically trigger a change observable with `Object.observe()`.	No direct replacement
`Object.prototype.watch()`	Attach a handler callback to a property that gets called when the property is assigned.	`Proxy`
`Object.prototype.unwatch()`	Remove watch handlers on a property.	`Proxy`

String

- Non-standard String generic methods like `String.slice(myStr, 0, 12)`, `String.replace(myStr, /\./g, "!")`, etc. were introduced in Firefox 1.5 (JavaScript 1.6), deprecated in Firefox 53, and removed in Firefox 68. You can use methods on `String.prototype` together with `Function.call` instead.
- `String.prototype.quote` was removed in Firefox 37.
- The non-standard `flags` parameter in `String.prototype.search`, `String.prototype.match`, and `String.prototype.replace` is obsolete.

WeakMap

- `WeakMap.prototype.clear()` was added in Firefox 20 and removed in Firefox 46. It is not possible to traverse all keys in a `WeakMap`.

Date

- `Date.prototype.toLocaleFormat()`, which used a format string in the same format expected by the `strftime()` function in C, is obsolete. Use `toLocaleString()` or `Intl.DateTimeFormat` instead.

Array

- Non-standard Array generic methods like `Array.slice(myArr, 0, 12)`, `Array.forEach(myArr, myFn)`, etc. have been introduced in Firefox 1.5 (JavaScript 1.6), deprecated in Firefox 68, and removed in Firefox 71. You can use methods on `Array.prototype` together with `Function.call` instead.

Property	Description	Alternative
`Array.observe()`	Asynchronously observing changes to Arrays.	`Proxy`
`Array.unobserve()`	Remove observers.	`Proxy`

Number

- `Number.toInteger()` is obsolete. Use `Math.floor`, `Math.round`, or other methods instead.

Proxy

- `Proxy.create` and `Proxy.createFunction` are obsolete. Use the `Proxy()` constructor instead.
- The following traps are obsolete:
  - `hasOwn` (bug 980565, Firefox 33).
  - `getEnumerablePropertyKeys` (bug 783829, Firefox 37)
  - `getOwnPropertyNames` (bug 1007334, Firefox 33)
  - `keys` (bug 1007334, Firefox 33)

ParallelArray

- `ParallelArray` is obsolete.

Statements

- `for each...in` is obsolete. Use `for...of` instead.
- let blocks and let expressions are obsolete.

- Expression closures (`function () 1` as a shorthand of `function () { return 1; }`) are obsolete. Use regular `functions` or arrow functions instead.

Acquiring source text

The `toSource()` methods of arrays, numbers, strings, etc. and the `uneval()` global function are obsolete. Use `toString()`, or write your own serialization method instead.

Legacy generator and iterator

Legacy generator function statements and legacy generator function expressions are removed. The legacy generator function syntax reuses the `function` keyword, which automatically becomes a generator function when there are one or more `yield` expressions in the body — this is now a syntax error. Use `function*` statements and `function*` expressions instead.

Array comprehensions and generator comprehensions are removed.

```js
// Legacy array comprehensions
[for (x of iterable) x]
[for (x of iterable) if (condition) x]
[for (x of iterable) for (y of iterable) x + y]

// Legacy generator comprehensions
(for (x of iterable) x)
(for (x of iterable) if (condition) x)
(for (x of iterable) for (y of iterable) x + y)
```

Firefox, prior to version 26, implemented another iterator protocol that is similar to the standard Iterator protocol. An object is a legacy iterator when it implements a `next()` method, which produces a value on each call and throws a `StopIteration` object at the end of iteration. This legacy iterator protocol differs from the standard iterator protocol:

- The value was returned directly as the return value of calls to `next()`, instead of the `value` property of the `IteratorResult` object.
- Iteration termination was expressed by throwing a `StopIteration` object, instead of through the `done` property of the `IteratorResult` object.

This feature, along with the `StopIteration` global constructor, was removed in Firefox 58+. For future-facing usages, consider using `for...of` loops and the iterator protocol.

Sharp variables

Sharp variables are obsolete. To create circular structures, use temporary variables instead.